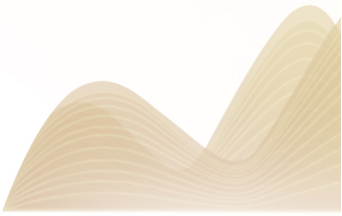
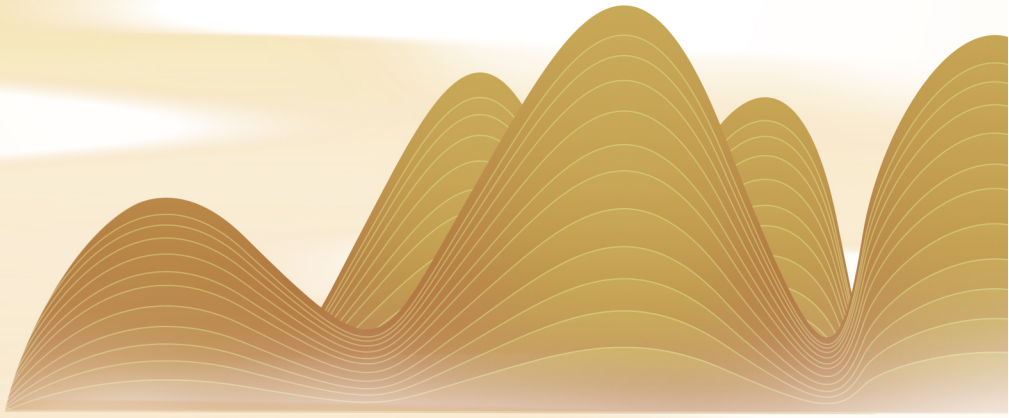
阿里云开发者社区
ALIBABA CLOUD DEVELOPER COMMUNITY

Java 工程师必读手册

一位阿里程序员的Java编码之道

作 者 陈昌毅（常意）





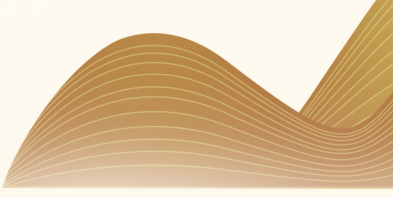
定风波·问道

莫惧山高道路**艰**，会当临顶俱欢**颜**。

热血少年身抖**擞**，先**走**，恰如一箭啸云**天**。

人困马乏车欲**倒**，还**早**，暗生惆怅望前**川**。

心定意坚终不**弃**，登**矣**，从今诸事有何**难**。



阿里云开发者“藏经阁”
海量电子手册免费下载

卷首语

古语有云：

> *道为术之灵，术为道之体；以道统术，以术得道。*

其中：“道”指“规律、道理、理论”，“术”指“方法、技巧、技术”。意思是：“道”是“术”的灵魂，“术”是“道”的实体；可以用“道”来统管“术”，也可以从“术”中获得“道”。

工匠追求“术”到极致，其实就是在寻“道”，且离悟“道”也就不远了，亦或是已经得道，这就是“工匠精神”——一种追求“以术得道”的精神。如果一个工匠只满足于“术”，不能追求“术”到极致去悟“道”，那只是一个靠“术”养家糊口的工匠而已。作者根据多年来的实践探索，总结了大量的 Java 编码之“术”，试图阐述出心中的 Java 编码之“道”。

目录

Java 经验之谈系列 5

编码方法论 5

那些年，我们踩过的那些 Java 坑 21

那些年，我们见过的 Java 服务端“乱象” 44

那些年，我们见过的 Java 服务端“问题” 63

Java 编程技巧系列 87

Java 编程技巧之输入输出参数 87

Java 编程技巧之数据结构 118

Java 编程技巧之样板代码 138

Java 单元测试系列 161

Java 单元测试技巧之 PowerMock 161

Java 单元测试技巧之编写流程 187

Java 单元测试技巧之 JSON 序列化 235

Java 极客主义系列 270

Java 代码精简之道 270

Java 代码高效之道 312

Java 函数优雅之道 347

最完美的 Lambda 表达式只有一行 382

卷尾语 410

➢ Java 经验之谈系列

编码方法论

导读

Don Roberts 提出的一条重构准则：

第一次做某件事时只管去做；
第二次做类似的事时会产生反感，但无论如何还是可以去做；
第三次再做类似的事时，你就应该重构。

编码也是如此，当多次编写类似的代码时，我们需要考虑是否有一种方法能够提高编码速度。作者多年来致力于敏捷开发，总结了一套编码的方法论，有助于程序员“快速、优质、高效”地进行编码。

方法 1：手工编写代码

大多数刚学习 Java 的程序员，都会怀着一种崇敬的仪式感，一字一句地在开发工具上敲出以下代码：

```
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello world!");
    }
}
```

没错，这就是经典的“Hello world”，这也是大多数人手工编写的第一个程序。

手工编写代码，更能体现一个程序员的基本素质。有很多公司，都把上机编程考试作为面试的重要手段之一。面试者需要根据题目的要求，挑选一款熟悉的编程工具（比如 Eclipse），手工编写代码并调试运行通过。在整个过程中，不能通过网络搜索答案，不能查看联机帮助文档，要求面试者必须手工编写代码，主要是考察面试者手工编写代码的能力——语法、函数、逻辑、思维、算法以及动手能力。

手工编写代码，是一个优秀程序员必须具备的基础能力。手工编写代码正如提笔写文章，语法就是遣词造句的方法、函数就是组成文章的词句、类库就是据经引典的掌故、架构就是行文表述的体裁、功能就是写作文章的主旨、算法就是组织语言的逻辑……所以，只要掌握一门程序语言的语法、学习一堆基础类库的函数、引用一些所需的第三方类库、选择一款成熟稳定的架构、明确一下产品需求的功能、挑选一种实现逻辑的算法……手工编写代码就会像写文章一样手到擒来。

方法 2：复制粘贴代码

常言道：“熟读唐诗三百首，不会作诗也会吟。”编码也是同样的道理，编码的第一步就是模仿，简单地说就是“抄代码”——复制粘贴代码。

复制粘贴代码是一门艺术，用好了编码会事半功倍。但是，没有检验过的东西，终究是不可全信的。当看到需要的代码时，在复制粘贴前，我们都需要仔细研读、认真思考、详细甄别……很多东西，都是仁者见仁、智者见智的东西，适合别的场景但不一定适合你的场景。作为一名合格的程序员，切不可一味地“拿来主义”。

1. 为什么要复制粘贴代码

1) 复制粘贴现有代码，可以节省开发时间；
2) 复制粘贴稳定代码，可以降低系统故障风险；
3) 复制粘贴网络代码，可以把别人的成果化为己用。

2. 复制粘贴代码带来问题

1) 你对复制的代码理解程度是多少？实现逻辑是否合理？能不能稳定运行？存在多少潜在的 Bug?
2) 这个代码在项目中已经复制粘贴了多少次？根据“三则重构”原则，你是否需要对这些相同代码进行重构？

3) 代码被复制粘贴次数越多，带来的代码维护问题越多。多个代码版本的更改和修正，要保持这些代码的同步，就必须需要在每一处进行同样的修改，增加了维护的成本和风险。

总之，复制粘贴代码，跟其它编码方法一样，没有优劣对错之分。它只是一种方法，你可以善用，也可以滥用。如果我们用到了复制粘贴，我们就必须为结果负责。

方法 3：用文本替换生成代码

1. 生成代码样例

已经编写好的用户查询相关代码：

```
/** 查询用户服务函数 */
public PageData<UserVO> queryUser(QueryUserParameterVO parameter) {
    Long totalCount = userDAO.countByParameter(parameter);
    List<UserVO> userList = null;
    if (Objects.nonNull(totalCount) && totalCount.compareTo(0L) > 0) {
        userList = userDAO.queryByParameter(parameter);
    }
    return new PageData<>(totalCount, userList);
}

/** 查询用户控制器函数 */
@RequestMapping(path = "/queryUser", method = RequestMethod.POST)
public Result<PageData<UserVO>> queryUser(@Valid @RequestBody QueryUserParameterVO
parameter) {
    PageData<UserVO> pageData = userService.queryUser(parameter);
    return Result.success(pageData);
}
```

如果我们要编写公司查询相关代码，其代码形式与用户查询类似，整理出替换关系如下：

1) 把“用户”替换为“公司”；
2) 把“User”替换为“Company”；
3) 把“user”替换为“company”。

利用 Notepad、EditPlus 等文本编辑器，选择区分大小写，进行普通文本替换，最终得到结果如下：

```
/** 查询公司服务函数 */
public PageData<CompanyVO> queryCompany(QueryCompanyParameterVO parameter) {
    Long totalCount = companyDAO.countByParameter(parameter);
    List<CompanyVO> companyList = null;
    if (Objects.nonNull(totalCount) && totalCount.compareTo(0L) > 0) {
        companyList = companyDAO.queryByParameter(parameter);
    }
    return new PageData<>(totalCount, companyList);
}

/** 查询公司控制器函数 */
@RequestMapping(path = "/queryCompany", method = RequestMethod.POST)
public Result<PageData<CompanyVO>> queryCompany(@Valid @RequestBody
QueryCompanyParameterVO parameter) {
    PageData<CompanyVO> pageData = companyService.queryCompany(parameter);
    return Result.success(pageData);
}
```

利用文本替换生成代码，整段代码生成时间不会超过 1 分钟。

2. 主要优缺点

1) 主要优点

- 生成代码速度较快。

2) 主要缺点

- 必须编写样例代码；
- 只适用于文本替换的情景。

方法 4：用 Excel 公式生成代码

Excel 的公式非常强悍，可以用于编写一些公式化的代码。

1. 利用 Excel 公式生成模型类

从 WIKI 上拷贝接口模型定义到 Excel 里，样例数据内容如下：

	A	B	C	D	E	F
1	序号	字段名称	字段类型	字段描述	是否可空	附加信息
2	1	id	Long	用户标识	否	
3	2	name	String	用户名称	否	
4	3	sex	Integer	用户性别	否	0:未知;1:男;2:女
5	4	description	String	用户描述	是	

编写 Excel 公式如下：

```
= "/** "&D6&IF(ISBLANK(F6), "", "("&F6&")")&" */ "&IF(E6 = "否", IF(C6 = "String",
"@NotBlank", "@NotNull"), "")&" private "&C6&" "&B6&";"
```

利用公式生成代码如下：

```
/** 用户标识 */ @NotNull private Long id;
/** 用户名称 */ @NotBlank private String name;
/** 用户性别(0:未知;1:男;2:女) */ @NotNull private Integer sex;
/** 用户描述 */  private String description;
```

创建模型类，整理代码如下：

```
/** 用户DO类 */
public class UserDO {
    /** 用户标识 */
    @NotNull
    private Long id;
    /** 用户名称 */
    @NotBlank
    private String name;
    /** 用户性别(0:未知;1:男;2:女) */
    @NotNull
    private Integer sex;
    /** 用户描述 */
    private String description;
    ......
}
```

2. 利用 Excel 公式生成枚举类

从 WIKI 上拷贝枚举定义到 Excel 里，样例数据内容如下：

	A	B	C	D
1	序号	字段取值	字段名称	字段描述
2	1	0	NONE	空
3	2	1	MAN	男
4	3	2	WOMAN	女

编写 Excel 公式如下：

```
="/** "&D2&"("&B2&") */"&C2&"("&B2&", """&D2&"""),"
```

利用公式生成代码如下：

```
/** 空(0) */NONE(0, "空"),
/** 男(1) */MAN(1, "男"),
/** 女(2) */WOMAN(2, "女"),
```

创建枚举类，整理代码如下：

```
/** 用户性别枚举 */
public enum UserSex {
    /** 枚举定义 */
    /** 空(0) */
    NONE(0, "空"),
    /** 男(1) */
    MAN(1, "男"),
    /** 女(2) */
    WOMAN(2, "女");
    ......
}
```

3. 利用 Excel 公式生成数据库语句

用 Excel 整理的公司列表如下，需要整理成 SQL 语句直接插入数据库：

	A	B	C	D	E
1	序号	公司名称	公司地址	公司电话	公司邮箱
2	1	高德	首开大厦	(010)11111111	gaode@xxx.com
3	2	阿里云	绿地中心	(010)22222222	aliyun@xxx.com
4	3	菜鸟	阿里中心	(010)33333333	cainiao@xxx.com

编写 Excel 公式如下：

```
= "('"&B2&"', '"&C2&"', '"&D2&"', '"&E2&"'),"
```

利用公式生成 SQL 如下：

```
('高德', '首开大厦', '(010)11111111', 'gaode@xxx.com'),
('阿里云', '绿地中心', '(010)22222222', 'aliyun@xxx.com'),
('菜鸟', '阿里中心', '(010)33333333', 'cainiao@xxx.com'),
```

添加 into 语句头，整理 SQL 如下：

```
insert into t_company(name, address, phone, email) values
('高德', '首开大厦', '(010)11111111', 'gaode@xxx.com'),
('阿里云', '绿地中心', '(010)22222222', 'aliyun@xxx.com'),
('菜鸟', '阿里中心', '(010)33333333', 'cainiao@xxx.com');
```

4. 主要优缺点

1) 主要优点

- 适用于表格化数据的代码生成；
- 写好公式后，拖拽生成代码，生成速度较快。

2) 主要缺点

- 不适用于复杂功能的代码生成。

方法 5：用工具生成代码

用工具生成代码，顾名思义就是借用已有的工具生成代码。很多开发工具都提供一些工具生成代码，比如：生成构造函数，重载基类/接口函数，生成 Getter/Setter 函数，生成 toString 函数……能够避免很多手敲代码。还有一些生成代码插件，也可以生成满足某些应用场景的代码。

这里以 mybatis-generator 插件生成代码为例，介绍如何利用工具生成代码。

1. 安装运行插件

具体方法这里不再累述，自行上网搜索文档了解。

2. 生成代码样例

1) 生成模型类代码

文件 User.java 内容：

```
......
public class User {
    private Long id;
    private String user;
    private String password;
    private Integer age;
    ......
}
```

2) 生成映射接口代码

文件 UserMapper.java 内容：

```
......
public interface UserMapper {
    User selectByPrimaryKey(Long id);
    ......
}
```

3) 生成映射 XML 代码

文件 UserMapper.xml 内容：

```
......
<mapper namespace="com.test.dao.UserMapper" >
  <resultMap id="BaseResultMap" type="com.test.pojo.User" >
    <id column="id" property="id" jdbcType="BIGINT" />
    <result column="user" property="user" jdbcType="VARCHAR" />
    <result column="password" property="password"
jdbcType="VARCHAR" />
    <result column="age" property="age" jdbcType="INTEGER" />
  </resultMap>
  <sql id="Base_Column_List" >
    id, user, password, age
  </sql>
  <select id="selectByPrimaryKey" resultMap="BaseResultMap"
parameterType="java.lang.Long" >
    select
    <include refid="Base_Column_List" />
    from test_user
    where id = #{id,jdbcType=BIGINT}
  </select>
  ......
</mapper>
```

3. 主要优缺点

1) 主要优点

- 利用生成代码插件，生成代码速度较快；
- 利用插件配置文件，控制生成想要的功能代码。

2) 主要缺点

- 需要时间研究和熟悉生成代码插件的使用；
- 生成的代码不一定满足代码规范，每次生成后需进行代码合规；
- 重新生成代码后，容易覆盖自定义代码（建议维护单独的生成代码库，通过 DIFF 工具比较代码差异，然后再赋值粘贴差异代码）。

方法 6：用代码生成代码

用代码生成代码，就是自己编写代码，按照自己的格式生成代码。下面，以生成基于 MyBatis 的数据库访问代码为例说明。

1. 查询表格信息

首先，我们要从数据库中拿到我们生成代码所需要的表和列相关信息。

1) 查询表信息

- 查询表信息语句：

```
select t.table_name as '表名称'
, t.table_comment as '表备注'
from information_schema.tables t
where t.table_schema = ?
and t.table_type = 'BASE TABLE'
and t.table_name = ?;
```

其中，第 1 个问号赋值数据库名称，第 2 个问号赋值表名称。

- 查询表信息结果：

序号	表名称	表备注
1	org_company	组织公司表

2) 查询列信息

- 查询列信息语句：

```
select c.column_name as '列名称'
, c.column_comment as '列备注'
, c.data_type as '数据类型'
, c.character_maximum_length as '字符长度'
, c.numeric_precision as '数字精度'
, c.numeric_scale as '数字范围'
, c.column_default as ''
, c.is_nullable as '是否可空'
, c.column_key as '列键名'
from information_schema.columns c
where c.table_schema = ?
and c.table_name = ?
order by c.ordinal_position;
```

其中，第 1 个问号赋值数据库名称，第 2 个问号赋值表名称。

- 查询列信息结果：

序号	列名称	列备注	数据类型	字符长度	数字精度	数字范围	是否可空	列键名
1	id	公司标识	bigint		19	0	NO	PRI
2	name	公司名称	varchar	50			NO	
3	address	联系地址	varchar	200			YES	
4	description	公司描述	text	65535			YES	

2. 编写生成代码

1) 编写生成模型类代码

```
/** 生成模型类文件函数 */
private void generateModelClassFile(File dir, Table table, List<Column> columnList)
throws Exception {
    try (PrintWriter writer = new PrintWriter(new File(dir, className + "DO.java")))
{
        String className = getClassName(table.getTableName());
        String classComments = getClassComment(table.getTableComment());
        writer.println("package " + groupName + "." + systemName + ".database;");
        ......
        writer.println("/** " + classComments + "DO类 */");
        writer.println("@Getter");
        writer.println("@Setter");
        writer.println("@ToString");
        writer.println("public class " + className + "DO {");
        for (Column column : columnList) {
            String fieldType = getFieldType(column);
            String fieldName = getFieldName(column.getColumnName());
            String fieldComment = getFieldComment(column);
            writer.println("\t/** " + fieldComment + " */");
            writer.println("\tprivate " + fieldType + " " + fieldName + ";");
        }
        writer.println("}");
    }
}
```

2) 编写生成 DAO 接口代码

```
/** 生成DAO接口文件函数 */
private void generateDaoInterfaceFile(File dir, Table table, List<Column> columnList,
List<Column> pkColumnList) throws Exception {
    try (PrintWriter writer = new PrintWriter(new File(dir, className + "DAO.java")))
{
        String className = getClassName(table.getTableName());
        String classComments = getClassComment(table.getTableComment());
        writer.println("package " + groupName + "." + systemName + ".database;");
        ......
        writer.println("/** " + classComments + "DAO接口 */");
        writer.println("public interface " + className + "DAO {");
        writer.println("\t/** 获取" + classComments + "函数 */");
        writer.print("\tpublic " + className + "DO get(");
        boolean isFirst = true;
        for (Column pkColumn : pkColumnList) {
            if (!isFirst) {
                writer.print(", ");
            } else {
                isFirst = false;
            }
            String fieldType = getFieldType(pkColumn);
            String fieldName = getFieldName(pkColumn.getColumnName());
            writer.print("@Param(\"" + fieldName + "\") " + fieldType + " " +
fieldName);
        }
        writer.println(");");
        ......
        writer.println("}");
    }
}
```

3) 编写生成 DAO 映射代码

```
/** 生成DAO映射文件函数 */
private void generateDaoMapperFile(File dir, Table table, List<Column> columnList, List<Column> pkColumnList) throws Exception {
    try (PrintWriter writer = new PrintWriter(new File(dir, className + "DAO.xml"))) {
        String className = getClassName(table.getTableName());
        String classComments = getClassComment(table.getTableComment());
        writer.println("<?xml version=\"1.0\" encoding=\"UTF-8\"?>");
        ......
        writer.println("<!-- " + classComments + "映射 -->");
        writer.println("<mapper namespace=\"" + groupName + "." + systemName + ".database." + className + "DAO\">");
        writer.println("\t<!-- 所有字段语句 -->");
        writer.println("\t<sql id=\"fields\">");
        if (CollectionUtils.isNotEmpty(columnList)) {
            boolean isFirst = true;
            String columnName = getColumnName(pkColumn.getColumnName());
            for (Column column : columnList) {
                if (isFirst) {
                    isFirst = false;
                    writer.println("\t\t" + columnName);
                } else {
                    writer.println("\t\t, " + columnName);
                }
            }
        }
        writer.println("\t</sql>");
        writer.println("\t<!-- 获取" + classComments + "函数语句 -->");
        writer.println("\t<select id=\"get\" resultType=\"" + groupName + "." + systemName + ".database." + className + "DO\">");
        writer.println("\t\tselect");
        writer.println("\t\t<include refid=\"fields\"/>");
        writer.println("\t\tfrom " + table.getTableName());
        boolean isFirst = true;
        for (Column pkColumn : pkColumnList) {
            String columnName = getColumnName(pkColumn.getColumnName());
            String fieldName = getFieldName(pkColumn.getColumnName());
            writer.print("\t\t");
            if (isFirst) {
                writer.print("where");
                isFirst = false;
            } else {
                writer.print("and");
            }
            writer.println(" " + columnName + " = #{" + fieldName + "}");
        }
        writer.println("\t</select>");
        writer.println("</mapper>");
    }
}
```

3. 生成相关代码

1) 生成的模型类代码

```java
/** 组织公司DO类 */
@Getter
@Setter
@ToString
public class OrgCompanyDO {
    /** 公司标识 */
    private Long id;
    /** 公司名称 */
    private String name;
    /** 联系地址 */
    private String address;
    /** 公司描述 */
    private String description;
}
```

2) 生成的 DAO 接口代码

```java
/** 组织公司DAO接口 */
public interface OrgCompanyDAO {
    /** 获取组织公司函数 */
    public OrgCompanyDO get(@Param("id") Long id);
}
```

3) 生成的 DAO 映射代码

```xml
<!-- 组织公司映射 -->
<mapper namespace="xxx.database.OrgCompanyDAO">
    <!-- 所有字段语句 -->
    <sql id="fields">
        id
        , name
        , address
        , description
    </sql>
    <!-- 获取组织公司函数语句 -->
    <select id="get" resultType="xxx.database.OrgCompanyDO">
        select
        <include refid="fields"/>
        from org_company
        where id = #{id}
    </select>
</mapper>
```

4. 主要优缺点

1) 主要优点

- 代码格式可以定制，保证生成代码合规；
- 代码功能可以定制，只生成需要的代码；
- 经过前期代码沉淀后，后期能够直接使用。

2) 主要缺点

- 需要研究数据来源，保证能获取到生成代码所需的数据；
- 需要建立数据模型、编写生成代码，耗费时间比较长。

终极方法：无招胜有招

编码的终极方法，是不是直接对着电脑说需求，然后电脑就自动生成代码了？未来科技发展到一定水平后，这种情况或许会变成现实。但是，目前这种情况是不现实的。现实中，想要做到“大口一张、代码就来”，除非你是老板、产品经理或者技术管理者。

编码的终极方法是“无招胜有招”，“无招”并不是不讲究“招式”，而是不拘泥于某一“招式”，信手拈来合适的“招式”为宜。本文中列举的各种编码方法，没有高低优劣之分，只有合不合适之说。所以，灵活地运用各种编码方法，就是编码的终极方法。

代码规范化

在上面的各种编码方法中，很多方法都需要手工编写样例代码。如果你的代码不遵循代码规范，就很难发现代码之间的共性，并抽象出能够作为标准的样例代码；如果作为标准的样例代码不满足代码规范，必然导致生成的代码也不满足代码规范，于是把这些不规范放大了十倍、百倍甚至千倍。所以，代码规范化是编码的重中之重。

后记

在构思这篇文章的时候,在网上看见这么一个梗:一位网友讽刺一位阿里人的简历，满篇都是“沉淀了一套 XX 方法论，为 XX 业务赋能。”，用了流行语“赋能”显得

很“高大上”。姑且不论他的简历如何，能够从方法论上着手的人，一定有值得我们学习的地方。这里，我也来蹭一下这个梗，就取一个高大上的名字《编码方法论，赋能你我他》。

那些年，我们踩过的那些 Java 坑

前言

中国有句老话叫“事不过三”，指一个人犯了同样的错误，一次两次三次还可以原谅，超过三次就不可原谅了。有人指出这个“三”是虚数，用来泛指多次，所以“事不过三”不包括“三”。至于“事不过三”包不包括“三”，可能跟每个人的底线有关系，属于哲学范畴，不在本文的讨论范围之内。

写代码也是如此，同一个代码“坑”，踩第一次叫“长了经验”，踩第二次叫“加深印象”，踩第三次叫“不长心眼”，踩三次以上就叫“无可救药”。在本文中，笔者总结了一些 Java 坑，描述了问题现象，进行了问题分析，给出了避坑方法。希望大家在日常工作中，遇到了这类 Java 坑，能够提前避让开来。

一、 对象比较方法

JDK1.7 提供的 Objects.equals 方法，非常方便地实现了对象的比较，有效地避免了繁琐的空指针检查。

1. 问题现象

在 JDK1.7 之前，在判断一个短整型、整型、长整型包装数据类型与常量是否相等时，我们一般这样写：

```
Short shortValue = (short)12345;
System.out.println(shortValue == 12345); // true
Integer intValue = 12345;
System.out.println(intValue == 12345); // true
Long longValue = 12345L;
System.out.println(longValue == 12345); // true
```

从 JDK1.7 之后，提供了 Objects.equals 方法，并推荐使用函数式编程，更改代码如下：

```
Short shortValue = (short)12345;
System.out.println(Objects.equals(shortValue, 12345)); // false
Integer intValue = 12345;
System.out.println(Objects.equals(intValue, 12345)); // true
Long longValue = 12345L;
System.out.println(Objects.equals(longValue, 12345)); // false
```

为什么直接把==替换为 Objects.equals 方法就会导致输出结果不一样？

2. 问题分析

通过反编译第一段代码，我们得到语句“System.out.println(shortValue == 12345);”的字节码指令如下：

```
7   getstatic java.lang.System.out : java.io.PrintStream [22]
10  aload_1 [shortValue]
11  invokevirtual java.lang.Short.shortValue() : short [28]
14  sipush 12345
17  if_icmpne 24
20  iconst_1
21  goto 25
24  iconst_0
25  invokevirtual java.io.PrintStream.println(boolean) : void [32]
```

原来，编译器会判断包装数据类型对应的基本数据类型，并采用这个基本数据类型的指令进行比较（比如上面字节码指令中的 sipush 和 if_icmpne 等），相当于编译器自动对常量进行了数据类型的强制转化。

为什么采用 Objects.equals 方法后，编译器不自动对常量进行数据类型的强制转化？通过反编译第二段代码，我们得到语句“System.out.println(Objects.equals(shortValue，12345));”的字节码指令如下：

```
7   getstatic java.lang.System.out : java.io.PrintStream [22]
10  aload_1 [shortValue]
11  sipush 12345
14  invokestatic java.lang.Integer.valueOf(int) : java.lang.Integer [28]
17  invokestatic java.util.Objects.equals(java.lang.Object, java.lang.Object) : boolean
[33]
20  invokevirtual java.io.PrintStream.println(boolean) : void [39]
```

原来，编译器根据字面意思，认为常量 12345 默认基本数据类型是 int，所以会自动转化为包装数据类型 Integer。

在 Java 语言中，整数的默认数据类型是 int，小数的默认数据类型是 double。

通过分析 Objects.equals 方法的源代码可知：语句“System.out.println(Objects.equals(shortValue，12345))”，因为 Objects.equals 的两个参数对象类型不一致，一个是包装数据类型 Short，另一个是包装数据类型 Integer，所以最终的比较结果必然是 false；而语句“System.out.println(Objects.equals(intValue，12345))”，因为 Objects.equals 的两个参数对象类型一致，都是包装数据类型 Integer 且取值相同，所以最终的比较结果必然是 true。

3. 避坑方法

1) 保持良好的编码习惯，避免数据类型的自动转化

为了避免数据类型自动转化，更科学的写法是直接声明常量为对应的基本数据类型。

第一段代码可以这样写：

```
Short shortValue = (short)12345;
System.out.println(shortValue == (short)12345); // true
Integer intValue = 12345;
System.out.println(intValue == 12345); // true
Long longValue = 12345L;
System.out.println(longValue == 12345L); // true
```

第二段代码可以这样写：

```
Short shortValue = (short)12345;
System.out.println(Objects.equals(shortValue, (short)12345)); // true
Integer intValue = 12345;
System.out.println(Objects.equals(intValue, 12345)); // true
Long longValue = 12345L;
System.out.println(Objects.equals(longValue, 12345L)); // true
```

2) 借助开发工具或插件，及早地发现数据类型不匹配问题

在 Eclipse 的问题窗口中，我们会看到这样的提示：

```
Unlikely argument type for equals(): int seems to be unrelated to Short
Unlikely argument type for equals(): int seems to be unrelated to Long
```

3) 进行常规性单元测试，尽量把问题发现在研发阶段

“勿以善小而不为”，不要因为改动很小就不需要进行单元测试了，往往 Bug 都出现在自己过度自信的代码中。像这种问题，只要进行一次单元测试，是完全可以发现问题的。

注意：

进行必要单元测试，适用于以下所有案例，所以下文不再累述。

二、 三元表达式拆包

三元表达式是 Java 编码中的一个固定语法格式：“条件表达式？表达式 1：表达式 2”。三元表达式的逻辑为：“如果条件表达式成立，则执行表达式 1，否则执行表达式 2”。

1. 问题现象

```
boolean condition = false;
Double value1 = 1.0D;
Double value2 = 2.0D;
Double value3 = null;
Double result = condition ? value1 * value2 : value3; // 抛出空指针异常
```

当条件表达式 condition 等于 false 时，直接把 Double 对象 value3 赋值给 Double 对象 result，按道理没有任何问题，为什么会抛出空指针异常？

2. 问题分析

通过反编译代码，我们得到语句“Double result=condition?value1*value2:value3;”的字节码指令如下：

```
17  iload_1 [condition]
18  ifeq 33
21  aload_2 [value1]
22  invokevirtual java.lang.Double.doubleValue() : double [24]
25  aload_3 [value2]
26  invokevirtual java.lang.Double.doubleValue() : double [24]
29  dmul
30  goto 38
33  aload 4 [value3]
35  invokevirtual java.lang.Double.doubleValue() : double [24]
38  invokestatic java.lang.Double.valueOf(double) : java.lang.Double [16]
41  astore 5 [result]
```

在第 33 行，加载 Double 对象 value3 到操作数栈中；在第 35 行，调用 Double 对象 value3 的 doubleValue 方法。这个时候，由于 value3 是空对象 null，调用 doubleValue 方法必然抛出抛出空指针异常。但是，为什么要把空对象 value3 转化为基础数据类型 double 呢？

查阅相关资料，得到**三元表达式的类型转化规则**：

- 若两个表达式类型相同，返回值类型为该类型；
- 若两个表达式类型不同，但类型不可转换，返回值类型为 Object 类型；
- 若两个表达式类型不同，但类型可以转化，先把包装数据类型转化为基本数据类型，然后按照基本数据类型的转换规则（byte<short(char)<int<long<float<double）来转化，返回值类型为优先级最高的基本数据类型。

根据规则分析，表达式 1（value1 * value2）的类型为基础数据类型 double，表达式 2（value3）的类型为包装数据类型 Double，根据三元表达式的类型转化规则判断，最终的表达式类型为基础数据类型 double。所以，当条件表达式 condition 为

false 时，需要把空 Double 对象 value3 转化为基础数据类型 double，于是就调用了 value3 的 doubleValue 方法进行拆包，当然会抛出空指针异常。

3. 避坑方法

1) 尽量避免使用三元表达式，可以采用 if-else 语句代替

如果三元表达式中有包装数据类型的算术计算，可以考虑利用 if-else 语句代替。改写代码如下：

```
if (condition) {
    result = value1 * value2;
} else {
    result = value3;
}
```

2) 尽量使用基本数据类型，避免包装数据类型的拆装包

如果在三元表达式中有算术计算，尽量使用基本数据类型，避免包装数据类型的拆装包。改写代码如下：

```
boolean condition = false;
double value1 = 1.0D;
double value2 = 2.0D;
double value3 = 3.0D;
double result = condition ? value1 * value2 : value3;
```

三、 泛型对象赋值

Java 泛型是 JDK1.5 中引入的一个新特性，其本质是参数化类型，即把数据类型做为一个参数使用。

1. 问题现象

在做用户数据分页查询时，因为笔误编写了如下代码：

1) PageDataVO.java

```
/** 分页数据VO类 */
@Getter
@Setter
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class PageDataVO<T> {
    /** 总共数量 */
    private Long totalCount;
    /** 数据列表 */
    private List<T> dataList;
}
```

2) UserDAO.java

```
/** 用户DAO接口 */
@Mapper
public interface UserDAO {
    /** 统计用户数量 */
    public Long countUser(@Param("query") UserQueryVO query);
    /** 查询用户信息 */
    public List<UserDO> queryUser(@Param("query") UserQueryVO query);
}
```

3) UserService.java

```
/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 查询用户信息 */
    public PageDataVO<UserVO> queryUser(UserQueryVO query) {
        List<UserDO> dataList = null;
        Long totalCount = userDAO.countUser(query);
        if (Objects.nonNull(totalCount) && totalCount.compareTo(0L) > 0) {
            dataList = userDAO.queryUser(query);
        }
        return new PageDataVO(totalCount, dataList);
    }
}
```

以上代码没有任何编译问题，但是却把 UserDO 中一些涉密字段返回给前端。细心的读者可能已经发现了，在 UserService 类的 queryUser 方法的语句“return new PageDataVO(totalCount，dataList);”中，我们把 List<UserDO>对象 dataList 赋值给了 PageDataVO<UserVO>的 List<UserVO>字段 dataList。

问题是：为什么开发工具不报编译错误啦？

2. 问题分析

由于历史原因，参数化类型和原始类型需要兼容。我们以 ArrayList 举例子，来看看如何兼容的。

以前的写法：

```
ArrayList list = new ArrayList();
```

现在的写法：

```
ArrayList<String> list = new ArrayList<String>();
```

考虑到与以前的代码兼容，各种对象引用之间传值，必然会出现以下的情况：

```
// 第一种情况
ArrayList list1 = new ArrayList<String>();
// 第二种情况
ArrayList<String> list2 = new ArrayList();
```

所以，Java 编译器对以上两种类型进行了兼容，不会出现编译错误，但会出现编译告警。但是，我的开发工具在编译时真没出现过告警。

再来分析我们遇到的问题，实际上同时命中了两种情况：

- 把 List<UserDO>对象赋值给 List，命中了第一种情况；

- 把 PageDataVO 对象赋值给 PageDataVO<UserVO>，命中了第二种情况。

最终的效果就是：我们神奇地把 List<UserDO>对象赋值给了 List<UserVO>。
问题的根源就是：我们在初始化 PageDataVO 对象时，没有要求强制进行类型检查。

3. 避坑方法

1) 在初始化泛型对象时，推荐使用 diamond 语法

在《阿里巴巴 Java 开发手册》中，有这么一条推荐规则：

【推荐】集合泛型定义时，在 JDK7 及以上，使用 diamond 语法或全省略。

说明：
菱形泛型，即 diamond，直接使用<>来指代前边已经指定的类型。

正例：

```
// <> diamond 方式
HashMap<String, String> userCache = new HashMap<>(16);
// 全省略方式
ArrayList<User> users = new ArrayList(10);
```

其实，初始化泛型对象时，全省略是不推荐的。这样会避免类型检查，从而造成上面的问题。

在初始化泛型对象时，推荐使用 diamond 语法，代码如下：

```
return new PageDataVO<>(totalCount, dataList);
```

现在，在 Eclipse 的问题窗口中，我们会看到这样的错误：

```
Cannot infer type arguments for PageDataVO<>
```

于是，我们就知道忘记把 List<UserDO>对象转化为 List<UserVO>对象了。

四、 泛型属性拷贝

Spring 的 BeanUtils.copyProperties 方法，是一个很好用的属性拷贝工具方法。

1. 问题现象

根据数据库开发规范，数据库表格必须包含 id，gmt_create，gmt_modified 三个字段。其中，id 这个字段，可能根据数据量不同，采用 int 或 long 类型。

首先，定义了一个 BaseDO 基类：

```
/** 基础DO类 */
@Getter
@Setter
@ToString
public class BaseDO<T> {
    private T id;
    private Date gmtCreate;
    private Date gmtModified;
}
```

针对 user 表，定义了一个 UserDO 类：

```
/** 用户DO */
@Getter
@Setter
@ToString
public static class UserDO extends BaseDO<Long> {
    private String name;
    private String description;
}
```

对于查询接口，定义了一个 UserVO 类：

```
/** 用户VO类 */
@Getter
@Setter
@ToString
public static class UserVO {
    private Long id;
    private String name;
    private String description;
}
```

实现查询用户服务接口，实现代码如下：

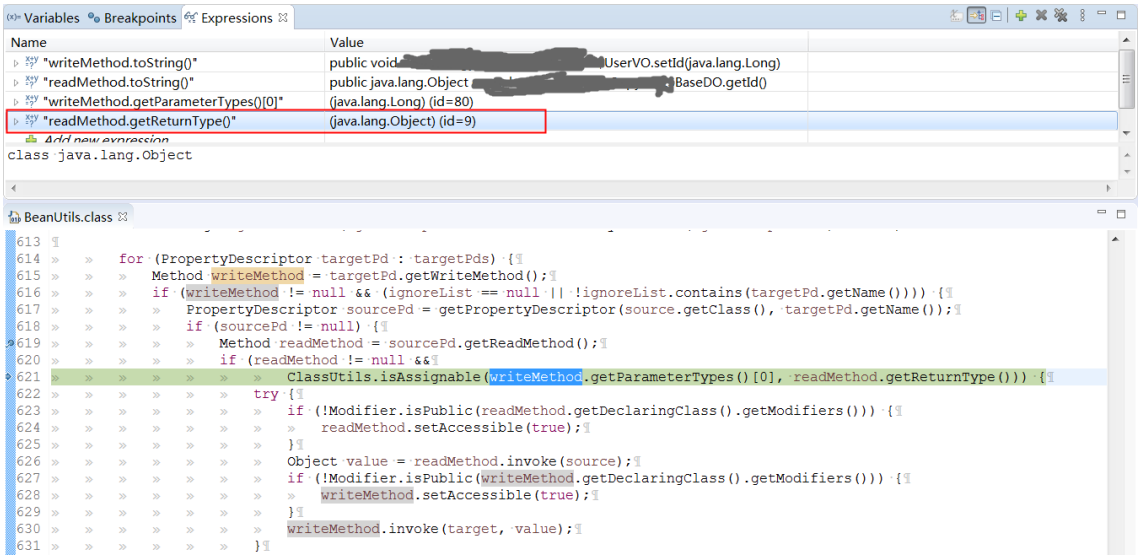
```
/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 查询用户 */
    public List<UserVO> queryUser(UserQueryVO query) {
        // 查询用户信息
        List<UserDO> userDOList = userDAO.queryUser(query);
        if (CollectionUtils.isEmpty()) {
            return Collections.emptyList();
        }

        // 转化用户列表
        List<UserVO> userVOList = new ArrayList<>(userDOList.size());
        for (UserDO userDO : userDOList) {
            UserVO userVO = new UserVO();
            BeanUtils.copyProperties(userDO, userVO);
            userVOList.add(userVO);
        }

        // 返回用户列表
        return userVOList;
    }
}
```

通过测试，我们会发现一个问题——调用查询用户服务接口，用户 ID 的值并没有返回。

```
[{"description":"This is a tester.","name":"tester"},...]
```

2. 问题分析

通过 Debug 模式运行，进入到 BeanUtils.copyProperties 工具方法内部，得到以下内容：



原来，UserDO 类的 getId 方法返回类型不是 Long 类型，而是被泛型还原成了 Object 类型。而下面的 ClassUtils.isAssignable 工具方法，判断是否能够把 Object 类型赋值给 Long 类型，当然会返回 false 导致不能进行属性拷贝。

为什么作者不考虑“先获取属性值，再判断能否赋值”？建议代码如下：

```
Object value = readMethod.invoke(source);
if (Objects.nonNull(value) && ClassUtils.isAssignable(writeMethod.getParameterTypes()[0],
value.getClass())) {
    ... // 赋值相关代码
}
```

3. 避坑方法

1) 不要盲目地相信第三方工具包，任何工具包都有可能存在问题

在 Java 中，存在很多第三方工具包，比如：Apache 的 commons-lang3、commons-collections，Google 的 guava……都是很好用的第三方工具包。但是，不要盲目地相信第三方工具包，任何工具包都有可能存在问题。

2) 如果需要拷贝的属性较少，可以手动编码进行属性拷贝

用 BeanUtils.copyProperties 反射拷贝属性，主要优点是节省了代码量，主要缺点是导致程序性能下降。所以，如果需要拷贝的属性较少，可以手动编码进行属性拷贝。

五、 Set 对象排重

在 Java 语言中，Set 数据结构可以用于对象排重，常见的 Set 类有 HashSet、LinkedHashSet 等。

1. 问题现象

编写了一个城市辅助类，从 CSV 文件中读取城市数据：

```
/** 城市辅助类 */
@Slf4j
public class CityHelper {
  /** 读取城市 */
    public static Collection<City> readCities(String fileName) {
        try (FileInputStream stream = new FileInputStream(fileName);
            InputStreamReader reader = new InputStreamReader(stream, "GBK");
            CSVParser parser = new CSVParser(reader, CSVFormat.DEFAULT.withHeader())) {
            Set<City> citySet = new HashSet<>(1024);
            Iterator<CSVRecord> iterator = parser.iterator();
            while (iterator.hasNext()) {
                citySet.add(parseCity(iterator.next()));
            }
            return citySet;
        } catch (IOException e) {
            log.warn("读取所有城市异常", e);
        }
        return Collections.emptyList();
    }

  /** 解析城市 */
    private static City parseCity(CSVRecord record) {
        City city = new City();
        city.setCode(record.get(0));
        city.setName(record.get(1));
        return city;
    }

    /** 城市类 */
    @Getter
    @Setter
    @ToString
    private static class City {
        /** 城市编码 */
        private String code;
        /** 城市名称 */
        private String name;
    }
}
```

代码中使用 HashSet 数据结构，目的是为了避免城市数据重复，对读取的城市数据进行强制排重。

当输入文件内容如下时：

```
编码,名称
010,北京
020,广州
010,北京
```

解析后的 JSON 结果如下：

```
[{"code":"010","name":"北京"},{"code":"020","name":"广州"},{"code":"010","name":"北京"}]
```

但是，并没有对城市“北京”进行排重。

2. 问题分析

当向集合 Set 中增加对象时，首先集合计算要增加对象的 hashCode，根据该值来得到一个位置用来存放当前对象。如在该位置没有一个对象存在的话，那么集合 Set 认为该对象在集合中不存在，直接增加进去。如果在该位置有一个对象存在的话，接着将准备增加到集合中的对象与该位置上的对象进行 equals 方法比较：如果该 equals 方法返回 false，那么集合认为集合中不存在该对象，就把该对象放在这个对象之后；如果 equals 方法返回 true，那么就认为集合中已经存在该对象了，就不会再将该对象增加到集合中了。

所以，在哈希表中判断两个元素是否重复要使用到 hashCode 方法和 equals 方法。hashCode 方法决定数据在表中的存储位置，而 equals 方法判断表中是否存在相同的数据。

分析上面的问题，由于没有重写 City 类的 hashCode 方法和 equals 方法，就会采用 Object 类的 hashCode 方法和 equals 方法。其实现如下：

```
public native int hashCode();
public boolean equals(Object obj) {
    return (this == obj);
}
```

可以看出：Object 类的 hashCode 方法是一个本地方法，返回的是对象地址；Object 类的 equals 方法只比较对象是否相等。所以，对于两条完全一样的北京数据，由于在解析时初始化了不同的 City 对象，导致 hashCode 方法和 equals 方法值都不一样，必然被 Set 认为是不同的对象，所以没有进行排重。

那么，我们就重写把 City 类的 hashCode 方法和 equals 方法，代码如下：

```java
/** 城市类 */
@Getter
@Setter
@ToString
private static class City {
    /** 城市编码 */
    private String code;
    /** 城市名称 */
    private String name;

    /** 判断相等 */
    @Override
    public boolean equals(Object obj) {
        if (obj == this) {
            return true;
        }
        if (Objects.isNull(obj)) {
            return false;
        }
        if (obj.getClass() != this.getClass()) {
            return false;
        }
        return Objects.equals(this.code, ((City)obj).code);
    }

    /** 哈希编码 */
    @Override
    public int hashCode() {
        return Objects.hashCode(this.code);
    }
}
```

重新支持测试程序，解析后的 JSON 结果如下：

```
[{"code":"010","name":"北京"},{"code":"020","name":"广州"}]
```

结果正确，已经对城市“北京”进行排重。

3. 避坑方法

1) 当确定数据唯一时，可以使用 List 代替 Set

当确定解析的城市数据唯一时，就没有必要进行排重操作，可以直接使用 List 来存储。

```
List<City> cityList = new ArrayList<>(1024);
Iterator<CSVRecord> iterator = parser.iterator();
while (iterator.hasNext()) {
    cityList.add(parseCity(iterator.next()));
}
return cityList;
```

2) 当确定数据不唯一时，可以使用 Map 代替 Set

当确定解析的城市数据不唯一时，需要安装城市名称进行排重操作，可以直接使用 Map 进行存储。为什么不建议实现 City 类的 hashCode 方法，再采用 HashSet 来实现排重呢？首先，不希望把业务逻辑放在模型 DO 类中；其次，把排重字段放在代码中，便于代码的阅读、理解和维护。

```
Map<String, City> cityMap = new HashMap<>(1024);
Iterator<CSVRecord> iterator = parser.iterator();
while (iterator.hasNext()) {
    City city = parseCity(iterator.next());
    cityMap.put(city.getCode(), city);
}
return cityMap.values();
```

3) 遵循 Java 语言规范，重写 hashCode 方法和 equals 方法

不重写 hashCode 方法和 equals 方法的自定义类不应该在 Set 中使用。

六、 公有方法代理

SpringCGLIB 代理生成的代理类是一个继承被代理类，通过重写被代理类中的非 final 的方法实现代理。所以，SpringCGLIB 代理的类不能是 final 类，代理的方法也不能是 final 方法，这是由继承机制限制的。

1. 问题现象

这里举例一个简单的例子，只有超级用户才有删除公司的权限，并且所有服务函数被 AOP 拦截处理异常。例子代码如下：

1) UserService.java

```
/** 用户服务类 */
@Service
public class UserService {
    /** 超级用户 */
    private User superUser;

  /** 设置超级用户 */
    public void setSuperUser(User superUser) {
        this.superUser = superUser;
    }

    /** 获取超级用户 */
    public final User getSuperUser() {
        return this.superUser;
    }
}
```

2) CompanyService.java

```
/** 公司服务类 */
@Service
public class CompanyService {
    /** 公司DAO */
    @Autowired
    private CompanyDAO companyDAO;
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /** 删除公司 */
    public void deleteCompany(Long companyId, Long operatorId) {
        // 设置超级用户
        userService.setSuperUser(new User(0L, "admin", "超级用户"));

        // 验证超级用户
        if (!Objects.equals(operatorId, userService.getSuperUser().getId())) {
            throw new ExampleException("只有超级用户才能删除公司");
        }

        // 删除公司信息
        companyDAO.delete(companyId, operatorId);
    }
}
```

当我们调用 CompanyService 的 deleteCompany 方法时，居然也抛出空指针异常（NullPointerException），因为调用 UserService 类的 getSuperUser 方法获取的超级用户为 null。但是，我们在 CompanyService 类的 deleteCompany 方法中，

每次都通过 UserService 类的 setSuperUser 方法强制指定了超级用户，按道理通过 UserService 类的 getSuperUser 方法获取到的超级用户不应该为 null。其实，这个问题也是由 AOP 代理导致的。

2. 问题分析

使用 SpringCGLIB 代理类时，Spring 会创建一个名为“UserService$$EnhancerBySpringCGLIB$$????????”的代理类。反编译这个代理类，得到以下主要代码：

```
public class UserService$$EnhancerBySpringCGLIB$$a2c3b345 extends UserService implements
SpringProxy, Advised, Factory {
    ......
    public final void setSuperUser(User var1) {
        MethodInterceptor var10000 = this.CGLIB$CALLBACK_0;
        if (var10000 == null) {
            CGLIB$BIND_CALLBACKS(this);
            var10000 = this.CGLIB$CALLBACK_0;
        }

        if (var10000 != null) {
            var10000.intercept(this, CGLIB$setSuperUser$0$Method, new Object[]{var1},
CGLIB$setSuperUser$0$Proxy);
        } else {
            super.setSuperUser(var1);
        }
    }
    ......
}
```

可以看出，这个代理类继承了 UserService 类，只代理了 setSuperUser 方法，但是没有代理 getSuperUser 方法。所以，当我们调用 setSuperUser 方法时，设置的是原始对象实例的 superUser 字段值；而当我们调用 getSuperUser 方法时，获取的是代理对象实例的 superUser 字段值。如果把这两个方法的 final 修饰符互换，同样存在获取超级用户为 null 的问题。

3. 避坑方法

1) 严格遵循 CGLIB 代理规范，被代理的类和方法不要加 final 修饰符

严格遵循 CGLIB 代理规范，被代理的类和方法不要加 final 修饰符，避免动态代理操作对象实例不同（原始对象实例和代理对象实例），从而导致数据不一致或空指针问题。

2） 缩小 CGLIB 代理类的范围，能不用被代理的类就不要被代理

缩小 CGLIB 代理类的范围，能不用被代理的类就不要被代理，即可以节省内存开销，又可以提高函数调用效率。

七、 公有字段代理

在 fastjson 强制升级到 1.2.60 时踩过一个坑，作者为了开发快速，在 ParseConfig 中定义了：

```
public class ParseConfig {
    public final SymbolTable symbolTable = new SymbolTable(4096);
    ......
}
```

在我们的项目中继承了该类，同时又被 AOP 动态代理了，于是一行代码引起了一场“血案”。

1. 问题现象

仍然使用上章的例子，但是把获取、设置方法删除，定义了一个公有字段。例子代码如下：

1） UserService.java

```
/** 用户服务类 */
@Service
public class UserService {
    /** 超级用户 */
    public final User superUser = new User(0L, "admin", "超级用户");
    ......
}
```

2) CompanyService.java

```
/** 公司服务类 */
@Service
public class CompanyService {
    /** 公司DAO */
    @Autowired
    private CompanyDAO companyDAO;
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /** 删除公司 */
    public void deleteCompany(Long companyId, Long operatorId) {
        // 验证超级用户
        if (!Objects.equals(operatorId, userService.superUser.getId())) {
            throw new ExampleException("只有超级用户才能删除公司");
        }

        // 删除公司信息
        companyDAO.delete(companyId, operatorId);
    }
}
```

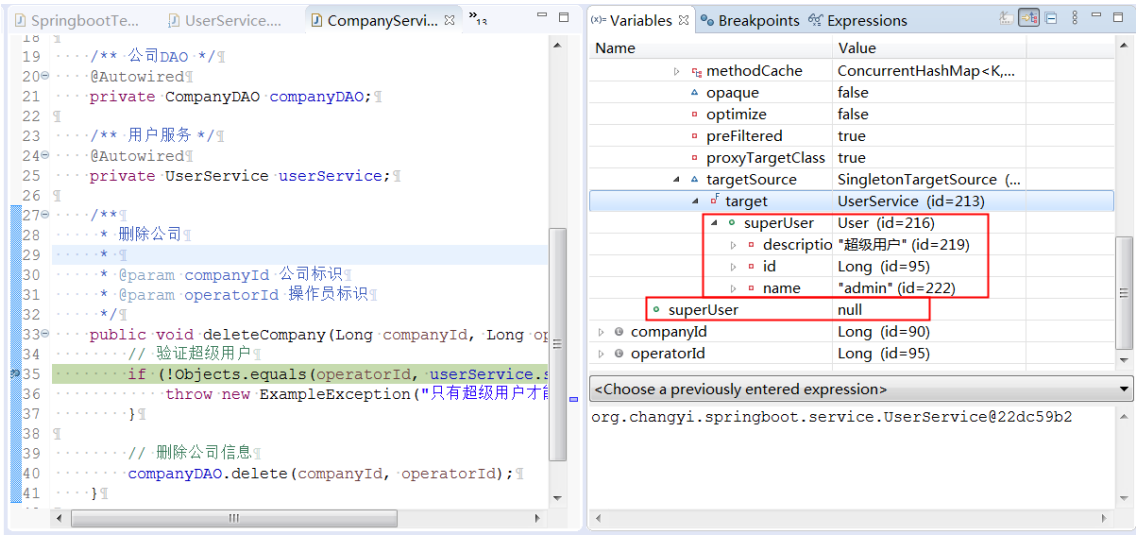
当我们调用 CompanyService 的 deleteCompany 方法时，居然抛出空指针异常（NullPointerException）。经过调试打印，发现是 UserService 的 superUser 变量为 null。如果把代理删除，就不会出现空指针异常，说明这个问题是由 AOP 代理导致的。

2. 问题分析

使用 SpringCGLIB 代理类时，Spring 会创建一个名为“UserService$$EnhancerBySpringCGLIB$$????????”的代理类。这个代理类继承了 UserService 类，并覆盖了 UserService 类中的所有非 final 的 public 的方法。但是，这个代理类并不调用 super 基类的方法；相反，它会创建的一个成员 userService 并指向原始的 UserService 类对象实例。

现在，内存中存在两个对象实例：一个是原始的 UserService 对象实例，另一个指向 UserService 的代理对象实例。这个代理类只是一个虚拟代理，它继承了 UserService 类，并且具有与 UserService 相同的字段，但是它从来不会去初始化和

使用它们。所以，一但通过这个代理类对象实例获取公有成员变量时，将返回一个默认值 null。



3. 避坑方法

1) 当确定字段不可变时，可以定义为公有静态常量

当确定字段不可变时，可以定义为公有静态常量，并用类名称+字段名称访问。类名称+字段名称访问公有静态常量，与类实例的动态代理无关。

2) 当确定字段不可变时，可以定义为私有成员变量

当确定字段不可变时，可以定义为私有成员变量，提供一个公有 Getter 方法获取该变量值。当该类实例被动态代理时，代理方法会调用被代理的 Getter 方法，从而返回被代理类的成员变量值。

3) 遵循 JavaBean 编码规范，不要定义公有成员变量

遵循 JavaBean 编码规范，不要定义公有成员变量。JavaBean 规范如下：

- JavaBean 类必须是一个公共类，并将其访问属性设置为 public，如：public class User{……}

- JavaBean 类必须有一个空的构造函数：类中必须有一个不带参数的公用构造器
- 一个 JavaBean 类不应有公共实例变量，类变量都为 private，如：private Integer id;
- 属性应该通过一组 getter/setter 方法来访问。

后记

最后，推荐大家阅读一下《阿里巴巴 Java 开发手册》，这本书让我受益匪浅。只要学习理解了《阿里巴巴 Java 开发手册》，就能在日常的 Java 开发工作中，避免踩到很多常识性的 Java 坑。

这里，作一首五言绝句赠与离开阿里的孤尽、夕华等前辈：

《赠别》

技术已封神，
声名四海闻。
功成身退去，
只为揽星辰。

那些年，我们见过的 Java 服务端“乱象”

导读

查尔斯·狄更斯在《双城记》中写道：“这是一个最好的时代，也是一个最坏的时代。”移动互联网的快速发展，出现了许多新机遇，很多创业者伺机而动；随着行业竞争加剧，互联网红利逐渐消失，很多创业公司九死一生。

笔者在初创公司摸爬滚打数年，接触了各式各样的 Java 微服务架构，从中获得了一些优秀的理念，但也发现了一些不合理的现象。现在，笔者总结了一些创业公司存在的 Java 服务端乱象，并尝试性地给出了一些不成熟的建议。

一、 使用 Controller 基类和 Service 基类

1. 现象描述

1) Controller 基类

常见的 Controller 基类如下：

```
/** 基础控制器类 */
public class BaseController {
    /** 注入服务相关 */
    /** 用户服务 */
    @Autowired
    protected UserService userService;
    ...

    /** 静态常量相关 */
    /** 手机号模式 */
    protected static final String PHONE_PATTERN = "/^[1]([3-9])[0-9]{9}$/";
    ...

    /** 静态函数相关 */
    /** 验证电话 */
    protected static vaildPhone(String phone) {...}
    ...
}
```

常见的 Controller 基类主要包含注入服务、静态常量和静态函数等，便于所有的 Controller 继承它，并在函数中可以直接使用这些资源。

2) Service 基类

常见的 Service 基类如下：

```
/** 基础服务类 */
public class BaseService {
    /** 注入DAO相关 */
    /** 用户DAO */
    @Autowired
    protected UserDAO userDAO;
    ...

    /** 注入服务相关 */
    /** 短信服务 */
    @Autowired
    protected SmsService smsService;
    ...

    /** 注入参数相关 */
    /** 系统名称 */
    @Value("${example.systemName}")
    protected String systemName;
    ...

    /** 静态常量相关 */
    /** 超级用户标识 */
    protected static final long SUPPER_USER_ID = 0L;
    ...

    /** 服务函数相关 */
    /** 获取用户函数 */
    protected UserDO getUser(Long userId) {...}
    ...

    /** 静态函数相关 */
    /** 获取用户名称 */
    protected static String getUserName(UserDO user) {...}
    ...
}
```

常见的 Service 基类主要包括注入 DAO、注入服务、注入参数、静态常量、服务函数、静态函数等，便于所有的 Service 继承它，并在函数中可以直接使用这些资源。

2. 论证基类必要性

首先，了解一下**里氏替换原则**：

- 里氏代换原则（Liskov Substitution Principle，简称 LSP）：所有引用基类（父类）的地方必须能透明地使用其子类的对象。

其次，了解一下**基类的优点**：

- 子类拥有父类的所有方法和属性，从而减少了创建子类的工作量；
- 提高了代码的重用性，子类拥有父类的所有功能；
- 提高了代码的扩展性，子类可以添加自己的功能。

所以，我们可以得出以下**结论**：

- Controller 基类和 Service 基类在整个项目中并没有直接被使用，也就没有可使用其子类替换基类的场景，所以不满足里氏替换原则；

- Controller 基类和 Service 基类并没有抽象接口函数或虚函数，即所有继承基类的子类间没有相关共性，直接导致在项目中仍然使用的是子类；

- Controller 基类和 Service 基类只关注了重用性，即子类能够轻松使用基类的注入 DAO、注入服务、注入参数、静态常量、服务函数、静态函数等资源。但是，忽略了这些资源的必要性，即这些资源并不是子类所必须的，反而给子类带来了加载时的性能损耗。

综上所述，Controller 基类和 Service 基类只是一个杂凑类，并不是一个真正意义上的基类，需要进行拆分。

3. 拆分基类的方法

由于 Service 基类比 Controller 基类更典型，本文以 Service 基类举例说明如何来拆分“基类”。

1) 把注入实例放入实现类

根据“使用即引入、无用则删除”原则，在需要使用的实现类中注入需要使用的 DAO、服务和参数。

```java
/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 短信服务 */
    @Autowired
    private SmsService smsService;

    /** 系统名称 */
    @Value("${example.systemName}")
    private String systemName;
    ...
}
```

2) 把静态常量放入常量类

对于静态常量，可以把它们封装到对应的常量类中，在需要时直接使用即可。

```java
/** 例子常量类 */
public class ExampleConstants {
    /** 超级用户标识 */
    public static final long SUPPER_USER_ID = 0L;
    ...
}
```

3) 把服务函数放入服务类

对于服务函数，可以把它们封装到对应的服务类中。在别的服务类使用时，可以注入该服务类实例，然后通过实例调用服务函数。

```java
/** 用户服务类 */
@Service
public class UserService {
    /** 获取用户函数 */
    public UserDO getUser(Long userId) {...}
    ...
}

/** 公司服务类 */
@Service
public class CompanyService {
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /** 获取管理员 */
    public UserDO getManager(Long companyId) {
        CompanyDO company = ...;
        return userService.getUser(company.getManagerId());
    }
    ...
}
```

4) 把静态函数放入工具类

对于静态函数，可以把它们封装到对应的工具类中，在需要时直接使用即可。

```java
/** 用户辅助类 */
public class UserHelper {
    /** 获取用户名称 */
    public static String getUserName(UserDO user) {...}
    ...
}
```

二、 把业务代码写在 Controller 中

1. 现象描述

我们会经常会在 Controller 类中看到这样的代码：

```
/** 用户控制器类 */
@Controller
@RequestMapping("/user")
public class UserController {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 获取用户函数 */
    @ResponseBody
    @RequestMapping(path = "/getUser", method = RequestMethod.GET)
    public Result<UserVO> getUser(@RequestParam(name = "userId", required = true) Long
userId) {
        // 获取用户信息
        UserDO userDO = userDAO.getUser(userId);
        if (Objects.isNull(userDO)) {
            return null;
        }

        // 拷贝并返回用户
        UserVO userVO = new UserVO();
        BeanUtils.copyProperties(userDO, userVO);
        return Result.success(userVO);
    }
    ...
}
```

编写人员给出的理由是：一个简单的接口函数，这么写也能满足需求，没有必要去封装成一个服务函数。

2. 一个特殊的案例

案例代码如下：

```
/** 测试控制器类 */
@Controller
@RequestMapping("/test")
public class TestController {
    /** 系统名称 */
    @Value("${example.systemName}")
    private String systemName;

    /** 访问函数 */
    @RequestMapping(path = "/access", method = RequestMethod.GET)
    public String access() {
        return String.format("系统(%s)欢迎您访问！", systemName);
    }
}
```

访问结果如下：

```
curl http://localhost:8080/test/access
系统(null)欢迎您访问!
```

为什么参数 systemName（系统名称）没有被注入值？《Spring Documentation》给出的解释是：

> *Note that actual processing of the @Value annotation is performed by a BeanPostProcessor.*
>
> *BeanPostProcessor interfaces are scoped per-container. This is only relevant if you are using container hierarchies. If you define a BeanPostProcessor in one container, it will only do its work on the beans in that container. Beans that are defined in one container are not post-processed by a BeanPostProcessor in another container, even if both containers are part of the same hierarchy.*

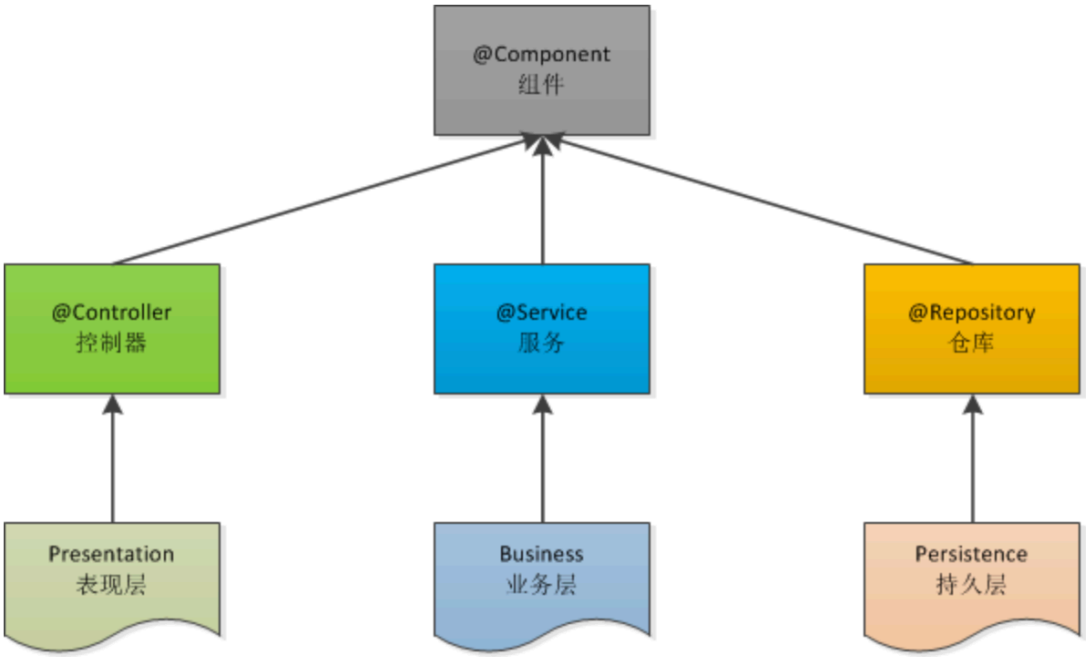
意思是说：

@Value 是通过 BeanPostProcessor 来处理的，而 WebApplicationContext 和 ApplicationContext 是单独处理的，所以 WebApplicationContext 不能使用父容器的属性值。

所以，Controller 不满足 Service 的需求，不要把业务代码写在 Controller 类中。

3. 服务端三层架构

SpringMVC 服务端采用经典的三层架构，即表现层、业务层、持久层，分别采用 @Controller、@Service、@Repository 进行类注解。



- **表现层（Presentation）**

  又称控制层（Controller），负责接收客户端请求，并向客户端响应结果，通常采用 HTTP 协议。

- **业务层（Business）**

  又称服务层（Service），负责业务相关逻辑处理，按照功能分为服务、作业等。

- **持久层（Persistence）**

  又称仓库层（Repository），负责数据的持久化，用于业务层访问缓存和数据库。

所以，把业务代码写入到 Controller 类中，是不符合 SpringMVC 服务端三层架构规范的。

三、 把持久层代码写在 Service 中

把持久层代码写在 Service 中，从功能上来看并没有什么问题，这也是很多人欣然接受的原因。

1. 引起以下主要问题

- 业务层和持久层混杂在一起，不符合 SpringMVC 服务端三层架构规范；
- 在业务逻辑中组装语句、主键等，增加了业务逻辑的复杂度；
- 在业务逻辑中直接使用第三方中间件，不便于第三方持久化中间件的替换；
- 同一对象的持久层代码分散在各个业务逻辑中，背离了面对对象的编程思想；
- 在写单元测试用例时，无法对持久层接口函数直接测试。

2. 把数据库代码写在 Service 中

这里以数据库持久化中间件 Hibernate 的直接查询为例。

现象描述：

```
/** 用户服务类 */
@Service
public class UserService {
    /** 会话工厂 */
    @Autowired
    private SessionFactory sessionFactory;

    /** 根据工号获取用户函数 */
    public UserVO getUserByEmpId(String empId) {
        // 组装HQL语句
        String hql = "from t_user where emp_id = '" + empId + "'";

        // 执行数据库查询
        Query query = sessionFactory.getCurrentSession().createQuery(hql);
        List<UserDO> userList = query.list();
        if (CollectionUtils.isEmpty(userList)) {
            return null;
        }

        // 转化并返回用户
        UserVO userVO = new UserVO();
        BeanUtils.copyProperties(userList.get(0), userVO);
        return userVO;
    }
}
```

建议方案：

```
/** 用户DAO类 */
@Repository
public class UserDAO {
     /** 会话工厂 */
    @Autowired
    private SessionFactory sessionFactory;

    /** 根据工号获取用户函数 */
    public UserDO getUserByEmpId(String empId) {
        // 组装HQL语句
        String hql = "from t_user where emp_id = '" + empId + "'";

        // 执行数据库查询
        Query query = sessionFactory.getCurrentSession().createQuery(hql);
        List<UserDO> userList = query.list();
        if (CollectionUtils.isEmpty(userList)) {
            return null;
        }

        // 返回用户信息
        return userList.get(0);
    }
}

/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 根据工号获取用户函数 */
    public UserVO getUserByEmpId(String empId) {
        // 根据工号查询用户
        UserDO userDO = userDAO.getUserByEmpId(empId);
        if (Objects.isNull(userDO)) {
            return null;
        }

        // 转化并返回用户
        UserVO userVO = new UserVO();
        BeanUtils.copyProperties(userDO, userVO);
        return userVO;
    }
}
```

关于插件：

阿里的 AliGenerator 是一款基于 MyBatis Generator 改造的 DAO 层代码自动生成工具。利用 AliGenerator 生成的代码，在执行复杂查询的时候，需要在业务代码中组装查询条件，使业务代码显得特别臃肿。

```
/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 获取用户函数 */
    public UserVO getUser(String companyId, String empId) {
        // 查询数据库
        UserParam userParam = new UserParam();
        userParam.createCriteria().andCompanyIdEqualTo(companyId)
            .andEmpIdEqualTo(empId)
            .andStatusEqualTo(UserStatus.ENABLE.getValue());
        List<UserDO> userList = userDAO.selectByParam(userParam);
        if (CollectionUtils.isEmpty(userList)) {
            return null;
        }

        // 转化并返回用户
        UserVO userVO = new UserVO();
        BeanUtils.copyProperties(userList.get(0), userVO);
        return userVO;
    }
}
```

个人不喜欢用 DAO 层代码生成插件，更喜欢用原汁原味的 MyBatis XML 映射，主要原因如下：

- 会在项目中导入一些不符合规范的代码；
- 只需要进行一个简单查询，也需要导入一整套复杂代码；
- 进行复杂查询时，拼装条件的代码复杂且不直观，不如在 XML 中直接编写 SQL 语句；
- 变更表格后需要重新生成代码并进行覆盖，可能会不小心删除自定义函数。

当然，既然选择了使用 DAO 层代码生成插件，在享受便利的同时也应该接受插件的缺点。

3. 把 Redis 代码写在 Service 中

现象描述：

```
/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;
    /** Redis模板 */
    @Autowired
    private RedisTemplate<String, String> redisTemplate;
    /** 用户主键模式 */
    private static final String USER_KEY_PATTERN = "hash::user::%s";

    /** 保存用户函数 */
    public void saveUser(UserVO user) {
        // 转化用户信息
        UserDO userDO = transUser(user);

        // 保存Redis用户
        String userKey = MessageFormat.format(USER_KEY_PATTERN, userDO.getId());
        Map<String, String> fieldMap = new HashMap<>(8);
        fieldMap.put(UserDO.CONST_NAME, user.getName());
        fieldMap.put(UserDO.CONST_SEX, String.valueOf(user.getSex()));
        fieldMap.put(UserDO.CONST_AGE, String.valueOf(user.getAge()));
        redisTemplate.opsForHash().putAll(userKey, fieldMap);

        // 保存数据库用户
        userDAO.save(userDO);
    }
}
```

建议方案：

```
/** 用户Redis类 */
@Repository
public class UserRedis {
    /** Redis模板 */
    @Autowired
    private RedisTemplate<String, String> redisTemplate;
    /** 主键模式 */
    private static final String KEY_PATTERN = "hash::user::%s";

    /** 保存用户函数 */
    public UserDO save(UserDO user) {
        String key = MessageFormat.format(KEY_PATTERN, userDO.getId());
        Map<String, String> fieldMap = new HashMap<>(8);
        fieldMap.put(UserDO.CONST_NAME, user.getName());
        fieldMap.put(UserDO.CONST_SEX, String.valueOf(user.getSex()));
        fieldMap.put(UserDO.CONST_AGE, String.valueOf(user.getAge()));
        redisTemplate.opsForHash().putAll(key, fieldMap);
    }
}

/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;
    /** 用户Redis */
    @Autowired
    private UserRedis userRedis;

    /** 保存用户函数 */
    public void saveUser(UserVO user) {
        // 转化用户信息
        UserDO userDO = transUser(user);

        // 保存Redis用户
        userRedis.save(userDO);

        // 保存数据库用户
        userDAO.save(userDO);
    }
}
```

把一个 Redis 对象相关操作接口封装为一个 DAO 类，符合面对对象的编程思想，也符合 SpringMVC 服务端三层架构规范，更便于代码的管理和维护。

四、 把数据库模型类暴露给接口

现象描述：

```
/** 用户DAO类 */
@Repository
public class UserDAO {
    /** 获取用户函数 */
    public UserDO getUser(Long userId) {...}
}

/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 获取用户函数 */
    public UserDO getUser(Long userId) {
        return userDAO.getUser(userId);
    }
}

/** 用户控制器类 */
@Controller
@RequestMapping("/user")
public class UserController {
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /** 获取用户函数 */
    @RequestMapping(path = "/getUser", method = RequestMethod.GET)
    public Result<UserDO> getUser(@RequestParam(name = "userId", required = true) Long
userId) {
        UserDO user = userService.getUser(userId);
        return Result.success(user);
    }
}
```

上面的代码，看上去是满足 SpringMVC 服务端三层架构的，唯一的问题就是把数据库模型类 UserDO 直接暴露给了外部接口。

1. 存在问题及解决方案

1) 存在问题

- 间接暴露数据库表格设计，给竞争对手竞品分析带来方便；
- 如果数据库查询不做字段限制，会导致接口数据庞大，浪费用户的宝贵流量；
- 如果数据库查询不做字段限制，容易把敏感字段暴露给接口，导致出现数据的安全问题；
- 如果数据库模型类不能满足接口需求，需要在数据库模型类中添加别的字段，导致数据库模型类跟数据库字段不匹配问题；
- 如果没有维护好接口文档，通过阅读代码是无法分辨出数据库模型类中哪些字段是接口使用的，导致代码的可维护性变差。

2) 解决方案

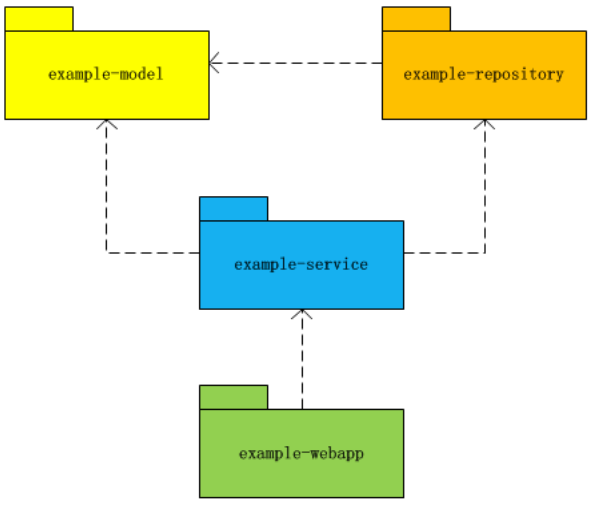
- 从管理制度上要求数据库和接口的模型类完全独立；
- 从项目结构上限制开发人员把数据库模型类暴露给接口。

2. 项目搭建的三种方式

下面，将介绍如何更科学地搭建 Java 项目，有效地限制开发人员把数据库模型类暴露给接口。

第 1 种：共用模型的项目搭建

共用模型的项目搭建，把所有模型类放在一个模型项目（example-model）中，其它项目（example-repository、example-service、example-website）都依赖该模型项目，关系图如下：



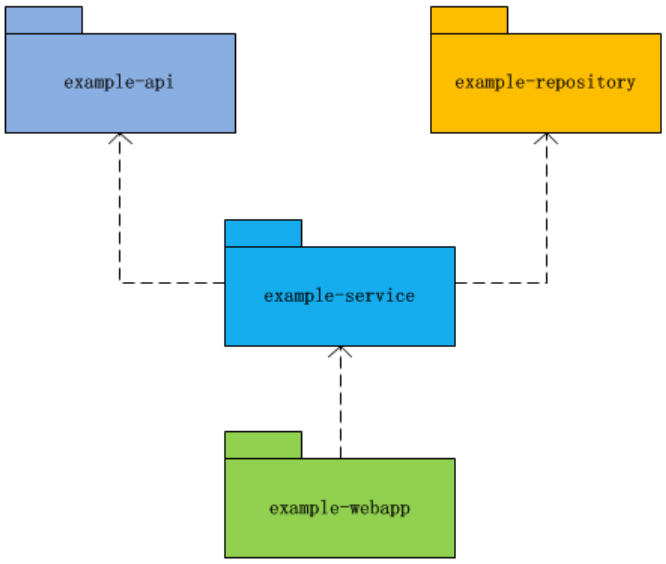
序号	项目名称	打包类型	项目功能
1	example-model	jar	定义了所有模型类，包括DO类和VO类等
2	example-repository	jar	对应持久层，实现了MySQL、Redis相关DAO等
3	example-service	jar	对应业务层，实现了Service、Job、Workflow等
4	example-webapp	war	对应表现层，实现了Controller、Interceptor、Filter等

- **风险**

表现层项目（example-webapp）可以调用业务层项目（example-service）中的任意服务函数，甚至于越过业务层直接调用持久层项目（example-repository）的 DAO 函数。

第 2 种：模型分离的项目搭建

模型分离的项目搭建，单独搭建 API 项目（example-api），抽象出对外接口及其模型 VO 类。业务层项目（example-service）实现了这些接口，并向表现层项目（example-webapp）提供服务。表现层项目（example-webapp）只调用 API 项目（example-api）定义的服务接口。



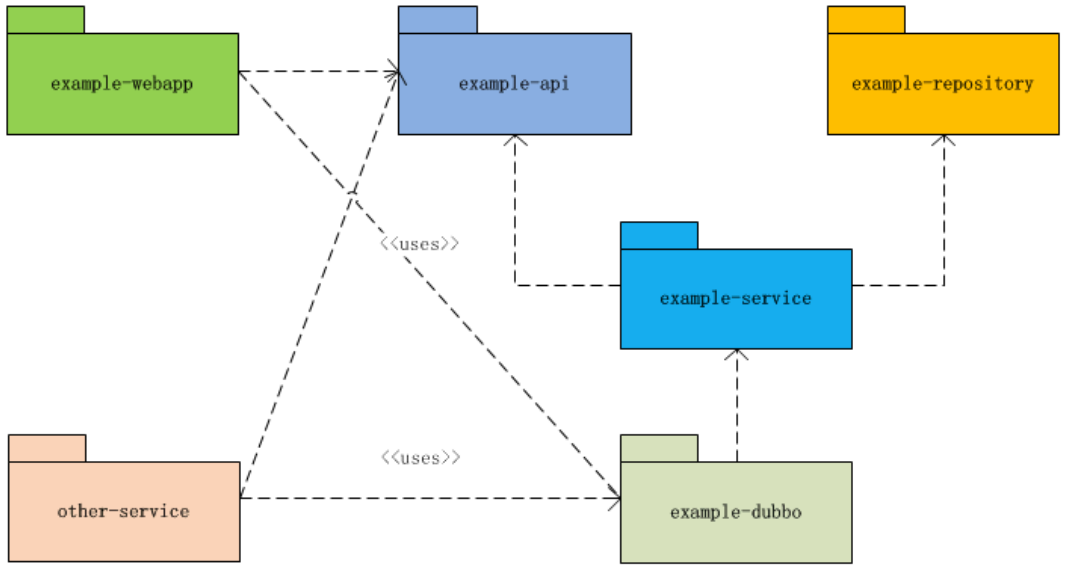
序号	项目名称	打包类型	项目功能
1	example-model	jar	定义了所有模型类，包括DO类和VO类等
2	example-repository	jar	对应持久层，实现了MySQL、Redis相关DAO等
3	example-service	jar	对应业务层，实现了Service、Job、Workflow等
4	example-webapp	war	对应表现层，实现了Controller、Interceptor、Filter等

- **风险**

表现层项目（example-webapp）仍然可以调用业务层项目（example-service）提供的内部服务函数和持久层项目（example-repository）的 DAO 函数。为了避免这种情况，只好管理制度上要求表现层项目（example-webapp）只能调用 API 项目（example-api）定义的服务接口函数。

第 3 种：服务化的项目搭建

服务化的项目搭，就是把业务层项目（example-service）和持久层项目（example-repository）通过 Dubbo 项目（example-dubbo）打包成一个服务，向业务层项目（example-webapp）或其它业务项目（other-service）提供 API 项目（example-api）中定义的接口函数。



序号	项目名称	打包类型	项目功能
1	example-api	jar	对应业务层的表现层，定义了对外开放接口和VO类
2	example-repository	jar	对应持久层，定义了DO类并实现了MySQL、Redis相关DAO等
3	example-service	jar	对应业务层，实现了Service、Job、Workflow等
4	example-dubbo	war	对应业务层的表现层，通过Dubbo提供服务
5	example-webapp	war	对应表现层，实现了Controller等，通过Dubbo调用服务
6	other-service	jar	对应其它项目的业务层，通过Dubbo调用服务

说明：

Dubbo 项目（example-dubbo）只发布 API 项目（example-api）中定义的服务接口，保证了数据库模型无法暴露。业务层项目（example-webapp）或其它业务项目（other-service）只依赖了 API 项目（example-api），只能调用该项目中定义的服务接口。

3. 一条不太建议的建议

有人会问：接口模型和持久层模型分离，接口定义了一个查询数据模型 VO 类，持久层也需要定义一个查询数据模型 DO 类；接口定义了一个返回数据模型 VO 类，持久层也需要定义一个返回数据模型 DO 类.....这样，对于项目早期快速迭代开发非常不利。能不能只让接口不暴露持久层数据模型，而能够让持久层使用接口的数据模型？

如果从 SpringMVC 服务端三层架构来说，这是不允许的，因为它会影响三层架构的独立性。但是，如果从快速迭代开发来说，这是允许的，因为它并不会暴露数据库模型类。所以，这是一条不太建议的建议。

```
/** 用户DAO类 */
@Repository
public class UserDAO {
    /** 统计用户函数 */
    public Long countByParameter(QueryUserParameterVO parameter) {...}
    /** 查询用户函数 */
    public List<UserVO> queryByParameter(QueryUserParameterVO parameter) {...}
}

/** 用户服务类 */
@Service
public class UserService {
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /** 查询用户函数 */
    public PageData<UserVO> queryUser(QueryUserParameterVO parameter) {
        Long totalCount = userDAO.countByParameter(parameter);
        List<UserVO> userList = null;
        if (Objects.nonNull(totalCount) && totalCount.compareTo(0L) > 0) {
            userList = userDAO.queryByParameter(parameter);
        }
        return new PageData<>(totalCount, userList);
    }
}

/** 用户控制器类 */
@Controller
@RequestMapping("/user")
public class UserController {
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /** 查询用户函数(parameter中包括分页参数startIndex和pageSize) */
    @RequestMapping(path = "/queryUser", method = RequestMethod.POST)
    public Result<PageData<UserVO>> queryUser(@Valid @RequestBody QueryUserParameterVO
parameter) {
        PageData<UserVO> pageData = userService.queryUser(parameter);
        return Result.success(pageData);
    }
}
```

后记

“仁者见仁、智者见智”，每个人都有自己的想法，而文章的内容也只是我的一家之言。

谨以此文献给那些我工作过的创业公司，是您们曾经放手让我去整改乱象，让我从中受益颇深并得以技术成长。

那些年，我们见过的 Java 服务端“问题”

导读

明代著名的心学集大成者王阳明先生在《传习录》中有云：

> *道无精粗，人之所见有精粗。如这一间房，人初进来，只见一个大规模如此。处久，便柱壁之类，一一看得明白。再久，如柱上有些文藻，细细都看出来。然只是一间房。*

是的，知识理论哪有什么精粗之分，只是人的认识程度不同而已。笔者在初创公司摸爬滚打数年，接触了各式各样的 Java 服务端架构，见得多了自然也就认识深了，就能分辨出各种方案的优劣了。这里，笔者总结了一些初创公司存在的 Java 服务端问题，并尝试性地给出了一些不成熟的解决方案。

一、 系统不是分布式

随着互联网的发展，计算机系统早就从单机独立工作过渡到多机器协同工作。计算机以集群的方式存在，按照分布式理论构建出庞大复杂的应用服务，早已深入人心并得到广泛地应用。但是，仍然有不少创业公司的软件系统停留在“单机版”。

1. 单机版系统抢单案例

这里，用并发性比较高的抢单功能为例说明：

```
// 抢取订单函数
public synchronized void grabOrder(Long orderId, Long userId) {
    // 获取订单信息
    OrderDO order = orderDAO.get(orderId);
    if (Objects.isNull(order)) {
        throw new BizRuntimeException(String.format("订单(%s)不存在", orderId));
    }

    // 检查订单状态
    if (!Objects.equals(order.getStatus, OrderStatus.WAITING_TO_GRAB.getValue())) {
        throw new BizRuntimeException(String.format("订单(%s)已被抢", orderId));
    }

    // 设置订单被抢
    orderDAO.setGrabed(orderId, userId);
}
```

以上代码，在一台服务器上运行没有任何问题。进入函数 grabOrder（抢取订单）时，利用 synchronized 关键字把整个函数锁定，要么进入函数前订单未被人抢取从而抢取订单成功，要么进入函数前订单已被抢取导致抢取订单失败，绝对不会出现进入函数前订单未被抢取而进入函数后订单又被抢取的情况。

但是，如果上面的代码在两台服务器上同时运行，由于 Java 的 synchronized 关键字只在一个虚拟机内生效，所以就会导致两个人能够同时抢取一个订单，但会以最后一个写入数据库的数据为准。所以，大多数的单机版系统，是无法作为分布式系统运行的。

2. 分布式系统抢单案例

添加分布式锁，进行代码优化：

```
// 抢取订单函数
public void grabOrder(Long orderId, Long userId) {
    Long lockId = orderDistributedLock.lock(orderId);
    try {
        grabOrderWithoutLock(orderId, userId);
    } finally {
        orderDistributedLock.unlock(orderId, lockId);
    }
}

// 不带锁的抢取订单函数
private void grabOrderWithoutLock(Long orderId, Long userId) {
    // 获取订单信息
    OrderDO order = orderDAO.get(orderId);
    if (Objects.isNull(order)) {
        throw new BizRuntimeException(String.format("订单(%s)不存在", orderId));
    }

    // 检查订单状态
    if (!Objects.equals(order.getStatus, OrderStatus.WAITING_TO_GRAB.getValue())) {
        throw new BizRuntimeException(String.format("订单(%s)已被抢", orderId));
    }

    // 设置订单被抢
    orderDAO.setGrabed(orderId, userId);
}
```

优化后的代码，在调用函数 grabOrderWithoutLock（不带锁的抢取订单）前后，利用分布式锁 orderDistributedLock（订单分布式锁）进行加锁和释放锁，跟单机版的 synchronized 关键字加锁效果基本一样。

3. 分布式系统的优缺点

分布式系统（Distributed System）是支持分布式处理的软件系统，是由通信网络互联的多处理机体系结构上执行任务的系统，包括分布式操作系统、分布式程序设计语言及其编译系统、分布式文件系统分布式数据库系统等。

1) 分布式系统的优点

- **可靠性、高容错性**
  一台服务器的崩溃，不会影响其它服务器，其它服务器仍能提供服务。

- **可扩展性**
  如果系统服务能力不足，可以水平扩展更多服务器。

- **灵活性**
  可以很容易的安装、实施、扩容和升级系统。

- **性能高**
  拥有多台服务器的计算能力，比单台服务器处理速度更快。

- **性价比高**
  分布式系统对服务器硬件要求很低，可以选用廉价服务器搭建分布式集群，从而得到更好的性价比。

2) 分布式系统的缺点

- **排查难度高**
  由于系统分布在多台服务器上，故障排查和问题诊断难度较高。

- **软件支持少**
  分布式系统解决方案的软件支持较少。

- **建设成本高**
  需要多台服务器搭建分布式系统。

曾经有不少的朋友咨询我：“找外包做移动应用，需要注意哪些事项？”

- **首先**，确定是否需要用分布式系统。软件预算有多少？预计用户量有多少？预计访问量有多少？是否只是业务前期试水版？单台服务器能否解决？是否接收短时间宕机？……如果综合考虑，单机版系统就可以解决的，那就不要采用分布式系统了。因为单机版系统和分布式系统的差别很大，相应的软件研发成本的差别也很大。

- **其次**，确定是否真正的分布式系统。分布式系统最大的特点，就是当系统服务能力不足时，能够通过水平扩展的方式，通过增加服务器来增加服务能力。然而，单机版系统是不支持水平扩展的，强行扩展就会引起一系列数据问题。由于单机版系统和分布式系统的研发成本差别较大，市面上的外包团队大多用单机版系统代替分布式系统交付。那么，如何确定你的系统是真正意义上的分布式系统呢？从软件上来说，是否采用了分布式软件解决方案；从硬件上来说，是否采用了分布式硬件部署方案。

4. 分布式软件解决方案

作为一个合格的分布式系统，需要根据实际需求采用相应的分布式软件解决方案。

1) 分布式锁

分布式锁是单机锁的一种扩展，主要是为了锁住分布式系统中的物理块或逻辑块，用以此保证不同服务之间的逻辑和数据的一致性。

目前，主流的分布式锁实现方式有 3 种：

- 基于数据库实现的分布式锁；
- 基于 Redis 实现的分布式锁；
- 基于 Zookeeper 实现的分布式锁。

2) 分布式消息

分布式消息中间件是支持在分布式系统中发送和接受消息的软件基础设施。常见的分布式消息中间件有 ActiveMQ、RabbitMQ、Kafka、MetaQ 等。

MetaQ（全称 Metamorphosis）是一个高性能、高可用、可扩展的分布式消息中间件，思路起源于 LinkedIn 的 Kafka，但并不是 Kafka 的一个拷贝。MetaQ 具有消息存储顺序写、吞吐量大和支持本地和 XA 事务等特性，适用于大吞吐量、顺序消息、广播和日志数据传输等场景。

3) 数据库分片分组

针对大数据量的数据库，一般会采用“分片分组”策略：

- **分片（shard）**：主要解决扩展性问题，属于水平拆分。引入分片，就引入了数据路由和分区键的概念。其中，分表解决的是数据量过大的问题，分库解决的是数据库性能瓶颈的问题。

- **分组（group）**：主要解决可用性问题，通过主从复制的方式实现，并提供读写分离策略用以提高数据库性能。

4) 分布式计算

分布式计算（Distributed computing）是一种“把需要进行大量计算的工程数据分割成小块，由多台计算机分别计算；在上传运算结果后，将结果统一合并得出数据结论”的科学。

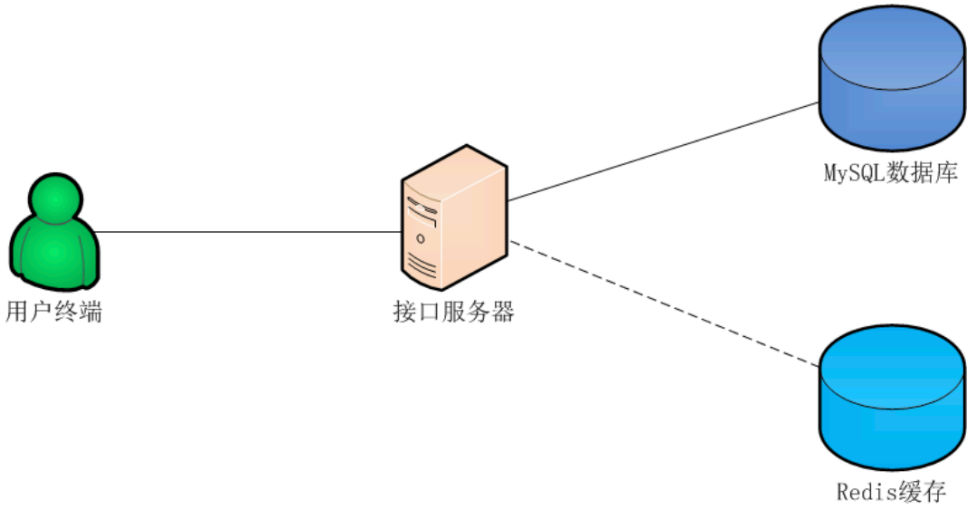
当前的高性能服务器在处理海量数据时，其计算能力、内存容量等指标都远远无法达到要求。在大数据时代，工程师采用廉价的服务器组成分布式服务集群，以集群协作的方式完成海量数据的处理，从而解决单台服务器在计算与存储上的瓶颈。Hadoop、Storm 以及 Spark 是常用的分布式计算中间件，Hadoop 是对非实时数据做批量处理的中间件，Storm 和 Spark 是对实时数据做流式处理的中间件。

除此之外，还有更多的分布式软件解决方案，这里就不再一一介绍了。

5. 分布式硬件部署方案

介绍完服务端的分布式软件解决方案，就不得不介绍一下服务端的分布式硬件部署方案。这里，只画出了服务端常见的接口服务器、MySQL 数据库、Redis 缓存，而忽略了其它的云存储服务、消息队列服务、日志系统服务.....

1) 一般单机版部署方案



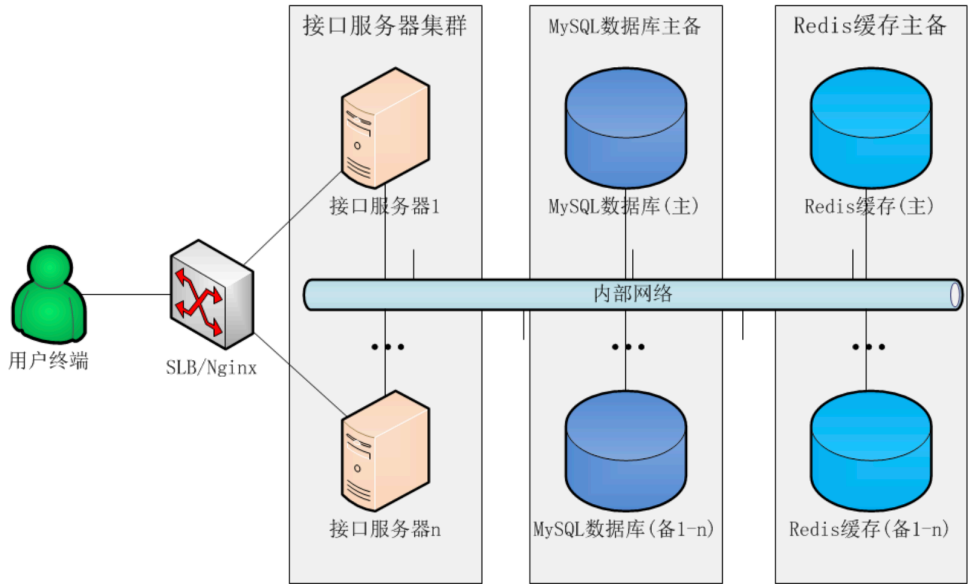
架构说明：

只有 1 台接口服务器、1 个 MySQL 数据库、1 个可选 Redis 缓存，可能都部署在同一台服务器上。

适用范围：

适用于演示环境、测试环境以及不怕宕机且日 PV 在 5 万以内的小型商业应用。

2) 中小型分布式硬件部署方案



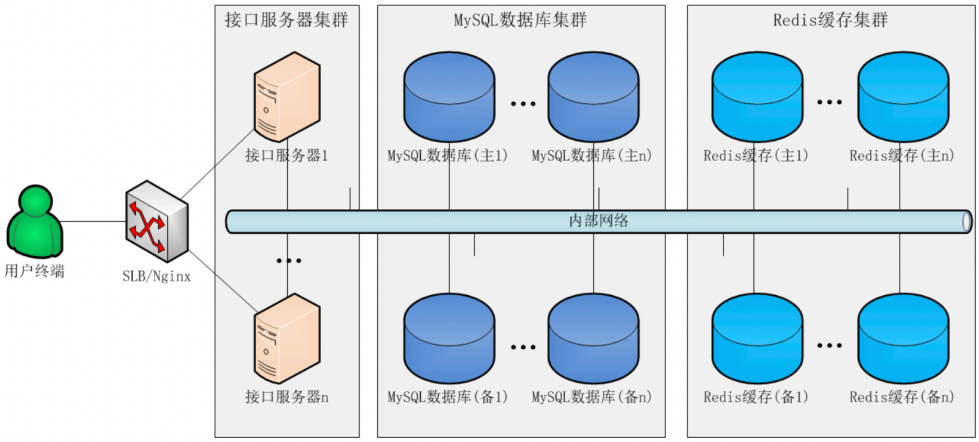
架构说明：

通过 SLB/Nginx 组成一个负载均衡的接口服务器集群，MySQL 数据库和 Redis 缓存采用了一主一备（或多备）的部署方式。

适用范围：

适用于日 PV 在 500 万以内的中小型商业应用。

3) 大型分布式硬件部署方案



架构说明：

通过 SLB/Nginx 组成一个负载均衡的接口服务器集群，利用分片分组策略组成一个 MySQL 数据库集群和 Redis 缓存集群。

适用范围：

适用于日 PV 在 500 万以上的大型商业应用。

二、 多线程使用不正确

多线程最主要目的就是“最大限度地利用 CPU 资源”，可以把串行过程变成并行过程，从而提高了程序的执行效率。

1. 一个慢接口案例

假设在用户登录时，如果是新用户，需要创建用户信息，并发放新用户优惠券。例子代码如下：

```
// 登录函数(示意写法)
public UserVO login(String phoneNumber, String verifyCode) {
    // 检查验证码
    if (!checkVerifyCode(phoneNumber, verifyCode)) {
        throw new ExampleException("验证码错误");
    }

    // 检查用户存在
    UserDO user = userDAO.getByPhoneNumber(phoneNumber);
    if (Objects.nonNull(user)) {
        return transUser(user);
    }

    // 创建新用户
    return createNewUser(user);
}

// 创建新用户函数
private UserVO createNewUser(String phoneNumber) {
    // 创建新用户
    UserDO user = new UserDO();
    ...
    userDAO.insert(user);

    // 绑定优惠券
    couponService.bindCoupon(user.getId(), CouponType.NEW_USER);

    // 返回新用户
    return transUser(user);
}
```

其中，绑定优惠券（bindCoupon）是给用户绑定新用户优惠券，然后再给用户发送推送通知。如果随着优惠券数量越来越多，该函数也会变得越来越慢，执行时间甚至超过 1 秒，并且没有什么优化空间。现在，登录（login）函数就成了名副其实的慢接口，需要进行接口优化。

2. 采用多线程优化

通过分析发现，绑定优惠券（bindCoupon）函数可以异步执行。首先想到的是采用多线程解决该问题，代码如下：

```
// 创建新用户函数
private UserVO createNewUser(String phoneNumber) {
    // 创建新用户
    UserDO user = new UserDO();
    ...
    userDAO.insert(user);

    // 绑定优惠券
    executorService.execute(()->couponService.bindCoupon(user.getId(),
CouponType.NEW_USER));

    // 返回新用户
    return transUser(user);
}
```

现在，在新线程中执行绑定优惠券（bindCoupon）函数，使用户登录（login）函数性能得到很大的提升。但是，如果在新线程执行绑定优惠券函数过程中，系统发生重启或崩溃导致线程执行失败，用户将永远获取不到新用户优惠券。除非提供用户手动领取优惠券页面，否则就需要程序员后台手工绑定优惠券。所以，用采用多线程优化慢接口，并不是一个完善的解决方案。

3. 采用消息队列优化

如果要保证绑定优惠券函数执行失败后能够重启执行，可以采用数据库表、Redis 队列、消息队列的等多种解决方案。由于篇幅优先，这里只介绍采用 MetaQ 消息队列解决方案，并省略了 MetaQ 相关配置仅给出了核心代码。

1) 消息生产者代码

```
// 创建新用户函数
private UserVO createNewUser(String phoneNumber) {
    // 创建新用户
    UserDO user = new UserDO();
    ...
    userDAO.insert(user);

    // 发送优惠券消息
    Long userId = user.getId();
    CouponMessageDataVO data = new CouponMessageDataVO();
    data.setUserId(userId);
    data.setCouponType(CouponType.NEW_USER);
    Message message = new Message(TOPIC, TAG, userId, JSON.toJSONBytes(data));
    SendResult result = metaqTemplate.sendMessage(message);
    if (!Objects.equals(result, SendStatus.SEND_OK)) {
        log.error("发送用户({})绑定优惠券消息失败:{}", userId, JSON.toJSONString(result));
    }

    // 返回新用户
    return transUser(user);
}
```

注意：

可能出现发生消息不成功，但是这种概率相对较低。

2) 消息消费者代码

```
// 优惠券服务类
@Slf4j
@Service
public class CouponService extends DefaultMessageListener<String> {
    // 消息处理函数
    @Override
    @Transactional(rollbackFor = Exception.class)
    public void onReceiveMessages(MetaqMessage<String> message) {
        // 获取消息体
        String body = message.getBody();
        if (StringUtils.isBlank(body)) {
            log.warn("获取消息({})体为空", message.getId());
            return;
        }

        // 解析消息数据
        CouponMessageDataVO data = JSON.parseObject(body, CouponMessageDataVO.class);
    if (Objects.isNull(data)) {
            log.warn("解析消息({})体为空", message.getId());
            return;
        }

        // 绑定优惠券
        bindCoupon(data.getUserId(), data.getCouponType());
    }
}
```

3) 解决方案优点

采集 MetaQ 消息队列优化慢接口解决方案的优点：

- 如果系统发生重启或崩溃，导致消息处理函数执行失败，不会确认消息已消费；由于 MetaQ 支持多服务订阅同一队列，该消息可以转到别的服务进行消费，亦或等到本服务恢复正常后再进行消费。
- 消费者可多服务、多线程进行消费消息，即便消息处理时间较长，也不容易引起消息积压；即便引起消息积压，也可以通过扩充服务实例的方式解决。
- 如果需要重新消费该消息，只需要在 MetaQ 管理平台上点击“消息验证”即可。

三、 流程定义不合理

1. 原有的采购流程

这是一个简易的采购流程，由库管系统发起采购，采购员开始采购，采购员完成采购，同时回流采集订单到库管系统。

其中，完成采购动作的核心代码如下：

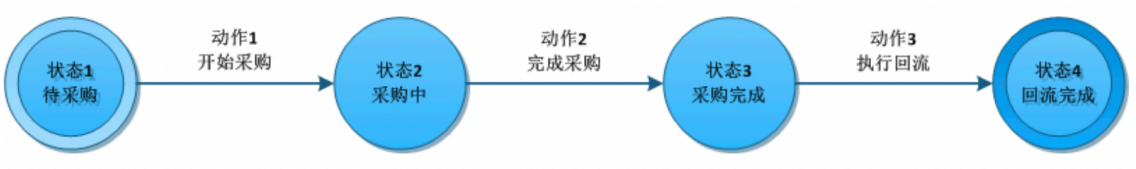
```
/** 完成采购动作函数(此处省去获取采购单/验证状态/锁定采购单等逻辑) */
public void finishPurchase(PurchaseOrder order) {
    // 完成相关处理
    ......

    // 回流采购单(调用HTTP接口)
    backflowPurchaseOrder(order);

    // 设置完成状态
    purchaseOrderDAO.setStatus(order.getId(), PurchaseOrderStatus.FINISHED.getValue());
}
```

由于函数 backflowPurchaseOrder（回流采购单）调用了 HTTP 接口，可能引起以下问题：

- 该函数可能耗费时间较长，导致完成采购接口成为慢接口；
- 该函数可能失败抛出异常，导致客户调用完成采购接口失败。

2. 优化的采购流程

通过需求分析，把“采购员完成采购并回流采集订单”动作拆分为“采购员完成采购”和“回流采集订单”两个独立的动作，把“采购完成”拆分为“采购完成”和“回流完成”两个独立的状态，更方便采购流程的管理和实现。



拆分采购流程的动作和状态后，核心代码如下：

```
/** 完成采购动作函数(此处省去获取采购单/验证状态/锁定采购单等逻辑) */
public void finishPurchase(PurchaseOrder order) {
    // 完成相关处理
    ......

    // 设置完成状态
    purchaseOrderDAO.setStatus(order.getId(), PurchaseOrderStatus.FINISHED.getValue());
}

/** 执行回流动作函数(此处省去获取采购单/验证状态/锁定采购单等逻辑) */
public void executeBackflow(PurchaseOrder order) {
    // 回流采购单(调用HTTP接口)
    backflowPurchaseOrder(order);

    // 设置回流状态
    purchaseOrderDAO.setStatus(order.getId(), PurchaseOrderStatus.BACKFLOWED.getValue());
}
```

其中，函数 executeBackflow（执行回流）由定时作业触发执行。如果回流采购单失败，采购单状态并不会修改为“已回流”；等下次定时作业执行时，将会继续执行回流动作；直到回流采购单成功为止。

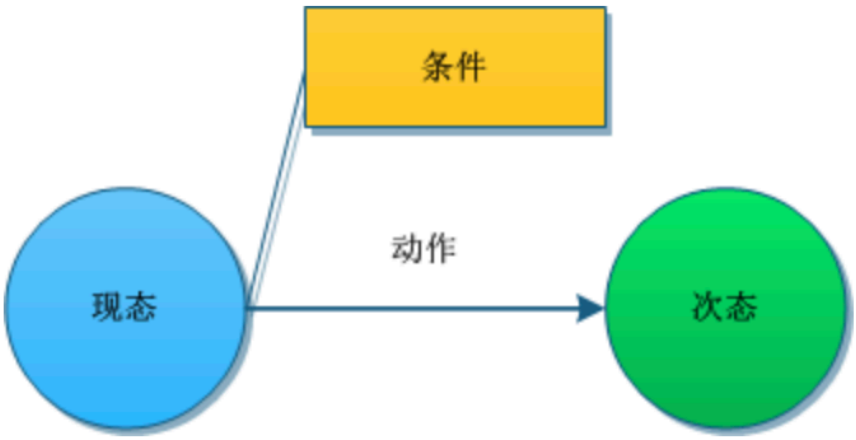
3. 有限状态机介绍

1) 概念

有限状态机（Finite-state machine，FSM），又称有限状态自动机，简称状态机，是表示有限个状态以及在这些状态之间的转移和动作等行为的一个数学模型。

2) 要素

状态机可归纳为 4 个要素：现态、条件、动作、次态。



- **现态**：指当前流程所处的状态，包括起始、中间、终结状态；
- **条件**：也可称为事件；当一个条件被满足时，将会触发一个动作并执行一次状态的迁移；
- **动作**：当条件满足后要执行的动作。动作执行完毕后，可以迁移到新的状态，也可以仍旧保持原状态；
- **次态**：当条件满足后要迁往的状态。“次态”是相对于“现态”而言的，“次态”一旦被激活，就转变成新的“现态”了。

3) 状态

状态表示流程中的持久状态，流程图上的每一个圈代表一个状态。

- **初始状态**：流程开始时的某一状态；
- **中间状态**：流程中间过程的某一状态；
- **终结状态**：流程完成时的某一状态。

使用建议：

- 状态必须是一个持久状态，而不能是一个临时状态；
- 终结状态不能是中间状态，不能继续进行流程流转；
- 状态划分合理，不要把多个状态强制合并为一个状态；
- 状态尽量精简，同一状态的不同情况可以用其它字段表示。

4) 动作

动作的三要素：角色、现态、次态，流程图上的每一条线代表一个动作。

- **角色**：谁发起的这个操作，可以是用户、定时任务等；
- **现态**：触发动作时当前的状态，是执行动作的前提条件；
- **次态**：完成动作后达到的状态，是执行动作的最终目标。

使用建议：

- 每个动作执行前，必须检查当前状态和触发动作状态的一致性；
- 状态机的状态更改，只能通过动作进行，其它操作都是不符合规范的；
- 需要添加分布式锁保证动作的原子性，添加数据库事务保证数据的一致性；
- 类似的动作（比如操作用户、请求参数、动作含义等）可以合并为一个动作，并根据动作执行结果转向不同的状态。

四、 系统间交互不科学

1. 直接通过数据库交互

在一些项目中，系统间交互不通过接口调用和消息队列，而是通过数据库直接访问。问其原因，回答道：“项目工期太紧张，直接访问数据库，简单又快捷”。

还是以上面的采购流程为例——采购订单由库管系统发起，由采购系统负责采购，采购完成后通知库管系统，库管系统进入入库操作。采购系统采购完成后，通知库管系统数据库的代码如下：

```
/** 执行回流动作函数(此处省去获取采购单/验证状态/锁定采购单等逻辑) */
public void executeBackflow(PurchaseOrder order) {
    // 完成原始采购单
    rawPurchaseOrderDAO.setStatus(order.getRawId(),
RawPurchaseOrderStatus.FINISHED.getValue());

    // 设置回流状态
    purchaseOrderDAO.setStatus(order.getId(), PurchaseOrderStatus.BACKFLOWED.getValue());
}
```

其中，通过 rawPurchaseOrderDAO（原始采购单 DAO）直接访问库管系统的数据库表，并设置原始采购单状态为已完成。

一般情况下，直接通过数据访问的方式是不会有问题的。但是，一旦发生竞态，就会导致数据不同步。有人会说，可以考虑使用同一分布式锁解决该问题。是的，这种解决方案没有问题，只是又在系统间共享了分布式锁。

1） 直接通过数据库交互的缺点

- 直接暴露数据库表，容易产生数据安全问题；
- 多个系统操作同一数据库表，容易造成数据库表数据混乱；
- 操作同一个数据库表的代码，分布在不同的系统中，不便于管理和维护；
- 具有数据库表这样的强关联，无法实现系统间的隔离和解耦。

2. 通过 Dubbo 接口交互

由于采购系统和库管系统都是内部系统，可以通过类似 Dubbo 的 RPC 接口进行交互。

1） 库管系统代码

```
/** 采购单服务接口 */
public interface PurchaseOrderService {
    /** 完成采购单函数 */
    public void finishPurchaseOrder(Long orderId);
}
/** 采购单服务实现 */
@Service("purchaseOrderService")
public class PurchaseOrderServiceImpl implements PurchaseOrderService {
    /** 完成采购单函数 */
    @Override
    @Transactional(rollbackFor = Exception.class)
    public void finishPurchaseOrder(Long orderId) {
        // 相关处理
        ...

        // 完成采购单
      purchaseOrderService.finishPurchaseOrder(order.getRawId());
    }
}
```

其中，库管系统通过 Dubbo 把 PurchaseOrderServiceImpl（采购单服务实现）以 PurchaseOrderService（采购单服务接口）定义的接口服务暴露给采购系统。这里，省略了 Dubbo 开发服务接口相关配置。

2) 采购系统代码

```
/** 执行回流动作函数(此处省去获取采购单/验证状态/锁定采购单等逻辑) */
public void executeBackflow(PurchaseOrder order) {
    // 完成采购单
    purchaseOrderService.finishPurchaseOrder(order.getRawId());

    // 设置回流状态
    purchaseOrderDAO.setStatus(order.getId(), PurchaseOrderStatus.BACKFLOWED.getValue());
}
```

其中，purchaseOrderService（采购单服务）为库管系统 PurchaseOrderService（采购单服务）在采购系统中的 Dubbo 服务客户端存根，通过该服务调用库管系统的服务接口函数 finishPurchaseOrder（完成采购单函数）。

这样，采购系统和库管系统自己的强关联，通过 Dubbo 就简单地实现了系统隔离和解耦。当然，除了采用 Dubbo 接口外，还可以采用 HTTPS、HSF、WebService 等同步接口调用方式，也可以采用 MetaQ 等异步消息通知方式。

3. 常见系统间交互协议

1) 同步接口调用

同步接口调用是以一种阻塞式的接口调用机制。常见的交互协议有：

- HTTP/HTTPS 接口；
- WebService 接口；
- Dubbo/HSF 接口；
- CORBA 接口。

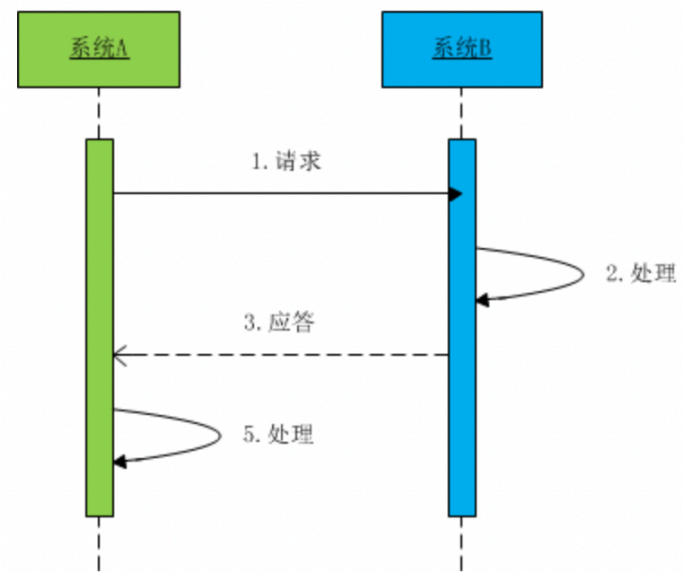
2) 异步消息通知

异步消息通知是一种通知式的信息交互机制。当系统发生某种事件时，会主动通知相应的系统。常见的交互协议有：

- MetaQ 的消息通知；
- CORBA 消息通知。

4. 常见系统间交互方式

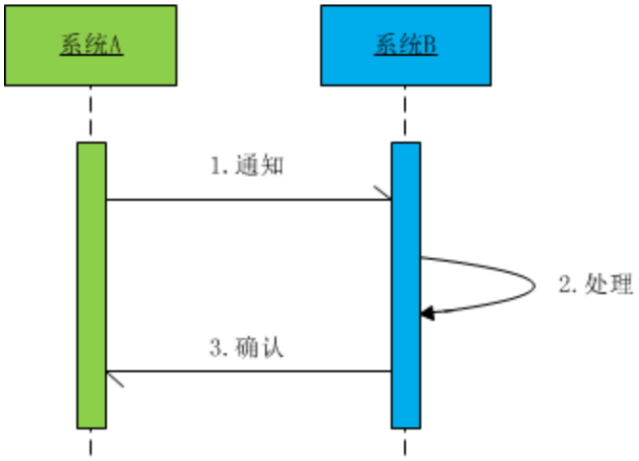
1) 请求-应答



- **适用范围**

适合于简单的耗时较短的接口同步调用场景，比如 Dubbo 接口同步调用。

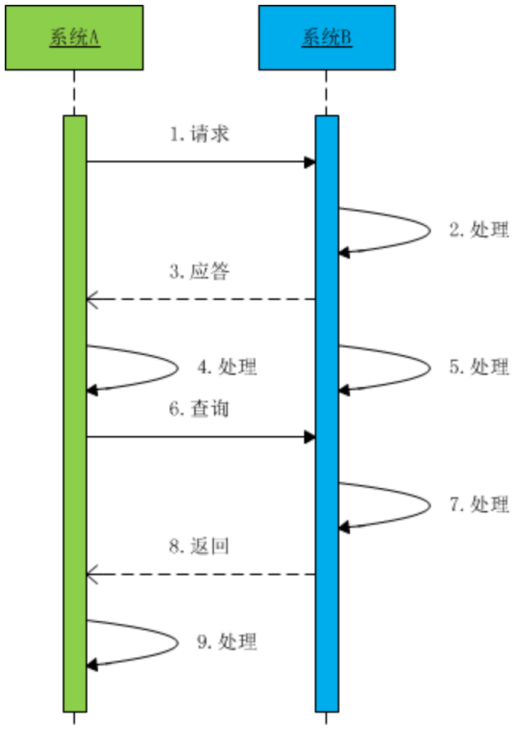
2) 通知-确认



- **适用范围**

适合于简单的异步消息通知场景，比如 MetaQ 消息通知。

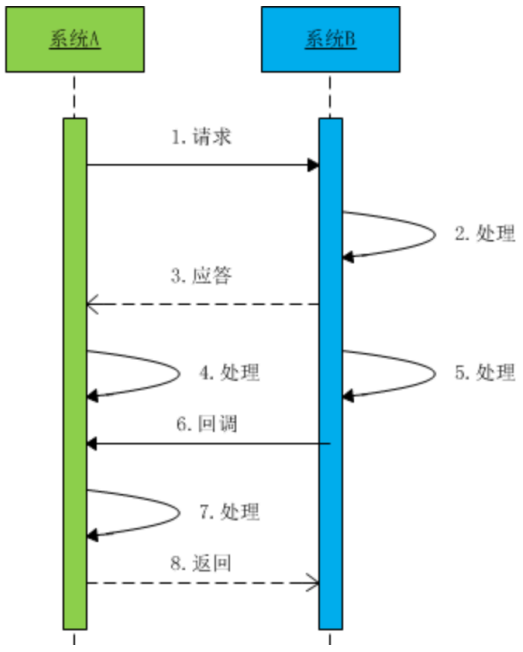
3) 请求-应答-查询-返回



- **适用范围**

适合于复杂的耗时较长的接口同步调用场景，比如提交作业任务并定期查询任务结果。

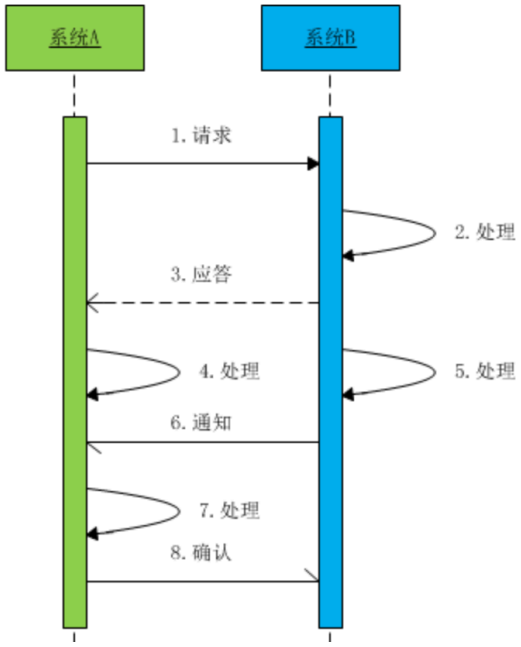
4) 请求-应答-回调



- **适用范围**

适合于复杂的耗时较长的接口同步调用和异步回调相结合的场景，比如支付宝的订单支付。

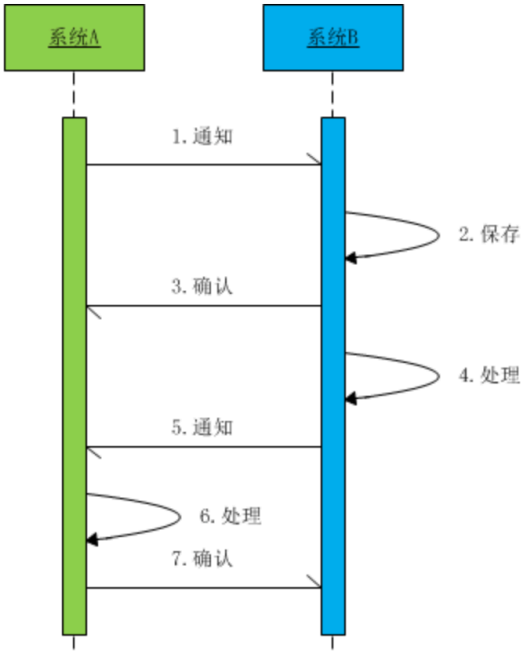
5) 请求-应答-通知-确认



- **适用范围**

适合于复杂的耗时较长的接口同步调用和异步消息通知相结合的场景，比如提交作业任务并等待完成消息通知。

6) 通知-确认-通知-确认



- **适用范围**

适合于复杂的耗时较长的异步消息通知场景。

五、 数据查询不分页

在数据查询时，由于未能对未来数据量做出正确的预估，很多情况下都没有考虑数据的分页查询。

1. 普通查询案例

以下是查询过期订单的代码：

```
/** 订单DAO接口 */
public interface OrderDAO {
    /** 查询过期订单函数 */
    @Select("select * from t_order where status = 5 and gmt_create <
date_sub(current_timestamp, interval 30 day)")
    public List<OrderDO> queryTimeout();
}

/** 订单服务接口 */
public interface OrderService {
    /** 查询过期订单函数 */
    public List<OrderVO> queryTimeout();
}
```

当过期订单数量很少时，以上代码不会有任何问题。但是，当过期订单数量达到几十万上千万时，以上代码就会出现以下问题：

- 数据量太大，导致服务端的内存溢出；
- 数据量太大，导致查询接口超时、返回数据超时等；
- 数据量太大，导致客户端的内存溢出。

所以，在数据查询时，特别是不能预估数据量的大小时，需要考虑数据的分页查询。

这里，主要介绍“设置最大数量”和“采用分页查询”两种方式。

2. 设置最大数量

“设置最大数量”是一种最简单的分页查询，相当于只返回第一页数据。例子代码如下：

```
/** 订单DAO接口 */
public interface OrderDAO {
    /** 查询过期订单函数 */
    @Select("select * from t_order where status = 5 and gmt_create <
date_sub(current_timestamp, interval 30 day) limit 0, #{maxCount}")
    public List<OrderDO> queryTimeout(@Param("maxCount") Integer maxCount);
}

/** 订单服务接口 */
public interface OrderService {
    /** 查询过期订单函数 */
    public List<OrderVO> queryTimeout(Integer maxCount);
}
```

适用于没有分页需求、但又担心数据过多导致内存溢出、数据量过大的查询。

3. 采用分页查询

“采用分页查询”是指定 startIndex（开始序号）和 pageSize（页面大小）进行数据查询，或者指定 pageIndex（分页序号）和 pageSize（页面大小）进行数据查询。

例子代码如下：

```
/** 订单DAO接口 */
public interface OrderDAO {
    /** 统计过期订单函数 */
    @Select("select count(*) from t_order where status = 5 and gmt_create <
date_sub(current_timestamp, interval 30 day)")
    public Long countTimeout();
    /** 查询过期订单函数 */
    @Select("select * from t_order where status = 5 and gmt_create <
date_sub(current_timestamp, interval 30 day) limit #{startIndex}, #{pageSize}")
    public List<OrderDO> queryTimeout(@Param("startIndex") Long startIndex,
@Param("pageSize") Integer pageSize);
}

/** 订单服务接口 */
public interface OrderService {
    /** 查询过期订单函数 */
    public PageData<OrderVO> queryTimeout(Long startIndex, Integer pageSize);
}
```

适用于真正的分页查询，查询参数 startIndex（开始序号）和 pageSize（页面大小）可由调用方指定。

4. 分页查询隐藏问题

假设，我们需要在一个定时作业（每 5 分钟执行一次）中，针对已经超时的订单（status=5，创建时间超时 30 天）进行超时关闭（status=10）。

实现代码如下：

```
/** 订单DAO接口 */
public interface OrderDAO {
    /** 查询过期订单函数 */
    @Select("select * from t_order where status = 5 and gmt_create <
date_sub(current_timestamp, interval 30 day) limit #{startIndex}, #{pageSize}")
    public List<OrderDO> queryTimeout(@Param("startIndex") Long startIndex,
@Param("pageSize") Integer pageSize);
    /** 设置订单超时关闭 */
    @Update("update t_order set status = 10 where id = #{orderId} and status = 5")
    public Long setTimeoutClosed(@Param("orderId") Long orderId)
}

/** 关闭过期订单作业类 */
public class CloseTimeoutOrderJob extends Job {
    /** 分页数量 */
    private static final int PAGE_COUNT = 100;
    /** 分页大小 */
    private static final int PAGE_SIZE = 1000;
    /** 作业执行函数 */
    @Override
    public void execute() {
        for (int i = 0; i < PAGE_COUNT; i++) {
            // 查询处理订单
            List<OrderDO> orderList = orderDAO.queryTimeout(i * PAGE_COUNT, PAGE_SIZE);
            for (OrderDO order : orderList) {
                // 进行超时关闭
                ......
                orderDAO.setTimeoutClosed(order.getId());
            }

            // 检查处理完毕
            if(orderList.size() < PAGE_SIZE) {
                break;
            }
        }
    }
}
```

粗看这段代码是没有问题的，尝试循环 100 次，每次取 1000 条过期订单，进行订单超时关闭操作，直到没有订单或达到 100 次为止。但是，如果结合订单状态一起看，就会发现从第二次查询开始，每次会忽略掉前 startIndex（开始序号）条应该处理的过期订单。这就是分页查询存在的隐藏问题：

- 当满足查询条件的数据，在操作中不再满足查询条件时，会导致后续分页查询中前 startIndex（开始序号）条满足条件的数据被跳过。

可以采用“设置最大数量”的方式解决，代码如下：

```java
/** 订单DAO接口 */
public interface OrderDAO {
    /** 查询过期订单函数 */
    @Select("select * from t_order where status = 5 and gmt_create <
date_sub(current_timestamp, interval 30 day) limit 0, #{maxCount}")
    public List<OrderDO> queryTimeout(@Param("maxCount") Integer maxCount);
    /** 设置订单超时关闭 */
    @Update("update t_order set status = 10 where id = #{orderId} and status = 5")
    public Long setTimeoutClosed(@Param("orderId") Long orderId)
}

/** 关闭过期订单作业(定时作业) */
public class CloseTimeoutOrderJob extends Job {
    /** 分页数量 */
    private static final int PAGE_COUNT = 100;
    /** 分页大小 */
    private static final int PAGE_SIZE = 1000;
    /** 作业执行函数 */
    @Override
    public void execute() {
        for (int i = 0; i < PAGE_COUNT; i++) {
            // 查询处理订单
            List<OrderDO> orderList = orderDAO.queryTimeout(PAGE_SIZE);
            for (OrderDO order : orderList) {
                // 进行超时关闭
                ......
                orderDAO.setTimeoutClosed(order.getId());
            }

            // 检查处理完毕
            if(orderList.size() < PAGE_SIZE) {
                break;
            }
        }
    }
}
```

后记

本文是《那些年，我们见过的 Java 服务端“乱象”》的姐妹篇，前文主要介绍的是 Java 服务端规范上的问题，而本文主要介绍的是 Java 服务端方案上的问题。

➢ Java 编程技巧系列

Java 编程技巧之输入输出参数

前言

软件开发方法学的泰斗肯特·贝克（Kent Beck）曾说过：

| *我不是一个伟大的程序员，我只是一个具有良好习惯的优秀程序员。*

养成良好的习惯，尤其是不断重构的习惯，是每一个优秀程序员都应该具备的素质。重构（Refactoring）就是在不改变软件现有功能的基础上，通过调整程序的结构、提高程序的质量、优化程序的性能……使其程序的设计模式和架构更趋合理，从而提高软件的稳定性、扩展性和维护性。

一、 一个需要重构的方法

1. 需求描述

需要把一个线串（一组经纬度坐标串），按照指定分段长度数组进行按比例划分。（由于指定线串的长度较小，可以近似地认为在几何平面上，无需进行球面距离换算。）

2. 代码实现

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {

    /** 常量相关 */
    /** 小数位数 */
    private static final int DIGIT_SCALE = 8;
    /** 放大比例 */
    private static final double ZOOM_SCALE = 10000000000L;
    /** 几何工厂 */
```

```
    private static final GeometryFactory GEOMETRY_FACTORY = new
GeometryFactory(new PrecisionModel(PrecisionModel.FLOATING));

    /**
     * 构造方法
     */
    private GeometryHelper() {
        throw new UnsupportedOperationException();
    }

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 检查分段数量
        if (Objects.isNull(segmentLengthes) || segmentLengthes.length < 1) {
            return new LineString[] {lineString};
        }

        // 计算总共长度
        double totalLength = Arrays.stream(segmentLengthes)
            .map(segmentLength -> Math.max(segmentLength, 0.0D))
            .sum();

        // 计算目标长度
        double lineLength = lineString.getLength();
        long[] targetLengthes = Arrays.stream(segmentLengthes)
            .mapToLong(segmentLength -> getTargetLength(lineLength, totalLength,
segmentLength))
            .toArray();

        // 初始化参数值
        int index = 1;
        Coordinate[] coordinates = lineString.getCoordinates();
        Coordinate coordinate = coordinates[0];
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
```

```
            // 添加线串坐标
            long addupLength = 0L;
            List<Coordinate> coordinateList = new ArrayList<>();
            coordinateList.add(coordinate);
            for (; index < coordinates.length; index++) {
                // 计算分段长度
                long segmentLength =
Math.round(coordinate.distance(coordinates[index]) * ZOOM_SCALE);

                // 根据长度处理
                boolean isBreak = true;
                int compareResult = Long.compare(addupLength + segmentLength,
targetLengthes[i]);
                // 根据长度处理：未达目标长度
                if (compareResult < 0) {
                    addupLength += segmentLength;
                    coordinate = coordinates[index];
                    coordinateList.add(coordinate);
                    isBreak = false;
                }
                // 根据长度处理：超过目标长度
                else if (compareResult > 0) {
                    long deltaLength = targetLengthes[i] - addupLength;
                    coordinate = buildMiddleCoordinate(coordinate,
coordinates[index], segmentLength, deltaLength);
                }
                // 根据长度处理：等于目标长度
                else {
                    index++;
                    coordinate = coordinates[index];
                }

                // 是否跳出循环
                if (isBreak) {
                    break;
                }
            }
            coordinateList.add(coordinate);

            // 设置线串对象
            lineStrings[i] = buildLineString(coordinateList);
        }

        // 添加最后一段
```

```
            lineStrings[length - 1] = buildLineString(coordinates, index,
coordinate);

            // 返回线串数组
            return lineStrings;
        }

        /**
         * 构建线串
         *
         * @param coordinates 坐标数组
         * @param index 当前序号
         * @param coordinate 当前坐标
         * @return 线串
         */
        private static LineString buildLineString(Coordinate[] coordinates, int
index, Coordinate coordinate) {
            List<Coordinate> coordinateList = new ArrayList<>();
            coordinateList.add(coordinate);
            coordinateList.addAll(Arrays.asList(ArrayUtils.subarray(coordinates,
index, coordinates.length)));
            return buildLineString(coordinateList);
        }

        /**
         * 构建线串
         *
         * @param coordinateList 坐标列表
         * @return 线串
         */
        private static LineString buildLineString(List<Coordinate> coordinateList)
{
            return GEOMETRY_FACTORY.createLineString(coordinateList.toArray(new
Coordinate[0]));
        }

        /**
         * 构建中间坐标
         *
         * @param coordinate1 坐标 1
         * @param coordinate2 坐标 2
         * @param segmentLength 分段长度
         * @param deltaLength 增量长度
         * @return 中间坐标
         */
```

```
    private static Coordinate buildMiddleCoordinate(Coordinate coordinate1,
Coordinate coordinate2,
        long segmentLength, long deltaLength) {
        double deltaScale = deltaLength * 1.0D / segmentLength;
        double middleX = round(coordinate1.x + (coordinate2.x - coordinate1.x) *
deltaScale, DIGIT_SCALE);
        double middleY = round(coordinate1.y + (coordinate2.y - coordinate1.y) *
deltaScale, DIGIT_SCALE);
        return new Coordinate(middleX, middleY);
    }

    /**
     * 获取目标长度
     *
     * @param lineLength 线路长度
     * @param totalLength 总共长度
     * @param segmentLength 段长度
     * @return 目标长度
     */
    private static long getTargetLength(double lineLength, double totalLength,
double segmentLength) {
        return Math.round(Math.max(segmentLength, 0.0D) * ZOOM_SCALE *
lineLength / totalLength);
    }

    /**
     * 四舍五入
     *
     * @param value 双精度浮点值
     * @param scale 保留小数位数
     * @return 四舍五入值
     */
    private static double round(double value, int scale) {
        return BigDecimal.valueOf(value).setScale(scale,
BigDecimal.ROUND_HALF_UP).doubleValue();
    }

}
```

3. 备注说明

在超过目标长度时，获取了一个中间坐标，由于数据精度的关系，这个坐标可能跟上一坐标或下一坐标重合。这里为了降低这块逻辑的复杂度，没有进行前后坐标的去重处理。

4. 存在问题

在方法 splitLineString（划分线串）中，存在一个两层循环，导致了方法逻辑复杂、层级较深、代码量大。如果把外层循环体提炼为一个方法，就能够使代码更简洁、更清晰、更容易维护。

二、 一次失败的重构经历

1. 理论依据

当看到一个方法定义过长时或者这段方法需要很多注释才能让人理解的时候，这时候就要考虑是不是把这个方法的部分代码提取出来，形成一个新的方法，方便调用和理解，同时也减小方法的粒度。我们把这种方法叫做提炼函数（Extract Function），在 Java 语言中可叫做提炼方法（Extract Method）。

2. 重构步骤

- 创建一个新方法，并根据这个方法的意图来命名；
- 将待提炼的代码段从原方法中拷贝到新方法中；
- 检查提炼的代码段，把缺少的变量添加到方法的参数中；
- 如果部分参数成对出现，可以把这些参数合并为一个参数；
- 如果方法需要有返回值，确定返回值的类型，并在合适的位置返回；
- 在原方法中，删除被提炼的代码段，替换为新方法的调用。

3. 代码实现

```
/**
 * 几何辅助类
 */
```

```
public final class GeometryHelper {

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        int index = 1;
        Coordinate[] coordinates = lineString.getCoordinates();
        Coordinate coordinate = coordinates[0];
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            lineStrings[i] = combineLineString(coordinates, index, coordinate,
targetLengthes[i]);
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates, index,
coordinate);

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param index 当前序号
     * @param coordinate 当前坐标
     * @param targetLength 目标长度
```

```
     * @return 线串
     */
    private static LineString combineLineString(Coordinate[] coordinates, int
index, Coordinate coordinate, long targetLength) {
        // 添加线串坐标
        long addupLength = 0L;
        List<Coordinate> coordinateList = new ArrayList<>();
        coordinateList.add(coordinate);
        for (; index < coordinates.length; index++) {
            // 计算分段长度
            long segmentLength =
Math.round(coordinate.distance(coordinates[index]) * ZOOM_SCALE);

            // 根据长度处理
            boolean isBreak = true;
            int compareResult = Long.compare(addupLength + segmentLength,
targetLength);
            // 根据长度处理：未达目标长度
            if (compareResult < 0) {
                addupLength += segmentLength;
                coordinate = coordinates[index];
                coordinateList.add(coordinate);
                isBreak = false;
            }
            // 根据长度处理：超过目标长度
            else if (compareResult > 0) {
                long deltaLength = targetLength - addupLength;
                coordinate = buildMiddleCoordinate(coordinate,
coordinates[index], segmentLength, deltaLength);
            }
            // 根据长度处理：等于目标长度
            else {
                index++;
                coordinate = coordinates[index];
            }

            // 是否跳出循环
            if (isBreak) {
                break;
            }
        }
        coordinateList.add(coordinate);

        // 返回线串对象
        return buildLineString(coordinateList);
```

```
    }

    /** 原有静态方法 */
    ......

}
```

4. 存在问题

粗看这段代码，似乎没有什么问题。但是，通过测试发现，并没有得到正确的结果。

5. 分析原因

在《Thinking in Java》中有这样一段话：

> *When you're passing primitives into a method，you get a distinct copy of the primitive. When you're passing a reference into a method, you get a copy of the reference.*
>
> *当您将基本类型传递到方法中时，您将得到该基本类型的副本。当您将对象引用传递到方法中时，您将得到该对象引用的副本。*

原来参数 index（当前序号）和 coordinate（当前坐标）在方法 combineLineString（组装线串）中的修改，只是对该方法中的参数副本进行修改，并没有体现到调用方法 splitLineString（划分线串）中，从而导致每次调用都在使用参数的初始化值。其实，这是在提取方法的过程中，没有考虑到参数的作用域。

6. 检查技巧

这里给出一个作者累试不爽的检查技巧——把提取方法的所有参数添加上 final 关键字，编译后观察到哪个参数出现编译错误，就说明这个参数是一个输入输出参数（Inout Parameter）。

7. 解决方案

在 Java 语言中，没有直接的输入输出参数机制，无法简单地实现参数的输入输出功能。所以，需要借助其它解决方案，来实现参数的输入输出功能。在这里，作者通过实践总结，给出了以下几种解决方案。

三、 利用方法参数实现

本章将从方法参数入手，实现参数的输入输出功能。

1. 利用参数类实现

理论依据：

引入参数对象（Introduce Parameter Object）：当一个方法的参数超过 3 个时，就可以考虑将参数封装成一个对象类。将参数封装成对象类后，提高了代码的可读性，并且该参数对象类也可以重用。以后，如果增加或删除参数，方法本身不需要修改，只需要修改参数对象类就可以了。

这里，可以利用引入参数对象重构方法，定义一个输入输出参数类，来实现参数的输入输出功能。

代码实现：

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
```

```
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        Coordinate[] coordinates = lineString.getCoordinates();
        InoutParameter inoutParameter = new InoutParameter(1, coordinates[0]);
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            lineStrings[i] = combineLineString(coordinates, inoutParameter,
targetLengthes[i]);
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates,
inoutParameter.getIndex(), inoutParameter.getCoordinate());

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param inoutParameter 输入输出参数
     * @param targetLength 目标长度
     * @return 线串
     */
    private static LineString combineLineString(Coordinate[] coordinates,
InoutParameter inoutParameter, long targetLength) {
        // 获取输入参数
        int index = inoutParameter.getIndex();
        Coordinate coordinate = inoutParameter.getCoordinate();

        // 添加线串坐标
        ......

        // 设置输出参数
        inoutParameter.setIndex(index);
```

```
        inoutParameter.setCoordinate(coordinate);

        // 返回线串对象
        return buildLineString(coordinateList);
    }

    /** 原有静态方法 */
    ......

    /**
     * 输入输出参数类
     */
    @Getter
    @Setter
    @ToString
    @NoArgsConstructor
    @AllArgsConstructor
    private static class InoutParameter {
        /** 当前序号 */
        private int index;
        /** 当前坐标 */
        private Coordinate coordinate;
    }

}
```

2. 利用单值数组实现

理论依据：

当您将对象引用传递到方法中时，您将得到该对象引用的副本。也就是说，当您将数组引用传递到方法中时，您将得到该数组引用的副本。但是，这两个数组引用都指向同一个数组，当修改副本数组引用中的值时，也能体现到原有数组引用中。

利用数组引用的这个特性，可以实现参数的输入输出功能。这里，引入了单值数组的概念，即一个数组只有一个值，用于传递输入输出参数值。

代码实现：

```
/**
 * 几何辅助类
```

```
 */
public final class GeometryHelper {

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        int[] indexHolder = new int[] {1};
        Coordinate[] coordinates = lineString.getCoordinates();
        Coordinate[] coordinateHolder = new Coordinate[] {coordinates[0]};
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            lineStrings[i] = combineLineString(coordinates, indexHolder,
coordinateHolder, targetLengthes[i]);
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates, indexHolder[0],
coordinateHolder[0]);

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param indexHolder 序号支撑
     * @param coordinateHolder 坐标支撑
```

```
     * @param targetLength 目标长度
     * @return 线串
     */
    private static LineString combineLineString(Coordinate[] coordinates, int[]
indexHolder, Coordinate[] coordinateHolder, long targetLength) {
        // 获取支撑取值
        int index = indexHolder[0];
        Coordinate coordinate = coordinateHolder[0];

        // 添加线串坐标
        ......

        // 设置支撑取值
        indexHolder[0] = index;
        coordinateHolder[0] = coordinate;

        // 返回线串对象
        return buildLineString(coordinateList);
    }

    /** 原有静态方法 */
    ......

}
```

3. 利用元组类实现

理论依据：

元组（Tuple）：Java 中的元组（Tuple）是一种数据结构，可以存放多个元素，并且每个元素的数据类型可以不同。Tuple 与 List 类似，但是不同的是，List 只能存储一种数据类型，而 Tuple 可存储多种数据类型。

可能你会质疑，Object 类型的 List 实际也是可以存储多种类型的啊？但是，在创建 List 时，需要指定元素数据类型，只能指定为 Object 类型；在获取的元素时，只能获取到 Object 类型的值，需要强制转化为对应的数据类型。而 Tuple 在创建时，可以直接指定多个元素数据类型；在获取元素时，无需进行数据类型的强制转化。

常用的元组工具包有：

a) Apache 的 commons-lang3 提供的元组类

- Pair<L, R>：MutablePair<L, R>，ImmutablePair<L, R>
- Triple<L, M, R>：MutableTriple<L, M, R>、ImmutableTriple<L, M, R>

b) JavaTuples 提供的元组类

- Unit<A>
- Pair<A,B>，KeyValue<A,B>
- Triplet<A,B,C>
- Quartet<A,B,C,D>
- Quintet<A,B,C,D,E>
- Sextet<A,B,C,D,E,F>
- Septet<A,B,C,D,E,F,G>
- Octet<A,B,C,D,E,F,G,H>
- Ennead<A,B,C,D,E,F,G,H,I>
- Decade<A,B,C,D,E,F,G,H,I,J>

随着元组的元数不断地增加，代码的阅读性也逐渐地下降。当元组的元数超过 3 个时，不如直接创建对象类，并给予合适类名和字段名，便于代码的理解和维护。所以，不建议使用 JavaTuples 中的元组类，而推荐使用 Apache 的 commons-lang3 提供的元组类。

代码实现：

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
```

```
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        Coordinate[] coordinates = lineString.getCoordinates();
        MutablePair<Integer, Coordinate> mutablePair = MutablePair.of(1,
coordinates[0]);
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            lineStrings[i] = combineLineString(coordinates, mutablePair,
targetLengthes[i]);
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates,
mutablePair.getLeft(), mutablePair.getRight());

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param mutablePair 当前配对
     * @param targetLength 目标长度
     * @return 线串
     */
    private static LineString combineLineString(Coordinate[] coordinates,
MutablePair<Integer, Coordinate> mutablePair,
        long targetLength) {
        // 获取配对取值
        int index = mutablePair.getLeft();
        Coordinate coordinate = mutablePair.getRight();

        // 添加线串坐标
        ......
```

```
        // 设置配对取值
        mutablePair.setLeft(index);
        mutablePair.setRight(coordinate);

        // 返回线串对象
        return buildLineString(coordinateList);
    }

    /** 原有静态方法 */
    ......

}
```

4. 利用支撑类实现

理论依据：

在上一节里，把所有输入输出参数放入到一个元组里，每一个输入输出参数没有一个具体的命名，造成了代码的理解和维护困难。如果每一个输入输出参数都定义一个元组，可以让代码维护者轻松地知道每一个参数的具体含义。所以，这里定义了自己的一元元组类——ObjectHolder（对象支撑类，也可以使用 javatuples 的 Unit 类），用于传递输入输出参数值。

代码实现：

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {
    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
```

```
        ......

        // 初始化参数值
        Coordinate[] coordinates = lineString.getCoordinates();
        ObjectHolder<Integer> indexHolder = new ObjectHolder<>(1);
        ObjectHolder<Coordinate> coordinateHolder = new
ObjectHolder<>(coordinates[0]);
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            lineStrings[i] = combineLineString(coordinates, indexHolder,
coordinateHolder, targetLengthes[i]);
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates,
indexHolder.getValue(), coordinateHolder.getValue());

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param indexHolder 序号支撑
     * @param coordinateHolder 坐标支撑
     * @param targetLength 目标长度
     * @return 线串
     */
    private static LineString combineLineString(Coordinate[] coordinates,
ObjectHolder<Integer> indexHolder, ObjectHolder<Coordinate> coordinateHolder,
long targetLength) {
        // 获取支撑取值
        int index = indexHolder.getValue();
        Coordinate coordinate = coordinateHolder.getValue();

        // 添加线串坐标
        ......

        // 设置支撑取值
        indexHolder.setValue(index);
```

```
        coordinateHolder.setValue(coordinate);

        // 返回线串对象
        return buildLineString(coordinateList);
    }

    /** 原有静态方法 */
    ......

}

/**
 * 对象支撑类
 */
@Getter
@Setter
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class ObjectHolder<T> {

    /** 对象取值 */
    private T value;

}
```

5. 利用其它方法实现

除此之外，还可以利用其它参数方法实现参数的输入输出功能：

1) 利用数组实现

首先，在调用函数中，定义一个对象数组，把所有输入输出参数存入对象数组中；其次，在被调用函数中，把这些参数从对象数组中取出来使用；再次，在被调用函数中，再把这些参数值存入对象数组中；最后，在调用函数中，把这些参数值从对象数组中取出来使用。

利用对象数组的问题是——代码可读性太差，而且在参数的存入和取出过程中，需要进行数据类型的强制转化。如果所有输入输出参数的类型一致，可以直接定义该类型的数组，从而避免了数据类型的强制转化。

2) 利用 Map 实现

首先，在调用函数中，定义一个对象 Map，把所有输入输出参数存入对象 Map 中；其次，在被调用函数中，把这些参数从对象 Map 中取出来使用；再次，在被调用函数中，再把这些参数值存入对象 Map 中；最后，在调用函数中，把这些参数值从对象 Map 中取出来使用。

利用对象 Map 实现，代码的可读性比利用对象数组实现更强，但是也存在同样的问题——在参数的存入和取出过程中，需要进行数据类型的强制转化。如果所有输入输出参数的类型一致，可以直接定义该类型的 Map，从而避免了数据类型的强制转化。但是，利用对象 Map 实现，还不如定义一个参数类更实用。

3) 利用原子类实现

JDK 中，提供了一套原子类——AtomicInteger、AtomicLong、AtomicDouble 等，可用于对应的基础类型和包装类型，实现对应参数的输入输出功能。实现方法跟 ObjectHolder 一样，这里不再累述。

四、 利用方法返回值实现

本章将从方法返回值入手，实现参数的输入输出功能。

1. 利用结果类实现

理论依据：

引入返回值对象（Introduce Return Object）：当一个方法的需要返回多个值时，就可以考虑将返回值封装成一个对象类。将返回值封装成对象类后，提高了代码的可读性，并且该返回值对象类也可以重用。以后，如果增加或删除返回值，方法本身不需要修改，只需要修改返回值对象类就可以了。

这里，可以利用引入返回值对象重构方法，定义一个返回值对象类，来实现参数的输入输出功能。

代码实现：

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        int index = 1;
        Coordinate[] coordinates = lineString.getCoordinates();
        Coordinate coordinate = coordinates[0];
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            ReturnResult result = combineLineString(coordinates, index,
coordinate, targetLengthes[i]);
            index = result.getIndex();
            coordinate = result.getCoordinate();
            lineStrings[i] = result.getLineString();
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates, index,
coordinate);

        // 返回线串数组
        return lineStrings;
    }
```

```
    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param index 当前序号
     * @param coordinate 当前坐标
     * @param targetLength 目标长度
     * @return 返回值
     */
    private static ReturnResult combineLineString(Coordinate[] coordinates, int
index, Coordinate coordinate, long targetLength) {
        // 添加线串坐标
        ......

        // 返回输出结果
        return new ReturnResult(index, coordinate,
buildLineString(coordinateList));
    }

    /** 原有静态方法 */
    ......

    /**
     * 返回值类
     */
    @Getter
    @Setter
    @ToString
    @NoArgsConstructor
    @AllArgsConstructor
    private static class ReturnResult {
        /** 当前序号 */
        private int index;
        /** 当前坐标 */
        private Coordinate coordinate;
        /** 线串对象 */
        private LineString lineString;
    }

}
```

2. 利用元组类实现

理论依据：

参考 3.3 章节的元组（Tuple）的定义和特性。元组（Tuple）可以用于方法的参数值，也可以用于方法的返回值。当一个方法需要返回多个值时，又不愿意定义自己的结果类时，可以采用元组（Tuple）实现多个值的返回。

代码实现：

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        int index = 1;
        Coordinate[] coordinates = lineString.getCoordinates();
        Coordinate coordinate = coordinates[0];
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            Triple<Integer, Coordinate, LineString> triple =
combineLineString(coordinates, index, coordinate, targetLengthes[i]);
            index = triple.getLeft();
```

```
            coordinate = triple.getMiddle();
            lineStrings[i] = triple.getRight();
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates, index,
coordinate);

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param index 当前序号
     * @param coordinate 当前坐标
     * @param targetLength 目标长度
     * @return 返回值
     */
    private static Triple<Integer, Coordinate, LineString>
combineLineString(Coordinate[] coordinates, int index, Coordinate coordinate,
long targetLength) {
        // 添加线串坐标
        ......

        // 返回输出结果
        return ImmutableTriple.of(index, coordinate,
buildLineString(coordinateList));
    }

    /** 原有静态方法 */
    ......

}
```

3. 利用其它方法实现

除此之外，还可以利用其它返回值方法实现参数的输入输出功能：

1） 利用数组实现

首先，在被调用方法中，定义一个对象数组，把多个返回值放入到对象数组中；最后，在调用函数中，把这些参数值从对象数组中取出来，并强制转化为对应的数据类型。

利用对象数组的问题是——代码可读性太差，而且在返回值的存入和取出过程中，需要进行数据类型的强制转化。如果所有返回值的数据类型一致，可以直接定义该类型的数组，从而避免了数据类型的强制转化。

2） 利用 Map 实现

首先，在被调用方法中，定义一个对象 Map，把多个返回值放入到对象 Map 中；最后，在调用函数中，把这些参数值从对象 Map 中取出来，并强制转化为对应的数据类型。

利用对象 Map 实现，代码的可读性比利用对象数组实现更强，但是也存在同样的问题——在返回值的存入和取出过程中，需要进行数据类型的强制转化。如果所有返回值的类型一致，可以直接定义该类型的 Map，从而避免了数据类型的强制转化。但是，利用对象 Map 实现，还不如定义一个返回值类更实用。

五、 利用类字段实现

本章将从类字段入手，实现参数的输入输出功能。

1. 利用线程本地变量实现

理论依据：

线程本地变量（ThreadLocal）：线程本地变量不同于它们的普通变量，因为访问某个变量的每个线程都有自己的局部变量，且独立于变量的初始化副本。线程本地变量实例通常是类中的私有静态字段，它希望将变量状态与某一个线程关联起来。要用类字段解决参数的输入输出问题，就必须考虑方法的线程安全性。这里，利用线程本地变量（ThreadLocal）来实现线程中输入输出参数值共享。

代码实现：

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {

    /** 属性相关 */
    /** 当前序号支撑 */
    private static final ThreadLocal<Integer> INDEX_HOLDER = new
ThreadLocal<>();
    /** 当前坐标支撑 */
    private static final ThreadLocal<Coordinate> COORDINATE_HOLDER = new
ThreadLocal<>();

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        INDEX_HOLDER.set(1);
        Coordinate[] coordinates = lineString.getCoordinates();
        COORDINATE_HOLDER.set(coordinates[0]);
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            lineStrings[i] = combineLineString(coordinates, targetLengthes[i]);
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates,
INDEX_HOLDER.get(), COORDINATE_HOLDER.get());
```

```
        // 清除支撑类
        INDEX_HOLDER.remove();
        COORDINATE_HOLDER.remove();

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param targetLength 目标长度
     * @return 线串
     */
    private static LineString combineLineString(Coordinate[] coordinates, long
targetLength) {
        // 获取支撑取值
        int index = INDEX_HOLDER.get();
        Coordinate coordinate = COORDINATE_HOLDER.get();

        // 添加线串坐标
        ......

        // 设置支持取值
        INDEX_HOLDER.set(index);
        COORDINATE_HOLDER.set(coordinate);

        // 返回线串对象
        return buildLineString(coordinateList);
    }

    /** 原有静态方法 */
    ......

}
```

2. 利用类成员变量实现

理论依据：

在上一章节中，利用线程本地变量（ThreadLocal）来实现线程中输入输出参数值共享，让方法的封装更复杂——需要从线程本地变量（ThreadLocal）读取和存储输入输出参数值。有没有一种更简单的方法，直接利用类成员变量实现输入输出参数值的共享呢？

答案是肯定的，可以把方法的封装和变量的定义封装到一个类中。这样，在每一个类实例中，都可以利用类成员变量来实现输入输出参数值的共享。但是，这个类是线程非安全的，必须在单线程中使用。

代码实现：

```
/**
 * 几何辅助类
 */
public final class GeometryHelper {

    // 原有构造方法
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public static LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        SplitLineStringAlgorithm algorithm = new SplitLineStringAlgorithm();
        return algorithm.splitLineString(lineString, segmentLengthes);
    }

}

/**
 * 划分线串算法类
 */
public class SplitLineStringAlgorithm {

    /** 属性相关 */
    /** 当前序号 */
    private int index;
```

```
    /** 当前坐标 */
    private Coordinate coordinate;

    /** 原有静态常量 */
    ......

    /**
     * 划分线串
     *
     * @param lineString 原始线串
     * @param segmentLengthes 分段长度数组
     * @return 线串数组
     */
    public LineString[] splitLineString(LineString lineString, double[]
segmentLengthes) {
        // 原有计算逻辑
        ......

        // 初始化参数值
        index = 1;
        Coordinate[] coordinates = lineString.getCoordinates();
        coordinate = coordinates[0];
        int length = targetLengthes.length;
        LineString[] lineStrings = new LineString[length];

        // 添加前面 N 段
        for (int i = 0; i < length - 1; i++) {
            lineStrings[i] = combineLineString(coordinates, targetLengthes[i]);
        }

        // 添加最后一段
        lineStrings[length - 1] = buildLineString(coordinates, index,
coordinate);

        // 返回线串数组
        return lineStrings;
    }

    /**
     * 组装线串
     *
     * @param coordinates 坐标数组
     * @param targetLength 目标长度
     * @return 线串
     */
```

```
    private LineString combineLineString(Coordinate[] coordinates, long
targetLength) {
        // 添加线串坐标
        ......

        // 返回线串对象
        return buildLineString(coordinateList);
    }

    /** 原有静态方法 */
    ......

}
```

六、 各种方法综合点评

下面，针对以上各种实现方法进行一个综合点评：

方法分类	方法名称	是否推荐	主要优点	主要缺点	适用场景
3.利用方法参数实现	3.1.利用参数类实现	是	可减少方法参数数量	需要自定义类	适用于输入输出参数较多场景。
	3.2.利用单值数组实现	否	无需自定义类	用数组代替变量	不建议使用。
	3.3.利用元组类实现	是	无需自定义类	需要操作元组对象	适用于输入输出参数较少的场景。
	3.4.利用支撑类实现	是	无需自定义类	需要操作支撑对象	适用于输入输出参数较少的场景。
4.利用方法返回值实现	4.1.利用结果类实现	是	支持返回值数多	需要自定义类	适用于返回值较多的场景。
	4.2.利用元组类实现	是	无需自定义类	需要操作元组对象	适用于返回值较少的场景。
5.利用类字段实现	5.1.利用线程本地变量实现	否	无需自定义类	使用ThreadLocal变量	不建议使用
	5.2.利用类成员变量实现	是	算法独立封装较灵活	直接使用成员变量	适用于复杂算法封装的场景。

总结如下：

- 各种实现方法有利有弊，应当根据具体的使用场景，来选择最适合的实现方法；

- 根据参数和返回值的类型选择实现方法：输入输出参数尽量使用方法参数实现，返回值尽量使用返回值实现；
- 根据参数和返回值的数量选择实现方法：数量少的尽量使用支撑类和元组类，数量多的尽量使用自定义类；
- 不建议使用一些取巧的实现方法，比如：3.2.利用单值数组实现、5.1.利用线程本地变量实现；
- 不推荐使用对象数组和对象 Map，Java 是强类型定义语言，不建议使用强制数据类型转化；
- 最适合本文中案例的实现方法是——3.4.利用支撑类实现。

后记

《庄子·养生主》有言：

> *吾生也有涯，而知也无涯。以有涯随无涯，殆已！*

意思是：我的生命是有限的，但知识却是无限的。用有限的生命去追求无限的知识，必然会失败的。

所以，知识并不是越多越好，而是“学而精之，精而深之，深而新之”。

Java 编程技巧之数据结构

导读

唐宋八大家之一欧阳修在《卖油翁》中写道：

> *翁取一葫芦置于地，以钱覆其口，徐以杓酌油沥之，自钱孔入，而钱不湿。因曰：“我亦无他，唯手熟尔。”*

编写代码的“老司机”也是如此，“老司机”之所以被称为“老司机”，原因也是“无他，唯手熟尔”。编码过程中踩过的坑多了，获得的编码经验也就多了，总结的编码技巧也就更多了。总结的编码技巧多了，凡事又能够举一反三，编码的速度自然就上来了。笔者从数据结构的角度，整理了一些 Java 编程技巧，以供大家学习参考。

一、 使用 HashSet 判断主键是否存在

HashSet 实现 Set 接口，由哈希表（实际上是 HashMap）实现，但不保证 set 的迭代顺序，并允许使用 null 元素。HashSet 的时间复杂度跟 HashMap 一致，如果没有哈希冲突则时间复杂度为 O(1)，如果存在哈希冲突则时间复杂度不超过 O(n)。所以，在日常编码中，可以使用 HashSet 判断主键是否存在。

案例

给定一个字符串（不一定全为字母），请返回第一个重复出现的字符。

```
/** 查找第一个重复字符 */
public static char findFirstRepeatedChar(String string) {
    // 检查空字符串
    if (Objects.isNull(string) || string.isEmpty()) {
        return null;
    }

    // 查找重复字符
    char[] charArray = string.toCharArray();
    Set charSet = new HashSet<>(charArray.length);
    for (char ch : charArray) {
        if (charSet.contains(ch)) {
            return ch;
        }
        charSet.add(ch);
    }

    // 默认返回为空
    return null;
}
```

其中，由于 Set 的 add 函数有个特性——如果添加的元素已经再集合中存在，则会返回 false。可以简化代码为：

```
if (!charSet.add(ch)) {
    return ch;
}
```

二、 使用 HashMap 存取键值映射关系

简单来说，HashMap 由数组和链表组成的，数组是 HashMap 的主体，链表则是主要为了解决哈希冲突而存在的。如果定位到的数组位置不含链表，那么查找、添加等操作很快，仅需一次寻址即可，其时间复杂度为 O（1）；如果定位到的数组包含链表，对于添加操作，其时间复杂度为 O（n）——首先遍历链表，存在即覆盖，不存在则新增；对于查找操作来讲，仍需要遍历链表，然后通过 key 对象的 equals 方法逐一对比查找。从性能上考虑，HashMap 中的链表出现越少，即哈希冲突越少，性能也就越好。所以，在日常编码中，可以使用 HashMap 存取键值映射关系。

案例

给定菜单记录列表，每条菜单记录中包含父菜单标识（根菜单的父菜单标识为 null），构建出整个菜单树。

```
/** 菜单 DO 类 */
@Setter
@Getter
@ToString
public static class MenuDO {
    /** 菜单标识 */
    private Long id;
    /** 菜单父标识 */
    private Long parentId;
    /** 菜单名称 */
    private String name;
    /** 菜单链接 */
    private String url;
}

/** 菜单 VO 类 */
@Setter
@Getter
@ToString
public static class MenuVO {
    /** 菜单标识 */
    private Long id;
    /** 菜单名称 */
    private String name;
```

```
    /** 菜单链接 */
    private String url;
    /** 子菜单列表 */
    private List<MenuVO> childList;
}

/** 构建菜单树函数 */
public static List<MenuVO> buildMenuTree(List<MenuDO> menuList) {
    // 检查列表为空
    if (CollectionUtils.isEmpty(menuList)) {
        return Collections.emptyList();
    }

    // 依次处理菜单
    int menuSize = menuList.size();
    List<MenuVO> rootList = new ArrayList<>(menuSize);
    Map<Long, MenuVO> menuMap = new HashMap<>(menuSize);
    for (MenuDO menuDO : menuList) {
        // 赋值菜单对象
        Long menuId = menuDO.getId();
        MenuVO menu = menuMap.get(menuId);
        if (Objects.isNull(menu)) {
            menu = new MenuVO();
            menu.setChildList(new ArrayList<>());
            menuMap.put(menuId, menu);
        }
        menu.setId(menuDO.getId());
        menu.setName(menuDO.getName());
        menu.setUrl(menuDO.getUrl());

        // 根据父标识处理
        Long parentId = menuDO.getParentId();
        if (Objects.nonNull(parentId)) {
            // 构建父菜单对象
            MenuVO parentMenu = menuMap.get(parentId);
            if (Objects.isNull(parentMenu)) {
                parentMenu = new MenuVO();
                parentMenu.setId(parentId);
                parentMenu.setChildList(new ArrayList<>());
                menuMap.put(parentId, parentMenu);
            }

            // 添加子菜单对象
            parentMenu.getChildList().add(menu);
        } else {
            // 添加根菜单对象
            rootList.add(menu);
        }
    }

    // 返回根菜单列表
    return rootList;
}
```

三、 使用 ThreadLocal 存储线程专有对象

ThreadLocal 提供了线程专有对象，可以在整个线程生命周期中随时取用，极大地方便了一些逻辑的实现。

常见的 ThreadLocal 用法主要有两种：

- 保存线程上下文对象，避免多层级参数传递；
- 保存非线程安全对象，避免多线程并发调用。

1. 保存线程上下文对象，避免多层级参数传递

这里，以 PageHelper 插件的源代码中的分页参数设置与使用为例说明。

1) 设置分页参数代码

```
/** 分页方法类 */
public abstract class PageMethod {
    /** 本地分页 */
    protected static final ThreadLocal<Page> LOCAL_PAGE = new ThreadLocal<Page>();

    /** 设置分页参数 */
    protected static void setLocalPage(Page page) {
        LOCAL_PAGE.set(page);
    }

    /** 获取分页参数 */
    public static <T> Page<T> getLocalPage() {
        return LOCAL_PAGE.get();
    }

    /** 开始分页 */
    public static <E> Page<E> startPage(int pageNum, int pageSize, boolean count, Boolean
reasonable, Boolean pageSizeZero) {
        Page<E> page = new Page<E>(pageNum, pageSize, count);
        page.setReasonable(reasonable);
        page.setPageSizeZero(pageSizeZero);
        Page<E> oldPage = getLocalPage();
        if (oldPage != null && oldPage.isOrderByOnly()) {
            page.setOrderBy(oldPage.getOrderBy());
        }
        setLocalPage(page);
        return page;
    }
}
```

2) 使用分页参数代码

```
/** 虚辅助方言类 */
public abstract class AbstractHelperDialect extends AbstractDialect implements Constant {
    /** 获取本地分页 */
    public <T> Page<T> getLocalPage() {
        return PageHelper.getLocalPage();
    }

    /** 获取分页SQL */
    @Override
    public String getPageSql(MappedStatement ms, BoundSql boundSql, Object parameterObject,
RowBounds rowBounds, CacheKey pageKey) {
        String sql = boundSql.getSql();
        Page page = getLocalPage();
        String orderBy = page.getOrderBy();
        if (StringUtil.isNotEmpty(orderBy)) {
            pageKey.update(orderBy);
            sql = OrderByParser.converToOrderBySql(sql, orderBy);
        }
        if (page.isOrderByOnly()) {
            return sql;
        }
        return getPageSql(sql, page, pageKey);
    }
    ...
}
```

3) 使用分页插件代码

```
/** 查询用户函数 */
public PageInfo<UserDO> queryUser(UserQuery userQuery, int pageNum, int pageSize) {
  PageHelper.startPage(pageNum, pageSize);
  List<UserDO> userList = userDAO.queryUser(userQuery);
  PageInfo<UserDO> pageInfo = new PageInfo<>(userList);
  return pageInfo;
}
```

如果要把分页参数通过函数参数逐级传给查询语句，除非修改 MyBatis 相关接口函数，否则是不可能实现的。

2. 保存非线程安全对象，避免多线程并发调用

在写日期格式化工具函数时，首先想到的写法如下：

```
/** 日期模式 */
private static final String DATE_PATTERN = "yyyy-MM-dd";

/** 格式化日期函数 */
public static String formatDate(Date date) {
    return new SimpleDateFormat(DATE_PATTERN).format(date);
}
```

其中，每次调用都要初始化 DateFormat 导致性能较低，把 DateFormat 定义成常量后的写法如下：

```
/** 日期格式 */
private static final DateFormat DATE_FORMAT = new SimpleDateFormat("yyyy-MM-dd");

/** 格式化日期函数 */
public static String formatDate(Date date) {
    return DATE_FORMAT.format(date);
}
```

由于 SimpleDateFormat 是非线程安全的，当多线程同时调用 formatDate 函数时，会导致返回结果与预期不一致。如果采用 ThreadLocal 定义线程专有对象，优化后的代码如下：

```
/** 本地日期格式 */
private static final ThreadLocal<DateFormat> LOCAL_DATE_FORMAT = new ThreadLocal<DateFormat>
() {
    @Override
    protected DateFormat initialValue() {
        return new SimpleDateFormat("yyyy-MM-dd");
    }
};

/** 格式化日期函数 */
public static String formatDate(Date date) {
    return LOCAL_DATE_FORMAT.get().format(date);
}
```

这是在没有线程安全的日期格式化工具类之前的实现方法。在 JDK8 以后，建议使用 DateTimeFormatter 代替 SimpleDateFormat，因为 SimpleDateFormat 是线程不安全的，而 DateTimeFormatter 是线程安全的。当然，也可以采用第三方提供的线程安全日期格式化函数，比如 apache 的 DateFormatUtils 工具类。

注意：

ThreadLocal 有一定的内存泄露的风险，尽量在业务代码结束前调用 remove 函数进行数据清除。

四、 使用 Pair 实现成对结果的返回

在 C/C++语言中，Pair（对）是将两个数据类型组成一个数据类型的容器，比如 std::pair。

Pair 主要有两种用途：

- 把 key 和 value 放在一起成对处理，主要用于 Map 中返回名值对，比如 Map 中的 Entry 类；
- 当一个函数需要返回两个结果时，可以使用 Pair 来避免定义过多的数据模型类。

第一种用途比较常见，这里主要说明第二种用途。

1. 定义模型类实现成对结果的返回

函数实现代码：

```
/** 点和距离类 */
@Setter
@Getter
@ToString
@AllArgsConstructor
public static class PointAndDistance {
    /** 点 */
    private Point point;
    /** 距离 */
    private Double distance;
}

/** 获取最近点和距离 */
public static PointAndDistance getNearestPointAndDistance(Point point, Point[] points) {
    // 检查点数组为空
    if (ArrayUtils.isEmpty(points)) {
        return null;
    }

    // 获取最近点和距离
    Point nearestPoint = points[0];
    double nearestDistance = getDistance(point, points[0]);
    for (int i = 1; i < points.length; i++) {
        double distance = getDistance(point, point[i]);
        if (distance < nearestDistance) {
            nearestDistance = distance;
            nearestPoint = point[i];
        }
    }

    // 返回最近点和距离
    return new PointAndDistance(nearestPoint, nearestDistance);
}
```

函数使用案例：

```
Point point = ...;
Point[] points = ...;
PointAndDistance pointAndDistance = getNearestPointAndDistance(point, points);
if (Objects.nonNull(pointAndDistance)) {
    Point point = pointAndDistance.getPoint();
    Double distance = pointAndDistance.getDistance();
    ...
}
```

2. 使用 Pair 类实现成对结果的返回

在 JDK 中，没有提供原生的 Pair 数据结构，也可以使用 Map::Entry 代替。不过，Apache 的 commons-lang3 包中的 Pair 类更为好用，下面便以 Pair 类进行举例说明。

函数实现代码：

```
/** 获取最近点和距离 */
public static Pair<Point, Double> getNearestPointAndDistance(Point point, Point[] points) {
    // 检查点数组为空
    if (ArrayUtils.isEmpty(points)) {
        return null;
    }

    // 获取最近点和距离
    Point nearestPoint = points[0];
    double nearestDistance = getDistance(point, points[0]);
    for (int i = 1; i < points.length; i++) {
        double distance = getDistance(point, point[i]);
        if (distance < nearestDistance) {
            nearestDistance = distance;
            nearestPoint = point[i];
        }
    }

    // 返回最近点和距离
    return Pair.of(nearestPoint, nearestDistance);
}
```

函数使用案例：

```
Point point = ...;
Point[] points = ...;
Pair<Point, Double> pair = getNearestPointAndDistance(point, points);
if (Objects.nonNull(pair)) {
    Point point = pair.getLeft();
    Double distance = pair.getRight();
    ...
}
```

此外，还有 Triple 类可以支持三个对象一起返回。

五、 定义 Enum 类实现取值和描述

在 C++、Java 等计算机编程语言中，枚举类型（Enum）是一种特殊数据类型，能够为一个变量定义一组预定义的常量。在使用枚举类型的时候，枚举类型变量取值必须为其预定义的取值之一。

1. 用 class 关键字实现的枚举类型

在 JDK5 之前，Java 语言不支持枚举类型，只能用类（class）来模拟实现枚举类型。

```
/** 订单状态枚举 */
public final class OrderStatus {
    /** 属性相关 */
    /** 状态取值 */
    private final int value;
    /** 状态描述 */
    private final String description;

    /** 常量相关 */
    /** 已创建(1) */
    public static final OrderStatus CREATED = new OrderStatus(1, "已创建");
    /** 进行中(2) */
    public static final OrderStatus PROCESSING = new OrderStatus(2, "进行中");
    /** 已完成(3) */
    public static final OrderStatus FINISHED = new OrderStatus(3, "已完成");

    /** 构造函数 */
    private OrderStatus(int value, String description) {
        this.value = value;
        this.description = description;
    }

    /** 获取状态取值 */
    public int getValue() {
        return value;
    }

    /** 获取状态描述 */
    public String getDescription() {
        return description;
    }
}
```

2. 用 enum 关键字实现的枚举类型

JDK5 提供了一种新的类型——Java 的枚举类型，关键字 enum 可以将一组具名的值的有限集合创建为一种新的类型，而这些具名的值可以作为常量使用，这是一种非常有用的功能。

```
/** 订单状态枚举 */
public enum OrderStatus {
    /** 常量相关 */
    /** 已创建(1) */
    CREATED(1, "已创建"),
    /** 进行中(2) */
    PROCESSING(2, "进行中"),
    /** 已完成(3) */
    FINISHED(3, "已完成");

    /** 属性相关 */
    /** 状态取值 */
    private final int value;
    /** 状态描述 */
    private final String description;

    /** 构造函数 */
    private OrderStatus(int value, String description) {
        this.value = value;
        this.description = description;
    }

    /** 获取状态取值 */
    public int getValue() {
        return value;
    }

    /** 获取状态描述 */
    public String getDescription() {
        return description;
    }
}
```

其实，Enum 类型就是一个语法糖，编译器帮我们做了语法的解析和编译。通过反编译，可以看到 Java 枚举编译后实际上是生成了一个类，该类继承了 java.lang.Enum<E>，并添加了 values()、valueOf()等枚举类型通用方法。

六、 定义 Holder 类实现参数的输出

在很多语言中，函数的参数都有输入（in）、输出（out）和输入输出（inout）之分。在 C/C++语言中，可以用对象的引用（&）来实现函数参数的输出（out）和输入输出（inout）。

但在 Java 语言中，虽然没有提供对象引用类似的功能，但是可以通过修改参数的字段值来实现函数参数的输出（out）和输入输出（inout）。

这里，我们叫这种输出参数对应的数据结构为 Holder（支撑）类。

1. Holder 类实现代码

```
/** 长整型支撑类 */
@Getter
@Setter
@ToString
public class LongHolder {
    /** 长整型取值 */
    private long value;

    /** 构造函数 */
    public LongHolder() {}

    /** 构造函数 */
    public LongHolder(long value) {
        this.value = value;
    }
}
```

2. Holder 类使用案例

```
/** 静态常量 */
/** 页面数量 */
private static final int PAGE_COUNT = 100;
/** 最大数量 */
private static final int MAX_COUNT = 1000;

/** 处理过期订单 */
public void handleExpiredOrder() {
  LongHolder minIdHolder = new LongHolder(0L);
    for (int pageIndex = 0; pageIndex < PAGE_COUNT; pageIndex++) {
        if (!handleExpiredOrder(pageIndex, minIdHolder)) {
          break;
        }
    }
}

/** 处理过期订单 */
private boolean handleExpiredOrder(int pageIndex, LongHolder minIdHolder) {
  // 获取最小标识
  Long minId = minIdHolder.getValue();

  // 查询过期订单(按id从小到大排序)
  List<OrderDO> orderList = orderDAO.queryExpired(minId, MAX_COUNT);
  if (CollectionUtils.isEmpty(taskTagList)) {
    return false;
  }

  // 设置最小标识
    int orderSize = orderList.size();
  minId = orderList.get(orderSize - 1).getId();
  minIdHolder.setValue(minId);

    // 依次处理订单
    for (OrderDO order : orderList) {
        ...
    }

  // 判断还有订单
  return orderSize >= PAGE_SIZE;
}
```

其实，可以实现一个泛型支撑类，适用于更多的数据类型。

七、 定义 Union 类实现数据体的共存

在 C/C++语言中，联合体（union），又称共用体，类似结构体（struct）的一种数据结构。联合体（union）和结构体（struct）一样，可以包含很多种数据类型和变量，两者区别如下：

- **结构体（struct）**中所有变量是“共存”的，同时所有变量都生效，各个变量占据不同的内存空间；

- **联合体（union）**中是各变量是“互斥”的，同时只有一个变量生效，所有变量占据同一块内存空间。

当多个数据需要共享内存或者多个数据每次只取其一时，可以采用联合体（union）。

在 Java 语言中，没有联合体（union）和结构体（struct）概念，只有类（class）的概念。众所众知，结构体（struct）可以用类（class）来实现。其实，联合体（union）也可以用类（class）来实现。但是，这个类不具备“多个数据需要共享内存”的功能，只具备“多个数据每次只取其一”的功能。

这里，以微信协议的客户消息为例说明。根据我多年来的接口协议封装经验，主要有以下两种实现方式。

1. 使用函数方式实现 Union

1） Union 类实现

```
/** 客户消息类 */
@ToString
public class CustomerMessage {

    /** 属性相关 */
    /** 消息类型 */
    private String msgType;
    /** 目标用户 */
    private String toUser;

    /** 共用体相关 */
    /** 新闻内容 */
    private News news;
    ...

    /** 常量相关 */
    /** 新闻消息 */
    public static final String MSG_TYPE_NEWS = "news";
    ...

    /** 构造函数 */
    public CustomerMessage() {}

    /** 构造函数 */
    public CustomerMessage(String toUser) {
        this.toUser = toUser;
    }

    /** 构造函数 */
    public CustomerMessage(String toUser, News news) {
        this.toUser = toUser;
        this.msgType = MSG_TYPE_NEWS;
        this.news = news;
    }
```

```
/** 清除消息内容 */
private void removeMsgContent() {
    // 检查消息类型
    if (Objects.isNull(msgType)) {
        return;
    }

    // 清除消息内容
    if (MSG_TYPE_NEWS.equals(msgType)) {
        news = null;
    } else if (...) {
     ...
     }
    msgType = null;
}

/** 检查消息类型 */
private void checkMsgType(String msgType) {
    // 检查消息类型
    if (Objects.isNull(msgType)) {
        throw new IllegalArgumentException("消息类型为空");
    }

    // 比较消息类型
    if (!Objects.equals(msgType, this.msgType)) {
        throw new IllegalArgumentException("消息类型不匹配");
    }
}

/** 设置消息类型函数 */
public void setMsgType(String msgType) {
    // 清除消息内容
    removeMsgContent();

    // 检查消息类型
    if (Objects.isNull(msgType)) {
        throw new IllegalArgumentException("消息类型为空");
    }

    // 赋值消息内容
    this.msgType = msgType;
    if (MSG_TYPE_NEWS.equals(msgType)) {
        news = new News();
    } else if (...) {
     ...
     } else {
        throw new IllegalArgumentException("消息类型不支持");
    }
}

/** 获取消息类型 */
public String getMsgType() {
    // 检查消息类型
    if (Objects.isNull(msgType)) {
        throw new IllegalArgumentException("消息类型无效");
    }

    // 返回消息类型
    return this.msgType;
}

/** 设置新闻 */
public void setNews(News news) {
    // 清除消息内容
```

```
        removeMsgContent();

        // 赋值消息内容
        this.msgType = MSG_TYPE_NEWS;
        this.news = news;
    }

    /** 获取新闻 */
    public News getNews() {
        // 检查消息类型
        checkMsgType(MSG_TYPE_NEWS);

        // 返回消息内容
        return this.news;
    }

    ...
}
```

2) Union 类使用

```
String accessToken = ...;
String toUser = ...;
List<Article> articleList = ...;
News news = new News(articleList);
CustomerMessage customerMessage = new CustomerMessage(toUser, news);
wechatApi.sendCustomerMessage(accessToken, customerMessage);
```

3) 主要优缺点

- 优点：更贴近 C/C++语言的联合体（union）；
- 缺点：实现逻辑较为复杂，参数类型验证较多。

2. 使用继承方式实现 Union

1) Union 类实现

```
/** 客户消息类 */
@Getter
@Setter
@ToString
public abstract class CustomerMessage {
    /** 属性相关 */
    /** 消息类型 */
    private String msgType;
    /** 目标用户 */
    private String toUser;

    /** 常量相关 */
    /** 新闻消息 */
    public static final String MSG_TYPE_NEWS = "news";
    ...
```

```
    /** 构造函数 */
    public CustomerMessage(String msgType) {
        this.msgType = msgType;
    }

    /** 构造函数 */
    public CustomerMessage(String msgType, String toUser) {
        this.msgType = msgType;
        this.toUser = toUser;
    }
}

/** 新闻客户消息类 */
@Getter
@Setter
@ToString(callSuper = true)
public class NewsCustomerMessage extends CustomerMessage {

    /** 属性相关 */
    /** 新闻内容 */
    private News news;

    /** 构造函数 */
    public NewsCustomerMessage() {
        super(MSG_TYPE_NEWS);
    }

    /** 构造函数 */
    public NewsCustomerMessage(String toUser, News news) {
        super(MSG_TYPE_NEWS, toUser);
        this.news = news;
    }
}
```

2) Union 类使用

```
String accessToken = ...;
String toUser = ...;
List<Article> articleList = ...;
News news = new News(articleList);
CustomerMessage customerMessage = new NewsCustomerMessage(toUser, news);
wechatApi.sendCustomerMessage(accessToken, customerMessage);
```

3) 主要优缺点

- 优点：使用虚基类和子类进行拆分，各个子类对象的概念明确；
- 缺点：与 C/C++语言的联合体（union）差别大，但是功能上大体一致。

在 C/C++语言中，联合体并不包括联合体当前的数据类型。但在上面实现的 Java 联合体中，已经包含了联合体对应的数据类型。所以，从严格意义上说，Java 联合体并不是真正的联合体，只是一个具备“多个数据每次只取其一”功能的类。

八、 使用泛型屏蔽类型的差异性

在 C++语言中，有个很好用的模板（template）功能，可以编写带有参数化类型的通用版本，让编译器自动生成针对不同类型的具体版本。而在 Java 语言中，也有一个类似的功能叫泛型（generic）。在编写类和方法的时候，一般使用的是具体的类型，而用泛型可以使类型参数化，这样就可以编写更通用的代码。

许多人都认为，C++模板（template）和 Java 泛型（generic）两个概念是等价的，其实实现机制是完全不同的。C++模板是一套宏指令集，编译器会针对每一种类型创建一份模板代码副本；Java 泛型的实现基于”类型擦除”概念，本质上是一种进行类型限制的语法糖。

1. 泛型类

以支撑类为例，定义泛型的通用支撑类：

```
/** 通用支撑类 */
@Getter
@Setter
@ToString
public class GenericHolder<T> {
    /** 通用取值 */
    private T value;

    /** 构造函数 */
    public GenericHolder() {}

    /** 构造函数 */
    public GenericHolder(T value) {
        this.value = value;
    }
}
```

2. 泛型接口

定义泛型的数据提供者接口：

```
/** 数据提供者接口 */
public interface DataProvider<T> {
    /** 获取数据函数 */
    public T getData();
}
```

3. 泛型方法

定义泛型的浅拷贝函数：

```
/** 浅拷贝函数 */
public static <T> T shallowCopy(Object source, Class<T> clazz) throws BeansException {
    // 判断源对象
    if (Objects.isNull(source)) {
        return null;
    }

    // 新建目标对象
    T target;
    try {
        target = clazz.newInstance();
    } catch (Exception e) {
        throw new BeansException("新建类实例异常", e);
    }

    // 拷贝对象属性
    BeanUtils.copyProperties(source, target);

    // 返回目标对象
    return target;
}
```

4. 泛型通配符

泛型通配符一般是使用“？”代替具体的类型实参，可以把“？”看成所有类型的父类。当具体类型不确定的时候，可以使用泛型通配符“？”；当不需要使用类型的具体功能，只使用 Object 类中的功能时，可以使用泛型通配符“？”。

```
/** 打印取值函数 */
public static void printValue(GenericHolder<?> holder) {
    System.out.println(holder.getValue());
}
/** 主函数 */
public static void main(String[] args) {
    printValue(new GenericHolder<>(12345));
    printValue(new GenericHolder<>("abcde"));
}
```

在 Java 规范中，不建议使用泛型通配符“？”，上面函数可以改为：

```
/** 打印取值函数 */
public static <T> void printValue(GenericHolder<T> holder) {
    System.out.println(holder.getValue());
}
```

5. 泛型上下界

在使用泛型的时候，我们还可以为传入的泛型类型实参进行上下界的限制，如：类型实参只准传入某种类型的父类或某种类型的子类。泛型上下界的声明，必须与泛型的声明放在一起。

1) 上界通配符（extends）

上界通配符为”extends”，可以接受其指定类型或其子类作为泛参。其还有一种特殊的形式，可以指定其不仅要是指定类型的子类，而且还要实现某些接口。例如：List<? extends A>表明这是 A 某个具体子类的 List，保存的对象必须是 A 或 A 的子类。对于 List<? extends A>列表，不能添加 A 或 A 的子类对象，只能获取 A 的对象。

2) 下界通配符（super）

下界通配符为”super”，可以接受其指定类型或其父类作为泛参。例如：List<? super A>表明这是 A 某个具体父类的 List，保存的对象必须是 A 或 A 的超类。对于 List<? super A>列表，能够添加 A 或 A 的子类对象，但只能获取 Object 的对象。

PECS（Producer Extends Consumer Super）原则

- 作为生产者提供数据（往外读取）时，适合用上界通配符（extends）；
- 作为消费者消费数据（往里写入）时，适合用下界通配符（super）。

在日常编码中，比较常用的是上界通配符（extends），用于限定泛型类型的父类。例子代码如下：

```java
/** 数字支撑类 */
@Getter
@Setter
@ToString
public class NumberHolder<T extends Number> {
    /** 通用取值 */
    private T value;

    /** 构造函数 */
    public NumberHolder() {}

    /** 构造函数 */
    public NumberHolder(T value) {
        this.value = value;
    }
}

/** 打印取值函数 */
public static <T extends Number> void printValue(GenericHolder<T> holder) {
    System.out.println(holder.getValue());
}
```

后记

笔者曾在通信行业从业十余年，接入了各类网管和设备的北向接口协议上百余种，涉及到传输、交换、接入、电源、环境等专业，接触了 CORBA、HTTP/HTTPS、WebService、Socket TCP/UDP、串口 RS232/485 等接口，总结出一套接口协议封装的”方法论”。其中，把接口协议文档中的数据格式转化为 Java 的枚举、结构体、联合体等数据结构，是接口协议封装中极其重要的一步。

Java 编程技巧之样板代码

前言

北宋科学家沈括在《梦溪笔谈》第十八卷《技艺》中这样描述“活字印刷术”：

> *庆历中，有布衣毕昇，又为活版。其法用胶泥刻字，薄如钱唇，每字为一印，火烧令坚……若止印三、二本，未为简易；若印数十百千本，则极为神速。*

在日常编码的过程中，我们可以总结出很多“样板代码”，就像“活字印刷术”中的“活字”一样。当我们编写新的代码时，需要用到这些“活字”，就把“样板代码”拷贝过来，修改替换一下就可以了，写起代码来“极为神速”。“样板代码”其实就是一种样例、一种模式、一种经验……总结的“样板代码”越多，编写代码的格式越规范、质量越高、速度越快。

这里，作者总结了几种常见 Java 的“样板代码”，希望起到抛砖引玉的作用，希望大家不断总结和完善，形成自己的样板代码库。

一、 样板代码简介

1. 什么是样板代码？

样板代码（Boilerplate Code），通常是指一堆具有固定模式的代码块，可以被广泛地应用到各个程序模块。

例如，读取文件就是典型的样板代码：

```
try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
    String line;
    while (Objects.nonNull(line = reader.readLine())) {
        // 处理一行
        ...
    }
} catch (IOException e) {
    String message = String.format("读取文件(%s)异常", fileName);
    log.error(message, e);
    throw new ExampleException(message, e);
}
```

2. 样板代码有什么用？

样板（Boilerplate），可以拆分为样例（Example）和模式（Pattern）两个单词进行理解——样例（Example）指可以当成一种标准范例，模式（Pattern）指可以作为一种解决方案。当遇到类似的案例时，就把样板代码拷贝过去，根据实际情况进行修改，该案例就被轻松解决了。

样板代码的主要作用：

- 提供一种标准样例：可以用于新人学习，能够快速上手并使用；
- 提供一种解决方案：遇到类似案例时，可以快速利用该方案进行解决；
- 有助于不断积累经验：当发现一种样例代码时，都会不断地进行优化，力求达到最佳样例；
- 有助于提高代码质量：样板代码必然通过了时间考验，存在 BUG 和出错的几率相对比较低；
- 有助于提高编码速度：利用样板代码编码，只是复制粘贴修改代码，编码速度大幅提高；
- 有助于统一代码样式：心中有了样板代码，就能保证每次都写出统一样式的代码。

3. 如何编写样板代码？

适合于样板代码的编写方法有：

1) 复制粘贴生成代码

利用复制粘贴样板代码，用好了编码会事半功倍。

2) 用文本替换生成代码

利用文本替换生成代码，可以很快生成一段新代码。

3) 用 Excel 公式生成代码

把样板代码先公式化，传入不同的参数，生成不同的代码。

4) 用工具或插件生成代码

很多开发工具或插件都提供一些工具生成代码，比如：生成构造方法、重载基类/接口方法、生成 Getter/Setter 方法、生成 toString 方法、生成数据库访问方法……能够避免很多手敲代码。

5) 用代码生成代码

用代码生成代码，就是自己编写代码，按照自己的样板代码格式生成代码。

4. 如何减少样板代码？

样板代码（Boilerplate Code）具有很大的重复性，通常被认为是一种冗余而又不得不写的代码。其实不然，有些样板代码不能减少，只是我们还没有遇到合适的解决方案而已。通常情况下，我们可以通过以下几种方式减少样板代码：

1) 利用注解减少样板代码

比如，JavaBean 模型类中的 Getter/Setter 就是样板代码，我们可以通过 Lombok 的@Getter/@Setter 注解来减少这样的样板代码。

原始代码：

```
public class User {
    private Long id;
    ...
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    ...
}
```

优化代码：

```
@Getter
@Setter
public class User {
    private Long id;
    ...
}
```

2) 利用框架减少样板代码

比如，MyBatis 是一款优秀的持久层框架，封装了获取数据库连接和声明、设置参数、获取结果集等所有 JDBC 操作。MyBatis 可以通过简单的 XML 或注解来配置和映射原始类型、接口和 Java POJO（Plain Old Java Objects，普通老式 Java 对象）为数据库中的记录。

原始代码：

```
/** 查询公司员工 */
public List<EmployeeDO> queryEmployee(Long companyId) {
    try (Connection connection = tddlDataSource.getConnection();
        PreparedStatement statement = connection.prepareStatement(QUERY_EMPLOYEE_SQL)) {
        statement.setLong(1, companyId);
        try (ResultSet result = statement.executeQuery()) {
            List<EmployeeDO> employeeList = new ArrayList<>();
            while (result.next()) {
                EmployeeDO employee = new EmployeeDO();
                employee.setId(result.getLong(1));
                employee.setName(result.getString(2));
                ...
                employeeList.add(employee);
            }
            return employeeList;
        }
    } catch (SQLException e) {
        String message = String.format("查询公司(%s)用户异常", companyId);
        log.error(message, e);
        throw new ExampleException(message, e);
    }
}
```

优化代码：

- **UserDAO.java**

```
@Mapper
public interface UserDAO {
    List<EmployeeDO> queryEmployee(@Param("companyId") Long companyId);
}
```

- **UserDAO.xml**

```
<mapper namespace="com.example.repository.UserDAO">
    <select id="queryEmployee" resultType="com.example.repository.UserDO">
        select id
        , name
        ...
        from t_user
        where company_id = #{companyId}
    </select>
</mapper>
```

3) 利用设计模式减少样板代码

利用设计模式，可以把一些重复性代码进行封装。比如，上面的读取文件行模式代码，就可以用模板方法进行封装。

原始代码：

```
try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
    String line;
    while (Objects.nonNull(line = reader.readLine())) {
        // 处理一行
        ...
    }
} catch (IOException e) {
    String message = String.format("读取文件(%s)异常", fileName);
    log.error(message, e);
    throw new ExampleException(message, e);
}
```

优化代码：

```
/** 定义方法 */
public static void readLine(String fileName, Consumer<String> lineConsumer) {
    try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
        String line;
        while (Objects.nonNull(line = reader.readLine())) {
            lineConsumer.accept(line);
        }
    } catch (IOException e) {
        String message = String.format("读取文件(%s)异常", fileName);
        log.error(message, e);
        throw new ExampleException(message, e);
    }
}

// 使用代码
readLine("example.txt", line -> {
    // 处理一行
    ...
});
```

5. 消灭不了的样板代码

如果样板代码可以被消灭，那么世界上就不存在样板代码了。即便是上一节提供的减少样板代码方法，也不能完全的消灭样板代码，因为这些样板代码依旧存在于框架和模式的实现中。所以，样板代码是消灭不了的。

既然不能消灭样板代码，那就应该合理地利用样板代码。提炼一段样板代码，若只用二三次，未为简便；若用数十百千次，则极为神速。下面，列举了几种常见 Java 的样板代码，描述了样板代码在日常编程中如何提炼和使用。

二、 定义工具类

1. 常用定义方式

通常，我们会如下定义工具类：

```
/** 例子工具类 */
public class ExampleHelper {
    /** 常量值 */
    public final static int CONST_VALUE = 123;
    /** 求和方法 */
    public static int sum(int a, int b) {
        return a + b;
    }
}
```

2. 存在一些问题

1) 修饰符顺序不规范

通过 SonarLint 插件扫描，会出现以下问题：

Rule key	Rule name	Description
java:S1124	Modifiers should be declared in the correct order（修饰符应该以正确的顺序声明）	Reorder the modifiers to comply with the Java Language Specification.（重新排序修饰符以符合Java语言规范。）

Java 语言规范建议使用“static final”，而不是“final static”。请记住这么一条规则：静态常量，静态（static）在前，常量（final）在后。

2) 工具类可以被继承覆盖

如果我们定义一个 MyExampleHelper 来继承 ExampleHelper：

```
public class MyExampleHelper extends ExampleHelper {
    /** 常量值 */
    public static final int CONST_VALUE = 321;

    /** 求和方法 */
    public static int sum(int a, int b) {
        return a * b;
    }
}
```

会发现，MyExampleHelper 会对 ExampleHelper 中的常量和方法进行覆盖，导致我们不知道是不是使用了 ExampleHelper 中的常量和方法。

对于 Apache 提供的工具类，很多同学都喜欢定义相同名称的工具类，并让这个工具类继承 Apache 的工具类，并在这个类中添加自己的实现方法。其实，我是非常不推荐这种做法的，因为你不知道——你调用的是 Apache 工具类提供的常量和方法，还是被覆盖的常量和方法。最好的办法，就是对工具类添加 final 关键字，让这个工具类不能被继承和覆盖。

3) 工具类可以被实例化

对于 ExampleHelper 工具类，我们可以这样使用：

```
int value = ExampleHelper.CONST_VALUE;
int sum = ExampleHelper.sum(1, 2);
```

也可以被这样使用：

```
ExampleHelper exampleHelper = new ExampleHelper();
int value = exampleHelper.CONST_VALUE;
int sum = exampleHelper.sum(1, 2);
```

对于工具类来说，没有必要进行实例化。所以，我们建议添加私有构造方法，并在方法中抛出 UnsupportedOperationException（不支持的操作异常）。

3. 最佳定义方式

根据以上存在问题及其解决方法，最佳定义的 ExampleHelper 工具类如下：

```
/** 例子工具类 */
public final class ExampleHelper {
    /** 常量值 */
    public static final int CONST_VALUE = 123;

    /** 构造方法 */
    private ExampleHelper() {
        throw new UnsupportedOperationException();
    }

    /** 求和方法 */
    public static int sum(int a, int b) {
        return a + b;
    }
}
```

三、 定义枚举类

1. 常用定义方式

通常，我们会如下定义枚举类：

```java
/** 例子枚举类 */
public enum ExampleEnum {
    /** 枚举相关 */
    ONE(1, "one(1)"),
    TWO(2, "two(2)"),
    THREE(3, "two(3)");

    /** 属性相关 */
    private Integer value;
    private String desc;

    /** 构造方法 */
    private ExampleEnum(Integer value, String desc) {
        this.value = value;
        this.desc = desc;
    }

    /** 获取取值 */
    public Integer getValue() {
        return value;
    }

    /** 获取描述 */
    public String getDesc() {
        return desc;
    }
}
```

2. 一些优化建议

1) 修饰符 private 可缺省

通过 SonarLint 插件扫描，会出现以下问题：

Rule key	Rule name	Description
java:S2333	Redundant modifiers should not be used（不应该使用多余的修饰符）	"private" is redundant in this context.（private在上下文中是多余的。）

根据建议，应该删除构造方法前多余的 private 修饰符。

2) 建议使用基础类型

用包装类型 Integer 保存枚举取值，本身并没有什么问题。但是，本着能用基础类型就用基础类型的规则，所以建议使用基础类型 int。

3) 建议使用 final 字段

假设，我们要实现一个静态方法，可能一不小心就把枚举值给修改了：

```
/** 修改取值 */
public static void modifyValue() {
    for (ExampleEnum value : values()) {
        value.value++;
    }
}
```

如果调用了 modifyValue 方法，就会把枚举值修改，导致应用程序出错。为了避免这样的情况出现，我们建议对字段添加 final 修饰符，从而避免字段值被恶意篡改。

3. 最佳定义方式

```
/** 例子枚举类 */
public enum ExampleEnum {
    /** 枚举相关 */
    ONE(1, "one(1)"),
    TWO(2, "two(2)"),
    THREE(3, "two(3)");

    /** 字段相关 */
    private final int value;
    private final String desc;

    /** 构造方法 */
    ExampleEnum(int value, String desc) {
        this.value = value;
        this.desc = desc;
    }

    /** 获取取值 */
    public int getValue() {
        return value;
    }

    /** 获取描述 */
    public String getDesc() {
        return desc;
    }
}
```

四、 定义模型类

下面，以定义 User（用户）模型类为例，从 JavaBean 模式、重载构造方法、Builder 模式 3 种方式，来说明模型类的定义方法以及优缺点。

假设：User（用户）模型类共有 4 个属性——id（标识）、name（名称）、age（年龄）、desc（描述），其中必填属性为——id（标识）、name（名称），可填属性为——age（年龄）、desc（描述）。

1. JavaBean 模式

JavaBean 是一个遵循特定写法的 Java 类，它通常具有如下特点：

- 必须具有一个无参的构造方法；
- 所有属性字段必须是私有的；
- 所有属性字段必须通过遵循一种命名规范的 Getter/Setter 方法开放出来。

通过 JavaBean 模式定义的 User（用户）模型类如下：

```
/** 用户类 */
public class User {
    private Long id;
    private String name;
    private Integer age;
    private String desc;

    public Long getId() {return id;}
    public void setId(Long id) {this.id = id;}
    public String getName() {return name;}
    public void setName(String name) {this.name = name;}
    public Integer getAge() {return age;}
    public vid setAge(Integer age) {this.age = age;}
    public String getDesc() {return desc;}
    public void setDesc(String desc) {this.desc = desc;}
}
```

注意：
也可以通过 Lombok 的@Getter/@Setter 注解生成对应个 Getter/Setter 方法。

1） 使用代码

```
User user = new User();
user.setId(1L);
user.setName("alibaba");
user.setAge(102);
user.setDesc("test");
verifyUser(user);
```

2) 主要优点

- 代码非常简单，只有私有属性字段及其公有 Getter/Setter 方法；
- 赋值对象代码可读性较强，明确地知道哪个属性字段对应哪个值；
- 非常简单实用，被广泛地用于 HSF、Dubbo、MyBatis 等中间件。

3) 主要缺点

- 由于可以通过 Setter 方法设置属性字段，所以不能定义为不可变类；
- 由于每个字段分别设置，所以不能保证字段必填，必须设置完毕后进行统一验证。

2. 重载构造方法

通过“重载构造方法”定义 User（用户）模型类如下：

```java
/** 用户类 */
public final class User {
    private Long id;
    private String name;
    private Integer age;
    private String desc;

    public User(Long id, String name) {
        this(id, name, null);
    }
    public User(Long id, String name, Integer age) {
        this(id, name, age, null);
    }
    public User(Long id, String name, Integer age, String desc) {
        Assert.notNull(id, "标识不能为空");
        Assert.notNull(name, "名称不能为空");
        this.id = id;
        this.name = name;
        this.age = age;
        this.desc = desc;
    }

    public Long getId() {return id;}
    public String getName() {return name;}
    public Integer getAge() {return age;}
    public String getDesc() {return desc;}
}
```

1) 使用代码

```java
User user1 = new User(1L, "alibaba");
User user2 = new User(1L, "alibaba", 102, "test");
```

2) 主要优点

- 初始化对象代码简洁，只有简单的一行代码；
- 可以定义为不可变类，初始化后属性字段值不可变更；
- 可以在构造方法内进行不可空验证。

3) 主要缺点

- 重载构造方法数量过多，无法覆盖必填字段和非必填字段的所有组合；
- 初始化对象代码可读性差，无法看出哪个属性字段对应哪个值；
- 如果删除某个字段，初始化对象代码可能不会报错，导致出现赋值错误问题。

3. Builder 模式

```
/** 用户类 */
public final class User {
    private Long id;
    private String name;
    private Integer age;
    private String desc;

    private User(Builder builder) {
        this.id = builder.id;
        this.name = builder.name;
        this.age = builder.age;
        this.desc = builder.desc;
    }
    public static Builder newBuilder(Long id, String name) {
        return new Builder(id, name);
    }

    public Long getId() {return id;}
    public String getName() {return name;}
    public Integer getAge() {return age;}
    public String getDesc() {return desc;}

    public static class Builder {
        private Long id;
        private String name;
        private Integer age;
        private String desc;

        private Builder(Long id, String name) {
            Assert.notNull(id, "标识不能为空");
            Assert.notNull(name, "名称不能为空");
            this.id = id;
            this.name = name;
        }
        public Builder age(Integer age) {
            this.age = age;
            return this;
        }
        public Builder desc(String desc) {
            this.desc = desc;
            return this;
        }
        public User build() {
            return new User(this);
        }
    }
}

```

注意：

可以采用 Lombok 的@Builder 注解简化代码。

1) 使用代码

```
User user = User.newBuilder(1L, "alibaba").age(102).desc("test").build();
```

2) 主要优点

- 明确了必填参数和可选参数，在构造方法中进行验证；
- 可以定义为不可变类，初始化后属性字段值不可变更；
- 赋值代码可读性较好，明确知道哪个属性字段对应哪个值；
- 支持链式方法调用，相比于调用 Setter 方法，代码更简洁。

3) 主要缺点

- 代码量较大，多定义了一个 Builder 类，多定义了一套属性字段，多实现了一套赋值方法；
- 运行效率低，需要先创建 Builder 实例，再赋值属性字段，再创建目标实例，最后拷贝属性字段。

五、 定义集合常量

在编码中，经常使用到各种集合常量，比如 List（列表）常量、Set（集合）常量、Map（映射）常量等。

1. 普通定义方式

1) 定义代码

最简单的方法，就是直接定义一个普通的集合常量。

```
/** 例子工具类 */
public final class ExampleHelper {
    /** 常量值列表 */
    public static final List<Integer> CONST_VALUE_LIST = Arrays.asList(1, 2, 3);
    /** 常量值集合 */
    public static final Set<Integer> CONST_VALUE_SET = new HashSet<>(Arrays.asList(1, 2,
3));
    /** 常量值映射 */
    public static final Map<Integer, String> CONST_VALUE_MAP;
    static {
        CONST_VALUE_MAP = new HashMap<>(MapHelper.DEFAULT);
        CONST_VALUE_MAP.put(1, "value1");
        CONST_VALUE_MAP.put(2, "value2");
        CONST_VALUE_MAP.put(3, "value3");
    }
    ...
}
```

2) 使用代码

使用也很方便，直接通过“类名.常量名”使用。

```
// 使用常量值集合
List<Integer> constValueList = ExampleHelper.CONST_VALUE_LIST;
Set<Integer> constValueSet = ExampleHelper.CONST_VALUE_SET;
Map<Integer, String> constValueMap = ExampleHelper.CONST_VALUE_MAP;
```

2. 存在主要问题

通过 SonarLint 插件扫描，会出现以下问题：

Rule key	Rule name	Description
java:S2386	Mutable fields should not be "public static"（可变字段不应为“公共静态”）	Make this member "protected".（将此成员设为“protected”。）

由于普通的集合对象（如 ArrayList、HashMap、HashSet 等）都是可变集合对象，即便是定义为静态常量，也可以通过操作方法进行修改。所以，上面方法定义的集合常量，并不是真正意义上的集合常量。其中，Arrays.asList 方法生成的内部 ArrayList 不能执行 add/remove/clear 方法，但是可以执行方法，也属于可变集合对象。

```
// 操作常量列表
ExampleHelper.CONST_VALUE_LIST.remove(3); // UnsupportedOperationException
ExampleHelper.CONST_VALUE_LIST.add(4); // UnsupportedOperationException
ExampleHelper.CONST_VALUE_LIST.set(1, 20); // [1,20,3]
ExampleHelper.CONST_VALUE_LIST.clear(); // UnsupportedOperationException

// 操作常量集合
ExampleHelper.CONST_VALUE_SET.remove(3); // [1,2]
ExampleHelper.CONST_VALUE_SET.add(3); // [1,2,3]
ExampleHelper.CONST_VALUE_SET.clear(); // []

// 操作常量映射
ExampleHelper.CONST_VALUE_MAP.remove(3); // {1:"value1",2:"value2"}
ExampleHelper.CONST_VALUE_MAP.put(3, "value3"); // {1:"value1",2:"value2",3:"value3"}
ExampleHelper.CONST_VALUE_MAP.clear(); // []
```

3. 最佳定义方式

在 JDK 中，Collections 工具类中提供一套方法，用于把可变集合对象变为不可变（不可修改，修改时会抛出 UnsupportedOperationException 异常）集合对象。所以，可以利用这套方法定义集合静态常量。

```
/** 例子工具类 */
public final class ExampleHelper {
    /** 常量值列表 */
    public static final List<Integer> CONST_VALUE_LIST =
Collections.unmodifiableList(Arrays.asList(1, 2, 3));
    /** 常量值集合 */
    public static final Set<Integer> CONST_VALUE_SET = Collections.unmodifiableSet(new
HashSet<>(Arrays.asList(1, 2, 3)));
    /** 常量值映射 */
    public static final Map<Integer, String> CONST_VALUE_MAP;
    static {
        Map<Integer, String> valueMap = new HashMap<>(MapHelper.DEFAULT);
        valueMap.put(1, "value1");
        valueMap.put(2, "value2");
        valueMap.put(3, "value3");
        CONST_VALUE_MAP = Collections.unmodifiableMap(valueMap);
    }
    ...
}
```

六、 定义数组常量

上一章介绍了如何定义集合常量，这一章就来介绍一下如何定义数组常量。

1. 定义公有数组常量

1) 定义代码

一般人定义数组常量，就会像下面代码一样，定义一个公有数组常量。

```
/** 例子工具类 */
public final class ExampleHelper {
    /** 常量值数组 */
    public static final int[] CONST_VALUES = new int[] {1, 2, 3};
    ...
}
```

2) 使用代码

使用也很方便，直接通过“类名.常量名”使用。

```
// 使用常量值数组
int[] constValues = ExampleHelper.CONST_VALUES;
```

3) 存在问题

但是，可以通过下标修改数组值，导致数组常量的值可变。所以，这种方法定义的数组常量，并不是一个真正意义上的数组常量。

```
// 修改常量值数组
ExampleHelper.CONST_VALUES[1] = 20; // [1, 20, 3]
```

2. 定义公有集合常量

1) 定义代码

可以通过上一章定义集合常量的方法，返回一个公有集合常量。

```
/** 例子工具类 */
public final class ExampleHelper {
    /** 常量值列表 */
    public static final List<Integer> CONST_VALUE_LIST =
        Collections.unmodifiableList(Arrays.asList(1, 2, 3));
    ...
}
```

2) 使用代码

要想得到数组常量，就把集合常量转化为数组常量。

```
// 使用常量值列表
int[] constValues = ExampleHelper.CONST_VALUE_LIST.stream()
    .mapToInt(Integer::intValue).toArray();
```

3) 存在问题

每一次都会把集合常量转化为数组常量，导致程序运行效率降低。

3. 最佳定义方式

最佳法“私有数组常量+公有克隆方法”的解决方案。如下代码所示：先定义一个私有数组常量，保证不会被外部类使用；在定义一个获取数组常量方法，并返回一个数组常量的克隆值。

1) 定义代码

这里，提供一个“私有数组常量+公有克隆方法”的解决方案。如下代码所示：先定义一个私有数组常量，保证不会被外部类使用；在定义一个获取数组常量方法，并返回一个数组常量的克隆值。

```
/** 例子工具类 */
public final class ExampleHelper {
    /** 常量值数组 */
    private static final int[] CONST_VALUES = new int[] {1, 2, 3};
    /** 获取常量值数组方法 */
    public static int[] getConstValues() {
        return CONST_VALUES.clone();
    }
    ...
}
```

2) 使用代码

由于每次返回的是一个克隆数组，即便修改了克隆数组的常量值，也不会导致原始数组常量值的修改。

```
// 使用常量值方法
int[] constValues = ExampleHelper.getConstValues(); // [1, 2, 3]
constValues[1] = 20; // [1, 20, 3]
constValues = ExampleHelper.getConstValues(); // [1, 2, 3]
```

七、 定义多条件表达式

1. 利用运算符&&（或||）直接拼接

1) 定义代码

有时候，我们会判断很多条件，需求用&&（或||）连接多个条件表达式。

```
/** 获取审核结果方法 */
private static Integer getAuditResult(AuditDataVO data) {
  if (isPassed(data.getAuditItem1())
    && isPassed(data.getAuditItem2())
        ...
    && isPassed(data.getAuditItem11())) {
    return AuditResult.PASSED;
  }
  return AuditResult.REJECTED;
}
```

2) 存在问题

通过 SonarLint 插件扫描，会存在 2 个问题：

Rule key	Rule name	Description
java:S1067	Expressions should not be too complex（表达式不能太复杂）	Reduce the number of conditional operators (11) used in the expression (maximum allowed 3).（减少表达式中使用的条件运算符（11个）的数量（最多允许3个）。）
java:S1541	Methods should not be too complex（方法不能太复杂）	The Cyclomatic Complexity of this method "getAuditResult" is 13 which is greater than 10 authorized.（方法"getAuditResult"的圈复杂度为13，大于10。）

其中，圈复杂度（Cyclomatic complexity，CC）也称为条件复杂度，是一种衡量代码复杂度的标准，其符号为 V（G）。

麦凯布最早提出一种称为“基础路径测试”（Basis Path Testing）的软件测试方式，测试程序中的每一线性独立路径，所需的测试用例个数即为程序的圈复杂度。

圈复杂度可以用来衡量一个模块判定结构的复杂程度，其数量上表现为独立路径的条数，也可理解为覆盖所有的可能情况最少使用的测试用例个数。

2. 利用运算符=和&&（或||）级联拼接

1) 定义代码

那么，就把&&（或||）连接符拆开，利用运算符=和&&（或||）级联进行拼接。

```
/** 获取审核结果方法 */
private static AuditResult getAuditResult(AuditDataVO data) {
  boolean isPassed = isPassed(data.getAuditItem1());
  isPassed = isPassed && isPassed(data.getAuditItem2());
    ...
  isPassed = isPassed && isPassed(data.getAuditItem11());
  if (isPassed) {
    return AuditResult.PASSED;
  }
  return AuditResult.REJECTED;
}
```

2) 存在问题

通过 SonarLint 插件扫描，还存在 1 个问题：

Rule key	Rule name	Description
java:S1541	Methods should not be too complex（方法不能太复杂）	The Cyclomatic Complexity of this method "getAuditResult" is 13 which is greater than 10 authorized.（方法“getAuditResult”的圈复杂度为13，大于10。）

也就是，利用运算符=和&&（或||）级联进行拼接，并不能减少方法的圈复杂度。

3. 利用动态无参数 Lambda 表达式列表

1) 定义代码

下面，利用动态无参数 Lambda 表达式列表优化，即把每个条件表达式作为 BooleanSupplier 对象存在列表中，然后依次执行条件表达式得出最后结果。

```
/** 获取审核结果方法 */
private static AuditResult getAuditResult(AuditDataVO data) {
  List<BooleanSupplier> supplierList = new ArrayList<>();
  supplierList.add(() -> isPassed(data.getAuditItem1()));
  supplierList.add(() -> isPassed(data.getAuditItem2()));
  ...
  supplierList.add(() -> isPassed(data.getAuditItem11()));
  for (BooleanSupplier supplier : supplierList) {
    if (!supplier.getAsBoolean()) {
      return AuditResult.REJECTED;
    }
  }
  return AuditResult.PASSED;
}
```

2) 存在问题

通过 SonarLint 插件扫描，没有提示任何问题。但是，每次都动态添加 Lambda 表达式，就会导致程序效率低下。那么，有没有把 Lambda 表达式静态化的方法呢？

4. 利用静态有参数 Lambda 表达式列表

1) 定义代码

要想固化 Lambda 表达式，就必须动态传入 AuditDataVO 对象。这里，采用“Predicate<AuditDataVO>”来接收 Lambda 表达式，在 Lambda 表达式中指定 AuditDataVO 对象 data。然后，在 for 循环中，依次指定 AuditDataVO 对象 data，并计算表达式的值。

```
/** 审核结果断言列表 */
private static final List<Predicate<AuditDataVO>> AUDIT_RESULT_PREDICATE_LIST =
  Collections.unmodifiableList(Arrays.asList(
    data -> isPassed(data.getAuditItem1()),
    data -> isPassed(data.getAuditItem2()),
    ...
    data -> isPassed(data.getAuditItem11())));

/** 获取审核结果方法 */
private static AuditResult getAuditResult(AuditDataVO data) {
  for (Predicate<AuditDataVO> predicate : AUDIT_RESULT_PREDICATE_LIST) {
    if (!predicate.test(data)) {
      return AuditResult.REJECTED;
    }
  }
  return AuditResult.PASSED;
}
```

2) 适用条件

- 适合于&&（或||）连接大量条件表达式的情况；
- 适合于每个条件表达式都需要传入相同参数的情况，如果每个条件表达式传入参数不同，只能使用动态无参数 Lambda 表达式列表方法；
- 如果需要传入两个参数，可以使用 BiPredicate 类型来接收 Lambda 表达式；如果需要传入多个参数，则需要自定义方法接口。

后记

高德从上一个财年开始，就狠抓代码质量（包括代码审查、代码规范、单元测试等），已经取得了显著成效。为了助力高德代码质量的提高，我系统性地梳理了服务端单元测试编写流程，总结出文章《Java 编程技巧之单元测试用例编写流程》。我写作该文章的宗旨就是——“读有所得、得有所思、思有所获”。所以，在阅读该文章后，只需依样画葫芦，就能轻而易举地编写出专家级的单元测试用例。

➢ Java 单元测试系列

Java 单元测试技巧之 PowerMock

前言

高德的技术大佬向老师在谈论方法论时说到：“复杂的问题要简单化，简单的问题要深入化。”

这句话让我感触颇深，这何尝不是一套编写代码的方法——把一个复杂逻辑拆分为许多简单逻辑，然后把每一个简单逻辑进行深入实现，最后把这些简单逻辑整合为复杂逻辑，总结为八字真言即是“化繁为简，由简入繁”。

编写 Java 单元测试用例，其实就是把“复杂的问题要简单化”——即把一段复杂的代码拆解成一系列简单的单元测试用例；写好 Java 单元测试用例，其实就是把“简单的问题要深入化”——即学习一套方法、总结一套模式并应用到实践中。这里，作者根据日常的工作经验，总结了一些 Java 单元测试技巧，以供大家交流和学习。

一、 准备环境

PowerMock 是一个扩展了其它如 EasyMock 等 mock 框架的、功能更加强大的框架。PowerMock 使用一个自定义类加载器和字节码操作来模拟静态方法、构造方法、final 类和方法、私有方法、去除静态初始化器等等。

1. 引入 PowerMock 包

为了引入 PowerMock 包，需要在 pom.xml 文件中加入下列 maven 依赖：

```
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-module-junit4</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-api-mockito2</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>
```

2. 集成 SpringMVC 项目

在 SpringMVC 项目中，需要在 pom.xml 文件中加入 JUnit 的 maven 依赖：

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
```

3. 集成 SpringBoot 项目

在 SpringBoot 项目中，需要在 pom.xml 文件中加入 JUnit 的 maven 依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

4. 一个简单的测试用例

这里，用 List 举例，模拟一个不存在的列表，但是返回的列表大小为 100。

```
public class ListTest {
    @Test
    public void testSize() {
        Integer expected = 100;
        List list = PowerMockito.mock(List.class);
        PowerMockito.when(list.size()).thenReturn(expected);
        Integer actual = list.size();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

二、 mock 语句

1. mock 方法

声明

- T PowerMockito.mock(Class<T> clazz);

用途

- 可以用于模拟指定类的对象实例。

当模拟非 final 类(接口、普通类、虚基类)的非 final 方法时,不必使用“@RunWith”和“@PrepareForTest”注解。当模拟 final 类或 final 方法时,必须使用“@RunWith”和“@PrepareForTest”注解。注解形如:

- @RunWith(PowerMockRunner.class);
- @PrepareForTest({TargetClass.class});

1) 模拟非 final 类普通方法

```
@Getter
@Setter
@ToString
public class Rectangle implements Sharp {
    private double width;
    private double height;
    @Override
    public double getArea() {
        return width * height;
    }
}

public class RectangleTest {
    @Test
    public void testGetArea() {
        double expectArea = 100.0D;
        Rectangle rectangle = PowerMockito.mock(Rectangle.class);
        PowerMockito.when(rectangle.getArea()).thenReturn(expectArea);
        double actualArea = rectangle.getArea();
        Assert.assertEquals("返回值不相等", expectArea, actualArea, 1E-6D);
    }
}
```

2) 模拟 final 类或 final 方法

```
@Getter
@Setter
@ToString
public final class Circle {
    private double radius;
    public double getArea() {
        return Math.PI * Math.pow(radius, 2);
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({Circle.class})
public class CircleTest {
    @Test
    public void testGetArea() {
        double expectArea = 3.14D;
        Circle circle = PowerMockito.mock(Circle.class);
        PowerMockito.when(circle.getArea()).thenReturn(expectArea);
        double actualArea = circle.getArea();
        Assert.assertEquals("返回值不相等", expectArea, actualArea, 1E-6D);
    }
}
```

2. mockStatic 方法

声明

- PowerMockito.mockStatic(Class<T> clazz);

用途

- 可以用于模拟类的静态方法，必须使用“@RunWith”和“@PrepareForTest”注解。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testIsEmpty() {
        String string = "abc";
        boolean expected = true;
        PowerMockito.mockStatic(StringUtils.class);
        PowerMockito.when(StringUtils.isEmpty(string)).thenReturn(expected);
        boolean actual = StringUtils.isEmpty(string);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

三、 spy 语句

如果一个对象，我们只希望模拟它的部分方法，而希望其它方法跟原来一样，可以使用“PowerMockito.spy”方法代替“PowerMockito.mock”方法。于是，通过 when 语句设置过的方法，调用的是模拟方法；而没有通过 when 语句设置的方法，调用的是原有方法。

1. spy 类

声明

- PowerMockito.spy(Class<T> clazz);

用途

- 用于模拟类的部分方法。

案例

```
public class StringUtils {
    public static boolean isNotEmpty(final CharSequence cs) {
        return !isEmpty(cs);
    }
    public static boolean isEmpty(final CharSequence cs) {
        return cs == null || cs.length() == 0;
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testIsNotEmpty() {
        String string = null;
        boolean expected = true;
        PowerMockito.spy(StringUtils.class);
        PowerMockito.when(StringUtils.isEmpty(string)).thenReturn(!expected);
        boolean actual = StringUtils.isNotEmpty(string);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2. spy 对象

声明

- T PowerMockito.spy(T object);

用途

- 用于模拟对象的部分方法。

案例

```
public class UserService {
    private Long superUserId;
    public boolean isNotSuperUser(Long userId) {
        return !isSuperUser(userId);
    }
    public boolean isSuperUser(Long userId) {
        return Objects.equals(userId, superUserId);
    }
}

@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Test
    public void testIsNotSuperUser() {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService.isSuperUser(userId)).thenReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

四、 when 语句

1. when().thenReturn()模式

声明

- PowerMockito.when(mockObject.someMethod(someArgs)).thenReturn(expectedValue);
- PowerMockito.when(mockObject.someMethod(someArgs)).thenThrow(expectedThrowable);

- PowerMockito.when(mockObject.someMethod(someArgs)).thenAnswer(expectedAnswer);
- PowerMockito.when(mockObject.someMethod(someArgs)).thenCallRealMethod();

用途

- 用于模拟对象方法，先执行原始方法，再返回期望的值、异常、应答，或调用真实的方法。

返回期望值：

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(index)).thenReturn(expected);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

返回期望异常：

```
public class ListTest {
    @Test(expected = IndexOutOfBoundsException.class)
    public void testGet() {
        int index = -1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(index)).thenThrow(new IndexOutOfBoundsException());
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

返回期望应答：

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(index)).thenAnswer(invocation -> {
            Integer value = invocation.getArgument(0);
            return value * 100;
        });
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

调用真实方法：

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> oldList = new ArrayList<>();
        oldList.add(expected);
        List<Integer> spylist = PowerMockito.spy(oldList);
        PowerMockito.when(spylist.get(index)).thenCallRealMethod();
        Integer actual = spylist.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2. doReturn().when()模式

声明

- PowerMockito.doReturn(expectedValue).when(mockObject).someMethod(someArgs);
- PowerMockito.doThrow(expectedThrowable).when(mockObject).someMethod(someArgs);
- PowerMockito.doAnswer(expectedAnswer).when(mockObject).someMethod(someArgs);
- PowerMockito.doNothing().when(mockObject).someMethod(someArgs);
- PowerMockito.doCallRealMethod().when(mockObject).someMethod(someArgs);

用途

- 用于模拟对象方法，直接返回期望的值、异常、应答，或调用真实的方法，无需执行原始方法。

注意：

千万不要使用以下语法：

- PowerMockito.doReturn(expectedValue).when(mockObject.someMethod(someArgs));
- PowerMockito.doThrow(expectedThrowable).when(mockObject.someMethod(someArgs));
- PowerMockito.doAnswer(expectedAnswer).when(mockObject.someMethod(someArgs));
- PowerMockito.doNothing().when(mockObject.someMethod(someArgs));
- PowerMockito.doCallRealMethod().when(mockObject.someMethod(someArgs));

虽然不会出现编译错误，但是在执行时会抛出 UnfinishedStubbingException 异常。

返回期望值：

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doReturn(expected).when(mockList).get(index);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

返回期望异常：

```
public class ListTest {
    @Test(expected = IndexOutOfBoundsException.class)
    public void testGet() {
        int index = -1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doThrow(new IndexOutOfBoundsException()).when(mockList).get(index);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

返回期望应答：

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doAnswer(invocation -> {
            Integer value = invocation.getArgument(0);
            return value * 100;
        }).when(mockList).get(index);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

模拟无返回值：

```
public class ListTest {
    @Test
    public void testClear() {
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doNothing().when(mockList).clear();
        mockList.clear();
        Mockito.verify(mockList).clear();
    }
}
```

调用真实方法：

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> oldList = new ArrayList<>();
        oldList.add(expected);
        List<Integer> spylist = PowerMockito.spy(oldList);
        PowerMockito.doCallRealMethod().when(spylist).get(index);
        Integer actual = spylist.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3. 两种模式的主要区别

两种模式都用于模拟对象方法，在 mock 实例下使用时，基本上是没有差别的。但是，在 spy 实例下使用时，when().thenReturn()模式会执行原方法，而 doReturn().when()模式不会执行原方法。

1) 测试服务类

```
@Slf4j
@Service
public class UserService {
    public long getUserCount() {
        log.info("调用获取用户数量方法");
        return 0L;
    }
}
```

2) 使用 when().thenReturn()模式

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Test
    public void testGetUserCount() {
        Long expected = 1000L;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService.getUserCount()).thenReturn(expected);
        Long actual = userService.getUserCount();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

在测试过程中，将会打印出“调用获取用户数量方法”日志。

3) 使用 doReturn().when()模式

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Test
    public void testGetUserCount() {
        Long expected = 1000L;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.doReturn(expected).when(userService).getUserCount();
        Long actual = userService.getUserCount();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

在测试过程中，不会打印出“调用获取用户数量方法”日志。

4. whenNew 模拟构造方法

声明

- PowerMockito.whenNew(MockClass.class).withNoArguments().thenReturn(expectedObject);
- PowerMockito.whenNew(MockClass.class).withArguments(someArgs).thenReturn(expectedObject);

用途

- 用于模拟构造方法。

案例

```
public final class FileUtils {
    public static boolean isFile(String fileName) {
        return new File(fileName).isFile();
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({FileUtils.class})
public class FileUtilsTest {
    @Test
    public void testIsFile() throws Exception {
        String fileName = "test.txt";
        File file = PowerMockito.mock(File.class);
        PowerMockito.whenNew(File.class).withArguments(fileName).thenReturn(file);
        PowerMockito.when(file.isFile()).thenReturn(true);
        Assert.assertTrue("返回值为假", FileUtils.isFile(fileName));
    }
}
```

注意：

需要加上注解“@PrepareForTest({FileUtils.class})”，否则模拟方法不生效。

五、 参数匹配器

在执行单元测试时，有时候并不关心传入的参数的值，可以使用参数匹配器。

1. 参数匹配器（any）

Mockito 提供“Mockito.anyInt()”、“Mockito.anyString”、“Mockito.any(Class<T> clazz)”等来表示任意值。

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(Mockito.anyInt())).thenReturn(expected);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2. 参数匹配器（eq）

当我们使用参数匹配器时，所有参数都应使用匹配器。如果要为某一参数指定特定值时，就需要使用“Mockito.eq()”方法。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testStartWith() {
        String string = "abc";
        String prefix = "b";
        boolean expected = true;
        PowerMockito.spy(StringUtils.class);
        PowerMockito.when(StringUtils.startsWith(Mockito.anyString(),
Mockito.eq(prefix))).thenReturn(expected);
        boolean actual = StringUtils.startsWith(string, prefix);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3. 附加匹配器

Mockito 的 AdditionalMatchers 类提供了一些很少使用的参数匹配器，我们可以进行参数大于（gt）、小于（lt）、大于等于（geq）、小于等于（leq）等比较操作，也可以进行参数与（and）、或（or）、非（not）等逻辑计算等。

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(AdditionalMatchers.geq(0))).thenReturn(expected);
        PowerMockito.when(mockList.get(AdditionalMatchers.lt(0))).thenThrow(new
IndexOutOfBoundsException());
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

六、 verify 语句

验证是确认在模拟过程中，被测试方法是否已按预期方式与其任何依赖方法进行了交互。

格式

- Mockito.verify(mockObject[,times(int)]).someMethod(somgArgs);

用途

- 用于模拟对象方法，直接返回期望的值、异常、应答，或调用真实的方法，无需执行原始方法。

案例

1. 验证调用方法

```
public class ListTest {
    @Test
    public void testGet() {
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doNothing().when(mockList).clear();
        mockList.clear();
        Mockito.verify(mockList).clear();
    }
}
```

2. 验证调用次数

```
public class ListTest {
    @Test
    public void testGet() {
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doNothing().when(mockList).clear();
        mockList.clear();
        Mockito.verify(mockList, Mockito.times(1)).clear();
    }
}
```

除 times 外，Mockito 还支持 atLeastOnce、atLeast、only、atMostOnce、atMost 等次数验证器。

3. 验证调用顺序

```
public class ListTest {
    @Test
    public void testAdd() {
        List<Integer> mockedList = PowerMockito.mock(List.class);
        PowerMockito.doReturn(true).when(mockedList).add(Mockito.anyInt());
        mockedList.add(1);
        mockedList.add(2);
        mockedList.add(3);
        InOrder inOrder = Mockito.inOrder(mockedList);
        inOrder.verify(mockedList).add(1);
        inOrder.verify(mockedList).add(2);
        inOrder.verify(mockedList).add(3);
    }
}
```

4. 验证调用参数

```
public class ListTest {
    @Test
    public void testArgumentCaptor() {
        Integer[] expecteds = new Integer[] {1, 2, 3};
        List<Integer> mockedList = PowerMockito.mock(List.class);
        PowerMockito.doReturn(true).when(mockedList).add(Mockito.anyInt());
        for (Integer expected : expecteds) {
            mockedList.add(expected);
        }
        ArgumentCaptor<Integer> argumentCaptor = ArgumentCaptor.forClass(Integer.class);
        Mockito.verify(mockedList, Mockito.times(3)).add(argumentCaptor.capture());
        Integer[] actuals = argumentCaptor.getAllValues().toArray(new Integer[0]);
        Assert.assertArrayEquals("返回值不相等", expecteds, actuals);
    }
}
```

5. 确保验证完毕

Mockito 提供“Mockito.verifyNoMoreInteractions”方法，在所有验证方法之后可以使用此方法，以确保所有调用都得到验证。如果模拟对象上存在任何未验证的调用，将会抛出“NoInteractionsWanted”异常。

```
public class ListTest {
    @Test
    public void testVerifyNoMoreInteractions() {
        List<Integer> mockedList = PowerMockito.mock(List.class);
        Mockito.verifyNoMoreInteractions(mockedList); // 执行正常
        mockedList.isEmpty();
        Mockito.verifyNoMoreInteractions(mockedList); // 抛出异常
    }
}
```

备注

“Mockito.verifyZeroInteractions”方法与“Mockito.verifyNoMoreInteractions”方法相同，但是目前已经被废弃。

6. 验证静态方法

Mockito 没有静态方法的验证方法，但是 PowerMock 提供这方面的支持。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testVerifyStatic() {
        PowerMockito.mockStatic(StringUtils.class);
        String expected = "abc";
        StringUtils.isEmpty(expected);
        PowerMockito.verifyStatic(StringUtils.class);
        ArgumentCaptor<String> argumentCaptor = ArgumentCaptor.forClass(String.class);
        StringUtils.isEmpty(argumentCaptor.capture());
        Assert.assertEquals("参数不相等", argumentCaptor.getValue(), expected);
    }
}
```

七、 私有属性

1. ReflectionTestUtils.setField 方法

在用原生 JUnit 进行单元测试时，我们一般采用“ReflectionTestUtils.setField`”方法设置私有属性值。

```
@Service
public class UserService {
    @Value("${system.userLimit}")
    private Long userLimit;
    public Long getUserLimit() {
        return userLimit;
    }
}

public class UserServiceTest {
    @Autowired
    private UserService userService;
    @Test
    public void testGetUserLimit() {
        Long expected = 1000L;
        ReflectionTestUtils.setField(userService, "userLimit", expected);
        Long actual = userService.getUserLimit();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

注意：

在测试类中，UserService 实例是通过@Autowired 注解加载的，如果该实例已经被动态代理，“ReflectionTestUtils.setField”方法设置的是代理实例，从而导致设置不生效。

2. Whitebox.setInternalState 方法

现在使用 PowerMock 进行单元测试时，可以采用“Whitebox.setInternalState”方法设置私有属性值。

```
@Service
public class UserService {
    @Value("${system.userLimit}")
    private Long userLimit;
    public Long getUserLimit() {
        return userLimit;
    }
}

@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @InjectMocks
    private UserService userService;
    @Test
    public void testGetUserLimit() {
        Long expected = 1000L;
        Whitebox.setInternalState(userService, "userLimit", expected);
        Long actual = userService.getUserLimit();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

注意：

需要加上注解“@RunWith(PowerMockRunner.class)”。

八、 私有方法

1. 模拟私有方法

1) 通过 when 实现

```
public class UserService {
    private Long superUserId;
    public boolean isNotSuperUser(Long userId) {
        return !isSuperUser(userId);
    }
    private boolean isSuperUser(Long userId) {
        return Objects.equals(userId, superUserId);
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({UserService.class})
public class UserServiceTest {
    @Test
    public void testIsNotSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService, "isSuperUser", userId).thenReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2) 通过 stub 实现

通过模拟方法 stub（存根），也可以实现模拟私有方法。但是，只能模拟整个方法的返回值，而不能模拟指定参数的返回值。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({UserService.class})
public class UserServiceTest {
    @Test
    public void testIsNotSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.stub(PowerMockito.method(UserService.class, "isSuperUser",
Long.class)).toReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        Assert.assertEquals("返回值不相等", expected, actual;
    }
}
```

2. 测试私有方法

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest9 {
    @Test
    public void testIsSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = new UserService();
        Method method = PowerMockito.method(UserService.class, "isSuperUser", Long.class);
        Object actual = method.invoke(userService, userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3. 验证私有方法

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({UserService.class})
public class UserServiceTest10 {
    @Test
    public void testIsNotSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService, "isSuperUser", userId).thenReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        PowerMockito.verifyPrivate(userService).invoke("isSuperUser", userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

这里，也可以用 Method 那套方法进行模拟和验证方法。

九、 主要注解

PowerMock 为了更好地支持 SpringMVC/SpringBoot 项目，提供了一系列的注解，大大地简化了测试代码。

1. @RunWith 注解

- @RunWith(PowerMockRunner.class)
- 指定 JUnit 使用 PowerMock 框架中的单元测试运行器。

2. @PrepareForTest 注解

- @PrepareForTest({ TargetClass.class })
- 当需要模拟 final 类、final 方法或静态方法时，需要添加@PrepareForTest 注解，并指定方法所在的类。如果需要指定多个类，在{}中添加多个类并用逗号隔开即可。

3. @Mock 注解

- @Mock 注解创建了一个全部 Mock 的实例，所有属性和方法全被置空（0 或者 null）。

4. @Spy 注解

- @Spy 注解创建了一个没有 Mock 的实例，所有成员方法都会按照原方法的逻辑执行，直到被 Mock 返回某个具体的值为止。

注意：

@Spy 注解的变量需要被初始化，否则执行时会抛出异常。

5. @InjectMocks 注解

@InjectMocks 注解创建一个实例，这个实例可以调用真实代码的方法，其余用@Mock 或@Spy 注解创建的实例将被注入到用该实例中。

```
@Service
public class UserService {
    @Autowired
    private UserDAO userDAO;
    public void modifyUser(UserVO userVO) {
        UserDO userDO = new UserDO();
        BeanUtils.copyProperties(userVO, userDO);
        userDAO.modify(userDO);
    }
}

@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    @InjectMocks
    private UserService userService;
    @Test
    public void testCreateUser() {
        UserVO userVO = new UserVO();
        userVO.setId(1L);
        userVO.setName("changyi");
        userVO.setDesc("test user");
        userService.modifyUser(userVO);
        ArgumentCaptor<UserDO> argumentCaptor = ArgumentCaptor.forClass(UserDO.class);
        Mockito.verify(userDAO).modify(argumentCaptor.capture());
        UserDO userDO = argumentCaptor.getValue();
        Assert.assertNotNull("用户实例为空", userDO);
        Assert.assertEquals("用户标识不相等", userVO.getId(), userDO.getId());
        Assert.assertEquals("用户名称不相等", userVO.getName(), userDO.getName());
        Assert.assertEquals("用户描述不相等", userVO.getDesc(), userDO.getDesc());
    }
}
```

6. @Captor 注解

@Captor 注解在字段级别创建参数捕获器。但是，在测试方法启动前，必须调用“MockitoAnnotations.openMocks(this)”进行初始化。

```
@Service
public class UserService {
    @Autowired
    private UserDAO userDAO;
    public void modifyUser(UserVO userVO) {
        UserDO userDO = new UserDO();
        BeanUtils.copyProperties(userVO, userDO);
        userDAO.modify(userDO);
    }
}

@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    @InjectMocks
    private UserService userService;
    @Captor
    private ArgumentCaptor<UserDO> argumentCaptor;
    @Before
    public void beforeTest() {
        MockitoAnnotations.openMocks(this);
    }
    @Test
    public void testCreateUser() {
        UserVO userVO = new UserVO();
        userVO.setId(1L);
        userVO.setName("changyi");
        userVO.setDesc("test user");
        userService.modifyUser(userVO);
        Mockito.verify(userDAO).modify(argumentCaptor.capture());
        UserDO userDO = argumentCaptor.getValue();
        Assert.assertNotNull("用户实例为空", userDO);
        Assert.assertEquals("用户标识不相等", userVO.getId(), userDO.getId());
        Assert.assertEquals("用户名称不相等", userVO.getName(), userDO.getName());
        Assert.assertEquals("用户描述不相等", userVO.getDesc(), userDO.getDesc());
    }
}
```

7. @PowerMockIgnore 注解

为了解决使用 PowerMock 后，提示 ClassLoader 错误。

十、 相关观点

1. 《Java 开发手册》规范

1) 好的单元测试必须遵守 AIR 原则

单元测试在线上运行时，感觉像空气（AIR）一样感觉不到，但在测试质量的保障上，却是非常关键的。好的单元测试宏观上来说，具有自动化、独立性、可重复执行的特点。

- A：Automatic（自动化）
- I：Independent（独立性）
- R：Repeatable（可重复）

2) 单元测试应该是全自动执行的，并且非交互式的。

测试用例通常是被定期执行的，执行过程必须完全自动化才有意义。输出结果需要人工检查的测试不是一个好的单元测试。单元测试中不准使用 System.out 来进行人肉验证，必须使用 assert 来验证。

3) 单元测试是可以重复执行的，不能受到外界环境的影响。

单元测试通常会被放到持续集成中，每次有代码 check in 时单元测试都会被执行。如果单测对外部环境（网络、服务、中间件等）有依赖，容易导致持续集成机制的不可用。

正例：

为了不受外界环境影响，要求设计代码时就把 SUT 的依赖改成注入，在测试时用 spring 这样的 DI 框架注入一个本地（内存）实现或者 Mock 实现。

4) 编写单元测试代码遵守 BCDE 原则，以保证被测试模块的交付质量。

- B：Border，边界值测试，包括循环边界、特殊取值、特殊时间点、数据顺序等。
- C：Correct，正确的输入，并得到预期的结果。
- D：Design，与设计文档相结合，来编写单元测试。
- E：Error，强制错误信息输入（如：非法数据、异常流程、业务允许外等），并得到预期的结果。

2. 为什么要使用 Mock?

根据网络相关资料，总结观点如下：

1) Mock 可以用来解除外部服务依赖，从而保证了测试用例的独立性。

现在的互联网软件系统，通常采用了分布式部署的微服务，为了单元测试某一服务而准备其它服务，存在极大的依耐性和不可行性。

2) Mock 可以减少全链路测试数据准备，从而提高了编写测试用例的速度。

传统的集成测试，需要准备全链路的测试数据，可能某些环节并不是你所熟悉的。最后，耗费了大量的时间和经历，并不一定得到你想要的结果。现在的单元测试，只需要模拟上游的输入数据，并验证给下游的输出数据，编写测试用例并进行测试的速度可以提高很多倍。

3) Mock 可以模拟一些非正常的流程，从而保证了测试用例的代码覆盖率。

根据单元测试的 BCDE 原则，需要进行边界值测试（Border）和强制错误信息输入（Error），这样有助于覆盖整个代码逻辑。在实际系统中，很难去构造这些边界值，也能难去触发这些错误信息。而 Mock 从根本上解决了这个问题：想要什么样的边界值，只需要进行 Mock；想要什么样的错误信息，也只需要进行 Mock。

4) Mock 可以不用加载项目环境配置，从而保证了测试用例的执行速度。

在进行集成测试时，我们需要加载项目的所有环境配置，启动项目依赖的所有服务接口。往往执行一个测试用例，需要几分钟乃至几十分钟。采用 Mock 实现的测试用例，不用加载项目环境配置，也不依赖其它服务接口，执行速度往往在几秒之内，大大地提高了单元测试的执行速度。

3. 单元测试与集成测试的区别

在实际工作中，不少同学用集成测试代替了单元测试，或者认为集成测试就是单元测试。这里，总结为了单元测试与集成测试的区别：

1) 测试对象不同

单元测试对象是实现了具体功能的程序单元，集成测试对象是概要设计规划中的模块及模块间的组合。

2) 测试方法不同

单元测试中的主要方法是基于代码的白盒测试，集成测试中主要使用基于功能的黑盒测试。

3) 测试时间不同

集成测试要晚于单元测试。

4) 测试内容不同

单元测试主要是模块内程序的逻辑、功能、参数传递、变量引用、出错处理及需求和设计中具体要求方面的测试；而集成测试主要验证各个接口、接口之间的数据传递关系，及模块组合后能否达到预期效果。

后记

《西江月·百途》

代码编辑已毕，谁能担保无遗。
单元测试解难题，查验流程合理。
质量提升效益，百途扛起旌旗。
俊杰志士聚一席，共创高德佳绩。

——谨以此词献给高德的《百途度量平台》。

Java 单元测试技巧之编写流程

前言

清代杰出思想家章学诚有一句名言：“学必求其心得，业必贵其专精。”

意思是：学习上一定要追求心得体会，事业上一定要贵以专注精深。做技术就是这样，一件事如果做到了极致，就必然会有所心得体会。作者最近在一个项目上，追求单元测试覆盖率到极致，所以才有了这篇心得体会。

上一篇文章《Java 单元测试技巧之 PowerMock》除了介绍单元测试基础知识外，主要介绍了“为什么要编写单元测试”。很多同学读完后，还是不能快速地编写单元测试用例。而这篇文章，立足于“如何来编写单元测试用例”，能够让同学们“有章可循”，能快速地编写出单元测试用例。

一、 编写单元测试用例

1. 测试框架简介

Mockito 是一个单元测试模拟框架，可以让你写出优雅、简洁的单元测试代码。Mockito 采用了模拟技术，模拟了一些在应用中依赖的复杂对象，从而把测试对象和依赖对象隔离开来。

PowerMock 是一个单元测试模拟框架，是在其它单元测试模拟框架的基础上做出扩展。通过提供定制的类加载器以及一些字节码篡改技术的应用，PowerMock 实现了对静态方法、构造方法、私有方法以及 final 方法的模拟支持等强大的功能。但是，正因为 PowerMock 进行了字节码篡改，导致部分单元测试用例并不被 JaCoco 统计覆盖率。

通过作者多年单元测试的编写经验，优先推荐使用 Mockito 提供的功能；只有在 Mockito 提供的功能不能满足需求时，才会采用 PowerMock 提供的功能；但是，不推荐使用影响 JaCoco 统计覆盖率的 PowerMock 功能。在本文中，我们也不会对影响 JaCoco 统计覆盖率的 PowerMock 功能进行介绍。

下面，将以 Mockito 为主、以 PowerMock 为辅，介绍一下如何编写单元测试用例。

2. 测试框架引入

为了引入 Mockito 和 PowerMock 包，需要在 maven 项目的 pom.xml 文件中加入以下包依赖：

```
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-core</artifactId>
    <version>${powermock.version}</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-api-mockito2</artifactId>
    <version>${powermock.version}</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-module-junit4</artifactId>
    <version>${powermock.version}</version>
    <scope>test</scope>
</dependency>
```

其中，powermock.version 为 2.0.9，为当前的最新版本，可根据实际情况修改。在 PowerMock 包中，已经包含了对应的 Mockito 和 JUnit 包，所以无需单独引入 Mockito 和 JUnit 包。

3. 典型代码案例

一个典型的服务代码案例如下：

```
/**
 * 用户服务类
 */
@Service
public class UserService {
    /** 服务相关 */
    /** 用户 DAO */
    @Autowired
    private UserDAO userDAO;
    /** 标识生成器 */
    @Autowired
    private IdGenerator idGenerator;

    /** 参数相关 */
    /** 可以修改 */
    @Value("${userService.canModify}")
    private Boolean canModify;
```

```
    /**
     * 创建用户
     *
     * @param userCreate 用户创建
     * @return 用户标识
     */
    public Long createUser(UserVO userCreate) {
        // 获取用户标识
        Long userId = userDAO.getIdByName(userCreate.getName());

        // 根据存在处理
        // 根据存在处理：不存在则创建
        if (Objects.isNull(userId)) {
            userId = idGenerator.next();
            UserDO create = new UserDO();
            create.setId(userId);
            create.setName(userCreate.getName());
            userDAO.create(create);
        }
        // 根据存在处理：已存在可修改
        else if (Boolean.TRUE.equals(canModify)) {
            UserDO modify = new UserDO();
            modify.setId(userId);
            modify.setName(userCreate.getName());
            userDAO.modify(modify);
        }
        // 根据存在处理：已存在禁修改
        else {
            throw new UnsupportedOperationException("不支持修改");
        }

        // 返回用户标识
        return userId;
    }
}
```

4. 测试用例编写

采用 Mockito 和 PowerMock 单元测试模拟框架，编写的单元测试用例如下：

1) UserServiceTest.java

```java
/**
 * 用户服务测试类
 */
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    /** 模拟依赖对象 */
    /** 用户DAO */
    @Mock
    private UserDAO userDAO;
    /** 标识生成器 */
    @Mock
    private IdGenerator idGenerator;

    /** 定义被测对象 */
    /** 用户服务 */
    @InjectMocks
    private UserService userService;

    /**
     * 在测试之前
     */
    @Before
    public void beforeTest() {
        // 注入依赖对象
        Whitebox.setInternalState(userService, "canModify", Boolean.TRUE);
    }

    /**
     * 测试：创建用户-新
     */
    @Test
    public void testCreateUserWithNew() {
    @Test
    public void testCreateUserWithNew() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getByName
        Mockito.doReturn(null).when(userDAO).getIdByName(Mockito.anyString());
        // 模拟依赖方法：idGenerator.next
        Long userId = 1L;
        Mockito.doReturn(userId).when(idGenerator).next();

        // 调用被测方法
        String text = ResourceHelper.getResourceAsString(getClass(), "userCreateVO.json");
        UserVO userCreate = JSON.parseObject(text, UserVO.class);
        Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreate));

        // 验证依赖方法
        // 验证依赖方法：userDAO.getByName
        Mockito.verify(userDAO).getIdByName(userCreate.getName());
        // 验证依赖方法：idGenerator.next
        Mockito.verify(idGenerator).next();
        // 验证依赖方法：userDAO.create
        ArgumentCaptor<UserDO> userCreateCaptor = ArgumentCaptor.forClass(UserDO.class);
        Mockito.verify(userDAO).create(userCreateCaptor.capture());
        text = ResourceHelper.getResourceAsString(getClass(), "userCreateDO.json");
        Assert.assertEquals("用户创建不一致", text, JSON.toJSONString(userCreateCaptor.getValue()));

        // 验证依赖对象
        Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
    }

    /**
     * 测试：创建用户-旧
     */
```

```
    /**
     * 测试：创建用户-旧
     */
    @Test
    public void testCreateUserWithOld() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getByName
        Long userId = 1L;
        Mockito.doReturn(userId).when(userDAO).getIdByName(Mockito.anyString());

        // 调用被测方法
        String text = ResourceHelper.getResourceAsString(getClass(), "userCreateVO.json");
        UserVO userCreate = JSON.parseObject(text, UserVO.class);
        Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreate));

        // 验证依赖方法
        // 验证依赖方法：userDAO.getByName
        Mockito.verify(userDAO).getIdByName(userCreate.getName());
        // 验证依赖方法：userDAO.modify
        ArgumentCaptor<UserDO> userModifyCaptor = ArgumentCaptor.forClass(UserDO.class);
        Mockito.verify(userDAO).modify(userModifyCaptor.capture());
        text = ResourceHelper.getResourceAsString(getClass(), "userModifyDO.json");
        Assert.assertEquals("用户修改不一致", text, JSON.toJSONString(userModifyCaptor.getValue()));

        // 验证依赖对象
        Mockito.verifyNoInteractions(idGenerator);
        Mockito.verifyNoMoreInteractions(userDAO);
    }

    /**
     * 测试：创建用户-异常
     */
    @Test
    public void testCreateUserWithException() {
      * 测试：创建用户-异常
      */
     @Test
     public void testCreateUserWithException() {
         // 注入依赖对象
         Whitebox.setInternalState(userService, "canModify", Boolean.FALSE);

         // 模拟依赖方法
         // 模拟依赖方法：userDAO.getByName
         Long userId = 1L;
         Mockito.doReturn(userId).when(userDAO).getIdByName(Mockito.anyString());

         // 调用被测方法
         String text = ResourceHelper.getResourceAsString(getClass(), "userCreateVO.json");
         UserVO userCreate = JSON.parseObject(text, UserVO.class);
         UnsupportedOperationException exception = Assert.assertThrows("返回异常不一致",
             UnsupportedOperationException.class, () -> userService.createUser(userCreate));
         Assert.assertEquals("异常消息不一致", "不支持修改", exception.getMessage());
     }
}
```

2) userCreateVO.json

```
{"name":"test"}
```

3) userCreateDO.json

```
{"id":1,"name":"test"}
```

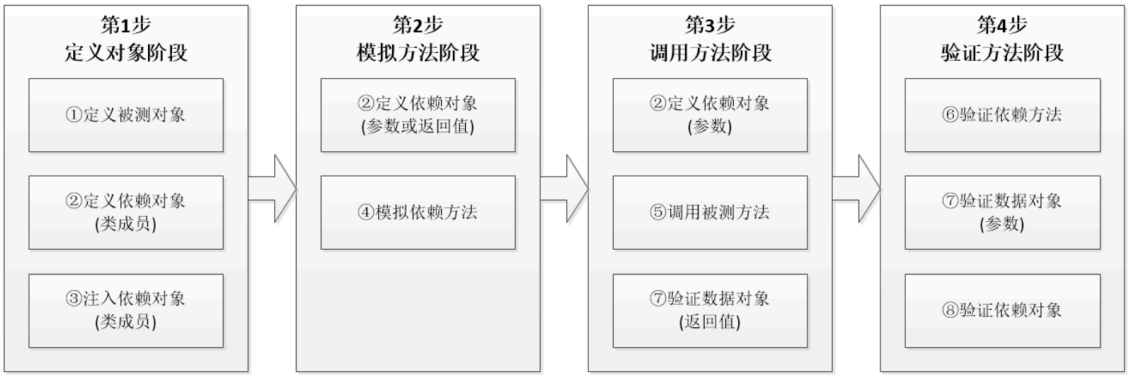
4) userModifyDO.json

```
{"id":1,"name":"test"}
```

通过执行以上测试用例，可以看到对源代码进行了 100%的行覆盖。

二、 测试用例编写流程

通过上一章编写 Java 类单元测试用例的实践，可以总结出以下 Java 类单元测试用例的编写流程：



上面一共有 3 个测试用例，这里仅以测试用例 testCreateUserWithNew（测试：创建用户-新）为例说明。

1. 定义对象阶段

第 1 步是定义对象阶段，主要包括定义被测对象、模拟依赖对象（类成员）、注入依赖对象（类成员）3 大部分。

1) 定义被测对象

在编写单元测试时，首先需要定义被测对象，或直接初始化、或通过 Spy 包装……其实，就是把被测试服务类进行实例化。

```
/** 定义被测对象 */
/** 用户服务 */
@InjectMocks
private UserService userService;
```

2) 模拟依赖对象（类成员）

在一个服务类中，我们定义了一些类成员对象——服务（Service）、数据访问对象（DAO）、参数（Value）等。在 Spring 框架中，这些类成员对象通过@Autowired、@Value 等方式注入，它们可能涉及复杂的环境配置、依赖第三方接口服务……但是，在单元测试中，为了解除对这些类成员对象的依赖，我们需要对这些类成员对象进行模拟。

```
/** 模拟依赖对象 */
/** 用户DAO */
@Mock
private UserDAO userDAO;
/** 标识生成器 */
@Mock
private IdGenerator idGenerator;
```

3) 注入依赖对象（类成员）

当模拟完这些类成员对象后，我们需要把这些类成员对象注入到被测试类的实例中。以便在调用被测试方法时，可能使用这些类成员对象，而不至于抛出空指针异常。

```
/** 定义被测对象 */
/** 用户服务 */
@InjectMocks
private UserService userService;

/**
 * 在测试之前
 */
@Before
public void beforeTest() {
  // 注入依赖对象
  Whitebox.setInternalState(userService, "canModify", Boolean.TRUE);
}
```

2. 模拟方法阶段

第 2 步是模拟方法阶段，主要包括模拟依赖对象（参数或返回值）、模拟依赖方法 2 大部分。

1) 模拟依赖对象（参数或返回值）

通常，在调用一个方法时，需要先指定方法的参数，然后获取到方法的返回值。所以，在模拟方法之前，需要先模拟该方法的参数和返回值。

```
Long userId = 1L;
```

2) 模拟依赖方法

在模拟完依赖的参数和返回值后，就可以利用 Mockito 和 PowerMock 的功能，进行依赖方法的模拟。如果依赖对象还有方法调用，还需要模拟这些依赖对象的方法。

```
// 模拟依赖方法
// 模拟依赖方法：userDAO.getByName
Mockito.doReturn(null).when(userDAO).getIdByName(Mockito.anyString());
// 模拟依赖方法：idGenerator.next
Mockito.doReturn(userId).when(idGenerator).next();
```

3. 调用方法阶段

第 3 步是调用方法阶段，主要包括模拟依赖对象（参数）、调用被测方法、验证参数对象（返回值）3 步。

1) 模拟依赖对象（参数）

在调用被测方法之前，需要模拟被测方法的参数。如果这些参数还有方法调用，还需要模拟这些参数的方法。

```
String text = ResourceHelper.getResourceAsString(getClass(), "userCreateVO.json");
UserVO userCreate = JSON.parseObject(text, UserVO.class);
```

2) 调用被测方法

在准备好参数对象后，就可以调用被测试方法了。如果被测试方法有返回值，需要定义变量接收返回值；如果被测试方法要抛出异常，需要指定期望的异常。

```
userService.createUser(userCreate)
```

3) 验证数据对象（返回值）

在调用被测试方法后，如果被测试方法有返回值，需要验证这个返回值是否符合预期；如果被测试方法要抛出异常，需要验证这个异常是否满足要求。

```
Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreate));
```

4. 验证方法阶段

第 4 步是验证方法阶段，主要包括验证依赖方法、验证数据对象（参数）、验证依赖对象 3 步。

1) 验证依赖方法

作为一个完整的测试用例，需要对每一个模拟的依赖方法调用进行验证。

```
// 验证依赖方法
// 验证依赖方法：userDAO.getByName
Mockito.verify(userDAO).getIdByName(userCreate.getName());
// 验证依赖方法：idGenerator.next
Mockito.verify(idGenerator).next();
// 验证依赖方法：userDAO.create
ArgumentCaptor<UserDO> userCreateCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).create(userCreateCaptor.capture());
```

2) 验证数据对象（参数）

对应一些模拟的依赖方法，有些参数对象是被测试方法内部生成的。为了验证代码逻辑的正确性，就需要对这些参数对象进行验证，看这些参数对象值是否符合预期。

```
text = ResourceHelper.getResourceAsString(getClass(), "userCreateDO.json");
Assert.assertEquals("用户创建不一致", text, JSON.toJSONString(userCreateCaptor.getValue()));
```

3) 验证依赖对象

作为一个完整的测试用例，应该保证每一个模拟的依赖方法调用都进行了验证。正好，Mockito 提供了一套方法，用于验证模拟对象所有方法调用都得到了验证。

```
// 验证依赖对象
Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
```

三、 定义被测对象

在编写单元测试时，首先需要定义被测对象，或直接初始化、或通过 Spy 包装……其实，就是把被测试服务类进行实例化。

1. 直接构建对象

直接构建一个对象，总是简单又直接。

```
UserService userService = new UserService();
```

2. 利用 Mockito.spy 方法

Mockito 提供一个 spy 功能，用于拦截那些尚未实现或不期望被真实调用的方法，默认所有方法都是真实方法，除非主动去模拟对应方法。所以，利用 spy 功能来定义被测对象，适合于需要模拟被测类自身方法的情况，适用于普通类、接口和虚基类。

```
UserService userService = Mockito.spy(new UserService());
UserService userService = Mockito.spy(UserService.class);
AbstractOssService ossService = Mockito.spy(AbstractOssService.class);
```

3. 利用@Spy 注解

@Spy 注解跟 Mockito.spy 方法一样，可以用来定义被测对象，适合于需要模拟被测类自身方法的情况，适用于普通类、接口和虚基类。@Spy 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Spy
    private UserService userService = new UserService();
    ...
}
```

注意：

@Spy 注解对象需要初始化。如果是虚基类或接口，可以用 Mockito.mock 方法实例化。

4. 利用@InjectMocks 注解

@InjectMocks 注解用来创建一个实例，并将其它对象（@Mock、@Spy 或直接定义的对象）注入到该实例中。所以，@InjectMocks 注解本身就可以用来定义被测对象。@InjectMocks 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @InjectMocks
    private UserService userService;
    ...
}
```

四、 模拟依赖对象

在编写单元测试用例时，需要模拟各种依赖对象——类成员、方法参数和方法返回值。

1. 直接构建对象

如果需要构建一个对象，最简单直接的方法就是——定义对象并赋值。

```
Long userId = 1L;
String userName = "admin";
UserDO user = new User();
user.setId(userId);
user.setName(userName);
List<Long> userIdList = Arrays.asList(1L, 2L, 3L);
```

2. 反序列化对象

如果对象字段或层级非常庞大，采用直接构建对象方法，可能会编写大量构建程序代码。这种情况，可以考虑反序列化对象，将会大大减少程序代码。由于 JSON 字符串可读性高，这里就以 JSON 为例，介绍反序列化对象。

1) 反序列化模型对象

```
String text = ResourceHelper.getResourceAsString(getClass(), "user.json");
UserDO user = JSON.parseObject(text, UserDO.class);
```

2) 反序列化集合对象

```
String text = ResourceHelper.getResourceAsString(getClass(), "userList.json");
List<UserDO> userList = JSON.parseArray(text, UserDO.class);
```

3) 反序列化映射对象

```
String text = ResourceHelper.getResourceAsString(getClass(), "userMap.json");
Map<Long, UserDO> userMap = JSON.parseObject(text, new TypeReference<Map<Long, UserDO>>()
{});
```

3. 利用 Mockito.mock 方法

Mockito 提供一个 mock 功能，用于拦截那些尚未实现或不期望被真实调用的方法，默认所有方法都已被模拟——方法为空并返回默认值（null 或 0），除非主动执行 doCallRealMethod 或 thenCallRealMethod 操作，才能够调用真实的方法。

利用 Mockito.mock 方法模拟依赖对象，主要用于以下几种情形：

- 只使用类实例，不使用类属性；
- 类属性太多，但使用其中少量属性（可以 mock 属性返回值）；
- 类是接口或虚基类，并不关心其具体实现类。

```
MockClass mockClass = Mockito.mock(MockClass.class);
List<Long> userIdList = (List<Long>)Mockito.mock(List.class);
```

4. 利用@Mock 注解

@Mock 注解跟 Mockito.mock 方法一样，可以用来模拟依赖对象，适用于普通类、接口和虚基类。@Mock 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    ...
}
```

5. 利用 Mockito.spy 方法

Mockito.spy 方法跟 Mockito.mock 方法功能相似，只是 Mockito.spy 方法默认所有方法都是真实方法，除非主动去模拟对应方法。

```
UserService userService = Mockito.spy(new UserService());
UserService userService = Mockito.spy(UserService.class);
AbstractOssService ossService = Mockito.spy(AbstractOssService.class);
```

6. 利用@Spy 注解

@Spy 注解跟 Mockito.spy 方法一样，可以用来模拟依赖对象，适用于普通类、接口和虚基类。@Spy 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class CompanyServiceTest {
    @Spy
    private UserService userService = new UserService();
    ...
}
```

注意：

@Spy 注解对象需要初始化。如果是虚基类或接口，可以用 Mockito.mock 方法实例化。

五、 注入依赖对象

当模拟完这些类成员对象后,我们需要把这些类成员对象注入到被测试类的实例中。以便在调用被测试方法时，可能使用这些类成员对象，而不至于抛出空指针异常。

1. 利用 Setter 方法注入

如果类定义了 Setter 方法，可以直接调用方法设置字段值。

```
userService.setMaxCount(100);
userService.setUserDAO(userDAO);
```

2. 利用 ReflectionTestUtils.setField 方法注入

JUnit 提供 ReflectionTestUtils.setField 方法设置属性字段值。

```
ReflectionTestUtils.setField(userService, "maxCount", 100);
ReflectionTestUtils.setField(userService, "userDAO", userDAO);
```

3. 利用 Whitebox.setInternalState 方法注入

PowerMock 提供 Whitebox.setInternalState 方法设置属性字段值。

```
Whitebox.setInternalState(userService, "maxCount", 100);
Whitebox.setInternalState(userService, "userDAO", userDAO);
```

4. 利用@InjectMocks 注解注入

@InjectMocks 注解用来创建一个实例，并将其它对象（@Mock、@Spy 或直接定义的对象）注入到该实例中。@InjectMocks 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    private Boolean canModify;

    @InjectMocks
    private UserService userService;
    ...
}
```

5. 设置静态常量字段值

有时候，我们需要对静态常量对象进行模拟，然后去验证是否执行了对应分支下的方法。比如：需要模拟 Lombok 的@Slf4j 生成的 log 静态常量。但是，Whitebox.setInternalState 方法和@InjectMocks 注解并不支持设置静态常量，需要自己实现一个设置静态常量的方法：

```
public final class FieldHelper {
    public static void setStaticFinalField(Class<?> clazz, String fieldName, Object
fieldValue) throws NoSuchFieldException, IllegalAccessException {
        Field field = clazz.getDeclaredField(fieldName);
        FieldUtils.removeFinalModifier(field);
        FieldUtils.writeStaticField(field, fieldValue, true);
    }
}
```

具体使用方法如下：

```
FieldHelper.setStaticFinalField(UserService.class, "log", log);
```

注意：

经过测试，该方法对于 int、Integer 等基础类型并不生效，应该是编译器常量优化导致。

六、 模拟依赖方法

在模拟完依赖的参数和返回值后，就可以利用 Mockito 和 PowerMock 的功能，进行依赖方法的模拟。如果依赖对象还有方法调用，还需要模拟这些依赖对象的方法。

1. 根据返回模拟方法

1) 模拟无返回值方法

```
Mockito.doNothing().when(userDAO).delete(userId);
```

2) 模拟方法单个返回值

```
Mockito.doReturn(user).when(userDAO).get(userId);
Mockito.when(userDAO.get(userId)).thenReturn(user);
```

3) 模拟方法多个返回值

直接列举出多个返回值：

```
Mockito.doReturn(record0, record1, record2, null).when(recordReader).read();
Mockito.when(recordReader.read()).thenReturn(record0, record1, record2, null);
```

转化列表为多个返回值：

```
List<Record> recordList = ...;
Mockito.doReturn(recordList.get(0), recordList.subList(1,
recordList.size()).toArray()).when(recordReader).read();
Mockito.when(recordReader.read()).thenReturn(recordList.get(0), recordList.subList(1,
recordList.size()).toArray());
```

4) 模拟方法定制返回值

可利用 Answer 定制方法返回值：

```
Map<Long, UserDO> userMap = ...;
Mockito.doAnswer(invocation -> userMap.get(invocation.getArgument(0)))
    .when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong()))
    .thenReturn(invocation -> userMap.get(invocation.getArgument(0)));
Mockito.when(userDAO.get(Mockito.anyLong()))
    .then(invocation -> userMap.get(invocation.getArgument(0)));
```

5) 模拟方法抛出单个异常

指定单个异常类型：

```
Mockito.doThrow(PersistenceException.class).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(PersistenceException.class);
```

指定单个异常对象：

```
Mockito.doThrow(exception).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(exception);
```

6) 模拟方法抛出多个异常

指定多个异常类型：

```
Mockito.doThrow(PersistenceException.class,
RuntimeException.class).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(PersistenceException.class,
RuntimeException.class);
```

指定多个异常对象：

```
Mockito.doThrow(exception1, exception2).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(exception1, exception2);
```

7) 直接调用真实方法

```
Mockito.doCallRealMethod().when(userService).getUser(userId);
Mockito.when(userService.getUser(userId)).thenCallRealMethod();
```

2. 根据参数模拟方法

Mockito 提供 do-when 语句和 when-then 语句模拟方法。

1) 模拟无参数方法

对于无参数的方法模拟：

```
Mockito.doReturn(deleteCount).when(userDAO).deleteAll();
Mockito.when(userDAO.deleteAll()).thenReturn(deleteCount);
```

2) 模拟指定参数方法

对于指定参数的方法模拟：

```
Mockito.doReturn(user).when(userDAO).get(userId);
Mockito.when(userDAO.get(userId)).thenReturn(user);
```

3) 模拟任意参数方法

在编写单元测试用例时，有时候并不关心传入参数的具体值，可以使用 Mockito 参数匹配器的 any 方法。Mockito 提供了 anyInt、anyLong、anyString、anyList、anySet、anyMap、any(Class<T> clazz)等方法来表示任意值。

```
Mockito.doReturn(user).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenReturn(user);
```

4) 模拟可空参数方法

Mockito 参数匹配器的 any 具体方法，并不能够匹配 null 对象。而 Mockito 提供一个 nullable 方法，可以匹配包含 null 对象的任意对象。此外，Mockito.any()方法也可以用来匹配可空参数。

```
Mockito.doReturn(user).when(userDAO)
    .queryCompany(Mockito.anyLong(), Mockito.nullable(Long.class));
Mockito.when(userDAO.queryCompany(Mockito.anyLong(), Mockito.<Long>any()))
    .thenReturn(user);
```

5) 模拟必空参数方法

同样，如果要匹配 null 对象，可以使用 isNull 方法，或使用 eq(null)。

```
Mockito.doReturn(user).when(userDAO).queryCompany(Mockito.anyLong(), Mockito.isNull());
Mockito.when(userDAO.queryCompany(Mockito.anyLong(), Mockito.eq(null))).thenReturn(user);
```

6) 模拟不同参数方法

Mockito 支持按不同的参数分别模拟同一方法。

```
Mockito.doReturn(user1).when(userDAO).get(1L);
Mockito.doReturn(user2).when(userDAO).get(2L);
...
```

注意：
如果一个参数满足多个模拟方法条件，会以最后一个模拟方法为准。

7) 模拟可变参数方法

对于一些变长度参数方法，可以按实际参数个数进行模拟：

```
Mockito.when(userService.delete(Mockito.anyLong()).thenReturn(true);
Mockito.when(userService.delete(1L, 2L, 3L).thenReturn(true);
```

也可以用 Mockito.any()模拟一个通用匹配方法：

```
Mockito.when(userService.delete(Mockito.<Long>any()).thenReturn(true);
```

注意：
Mockito.<T>any()并不等于 Mockito.any(Class<T> type)，前者可以匹配 null 和类型 T 的可变参数，后者只能匹配 T 必填参数。

3. 模拟其它特殊方法

1) 模拟 final 方法

PowerMock 提供对 final 方法的模拟，方法跟模拟普通方法一样。但是，需要把对应的模拟类添加到@PrepareForTest 注解中。

```
// 添加@PrepareForTest注解
@PrepareForTest({UserService.class})

// 跟模拟普通方法完全一致
Mockito.doReturn(userId).when(idGenerator).next();
Mockito.when(idGenerator.next()).thenReturn(userId);
```

2) 模拟私有方法

PowerMock 提供提对私有方法的模拟，但是需要把私有方法所在的类放在@PrepareForTest 注解中。

```
PowerMockito.doReturn(true).when(UserService.class, "isSuper", userId);
PowerMockito.when(UserService.class, "isSuper", userId).thenReturn(true);
```

3) 模拟构造方法

PowerMock 提供 PowerMockito.whenNew 方法来模拟构造方法，但是需要把使用构造方法的类放在@PrepareForTest 注解中。

```
PowerMockito.whenNew(UserDO.class).withNoArguments().thenReturn(userDO);
PowerMockito.whenNew(UserDO.class).withArguments(userId, userName).thenReturn(userDO);
```

4) 模拟静态方法

PowerMock 提供 PowerMockito.mockStatic 和 PowerMockito.spy 来模拟静态方法类，然后就可以模拟静态方法了。同样，需要把对应的模拟类添加到@PrepareForTest 注解中。

```
// 模拟对应的类
PowerMockito.mockStatic(HttpHelper.class);
PowerMockito.spy(HttpHelper.class);

// 模拟对应的方法
PowerMockito.when(HttpHelper.httpPost(SERVER_URL)).thenReturn(response);
PowerMockito.doReturn(response).when(HttpHelper.class, "httpPost", SERVER_URL);
PowerMockito.when(HttpHelper.class, "httpPost", SERVER_URL).thenReturn(response);
```

注意：

第一种方式不适用于 PowerMockito.spy 模拟的静态方法类。

七、 调用被测方法

在准备好参数对象后，就可以调用被测试方法了。

如果把方法按访问权限分类，可以简单地分为有访问权限和无访问权限两种。但实际上，Java 语言中提供了 public、protected、private 和缺失共 4 种权限修饰符，在不同的环境下又对应不同的访问权限。具体映射关系如下：

修饰符	本类	本包	子类	其它
public	有	有	有	有
protected	有	有	有	无
缺省	有	有	无	无
private	有	无	无	无

下面，将根据有访问权限和无访问权限两种情况，来介绍如何调用被测方法。

1. 调用构造方法

1) 调用有访问权限的构造方法

可以直接调用有访问权限的构造方法。

```
UserDO user = new User();
UserDO user = new User(1L, "admin");
```

2) 调用无访问权限的构造方法

调用无访问权限的构造方法，可以使用 PowerMock 提供的 Whitebox.invokeConstructor 方法。

```
Whitebox.invokeConstructor(NumberHelper.class);
Whitebox.invokeConstructor(User.class, 1L, "admin");
```

备注

该方法也可以调用有访问权限的构造方法，但是不建议使用。

2. 调用普通方法

1) 调用有访问权限的普通方法

可以直接调用有访问权限的普通方法。

```
userService.deleteUser(userId);
User user = userService.getUser(userId);
```

2) 调用无权限访问的普通方法

调用无访问权限的普通方法，可以使用 PowerMock 提供的 Whitebox.invokeMethod 方法。

```
User user = (User)Whitebox.invokeMethod(userService, "isSuper", userId);
```

也可以使用 PowerMock 提供 Whitebox.getMethod 方法和 PowerMockito.method 方法，可以直接获取对应类方法对象。然后，通过 Method 的 invoke 方法，可以调用没有访问权限的方法。

```
Method method = Whitebox.getMethod(UserService.class, "isSuper", Long.class);
Method method = PowerMockito.method(UserService.class, "isSuper", Long.class);
User user = (User)method.invoke(userService, userId);
```

备注

该方法也可以调用有访问权限的普通方法，但是不建议使用。

3. 调用静态方法

1) 调用有权限访问的静态方法

可以直接调用有访问权限的静态方法。

```
boolean isPositive = NumberHelper.isPositive(-1);
```

2) 调用无权限访问的静态方法

调用无权限访问的静态方法，可以使用 PowerMock 提供的 Whitebox.invokeMethod 方法。

```
String value = (String)Whitebox.invokeMethod(JSON.class, "toJSONString", object);
```

备注

该方法也可以调用有访问权限的静态方法，但是不建议使用。

八、 验证依赖方法

在单元测试中，验证是确认模拟的依赖方法是否按照预期被调用或未调用的过程。Mockito 提供了许多方法来验证依赖方法调用，给我们编写单元测试用例带来了很大的帮助。

1. 根据参数验证方法调用

1) 验证无参数方法调用

```
Mockito.verify(userDAO).deleteAll();
```

2) 验证指定参数方法调用

```
Mockito.verify(userDAO).delete(userId);
Mockito.verify(userDAO).delete(Mockito.eq(userId));
```

3) 验证任意参数方法调用

```
Mockito.verify(userDAO).delete(Mockito.anyLong());
```

4) 验证可空参数方法调用

```
Mockito.verify(userDAO).queryCompany(Mockito.anyLong(), Mockito.nullable(Long.class));
```

5) 验证必空参数方法调用

```
Mockito.verify(userDAO).queryCompany(Mockito.anyLong(), Mockito.isNull());
```

6) 验证可变参数方法调用

对于一些变长度参数方法，可以按实际参数个数进行验证：

```
Mockito.verify(userService).delete(Mockito.any(Long.class));
Mockito.verify(userService).delete(1L, 2L, 3L);
```

也可以用 Mockito.any()进行通用验证：

```
Mockito.verify(userService).delete(Mockito.<Long>any());
```

2. 验证方法调用次数

1) 验证方法默认调用 1 次

```
Mockito.verify(userDAO).delete(userId);
```

2) 验证方法从不调用

```
Mockito.verify(userDAO, Mockito.never()).delete(userId);
```

3) 验证方法调用 n 次

```
Mockito.verify(userDAO, Mockito.times(n)).delete(userId);
```

4) 验证方法调用至少 1 次

```
Mockito.verify(userDAO, Mockito.atLeastOnce()).delete(userId);
```

5) 验证方法调用至少 n 次

```
Mockito.verify(userDAO, Mockito.atLeast(n)).delete(userId);
```

6) 验证方法调用最多 1 次

```
Mockito.verify(userDAO, Mockito.atMostOnce()).delete(userId);
```

7) 验证方法调用最多 n 次

```
Mockito.verify(userDAO, Mockito.atMost(n)).delete(userId);
```

8) 验证方法调用指定 n 次

Mockito 允许按顺序进行验证方法调用，未被验证到的方法调用将不会被标记为已验证。

```
Mockito.verify(userDAO, Mockito.call(n)).delete(userId);
```

9) 验证对象及其方法调用 1 次

用于验证对象及其方法调用 1 次，如果该对象还有别的方法被调用或者该方法调用了多次，都将导致验证方法调用失败。

```
Mockito.verify(userDAO, Mockito.only()).delete(userId);
```

相当于：

```
Mockito.verify(userDAO).delete(userId);
Mockito.verifyNoMoreInteractions(userDAO);
```

3. 验证方法调用并捕获参数值

Mockito 提供 ArgumentCaptor 类来捕获参数值，通过调用 forClass(Class<T> clazz) 方法来构建一个 ArgumentCaptor 对象，然后在验证方法调用时来捕获参数，最后

获取到捕获的参数值并验证。如果一个方法有多个参数都要捕获并验证，那就需要创建多个 ArgumentCaptor 对象。

ArgumentCaptor 的主要接口方法：

- capture 方法，用于捕获方法参数；
- getValue 方法，用于获取捕获的参数值，如果捕获了多个参数值，该方法只返回最后一个参数值；
- getAllValues 方法，用户获取捕获的所有参数值。

1) 使用 ArgumentCaptor.forClass 方法定义参数捕获器

在测试用例方法中，直接使用 ArgumentCaptor.forClass 方法定义参数捕获器。

```
ArgumentCaptor<UserDO> userCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).modify(userCaptor.capture());
UserDO user = userCaptor.getValue();
```

注意：
定义泛型类的参数捕获器时，存在强制类型转换，会引起编译器警告。

2) 使用@Captor 注解定义参数捕获器

也可以用 Mockito 提供的@Captor 注解，在测试用例类中定义参数捕获器。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Captor
    private ArgumentCaptor<UserDO> userCaptor;
    @Test
    public void testModifyUser() {
        ...
        Mockito.verify(userDAO).modify(userCaptor.capture());
        UserDO user = userCaptor.getValue();
    }
}
```

注意：
定义泛型类的参数捕获器时，由于是 Mockito 自行初始化，不会引起编译器警告。

3) 捕获多次方法调用的参数值列表

```
ArgumentCaptor<UserDO> userCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO, Mockito.atLeastOnce()).modify(userCaptor.capture());
List<UserDO> userList = userCaptor.getAllValues();
```

4. 验证其它特殊方法

1) 验证 final 方法调用

final 方法的验证跟普通方法类似，这里不再累述。

2) 验证私有方法调用

PowerMockito 提供 verifyPrivate 方法验证私有方法调用。

```
PowerMockito.verifyPrivate(myClass, times(1)).invoke("unload", any(List.class));
```

3) 验证构造方法调用

PowerMockito 提供 verifyNew 方法验证构造方法调用。

```
PowerMockito.verifyNew(MockClass.class).withNoArguments();
PowerMockito.verifyNew(MockClass.class).withArguments(someArgs);
```

4) 验证静态方法调用

PowerMockito 提供 verifyStatic 方法验证静态方法调用。

```
PowerMockito.verifyStatic(StringUtils.class);
StringUtils.isEmpty(string);
```

九、 验证数据对象

Junit 测试框架中 Assert 类就是断言工具类，主要验证单元测试中实际数据对象与期望数据对象一致。在调用被测方法时，需要对返回值和异常进行验证；在验证方法调用时，也需要对捕获的参数值进行验证。

1. 验证数据对象空值

1) 验证数据对象为空

通过 Junit 提供的 Assert.assertNull 方法验证数据对象为空。

```
Assert.assertNull("用户标识必须为空", userId);
```

2) 验证数据对象非空

通过 Junit 提供的 Assert.assertNotNull 方法验证数据对象非空。

```
Assert.assertNotNull("用户标识不能为空", userId);
```

2. 验证数据对象布尔值

1) 验证数据对象为真

通过 Junit 提供的 Assert.assertTrue 方法验证数据对象为真。

```
Assert.assertTrue("返回值必须为真", NumberHelper.isPositive(1));
```

2) 验证数据对象为假

通过 Junit 提供的 Assert.assertFalse 方法验证数据对象为假。

```
Assert.assertFalse("返回值必须为假", NumberHelper.isPositive(-1));
```

3. 验证数据对象引用

在单元测试用例中，对于一些参数或返回值对象，不需要验证对象具体取值，只需要验证对象引用是否一致。

1) 验证数据对象一致

Junit 提供的 Assert.assertSame 方法验证数据对象一致。

```
UserDO expectedUser = ...;
Mockito.doReturn(expectedUser).when(userDAO).get(userId);
UserDO actualUser = userService.getUser(userId);
Assert.assertSame("用户必须一致", expectedUser, actualUser);
```

2) 验证数据对象不一致

Junit 提供的 Assert.assertNotSame 方法验证数据对象一致。

```
UserDO expectedUser = ...;
Mockito.doReturn(expectedUser).when(userDAO).get(userId);
UserDO actualUser = userService.getUser(otherUserId);
Assert.assertNotSame("用户不能一致", expectedUser, actualUser);
```

4. 验证数据对象值

Junit 提供 Assert.assertEquals、Assert.assertNotEquals、Assert.assertArrayEquals 方法组，可以用来验证数据对象值是否相等。

1) 验证简单数据对象

对于简单数据对象（比如：基础类型、包装类型、实现了 equals 的数据类型……），可以直接通过 Junit 的 Assert.assertEquals 和 Assert.assertNotEquals 方法组进行验证。

```
Assert.assertNotEquals("用户名称不一致", "admin", userName);
Assert.assertEquals("账户金额不一致", 10000.0D, accountAmount, 1E-6D);
```

2) 验证简单数组或集合对象

对于简单数组对象（比如：基础类型、包装类型、实现了 equals 的数据类型……），可以直接通过 Junit 的 Assert.assertArrayEquals 方法组进行验证。对于简单集合对象，也可以通过 Assert.assertEquals 方法验证。

```
Long[] userIds = ...;
Assert.assertArrayEquals("用户标识列表不一致", new Long[] {1L, 2L, 3L}, userIds);

List<Long> userIdList = ...;
Assert.assertEquals("用户标识列表不一致", Arrays.asList(1L, 2L, 3L), userIdList);
```

3) 验证复杂数据对象

对于复杂的 JavaBean 数据对象，需要验证 JavaBean 数据对象的每一个属性字段。

```
UserDO user = ...;
Assert.assertEquals("用户标识不一致", Long.valueOf(1L), user.getId());
Assert.assertEquals("用户名称不一致", "admin", user.getName());
Assert.assertEquals("用户公司标识不一致", Long.valueOf(1L), user.getCompany().getId());
...
```

4) 验证复杂数组或集合对象

对于复杂的 JavaBean 数组和集合对象，需要先展开数组和集合对象中每一个 JavaBean 数据对象，然后验证 JavaBean 数据对象的每一个属性字段。

```
List<UserDO> expectedUserList = ...;
List<UserDO> actualUserList = ...;
Assert.assertEquals("用户列表长度不一致", expectedUserList.size(), actualUserList.size());
UserDO[] expectedUsers = expectedUserList.toArray(new UserDO[0]);
UserDO[] actualUsers = actualUserList.toArray(new UserDO[0]);
for (int i = 0; i < actualUsers.length; i++) {
    Assert.assertEquals(String.format("用户(%s)标识不一致", i), expectedUsers[i].getId(),
actualUsers[i].getId());
    Assert.assertEquals(String.format("用户(%s)名称不一致", i), expectedUsers[i].getName(),
actualUsers[i].getName());
Assert.assertEquals("用户公司标识不一致", expectedUsers[i].getCompany().getId(),
 actualUsers[i].getCompany().getId());
    ...
}
```

5) 通过序列化验证数据对象

如上一节例子所示，当数据对象过于复杂时，如果采用 Assert.assertEquals 依次验证每个 JavaBean 对象、验证每一个属性字段，测试用例的代码量将会非常庞大。这里，推荐使用序列化手段简化数据对象的验证，比如利用 JSON.toJSONString 方法把复杂的数据对象转化为字符串，然后再使用 Assert.assertEquals 方法进行验证字符串。但是，序列化值必须具备有序性、一致性和可读性。

```
List<UserDO> userList = ...;
String text = ResourceHelper.getResourceAsString(getClass(), "userList.json");
Assert.assertEquals("用户列表不一致", text, JSON.toJSONString(userList));
```

通常使用 JSON.toJSONString 方法把 Map 对象转化为字符串，其中 key-value 的顺序具有不确定性，无法用于验证两个对象是否一致。这里，JSON 提供序列化选项 SerializerFeature.MapSortField（映射排序字段），可以用于保证序列化后的 key-value 的有序性。

```
Map<Long, Map<String, Object>> userMap = ...;
String text = ResourceHelper.getResourceAsString(getClass(), "userMap.json");
Assert.assertEquals("用户映射不一致", text, JSON.toJSONString(userMap,
SerializerFeature.MapSortField));
```

6) 验证数据对象私有属性字段

有时候，单元测试用例需要对复杂对象的私有属性字段进行验证。而 PowerMockito 提供的 Whitebox.getInternalState 方法，获取轻松地获取到私有属性字段值。

```
MapperScannerConfigurer configurer = myBatisConfiguration.buildMapperScannerConfigurer();
Assert.assertEquals("基础包不一致", "com.alibaba.example",
Whitebox.getInternalState(configurer, "basePackage"));
```

5. 验证异常对象内容

异常作为 Java 语言的重要特性，是 Java 语言健壮性的重要体现。捕获并验证异常数据内容，也是测试用例的一种。

1) 通过@Test 注解验证异常对象

Junit 的注解@Test 提供了一个 expected 属性，可以指定一个期望的异常类型，用来捕获并验证异常。但是，这种方式只能验证异常类型，并不能验证异常原因和消息。

```
@Test(expected = ExampleException.class)
public void testGetUser() {
    // 模拟依赖方法
    Mockito.doReturn(null).when(userDAO).get(userId);

    // 调用被测方法
    userService.getUser(userId);
}
```

2) 通过@Rule 注解验证异常对象

如果想要验证异常原因和消息，就需求采用@Rule 注解定义 ExpectedException 对象，然后在测试方法的前面声明要捕获的异常类型、原因和消息。

```
@Rule
private ExpectedException exception = ExpectedException.none();
@Test
public void testGetUser() {
    // 模拟依赖方法
    Long userId = 123L;
    Mockito.doReturn(null).when(userDAO).get(userId);

    // 调用被测方法
    exception.expect(ExampleException.class);
    exception.expectMessage(String.format("用户(%s)不存在", userId));
    userService.getUser(userId);
}
```

3) 通过 Assert.assertThrows 验证异常对象

在最新版的 JUnit 中，提供了一个更为简洁的异常验证方式——Assert.assertThrows 方法。

```
@Test
public void testGetUser() {
    // 模拟依赖方法
    Long userId = 123L;
    Mockito.doReturn(null).when(userDAO).get(userId);

    // 调用被测方法
    ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class,
() -> userService.getUser(userId));
    Assert.assertEquals("异常消息不一致", "处理异常", exception.getMessage());
}
```

十、 验证依赖对象

1. 验证模拟对象没有任何方法调用

Mockito 提供了 verifyNoInteractions 方法，可以验证模拟对象在被测试方法中没有任何调用。

```
Mockito.verifyNoInteractions(idGenerator, userDAO);
```

2. 验证模拟对象没有更多方法调用

Mockito 提供了 verifyNoMoreInteractions 方法，在验证模拟对象所有方法调用后使用，可以验证模拟对象所有方法调用是否都得到验证。如果模拟对象存在任何未验证的方法调用，就会抛出 NoInteractionsWanted 异常。

```
Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
```

备注

Mockito 的 verifyZeroInteractions 方法与 verifyNoMoreInteractions 方法功能相同，但是目前前者已经被废弃。

3. 清除模拟对象所有方法调用标记

在编写单元测试用例时，为了减少单元测试用例数和代码量，可以把多组参数定义在同一个单元测试用例中，然后用 for 循环依次执行每一组参数的被测方法调用。

为了避免上一次测试的方法调用影响下一次测试的方法调用验证，最好使用 Mockito 提供 clearInvocations 方法清除上一次的方法调用。

```
// 清除所有对象调用
Mockito.clearInvocations();
// 清除指定对象调用
Mockito.clearInvocations(idGenerator, userDAO);
```

十一、 典型案例及解决方案

这里，只收集了几个经典案例，解决了特定环境下的特定问题。

1. 测试框架特性导致问题

在编写单元测试用例时，或多或少会遇到一些问题，大多数是由于对测试框架特性不熟悉导致，比如：

- Mockito 不支持对静态方法、构造方法、final 方法、私有方法的模拟；
- Mockito 的 any 相关的参数匹配方法并不支持可空参数和空参数；
- 如果为 Mock 方法或 Mock 方法参数不匹配时，会返回默认值（基础类型为 0，对象类型为 null）；
- 采用 Mockito 的参数匹配方法或 Argument 的 captor 方法时，其它参数不能直接用常量或变量，必须使用 Mockito 的 eq 方法包装；
- 使用 when-then 语句模拟 Spy 对象方法会先执行真实方法，应该使用 do-when 语句；
- PowerMock 对静态方法、构造方法、final 方法、私有方法的模拟需要把对应的类添加到@PrepareForTest 注解中；
- PowerMock 模拟 JDK 的静态方法、构造方法、final 方法、私有方法时，需要把使用这些方法的类加入到@PrepareForTest 注解中，从而导致单元测试覆盖率不被统计；

- PowerMock 使用自定义的类加载器来加载类，可能导致系统类加载器认为有类型转换问题；需要加上@PowerMockIgnore({"javax.crypto.*"})注解，来告诉 PowerMock 这个包不要用 PowerMock 的类加载器加载，需要采用系统类加载器来加载；

- 如果遇到 Mock 对象静态常量初始化失败，可以利用注解 @SuppressStaticInitializationFor 抑制静态常量初始化。

对于这些问题，可以根据提示信息查阅相关资料解决，这里就不再累述了。

2. 捕获参数值已变更问题

在编写单元测试用例时，通常采用 ArgumentCaptor 进行参数捕获，然后对参数对象值进行验证。如果参数对象值没有变更，这个步骤就没有任何问题。但是，如果参数对象值在后续流程中发生变更，就会导致验证参数值失败。

1) 原始代码

```
public <T> void readData(RecordReader recordReader, int batchSize, Function<Record, T>
dataParser, Predicate<List<T>> dataStorage) {
    try {
        // 依次读取数据
        Record record;
        boolean isContinue = true;
        List<T> dataList = new ArrayList<>(batchSize);
        while (Objects.nonNull(record = recordReader.read()) && isContinue) {
            // 解析添加数据
            T data = dataParser.apply(record);
            if (Objects.nonNull(data)) {
                dataList.add(data);
            }

            // 批量存储数据
            if (dataList.size() == batchSize) {
                isContinue = dataStorage.test(dataList);
                dataList.clear();
            }
        }

        // 存储剩余数据
        if (CollectionUtils.isNotEmpty(dataList)) {
            dataStorage.test(dataList);
            dataList.clear();
        }
    } catch (IOException e) {
        String message = READ_DATA_EXCEPTION;
        log.warn(message, e);
        throw new ExampleException(message, e);
    }
}
```

2) 测试用例

```
@Test
public void testReadData() throws Exception {
    // 模拟依赖方法
    // 模拟依赖方法: recordReader.read
    Record record0 = Mockito.mock(Record.class);
    Record record1 = Mockito.mock(Record.class);
    Record record2 = Mockito.mock(Record.class);
    TunnelRecordReader recordReader = Mockito.mock(TunnelRecordReader.class);
    Mockito.doReturn(record0, record1, record2, null).when(recordReader).read();
    // 模拟依赖方法: dataParser.apply
    Object object0 = new Object();
    Object object1 = new Object();
    Object object2 = new Object();
    Function<Record, Object> dataParser = Mockito.mock(Function.class);
    Mockito.doReturn(object0).when(dataParser).apply(record0);
    Mockito.doReturn(object1).when(dataParser).apply(record1);
    Mockito.doReturn(object2).when(dataParser).apply(record2);
    // 模拟依赖方法: dataStorage.test
    Predicate<List<Object>> dataStorage = Mockito.mock(Predicate.class);
    Mockito.doReturn(true).when(dataStorage).test(Mockito.anyList());

    // 调用测试方法
    odpsService.readData(recordReader, 2, dataParser, dataStorage);

    // 验证依赖方法
    // 模拟依赖方法: recordReader.read
    Mockito.verify(recordReader, Mockito.times(4)).read();
    // 模拟依赖方法: dataParser.apply
    Mockito.verify(dataParser, Mockito.times(3)).apply(Mockito.any(Record.class));
    // 验证依赖方法: dataStorage.test
    ArgumentCaptor<List<Object>> recordListCaptor = ArgumentCaptor.forClass(List.class);
    Mockito.verify(dataStorage, Mockito.times(2)).test(recordListCaptor.capture());
    Assert.assertEquals("数据列表不一致", Arrays.asList(Arrays.asList(object0, object1),
Arrays.asList(object2)), recordListCaptor.getAllValues());
}
```

3) 问题现象

执行单元测试用例失败，抛出以下异常信息：

```
java.lang.AssertionError: 数据列表不一致 expected:<[[java.lang.Object@1e3469df,
java.lang.Object@79499fa], [java.lang.Object@48531d5]]> but was:<[[], []]>
```

4) 问题原因

由于参数 dataList 在调用 dataStorage.test 方法后，都被主动调用 dataList.clear 方法进行清空。由于 ArgumentCaptor 捕获的是对象引用，所以最后捕获到了同一个空列表。

5) 解决方案

可以在模拟依赖方法 dataStorage.test 时，保存传入参数的当前值进行验证。代码如下：

```
@Test
public void testReadData() throws Exception {
    // 模拟依赖方法
    ...
    // 模拟依赖方法：dataStorage.test
    List<Object> dataList = new ArrayList<>();
    Predicate<List<Object>> dataStorage = Mockito.mock(Predicate.class);
    Mockito.doAnswer(invocation -> dataList.addAll((List<Object>)invocation.getArgument(0)))
        .when(dataStorage).test(Mockito.anyList());

    // 调用测试方法
    odpsService.readData(recordReader, 2, dataParser, dataStorage);

    // 验证依赖方法
    ...
    // 验证依赖方法：dataStorage.test
    Mockito.verify(dataStorage, Mockito.times(2)).test(Mockito.anyList());
    Assert.assertEquals("数据列表不一致", Arrays.asList(object0, object1, object2), dataList);
}
```

3. 模拟 Lombok 的 log 对象问题

Lombok 的@Slf4j 注解，广泛地应用于 Java 项目中。

在某些代码分支里，可能只有 log 记录日志的操作，为了验证这个分支逻辑被正确执行，需要在单元测试用例中对 log 记录日志的操作进行验证。

1) 原始方法

```
@Slf4j
@Service
public class ExampleService {
    public void recordLog(int code) {
        if (code == 1) {
            log.info("执行分支1");
            return;
        }
        if (code == 2) {
            log.info("执行分支2");
            return;
        }
        log.info("执行默认分支");
    }
    ...
}
```

2) 测试用例

```
@RunWith(PowerMockRunner.class)
public class ExampleServiceTest {
    @Mock
    private Logger log;
    @InjectMocks
    private ExampleService exampleService;
    @Test
    public void testRecordLog1() {
        exampleService.recordLog(1);
        Mockito.verify(log).info("执行分支1");
    }
}
```

3) 问题现象

执行单元测试用例失败，抛出以下异常信息：

```
Wanted but not invoked:
logger.info("执行分支1");
```

4) 原因分析

经过调式跟踪，发现 ExampleService 中的 log 对象并没有被注入。通过编译发现，Lombok 的@Slf4j 注解在 ExampleService 类中生成了一个静态常量 log，而@InjectMocks 注解并不支持静态常量的注入。

5) 解决方案

采用作者实现的 FieldHelper.setStaticFinalField 方法，可以实现对静态常量的注入模拟对象。

```
@RunWith(PowerMockRunner.class)
public class ExampleServiceTest {
    @Mock
    private Logger log;
    @InjectMocks
    private ExampleService exampleService;
    @Before
    public void beforeTest() throws Exception {
        FieldHelper.setStaticFinalField(ExampleService.class, "log", log);
    }
    @Test
    public void testRecordLog1() {
        exampleService.recordLog(1);
        Mockito.verify(log).info("执行分支1");
    }
}
```

4. 兼容 Pandora 等容器问题

阿里巴巴的很多中间件，都是基于 Pandora 容器的，在编写单元测试用例时，可能会遇到一些坑。

1) 原始方法

```
@Slf4j
public class MetaqMessageSender {
    @Autowired
    private MetaProducer metaProducer;
    public String sendMetaqMessage(String topicName, String tagName, String messageKey,
String messageBody) {
        try {
            // 组装消息内容
            Message message = new Message();
            message.setTopic(topicName);
            message.setTags(tagName);
            message.setKeys(messageKey);
            message.setBody(messageBody.getBytes(StandardCharsets.UTF_8));

            // 发送消息请求
            SendResult sendResult = metaProducer.send(message);
            if (sendResult.getSendStatus() != SendStatus.SEND_OK) {
                String msg = String.format("发送标签(%s)消息(%s)状态错误(%s)", tagName,
messageKey, sendResult.getSendStatus());
                log.warn(msg);
                throw new ReconsException(msg);
            }
            log.info(String.format("发送标签(%s)消息(%s)状态成功:%s", tagName, messageKey,
sendResult.getMsgId()));

            // 返回消息标识
            return sendResult.getMsgId();
        } catch (MQClientException | RemotingException | MQBrokerException |
InterruptedException e) {
            // 记录消息异常
            Thread.currentThread().interrupt();
            String message = String.format("发送标签(%s)消息(%s)状态异常:%s", tagName,
messageKey, e.getMessage());
            log.warn(message, e);
            throw new ReconsException(message, e);
        }
    }
}
```

2) 测试用例

```
@RunWith(PowerMockRunner.class)
public class MetaqMessageSenderTest {
    @Mock
    private MetaProducer metaProducer;
    @InjectMocks
    private MetaqMessageSender metaqMessageSender;
    @Test
    public void testSendMetaqMessage() throws Exception {
        // 模拟依赖方法
        SendResult sendResult = new SendResult();
        sendResult.setMsgId("msgId");
        sendResult.setSendStatus(SendStatus.SEND_OK);
        Mockito.doReturn(sendResult).when(metaProducer).send(Mockito.any(Message.class));

        // 调用测试方法
        String topicName = "topicName";
        String tagName = "tagName";
        String messageKey = "messageKey";
        String messageBody = "messageBody";
        String messageId = metaqMessageSender.sendMetaqMessage(topicName, tagName,
messageKey, messageBody);
        Assert.assertEquals("messageId不一致", sendResult.getMsgId(), messageId);

        // 验证依赖方法
        ArgumentCaptor<Message> messageCaptor = ArgumentCaptor.forClass(Message.class);
        Mockito.verify(metaProducer).send(messageCaptor.capture());
        Message message = messageCaptor.getValue();
        Assert.assertEquals("topicName不一致", topicName, message.getTopic());
        Assert.assertEquals("tagName不一致", tagName, message.getTags());
        Assert.assertEquals("messageKey不一致", messageKey, message.getKeys());
        Assert.assertEquals("messageBody不一致", messageBody, new String(message.getBody()));
    }
}
```

3) 问题现象

执行单元测试用例失败，抛出以下异常信息：

```
java.lang.RuntimeException: com.alibaba.rocketmq.client.producer.SendResult was loaded by
org.powermock.core.classloader.javassist.JavassistMockClassLoader@5d43661b, it should be
loaded by Pandora Container. Can not load this fake sdk class.
```

4) 原因分析

基于 Pandora 容器的中间件，需要使用 Pandora 容器加载。在上面测试用例中，使用了 PowerMock 容器加载，从而导致抛出类加载异常。

5) 解决方案

首先，把 PowerMockRunner 替换为 PandoraBootRunner。其次，为了使@Mock、@InjectMocks 等 Mockito 注解生效，需要调用 MockitoAnnotations.initMocks(this)方法进行初始化。

```
@RunWith(PandoraBootRunner.class)
public class MetaqMessageSenderTest {
  ...
    @Before
    public void beforeTest() {
        MockitoAnnotations.initMocks(this);
    }
    ...
}
```

十二、 消除类型转换警告

在编写测试用例时，特别是泛型类型转换时，很容易产生类型转换警告。常见类型转换警告如下：

```
Type safety: Unchecked cast from Object to List<Object>
Type safety: Unchecked invocation forClass(Class<Map>) of the generic method
forClass(Class<S>) of type ArgumentCaptor
Type safety: The expression of type ArgumentCaptor needs unchecked conversion to conform to
ArgumentCaptor<Map<String,Object>>
```

作为一个有代码洁癖的轻微强迫症程序员，是绝对不容许这些类型转换警告产生的。于是，总结了以下方法来解决这些类型转换警告。

1. 利用注解初始化

Mockito 提供@Mock 注解来模拟类实例，提供@Captor 注解来初始化参数捕获器。由于这些注解实例是通过测试框架进行初始化的，所以不会产生类型转换警告。

问题代码：

```
Map<Long, String> resultMap = Mockito.mock(Map.class);
ArgumentCaptor<Map<String, Object>> parameterMapCaptor = ArgumentCaptor.forClass(Map.class);
```

建议代码：

```
@Mock
private Map<Long, String> resultMap;
@Captor
private ArgumentCaptor<Map<String, Object>> parameterMapCaptor;
```

2. 利用临时类或接口

我们无法获取泛型类或接口的 class 实例，但是很容易获取具体类的 class 实例。这个解决方案的思路是——先定义继承泛型类的具体子类，然后 mock、spy、forClass 以及 any 出这个具体子类的实例，然后把具体子类实例转换为父类泛型实例。

问题代码：

```
Function<Record, Object> dataParser = Mockito.mock(Function.class);
AbstractDynamicValue<Long, Integer> dynamicValue = Mockito.spy(AbstractDynamicValue.class);
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
ArgumentCaptor.forClass(ActionRequest.class);
```

建议代码：

```
/** 定义临时类或接口 */
private interface DataParser extends Function<Record, Object> {};
private static abstract class AbstractTemporaryDynamicValue extends
AbstractDynamicValue<Long, Integer> {};
private static class VoidActionRequest extends ActionRequest<Void> {};

/** 使用临时类或接口 */
Function<Record, Object> dataParser = Mockito.mock(DataParser.class);
AbstractDynamicValue<Long, Integer> dynamicValue =
Mockito.spy(AbstractTemporaryDynamicValue.class);
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
ArgumentCaptor.forClass(VoidActionRequest.class);
```

3. 利用 CastUtils.cast 方法

SpringData 包中提供一个 CastUtils.cast 方法，可以用于类型的强制转换。这个解决方案的思路是——利用 CastUtils.cast 方法屏蔽类型转换警告。

问题代码：

```
Function<Record, Object> dataParser = Mockito.mock(Function.class);
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
ArgumentCaptor.forClass(ActionRequest.class);
Map<Long, Double> scoreMap = (Map<Long, Double>)method.invoke(userService);
```

建议代码：

```
Function<Record, Object> dataParser = CastUtils.cast(Mockito.mock(Function.class));
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
CastUtils.cast(ArgumentCaptor.forClass(ActionRequest.class));
Map<Long, Double> scoreMap = CastUtils.cast(method.invoke(userService));
```

这个解决方案，不需要定义注解，也不需要定义临时类或接口，能够让测试用例代码更为精简，所以作者重点推荐使用。如果不愿意引入 SpringData 包，也可以自己参考实现该方法，只是该方法会产生类型转换警告。

注意：

CastUtils.cast 方法本质是——先转换为 Object 类型，再强制转换对应类型，本身不会对类型进行校验。所以，CastUtils.cast 方法好用，但是不要乱用，否则就是大坑（只有运行时才能发现问题）。

4. 利用类型自动转换方法

在 Mockito 中，提供形式如下的方法——泛型类型只跟返回值有关，而跟输入参数无关。这样的方法，可以根据调用方法的参数类型自动转换，而无需手动强制类型转换。如果手动强制类型转换，反而会产生类型转换警告。

```
<T> T getArgument(int index);
public static <T> T any();
public static synchronized <T> T invokeMethod(Object instance, String methodToExecute,
Object... arguments) throws Exception;
```

问题代码：

```
Mockito.doAnswer(invocation -> dataList.addAll((List<Object>)invocation.getArgument(0)))
    .when(dataStorage).test(Mockito.anyList());
Mockito.doThrow(e).when(workflow).beginToPrepare((ActionRequest<Void>)Mockito.any());
Map<Long, Double> scoreMap = (Map<Long, Double>)Whitebox.invokeMethod(userService,
"getScoreMap");
```

建议代码：

```
Mockito.doAnswer(invocation -> dataList.addAll(invocation.getArgument(0)))
    .when(dataStorage).test(Mockito.anyList());
Mockito.doThrow(e).when(workflow).beginToPrepare(Mockito.any());
Map<Long, Double> scoreMap = Whitebox.invokeMethod(userService, "getScoreMap");
```

其实，SpringData 的 CastUtils.cast 方法之所以这么强悍，也是采用了类型自动转化方法。

5. 利用 doReturn-when 语句代替 when-thenReturn 语句

Mockito 的 when-thenReturn 语句需要对返回类型强制校验，而 doReturn-when 语句不会对返回类型强制校验。利用这个特性，可以利用 doReturn-when 语句代替 when-thenReturn 语句解决类型转换警告。

问题代码：

```
List<String> valueList = Mockito.mock(List.class);
Mockito.when(listOperations.range(KEY, start, end)).thenReturn(valueList);
```

建议代码：

```
List<?> valueList = Mockito.mock(List.class);
Mockito.doReturn(valueList).when(listOperations).range(KEY, start, end);
```

6. 利用 Whitebox.invokeMethod 方法代替 Method.invoke 方法

JDK 提供的 Method.invoke 方法返回的是 Object 类型，转化为具体类型时需要强制转换，会产生类型转换警告。而 PowerMock 提供的 Whitebox.invokeMethod 方法返回类型可以自动转化，不会产生类型转换警告

问题代码：

```
Method method = PowerMockito.method(UserService.class, "getScoreMap");
Map<Long, Double> scoreMap = (Map<Long, Double>)method.invokeMethod(userService);
```

建议代码：

```
Map<Long, Double> scoreMap = Whitebox.invokeMethod(userService, "getScoreMap");
```

7. 利用 instanceof 关键字

在具体类型强制转换时，建议利用 instanceof 关键字先判断类型，否则会产生类型转换警告。

问题代码：

```
JSONArray jsonArray = (JSONArray)object;
...
```

建议代码：

```
if (object instanceof JSONArray) {
    JSONArray jsonArray = (JSONArray)object;
    ...
}
```

8. 利用 Class.cast 方法

在泛型类型强制转换时，会产生类型转换警告。可以采用泛型类的 cast 方法转换，从而避免产生类型转换警告。

问题代码：

```
public static <V> V parseValue(String text, Class<V> clazz) {
    if (Objects.equals(clazz, String.class)) {
        return (V)text;
    }
    return JSON.parseObject(text, clazz);
}
```

建议代码：

```
public static <V> V parseValue(String text, Class<V> clazz) {
    if (Objects.equals(clazz, String.class)) {
        return clazz.cast(text);
    }
    return JSON.parseObject(text, clazz);
}
```

9. 避免不必要的类型转换

有时候，没有必要进行类型转换，就尽量避免类型转换。比如：把 Object 类型转换为具体类型，但又把具体类型当 Object 类型使用，就没有必要进行类型转换。像这种情况，可以合并表达式或定义基类变量，从而避免不必要的类型转化。

问题代码：

```
Boolean isSupper = (Boolean)method.invokeMethod(userService, userId);
Assert.assertEquals("期望值不为真", Boolean.TRUE, isSupper);

List<UserVO> userList = (Map<Long, Double>)method.invokeMethod(userService, companyId);
Assert.assertEquals("期望值不一致", expectedJson, JSON.toJSONString(userList));
```

建议代码：

```
Assert.assertEquals("期望值不为真", Boolean.TRUE, method.invokeMethod(userService, userId));

Object userList = method.invokeMethod(userService, companyId);
Assert.assertEquals("期望值不一致", expectedJson, JSON.toJSONString(userList));
```

后记

《满江红·卓越工程》

夜色朦胧，高楼里、灯如白昼。
揉困眼，仰天长吼，又熬一宿。
美好生活仍未见，称心工作何时有？
累成狗、秃了少年头，人消瘦。
编单测，纠错谬。
核代码，查疏漏。
讲方法，打破加班魔咒。
规范流程提效率，工程实践争优秀。
从此后、阔步向前行，精神抖。

——谨以此词献给集团的《卓越工程》项目。

Java 单元测试技巧之 JSON 序列化

前言

《论语》中孔子有言：“工欲善其事，必先利其器。”

今年 7 月，作者希望迎接更大的挑战，从高德地图数据转岗到共享出行后，接手并维护了几个 Java 后端项目。在熟悉业务和代码的过程中，快速地对原有项目进行单元测试用例的补充，使其单元测试覆盖率达到 70%+甚至于 100%。有同事问我：“你写单元测试为什么这么快？”我微微一笑：“工欲善其事，必先利其器。而我快速编写 Java 单元测试用例的技巧就是——JSON 序列化。”

是的，做任何事情，都要讲究方式方法；只要方式方法对了，就会事半功倍。这里，作者系统性地总结了 JSON 序列化在编写 Java 单元测试用例中的使用技巧，希望能够让大家“读有所得、得有所思、思有所获”。

一、 冗长的单元测试代码

在编写单元测试用例的过程中，经常会出现以下冗长的单元测试代码。

1. 冗长的数据模拟代码

1) 模拟类属性值

在模拟类属性值时，会遇到以下的冗长代码：

```
Map<Long, String> languageMap = new HashMap<>(MapHelper.DEFAULT);
languageMap.put(1L, "Java");
languageMap.put(2L, "C++");
languageMap.put(3L, "Python");
languageMap.put(4L, "JavaScript");
... // 约几十行
Whitebox.setInternalState(developmentService, "languageMap", languageMap);
```

2) 模拟方法参数值

在模拟方法参数值时，会遇到以下的冗长代码：

```
List<UserCreateVO> userCreateList = new ArrayList<>();
UserCreateVO userCreate0 = new UserCreateVO();
userCreate0.setName("Changyi");
userCreate0.setTitle("Java Developer");
... // 约几十行
userCreateList.add(userCreate0);
UserCreateVO userCreate1 = new UserCreateVO();
userCreate1.setName("Tester");
userCreate1.setTitle("Java Tester");
... // 约几十行
userCreateList.add(userCreate1);
... // 约几十条
userService.batchCreate(userCreateList);
```

3) 模拟方法返回值

在模拟方法返回值时，会遇到以下的冗长代码：

```
Long companyId = 1L;
List<UserDO> userList = new ArrayList<>();
UserDO user0 = new UserDO();
user0.setId(1L);
user0.setName("Changyi");
user0.setTitle("Java Developer");
... // 约几十行
userList.add(user0);
UserDO user1 = new UserDO();
user1.setId(2L);
user1.setName("Tester");
user1.setTitle("Java Tester");
... // 约几十行
userList.add(user1);
... // 约几十条
Mockito.doReturn(userList).when(userDAO).queryByCompanyId(companyId);
```

2. 冗长的数据验证代码

1) 验证方法返回值

在验证方法返回值时，会遇到以下的冗长代码：

```
Long companyId = 1L;
List<UserVO> userList = userService.queryByCompanyId(companyId);
UserVO user0 = userList.get(0);
Assert.assertEquals("name不一致", "Changyi", user0.getName());
Assert.assertEquals("title不一致", "Java Developer", user0.getTitle());
... // 约几十行
UserVO user1 = userList.get(1);
Assert.assertEquals("name不一致", "Tester", user1.getName());
Assert.assertEquals("title不一致", "Java Tester", user1.getTitle());
... // 约几十行
... // 约几十条
```

2) 验证方法参数值

在验证方法参数值时，会遇到以下的冗长代码：

```
ArgumentCaptor<List<UserDO>> userCreateListCaptor =
CastUtils.cast(ArgumentCaptor.forClass(List.class));
Mockito.verify(userDAO).batchCreate(userCreateListCaptor.capture());
List<UserDO> userCreateList = userCreateListCaptor.getValue();
UserDO userCreate0 = userCreateList.get(0);
Assert.assertEquals("name不一致", "Changyi", userCreate0.getName());
Assert.assertEquals("title不一致", "Java Developer", userCreate0.getTitle());
... // 约几十行
UserDO userCreate1 = userCreateList.get(1);
Assert.assertEquals("name不一致", "Tester", userCreate1.getName());
Assert.assertEquals("title不一致", "Java Tester", userCreate1.getTitle());
... // 约几十行
... // 约几十条
```

二、 采用 JSON 序列化简化

常言道：“眼见为实，耳听为虚。”下面，就通过 JSON 序列化来简化上面的单元测试用例代码，让大家先睹为快。

1. 简化数据模拟代码

对于数据模拟，首先需要先加载 JSON 资源文件为字符串，然后通过 JSON 反序列化字符串为数据对象，最后用于模拟类属性值、方法参数值和方法返回值。这样，就精简了原来冗长的赋值语句。

1) 模拟类属性值

利用 JSON 反序列化，简化模拟类属性值代码如下：

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "languageMap.json");
Map<Long, String> languageMap = JSON.parseObject(text, new TypeReference<Map<Long, String>>
() {});
Whitebox.setInternalState(mobilePhoneService, "languageMap", languageMap);
```

其中，JSON 资源文件 languageMap.json 的内容如下：

```
{1:"Java",2:"C++",3:"Python",4:"JavaScript"...}
```

2) 模拟方法参数值

利用 JSON 反序列化，简化模拟方法参数值代码如下：

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
List<UserCreateVO> userCreateList = JSON.parseArray(text, UserCreateVO.class);
userService.batchCreate(userCreateList);
```

其中，JSON 资源文件 userCreateList.json 的内容如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

3) 模拟方法返回值

利用 JSON 反序列化，简化模拟方法返回值代码如下：

```
Long companyId = 1L;
String text = ResourceHelper.getResourceAsString(getClass(), path + "userList.json");
List<UserDO> userList = JSON.parseArray(text, UserDO.class);
Mockito.doReturn(userList).when(userDAO).queryByCompanyId(companyId);
```

其中，JSON 资源文件 userList.json 的内容如下：

```
[{"id":1,"name":"Changyi","title":"Java Developer"...},{"id":2,"name":"Tester","title":"Java
Tester"...},...]
```

2. 简化数据验证代码

对于数据验证，首先需要先加载 JSON 资源文件为字符串，然后通过 JSON 序列化数据对象为字符串，最后验证两字符串是否一致。这样，就精简了原来冗长的验证语句。

1) 验证方法返回值

利用 JSON 序列化，简化验证方法返回值代码如下：

```
Long companyId = 1L;
List<UserVO> userList = userService.queryByCompanyId(companyId);
String text = ResourceHelper.getResourceAsString(getClass(), path + "userList.json");
Assert.assertEquals("用户列表不一致", text, JSON.toJSONString(userList));
```

其中，JSON 资源文件 userList.json 的内容如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

2) 验证方法参数值

利用 JSON 序列化，简化验证方法参数值代码如下：

```
ArgumentCaptor<List<UserDO>> userCreateListCaptor =
CastUtils.cast(ArgumentCaptor.forClass(List.class));
Mockito.verify(userDAO).batchCreate(userCreateListCaptor.capture());
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
Assert.assertEquals("用户创建列表不一致", text,
JSON.toJSONString(userCreateListCaptor.getValue()));
```

其中，JSON 资源文件 userCreateList.json 的内容如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

三、 测试用例及资源命名

俗话说：“没有规矩，不成方圆。”所以，为了更好地利用 JSON 序列化技巧，首先对测试用例和资源文件进行规范化命名。

1. 测试类命名

按照行业惯例，测试类的命名应以被测试类名开头并以 Test 结尾。比如：UserService（用户服务类）的测试类需要命名为 UserServiceTest（用户服务测试类）。

单元测试类应该放在被测试类的同一工程的“src/test/java”目录下，并且要放在被测试类的同一包下。注意，单元测试类不允许写在业务代码目录下，否则在编译时没法过滤这些测试用例。

2. 测试方法命名

按照行业规范,测试方法命名应以test开头并以被测试方法结尾。比如:batchCreate（批量创建）的测试方法需要命名为 testBatchCreate（测试：批量创建），queryByCompanyId（根据公司标识查询）的测试方法需要命名为testQueryByCompanyId（测试：根据公司标识查询）。

当一个方法对应多个测试用例时，就需要创建多个测试方法，原有测试方法命名已经不能满足需求了。有人建议在原有的测试方法命名的基础上，添加 123 等序号表示不同的用例。比如：testBatchCreate1（测试：批量创建 1）、testBatchCreate2（测试：批量创建 2）.....但是，这种方法不能明确每个单元测试的用意。

这里，作者建议在原有的测试方法命名的基础上，添加“With+条件”来表达不同的测试用例方法。

1) 按照结果命名

- testBatchCreateWithSuccess（测试：批量创建-成功）；
- testBatchCreateWithFailure（测试：批量创建-失败）；
- testBatchCreateWithException（测试：批量创建-异常）；

2) 按照参数命名

- testBatchCreateWithListNull（测试：批量创建-列表为 NULL）；
- testBatchCreateWithListEmpty（测试：批量创建-列表为空）；
- testBatchCreateWithListNotEmpty（测试：批量创建-列表不为空）；

3) 按照意图命名

- testBatchCreateWithNormal（测试：批量创建-正常）；
- testBatchCreateWithGray（测试：批量创建-灰度）；
- testBatchCreateWithException（测试：批量创建-异常）；

当然,还有形成其它的测试方法命名方式,也可以把不同的测试方法命名方式混用,只要能清楚地表达出这个测试用例的涵义即可。

3. 测试类资源目录命名

这里,作者建议的资源目录命名方式为——以 test 开头且以被测试类名结尾。比如:UserService(用户服务类)的测试资源目录可以命名为 testUserService。

那么,这个资源目录应该放在哪儿了?作者提供了 2 个选择:

- 放在“src/test/java”目录下,跟测试类放在同一目录下——这是作者最喜欢的方式;
- 放在“src/test/resources”目录下,跟测试类放在同一目录下——建议 IDEA 用户采用这种方式。

4. 测试方法资源目录命名

在前面的小节中,我们针对测试方法进行了规范命名。这里,我们可以直接拿来使用——即用测试方法名称来命名测试目录。当然,这些测试方法资源目录应该放在测试类资源目录下。比如:测试类 UserServiceTest(用户服务测试类)的测试方法 testBatchCreateWithSuccess(测试:批量创建-成功)的测试资源目录就是 testUserService/testBatchCreateWithSuccess。

另外,也可以采用“测试方法名称”+“测试条件名称”二级目录的命名方式。比如:测试类 UserServiceTest(用户服务测试类)的测试方法 testBatchCreateWithSuccess(测试:批量创建-成功)的测试资源目录就是 testUserService/testBatchCreate/success。

这里,作者首推的是第一种方式,因为测试方法名称和资源目录名称能够保持一致。

5. 测试资源文件命名

在被测试代码中,所有参数、变量都已经有了命名。所以,建议优先使用这些参数和变量的名称,并加后缀“.json”标识文件格式。如果这些资源文件名称冲突,可

以添加前缀以示区分，比如：userCreateList 的资源文件名称为“userCreateList.json”。

另外，在测试用例代码中，把这些测试资源文件加载后，反序列化为对应的数据对象，这些数据对象的变量名称也应该跟资源文件名称保持一致。

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
List<UserCreateVO> userCreateList = JSON.parseArray(text, UserCreateVO.class);
userService.batchCreate(userCreateList);
```

6. 测试资源文件存储

在测试资源目录和名称定义好之后，就需要存入测试资源文件了。存储方式总结如下：

- 如果是测试类下所有测试用例共用的资源文件，建议存储在测试类资源目录下，比如：testUserService；
- 如果是测试用例独有的资源文件，建议存储在测试方法资源目录下，比如：testUserService/testBatchCreateWithSuccess；
- 如果是某一被测方法所有的测试用例共用的资源文件，建议存储在不带任何修饰的测试方法资源目录下，比如：testUserService/testBatchCreate；
- 如果测试类资源目录下只有一个测试方法资源目录，可以去掉这个测试方法资源目录，把所有资源文件存储在测试类资源目录下。

注意：
这里的资源文件不光是 JSON 资源文件，但也可以是其它类型的资源文件。

7. 文件名称过长

由于资源目录名称较长（大概超过 50 个字符），可能会导致 git 检出代码时出现以下错误：

```
git checkout develop
error: xxx/xxx: Filename too long
```

或者，在添加文件时出现以下错误：

```
git add .
error: open("xxx/xxx"): Filename too long
error: unable to index file 'xxx/xxx'
fatal: adding files failed
```

可以通过以下 git 设置参数解决：

```
git config --system core.longpaths true
```

当然，测试用例名称和资源目录名称没必要太长，可以进行一些精简使其小于等于 50 个字符。

8. JSON 资源文件格式

关于 JSON 资源文件是否格式化的建议：不要格式化 JSON 资源文件内容，否则会占用更多的代码行数，还会导致无法直接进行文本比较。

四、 测试资源使用案例

在上一章中，讲了测试用例和资源的命名规则以及存放方式。但是，只是文字的描述，没有什么体感。所有，这一章将举例一个完整的案例来实际说明。

1. 被测案例代码

以 UserService 的 createUser 方法为例说明：

```
/**
 * 用户服务类
 */
@Service
public class UserService {

    /**
 服务相关 */
    /** 用户 DAO */
    @Autowired
    private UserDAO userDAO;
    /** 标识生成器 */
    @Autowired
    private IdGenerator idGenerator;

    /** 参数相关 */
```

```
    /** 可以修改 */
    @Value("${userService.canModify}")
    private Boolean canModify;

    /**
     * 创建用户
     *
     * @param userCreate 用户创建
     * @return 用户标识
     */
    public Long createUser(UserVO userCreate) {
        // 获取用户标识
        Long userId = userDAO.getIdByName(userCreate.getName());

        // 根据存在处理
        // 根据存在处理：不存在则创建
        if (Objects.isNull(userId)) {
            userId = idGenerator.next();
            UserDO userCreateDO = new UserDO();
            userCreateDO.setId(userId);
            userCreateDO.setName(userCreate.getName());
            userDAO.create(userCreateDO);
        }
        // 根据存在处理：已存在可修改
        else if (Boolean.TRUE.equals(canModify)) {
            UserDO userModifyDO = new UserDO();
            userModifyDO.setId(userId);
            userModifyDO.setName(userCreate.getName());
            userDAO.modify(userModifyDO);
        }
        // 根据存在处理：已存在禁修改
        else {
            throw new UnsupportedOperationException("不支持修改");
        }

        // 返回用户标识
        return userId;
    }

}
```

2. 测试用例代码

编写完整的测试用例如下：

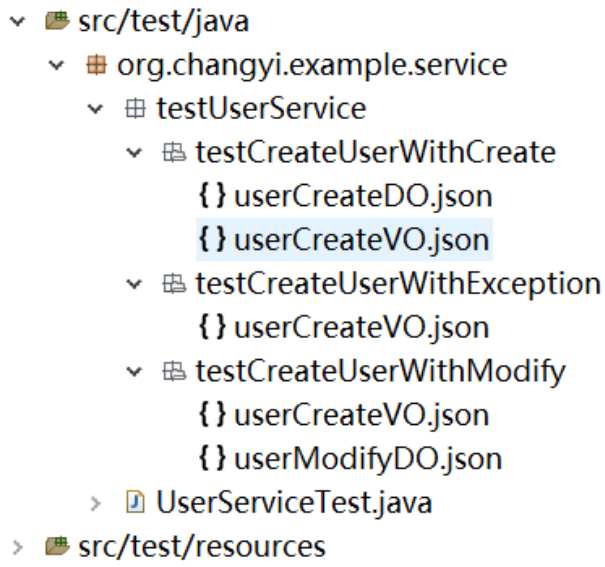
```java
/**
 * 用户服务测试类
 */
@RunWith(PowerMockRunner.class)
public class UserServiceTest {

    /** 模拟依赖对象 */
    /** 用户DAO */
    @Mock
    private UserDAO userDAO;
    /** 标识生成器 */
    @Mock
    private IdGenerator idGenerator;

    /** 定义测试对象 */
    /** 用户服务 */
    @InjectMocks
    private UserService userService;

    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH = "testUserService/";

    /**
     * 在测试之前
     */
    @Before
    public void beforeTest() {
        // 注入依赖对象
        Whitebox.setInternalState(userService, "canModify", Boolean.TRUE);
    }

    /**
     * 测试：创建用户-创建
     */
    /**
     * 测试：创建用户-创建
     */
    @Test
    public void testCreateUserWithCreate() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getByName
        Mockito.doReturn(null).when(userDAO).getIdByName(Mockito.anyString());
        // 模拟依赖方法：idGenerator.next
        Long userId = 1L;
        Mockito.doReturn(userId).when(idGenerator).next();

        // 调用测试方法
        String path = RESOURCE_PATH + "testCreateUserWithCreate/";
        String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateVO.json");
        UserVO userCreate = JSON.parseObject(text, UserVO.class);
        Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreate));

        // 验证依赖方法
        // 验证依赖方法：userDAO.getByName
        Mockito.verify(userDAO).getIdByName(userCreate.getName());
        // 验证依赖方法：idGenerator.next
        Mockito.verify(idGenerator).next();
        // 验证依赖方法：userDAO.create
        ArgumentCaptor<UserDO> userCreateCaptor = ArgumentCaptor.forClass(UserDO.class);
        Mockito.verify(userDAO).create(userCreateCaptor.capture());
        text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateDO.json");
        Assert.assertEquals("用户创建不一致", text, JSON.toJSONString(userCreateCaptor.getValue()));

        // 验证依赖对象
        // 验证依赖对象：idGenerator, userDAO
        Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
```

```
    // 验证依赖对象
    // 验证依赖对象：idGenerator, userDAO
    Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
}

/**
 * 测试：创建用户-修改
 */
@Test
public void testCreateUserWithModify() {
    // 模拟依赖方法
    // 模拟依赖方法：userDAO.getByName
    Long userId = 1L;
    Mockito.doReturn(userId).when(userDAO).getIdByName(Mockito.anyString());

    // 调用测试方法
    String path = RESOURCE_PATH + "testCreateUserWithModify/";
    String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateVO.json");
    UserVO userCreate = JSON.parseObject(text, UserVO.class);
    Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreate));

    // 验证依赖方法
    // 验证依赖方法：userDAO.getByName
    Mockito.verify(userDAO).getIdByName(userCreate.getName());
    // 验证依赖方法：userDAO.modify
    ArgumentCaptor<UserDO> userModifyCaptor = ArgumentCaptor.forClass(UserDO.class);
    Mockito.verify(userDAO).modify(userModifyCaptor.capture());
    text = ResourceHelper.getResourceAsString(getClass(), path + "userModifyDO.json");
    Assert.assertEquals("用户修改不一致", text, JSON.toJSONString(userModifyCaptor.getValue()));

    // 验证依赖对象
    // 验证依赖对象
    // 验证依赖对象：idGenerator
    Mockito.verifyZeroInteractions(idGenerator);
    // 验证依赖对象：userDAO
    Mockito.verifyNoMoreInteractions(userDAO);
}

/**
 * 测试：创建用户-异常
 */
@Test
public void testCreateUserWithException() {
    // 注入依赖对象
    Whitebox.setInternalState(userService, "canModify", Boolean.FALSE);

    // 模拟依赖方法
    // 模拟依赖方法：userDAO.getByName
    Long userId = 1L;
    Mockito.doReturn(userId).when(userDAO).getIdByName(Mockito.anyString());

    // 调用测试方法
    String path = RESOURCE_PATH + "testCreateUserWithException/";
    String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateVO.json");
    UserVO userCreate = JSON.parseObject(text, UserVO.class);
    UnsupportedOperationException exception = Assert.assertThrows("返回异常不一致",
        UnsupportedOperationException.class, () -> userService.createUser(userCreate));
    Assert.assertEquals("异常消息不一致", "不支持修改", exception.getMessage());

    // 验证依赖方法
    // 验证依赖方法：userDAO.getByName
    Mockito.verify(userDAO).getIdByName(userCreate.getName());

    // 验证依赖对象
    // 验证依赖对象：idGenerator
    Mockito.verifyZeroInteractions(idGenerator);
```

```
        // 验证依赖对象
        // 验证依赖对象：idGenerator
        Mockito.verifyZeroInteractions(idGenerator);
        // 验证依赖对象：userDAO
        Mockito.verifyNoMoreInteractions(userDAO);
    }

}
```

3. 资源文件目录

测试用例所涉及的资源文件目录如下：



其中，资源文件内容比较简单，这里就不再累述了。

4. POM 文件配置

根项目的 pom.xml 文件需要做以下配置：

```
<?xml version="1.0" encoding="UTF-8" ?>
<project                              xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
    ...

    <!-- 属性管理 -->
    <properties>
        ...
        <junit.version>4.13.1</junit.version>
        <mockito.version>3.3.3</mockito.version>
        <powermock.version>2.0.9</powermock.version>
    </properties>
```

```
    <!-- 依赖管理 -->
    <dependencyManagement>
        <dependencies>
            ...
            <!-- PowerMock -->
            <dependency>
                <groupId>junit</groupId>
                <artifactId>junit</artifactId>
                <version>${junit.version}</version>
                <scope>test</scope>
            </dependency>
            <dependency>
                <groupId>org.mockito</groupId>
                <artifactId>mockito-core</artifactId>
                <version>${mockito.version}</version>
                <scope>test</scope>
            </dependency>
            <dependency>
                <groupId>org.powermock</groupId>
                <artifactId>powermock-module-junit4</artifactId>
                <version>${powermock.version}</version>
                <scope>test</scope>
            </dependency>
            <dependency>
                <groupId>org.powermock</groupId>
                <artifactId>powermock-api-mockito2</artifactId>
                <version>${powermock.version}</version>
                <scope>test</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <!-- 构建管理 -->
    <build>
        <pluginManagement>
            <plugins>
                ...
                <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                    <artifactId>maven-resources-plugin</artifactId>
                    <version>2.6</version>
                    <executions>
                        ...
                        <execution>
                            <id>copy-test-resources</id>
                            <phase>compile</phase>
                            <goals>
                                <goal>copy-resources</goal>
                            </goals>
                            <configuration>
                                <encoding>UTF-8</encoding>
                                <outputDirectory>${project.build.directory}/test-
classes</outputDirectory>
                                <resources>
                                    <resource>
                                        <directory>src/test/java</directory>
                                        <includes>
                                            <include>**/*.txt</include>
                                            <include>**/*.csv</include>
                                            <include>**/*.json</include>
                                            <include>**/*.properties</include>
                                        </includes>
                                    </resource>
                                    <resource>
                                        <directory>src/test/resources</directory>
                                        <includes>
                                            <include>**/*.txt</include>
```

```
                                    <include>**/*.csv</include>
                                    <include>**/*.json</include>
                                    <include>**/*.properties</include>
                                </includes>
                            </resource>
                        </resources>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</pluginManagement>
</build>
</project>
```

简要说明如下：

- 在属性配置中，配置了单元测试所依赖的包版本；
- 在依赖配置中，配置了单元测试所依赖的包名称；
- 在构建配置中，配置了编译时需要拷贝目录下的资源文件（如果有其它的资源文件格式，需要在 pom 中配置添加）。

5. 工具类代码

在上面单元测试用例中，需要使用到一个工具类 ResourceHelper（资源赋值类），代码如下：

```java
/**
 * 资源辅助类
 */
public final class ResourceHelper {

    /**
     * 构造方法
     */
    private ResourceHelper() {
        throw new UnsupportedOperationException();
    }

    /**
     * 以字符串方式获取资源
     *
     * @param clazz 类
     * @param name 资源名称
     * @return 字符串
     */
    public static <T> String getResourceAsString(Class<T> clazz, String name) {
        try (InputStream is = clazz.getResourceAsStream(name)) {
            return IOUtils.toString(is, StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new RuntimeException(String.format("以字符串方式获取资源(%s)异常", name), e);
        }
    }

}
```

如果在集团内部，也可以直接导入作者提供的二方工具包：

```
<dependency>
  <groupId>com.amap</groupId>
  <artifactId>aostools-util</artifactId>
  <version>1.0.1</version>
</dependency>
```

五、 JSON 资源文件的来源

JSON 资源文件来源方式很多，作者根据实际操作经验，总结出以下几种以供大家参考。

1. 来源于自己组装

直接利用 JSON 编辑器或者纯文本编辑器，自己一个字段一个字段地编写 JSON 资源数据。

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

注意：

这种方式容易出现 JSON 格式错误及字符串转义问题。

2. 来源于代码生成

做为程序员，能够用程序生成 JSON 资源数据，就绝不手工组装 JSON 资源数据。下面，便是利用 Fastjson 的 JSON.toJSONString 方法生成 JSON 资源数据。

```
public static void main(String[] args) {
    List<UserCreateVO> userCreateList = new ArrayList<>();
    UserCreateVO userCreate0 = new UserCreateVO();
    userCreate0.setName("Changyi");
    userCreate0.setTitle("Java Developer");
    ... // 约几十行
    userCreateList.add(userCreate0);
    UserCreateVO userCreate1 = new UserCreateVO();
    userCreate1.setName("Tester");
    userCreate1.setTitle("Java Tester");
    ... // 约几十行
    userCreateList.add(userCreate1);
    ... // 约几十条
    System.out.println(JSON.toJSONString(userCreateList));
}
```

执行该程序后，生成的 JSON 资源数据如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

注意：

这种方式能够避免 JSON 格式错误及字符串转义问题。

3. 来源于线上日志

如果是事后补充单元测试，首先想到的就是利用线上日志。比如：

```
2021-08-31 18:55:40,867 INFO [UserService.java:34] - 根据公司标识(1)查询所有用户:
[{"id":1,"name":"Changyi","title":"Java Developer"...},{"id":2,"name":"Tester","title":"Java
Tester"...},...]
```

从上面的日志中，我们可以得到方法 userDAO.queryByCompanyId 的请求参数 companyId 取值为“1”，返回结果为：

```
"[{"id":1,"name":"Changyi","title":"Java
Developer"…},{"id":2,"name":"Tester","title":"Java Tester"…},…]"
```

注意：

要想得到现成的 JSON 资源数据，就必须输出完整的 JSON 数据内容。但是，由于 JSON 数据内容过大，一般不建议全部输出。所以，从线上日志中也不一定能够拿到现成的 JSON 资源数据。

4. 来源于集成测试

集成测试，就是把整个或部分项目环境运行起来，能够连接数据库、Redis、MetaQ、HSF 等所依赖的第三方服务环境，然后测试某一个方法的功能是否能够达到预期。

```
/**
 * 用户DAO测试类
 */
@Slf4j
@RunWith(PandoraBootRunner.class)
@DelegateTo(SpringJUnit4ClassRunner.class)
@SpringBootTest(classes = {ExampleApplication.class})
public class UserDaoTest {

    /** 用户DAO */
    @Resource
    private UserDAO userDAO;

    /**
     * 测试：根据公司标识查询
     */
    @Test
    public void testQueryByCompanyId() {
        Long companyId = 1L;
        List<UserDO> userList = userDAO.queryByCompanyId(companyId);
        log.info("userList={}", JSON.toJSONString(userList));
    }

}
```

执行上面集成测试用例，输出的日志内容如下：

```
2021-08-31 18:55:40,867 INFO [UserDaoTest.java:24] - userList=
[{"id":1,"name":"Changyi","title":"Java Developer"...},{"id":2,"name":"Tester","title":"Java
Tester"...},...]
```

上面日志中，userList 后面的就是我们需要的 JSON 资源数据。

我们也可以用集成测试得到方法内部的方法调用的参数值和返回值，具体方法如下：

- 首先，在源代码中添加日志输出语句；
- 然后，执行单元测试用例，得到对应的方法调用参数值和返回值；
- 最后，删除源代码中日志输出语句，恢复源代码为原来的样子。

5. 来源于测试过程

有一些数据，是由被测方法生成的，比如：方法返回值和调用参数。针对这类数据，可以在测试过程中生成，然后逐一进行数据核对，最后整理成 JSON 资源文件。

被测方法：

```
public void batchCreate(List<UserCreate> createList) {
    List<UserDO> userList = createList.stream()
        .map(UserService::convertUser).collect(Collectors.toList());
    userDAO.batchCreate(userList);
}
```

测试用例：

```
@Test
public void testBatchCreate() {
    // 调用测试方法
    List<UserCreate> createList = ...;
    userService.batchCreate(createList);

    // 验证测试方法
    ArgumentCaptor<List<UserDO>> userListCaptor =
CastUtils.cast(ArgumentCaptor.forClass(List.class));
    Mockito.verify(userDAO).batchCreate(userListCaptor.capture());
    Assert.assertEquals("用户列表不一致", "", JSON.toJSONString(userListCaptor.getValue()));
}
```

执行单元测试后，提示以下问题：

```
org.junit.ComparisonFailure: 用户列表不一致 expected:<[]> but was:
<[[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]]>
```

上面的错误信息中，后面括号中的就是我们需要需要的 JSON 资源数据。

注意：

一定要进行数据核对，这有可能是错误代码生成的错误数据。用错误数据去验证生成它的代码，当然不会测试出其中的问题。

六、 JSON 序列化技巧

这里以 Fastjson 为例，介绍一些 JSON 序列化技巧。

1. 序列化对象

利用 JSON.toJSONString 方法序列化对象：

```
UserVO user = ...;
String text = JSON.toJSONString(user);
```

2. 序列化数组

利用 JSON.toJSONString 方法序列化数组：

```
UserVO[] users = ...;
String text = JSON.toJSONString(users);
```

3. 序列化集合

利用 JSON.toJSONString 方法序列化集合（继承至 Collection，比如 List、Set 等集合）：

```
List<UserVO> userList = ...;
String text = JSON.toJSONString(userList);
```

4. 序列化映射

利用 JSON.toJSONString 方法序列化映射：

```
Map<Long, UserVO> userMap = ...;
String text = JSON.toJSONString(userMap, SerializerFeature.MapSortField);
```

其中，为了保证每次序列化的映射字符串一致，需要指定序列化参数 MapSortField 进行排序。

5. 序列化模板对象

利用 JSON.toJSONString 方法序列化模板对象：

```
Result<UserVO> result = ...;
String text = JSON.toJSONString(result);
```

6. 序列化指定属性字段

利用 JSON.toJSONString 方法序列化指定属性字段，主要通过设置属性预过滤器（SimplePropertyPreFilter）的包含属性字段列表（includes）实现。主要应用于只想验证某些字段的情况，比如只验证跟测试用例有关的字段。

指定所有类的属性字段：

利用 JSON.toJSONString 方法序列化指定所有类的属性字段：

```
UserVO user = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter();
filter.getIncludes().addAll(Arrays.asList("id", "name"));
String text = JSON.toJSONString(user, filter);
```

指定单个类的属性字段：

利用 JSON.toJSONString 方法序列化指定单个类的属性字段：

```
List<UserVO> userList = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter(UserVO.class);
filter.getIncludes().addAll(Arrays.asList("id", "name"));
String text = JSON.toJSONString(userList, filter);
```

指定多个类的属性字段：

利用 JSON.toJSONString 方法序列化指定多个类的属性字段：

```
Pair<UserVO, CompanyVO> userCompanyPair = ...;
SimplePropertyPreFilter userFilter = new SimplePropertyPreFilter(UserVO.class);
userFilter.getUncludes().addAll(Arrays.asList("id", "name"));
SimplePropertyPreFilter companyFilter = new SimplePropertyPreFilter(CompanyVO.class);
companyFilter.getIncludes().addAll(Arrays.asList("id", "name"));
String text = JSON.toJSONString(userCompanyPair, new SerializeFilter[]{userFilter,
companyFilter});
```

7. 序列化字段排除属性字段

利用 JSON.toJSONString 方法序列化过滤属性字段，主要通过设置属性预过滤器（SimplePropertyPreFilter）的排除属性字段列表（excludes）实现。主要应用于不想验证某些字段的情况，比如排除无法验证的随机属性字段。

排除所有类的属性字段：

利用 JSON.toJSONString 方法序列化排除所有类的属性字段：

```
UserVO user = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter();
filter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
String text = JSON.toJSONString(user, filter);
```

排除单个类的属性字段：

利用 JSON.toJSONString 方法序列化排除单个类的属性字段：

```
List<UserVO> userList = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter(UserVO.class);
filter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
String text = JSON.toJSONString(userList, filter);
```

排除多个类的属性字段：

利用 JSON.toJSONString 方法序列化排除多个类的属性字段：

```
Pair<UserVO, CompanyVO> userCompanyPair = ...;
SimplePropertyPreFilter userFilter = new SimplePropertyPreFilter(UserVO.class);
userFilter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
SimplePropertyPreFilter companyFilter = new SimplePropertyPreFilter(CompanyVO.class);
companyFilter.getExcludes().addAll(Arrays.asList("createTime", "modifyTime"));
String text = JSON.toJSONString(userCompanyPair, new SerializeFilter[]{userFilter,
companyFilter});
```

8. 自定义序列化

对应一些类对象，需要序列化为特殊格式文本，就必须自定义序列化器。比如：Geometry 序列化文本，通常采用 WKT（Well-known text）表示，便于用户快速阅读理解。

全局配置序列化器：

通过 JSON 序列化全局配置指定类序列化器：

```
Geometry geometry = ...;
SerializeConfig.getGlobalInstance().put(Geometry.class, new GeometrySerializer());
String text = JSON.toJSONString(geometry);
```

注意：
这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的序列化器。

特定配置序列化器：

通过 JSON 序列化特定配置指定类序列化器：

```
Geometry geometry = ...;
SerializeConfig config = new SerializeConfig();
config.put(Geometry.class, new GeometrySerializer());
String text = JSON.toJSONString(geometry, config);
```

注意：
这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的序列化器。

注解配置序列化器：

通过 JSON 序列化注解配置指定类序列化器：

```
public class User {
    ...
    @JSONField(serializeUsing = GeometrySerializer.class)
    private Geometry location;
    ...
}

User user = ...;
String text = JSON.toJSONString(user);
```

其中：GeometrySerializer 为自定义类，这里就不贴出具体实现了。

七、 JSON 反序列化技巧

这里以 Fastjson 为例，介绍一些 JSON 反序列化技巧。

1. 反序列化对象

利用 JSON.parseObject 方法反序列化对象：

```
String text = ...;
UserVO user = JSON.parseObject(text, UserVO.class);
```

2. 反序列化数组

利用 JSON.parseObject 方法反序列化数组：

```
String text = ...;
UserVO[] users = JSON.parseObject(text, UserVO[].class);
```

3. 反序列化集合

利用 JSON.parseArray 方法反序列化列表：

```
String text = ...;
List<UserVO> userList = JSON.parseArray(text, UserVO.class);
```

利用 JSON.parseObject 方法反序列化集合：

```
String text = ...;
Set<UserVO> userSet = JSON.parseObject(text, new TypeReference<Set<UserVO>>() {});
```

4. 反序列化映射

利用 JSON.parseObject 方法反序列化映射：

```
String text = ...;
Map<Long, UserVO> userList = JSON.parseObject(text, new TypeReference<Map<Long, UserVO>>()
{});
```

注意：

如果映射的 key 是复杂类型，这种方法反序列会报格式错误，需要自定义反序列化器。

5. 反序列化模板对象

利用 JSON.parseObject 方法反序列化模板对象：

```
String text = ...;
Result<UserVO> result = JSON.parseArray(text, new TypeReference<Result<UserVO>>() {});
```

6. 反序列化非公有字段

由于某些属性字段没有公有设置方法，或者以字段名称作为公有设置方法。当需要反序列化这些属性字段时，需要指定 SupportNonPublicField（支持非公有字段）反序列化参数。

```
String text = ...;
UserVO user = JSON.parseObject(text, UserVO.class, Feature.SupportNonPublicField);
```

7. 反序列化 Builder 模式类

有些同学喜欢用 Builder 模式，导致实体类并没有公有构造方法。当利用 Fastjson 反序列化这些类是，就会出现以下问题：

```
com.alibaba.Fastjson.JSONException: default constructor not found. class com.example.User
```

只要对应的 Builder 类有默认构造方法，就可以采用下面的方式序列化。

```
String text = ...;
User user = JSON.parseObject(text, User.UserBuilder.class,
Feature.SupportNonPublicField).build();
```

首先通过 JSON.parseObject 方法+SupportNonPublicField 参数反序列化 Builder 对象，然后通过 Builder 对象的 build 方法来构造实体对象。

如果对应的 Builder 类没有默认构造方法，或者需要反序列化模板对象时，需要自定义 JSON 反序列化器。

8. 反序列化丢失字段值

Fastjson 支持没有默认构造方法的类的反序列化，但存在丢失字段值的问题。

```
@Getter
@Setter
@ToString
class User {
    private Long id;
    private String name;
    public User(Long id) {
        this.id = id;
    }
}

String text = "{\"id\":123,\"name\":\"test\"}";
User user = JSON.parseObject(text, User.class); // 会丢失name值
```

咨询过 Fastjson 维护人员，目前还没有解决这个 bug，有待后续版本中解决。如果要反序列化这种类，可以考虑添加默认构造方法或自定义反序列化器。

9. 自定义反序列化器

对应一些类对象，需要把特殊格式文本反序列化为对象，就必须自定义反序列化器。比如：Geometry 序列化文本，通常采用 WKT（Well-known text）表示，便于用户快速阅读理解。

1) 全局配置反序列化器

通过 JSON 序列化全局配置指定类反序列化器：

```
String text = ...;
ParserConfig.getGlobalInstance().putDeserializer(Geometry.class, new
GeometryDeserializer());
Geometry geometry = JSON.parseObject(text, Geometry.class);
```

注意：

这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的反序列化器。

2) 特定配置反序列化器

通过 JSON 序列化特定配置指定类反序列化器：

```
String text = ...;
ParserConfig config = new ParserConfig();
config.putDeserializer(Geometry.class, new GeometryDeserializer());
Geometry geometry = JSON.parseObject(text, Geometry.class, config);
```

注意：

这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的反序列化器。

3) 注解配置反序列化器

通过 JSON 序列化注解配置指定类反序列化器：

```
public class User {
    ...
    @JSONField(deserializeUsing = GeometryDeserializer.class)
    private Geometry location;
    ...
}

String text = ...;
User user = JSON.parseObject(text, User.class);
```

其中

GeometryDeserializer 为自定义类，这里就不贴出具体实现了。

八、 不必要的 JSON 序列化

以上章节，都是说 JSON 资源文件在单元测试中如何运用，如何利用 JSON 资源文件把单元测试编写得更优雅。有时候，任何手段都有两面性，过渡依赖 JSON 资源文件测试，也会把单元测试复杂化。这里，作者总结了几个例子以示说明。

1. 完全透传的对象

1) 完全透传的参数对象

在测试方法中，有些参数没有被任何修改，只是完全被透传而已。

被测方法：

```
public void batchCreate(List<UserCreate> createList) {
    userDAO.batchCreate(createList);
}
```

测试用例：

```
@Test
public void testBatchCreate() {
    // 调用测试方法
    List<UserCreate> createList = new ArrayList<>();
    userService.batchCreate(createList);

    // 验证测试方法
    Mockito.verify(userDAO).batchCreate(createList);
}
```

其中，不需要 ArgumentCaptor 去捕获 userDAO.batchCreate 的参数并验证参数值，这里只需要验证 createList 是不是同一个对象即可。

2) 完全透传的返回对象

在测试方法中，有些返回值没有被任何修改，只是完全被透传而已。

被测方法：

```
public List<UserVO> queryByCompanyId(Long companyId) {
    return userDAO.queryByCompanyId(companyId);
}
```

测试用例：

```
@Test
public void testQueryByCondition() {
    // 模拟依赖方法
    Long companyId = 1L;
    List<UserVO> userList = new ArrayList<>();
    Mockito.doReturn(userList).when(userDAO).queryByCompanyId(companyId);

    // 调用测试方法
    Assert.assertEquals("用户列表不一致", userList, userService.queryByCompanyId(companyId));
}
```

其中，userList 对象不需要构造数据，只需要验证是不是同一个对象即可。

2. 完全透传的属性

1) 完全透传的参数值属性

在测试方法中，有些参数值属性没有被任何修改，只是完全被透传而已。

被测方法：

```
public void handleResult(Result<UserVO> result) {
    if (!result.isSuccess()) {
        metaProducer.sendCouponMessage(result.getData());
    }
}
```

测试用例：

```
@Test
public void testHandleResultWithSuccess() {
    // 调用测试方法
    UserVO user = new UserVO();
    Result<UserVO> result = Result.success(user);
    userService.handleResult(result);

    // 验证依赖方法
    Mockito.verify(metaProducer).sendCouponMessage(user);
}
```

其中，user 对象不需要构造数据，只需要验证是不是同一个对象即可。

2) 完全透传的返回值属性

在测试方法中，有些返回值属性没有被任何修改，只是完全被透传而已。

被测方法：

```
public UserVO get(Long userId) {
    Result<UserVO> result = userHsfService.get(userId);
    if (!result.isSuccess()) {
        throw new ExmapleException(String.format("获取用户(%s)失败:%s", userId,
result.getMessage()));
    }
    return result.getData();
}
```

测试用例：

```
@Test
public void testGetWithSuccess() {
    // 模拟依赖方法
    Long userId = 123L;
    UserVO user = UserVO();
    Mockito.doReturn(Result.success(user)).when(userHsfService).get(userId);

    // 调用测试方法
    Assert.assertEquals("用户信息不一致", user, userService.get(userId));
}
```

其中，user 对象不需要构造数据，只需要验证是不是同一个对象即可。

3. 仅用少数字段的对象

1) 仅用少数字段的参数值对象

在测试方法中，有些参数值对象字段虽多，但只会用到其中少数字段。

被测方法：

```
public void create(UserCreate userCreate) {
    Boolean exist = userDAO.existByName(userCreate.getName());
    if (Boolean.TRUE.equals(exist)) {
        throw new ExmapleException(String.format("用户(%s)已存在", userCreate.getName()));
    }
    userDAO.create(userCreate);
}
```

测试用例：

```
@Test
public void testCreateWithException() {
    UserCreate userCreate = new UserCreate();
    userCreate.setName("changyi");
    ExmapleException exception = Assert.assertThrows("异常类型不一致", ExmapleException.class,
() -> userService.create(userCreate));
    Assert.assertEquals("异常消息不一致", String.format("用户(%s)已存在", userCreate.getName()),
exception.getMessage());
}
```

其中，不需要构造参数值 userCreate 的所有属性字段，只需构造使用到的 name 属性字段即可。

2) 仅用少数字段的返回值对象

在测试方法中，有些返回值对象字段虽多，但只会用到其中少数字段。

被测方法：

```
public boolean isVip(Long userId) {
    UserDO user = userDAO.get(userId);
    return VIP_ROLE_ID_SET.contains(user.getRoleId());
}
```

测试用例：

```
@Test
public void testIsVipWithTrue() {
    // 模拟依赖方法
    Long userId = 123L;
    UserDO user = new UserDO();
    user.setRoleId(VIP_ROLE_ID);
    Mockito.doReturn(user).when(userDAO).get(userId);

    // 调用测试方法
    Assert.assertTrue("返回值不为真", userService.isVip());
}
```

其中，不需要构造返回值 user 的所有属性字段，只需构造使用到的 roleId 属性字段即可。

4. 使用 new 还是 mock 初始化对象？

在上面案例中，我们都采用 new 来初始化对象并采用 set 来模拟属性值的。有些同学会问，为什么不采用 mock 来初始化对象、用 doReturn-when 来模拟属性值？我想说，都是一样的效果，只是前者显得更简洁而已。

关于使用 new 还是 mock 初始化对象，这个问题在网上一直有争论，双方都各有自己的理由。

这里，按照作者的个人使用习惯，进行了简单的归纳总结如下：

使用情形	采用new初始化对象	采用mock初始化对象
实体类	首选	可以
接口类	不可以	首选
虚基类	不可以	首选
简单方法	首选	可以
复杂方法	可以	首选

九、 JSON 结合 Mockito 妙用

上面已经介绍过，JSON 序列化在编写 Java 单元测试用例时最大的妙用有两点：

- JSON 反序列化字符串为数据对象，大大减少了数据对象的模拟代码；
- JSON 序列化数据对象为字符串，把数据对象验证简化为字符串验证，大大减少了数据对象的验证代码。

除此之外，JSON 序列化结合 Mockito，往往会起到意想不到的效果，能产生一些非常巧妙有效的用法。

1. 模拟方法返回多个值

当一个方法需要多次调用，但返回值跟输入参数无关，只跟调用顺序有关的时，可以用数组来模拟方法返回值。先加载一个列表 JSON 资源文件，通过 JSON.parseObject 方法转化为数组，然后利用 Mockito 的 doReturn-when 或 when-thenReturn 语法来模拟方法返回多个值。

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "recordList.json");
Record[] records = JSON.parseObject(text, Record[].class);
Mockito.doReturn(records[0], ArrayUtils.subarray(records, 1,
records.length)).when(recordReader).read();
```

2. 模拟方法返回对应值

当一个方法需要多次调用，但返回值跟调用顺序有关，只能调输入参数有关的时，可以用映射来模拟方法返回值。先加载一个映射 JSON 资源文件，通过 JSON.parseObject 方法转化为映射，然后利用 Mockito 的 doAnswer-when 或

when-thenAnswer 语法来模拟方法返回对应值（根据指定参数返回映射中的对应值）。

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "roleMap.json");
Map<Long, String> roleIdMap = JSON.parseObject(text, new TypeReference<Map<Long, String>>()
{});
Mockito.doAnswer(invocation ->
userMap.get(invocation.getArgument(0))).when(roleService).get(roleId);
```

3. 验证多次方法调用参数

当验证一个方法调用参数时，需要用 ArgumentCaptor 来捕获这个参数，然后通过 getValue 方法验证这个参数。如果这个方法被多次调用，就没有必要依次验证了，可以通过 getAllValues 方法获取一个列表，然后通过 JSON.toJSONString 转化为 JSON 字符串，然后跟 JSON 资源文件进行统一验证。

```
ArgumentCaptor<UserCreateVO> userCreateCaptor = ArgumentCaptor.forClass(UserCreateVO.class);
Mockito.verify(userDAO, Mockito.atLeastOnce()).create(userCreateCaptor.capture());
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
Assert.assertEquals("用户创建列表不一致", text,
JSON.toJSONString(userCreateCaptor.getAllValues()));
```

当然，二者结合的妙用不仅限于此，还有更多作者没有总结到的。

后记

闲暇之余，填词一首自娱自乐，借以致敬广大的码农们：

《卜算子·致码农》

业务要精通，
眼界需开阔。
谈笑风生略几回，
草案心头过。
编写守规则，
测试知结果。
奋笔疾书数万行，
代码无一错。

其中，第二段便说明了代码规范和单元测试的重要性——只有做到“编写守规则，测试知结果”，才能保障“奋笔疾书数万行，代码无一错”。

➢ Java 极客主义系列

Java 代码精简之道

前言

古语有云：

> *道为术之灵，术为道之体；以道统术，以术得道。*

其中：“道”指“规律、道理、理论”，“术”指“方法、技巧、技术”。意思是：“道”是“术”的灵魂，“术”是“道”的实体；可以用“道”来统管“术”，也可以从“术”中获得“道”。

在拜读大佬“孤尽”的文章《Code Review 是苦涩但有意思的修行》时，感受最深的一句话就是：“优质的代码一定是少即是多的精兵原则”，这就是大佬的代码精简之“道”。

工匠追求“术”到极致，其实就是在寻“道”，且离悟“道”也就不远了，亦或是已经得道，这就是“工匠精神”——一种追求“以术得道”的精神。如果一个工匠只满足于“术”，不能追求“术”到极致去悟“道”，那只是一个靠“术”养家糊口的工匠而已。作者根据多年来的实践探索，总结了大量的 Java 代码精简之“术”，试图阐述出心中的 Java 代码精简之“道”。

一、 利用语法

1. 利用三元表达式

普通：

```
String title;
if (isMember(phone)) {
    title = "会员";
} else {
    title = "游客";
}
```

精简：

```
String title = isMember(phone) ? "会员" : "游客";
```

注意：

对于包装类型的算术计算，需要注意避免拆包时的空指针问题。

2. 利用 for-each 语句

从 Java 5 起，提供了 for-each 循环，简化了数组和集合的循环遍历。For-each 循环允许你无需保持传统 for 循环中的索引就可以遍历数组，或在使用迭代器时无需在 while 循环中调用 hasNext 方法和 next 方法就可以遍历集合。

普通：

```
double[] values = ...;
for(int i = 0; i < values.length; i++) {
    double value = values[i];
    // TODO: 处理value
}

List<Double> valueList = ...;
Iterator<Double> iterator = valueList.iterator();
while (iterator.hasNext()) {
    Double value = iterator.next();
    // TODO: 处理value
}
```

精简：

```
double[] values = ...;
for(double value : values) {
    // TODO: 处理value
}

List<Double> valueList = ...;
for(Double value : valueList) {
    // TODO: 处理value
}
```

3. 利用 try-with-resource 语句

所有实现 Closeable 接口的“资源”，均可采用 try-with-resource 进行简化。

普通：

```
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("cities.csv"));
    String line;
    while ((line = reader.readLine()) != null) {
        // TODO: 处理line
    }
} catch (IOException e) {
    log.error("读取文件异常", e);
} finally {
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            log.error("关闭文件异常", e);
        }
    }
}
```

精简：

```
try (BufferedReader reader = new BufferedReader(new FileReader("test.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        // TODO: 处理line
    }
} catch (IOException e) {
    log.error("读取文件异常", e);
}
```

4. 利用 return 关键字

利用 return 关键字，可以提前函数返回，避免定义中间变量。

普通：

```
public static boolean hasSuper(@NonNull List<UserDO> userList) {
    boolean hasSuper = false;
    for (UserDO user : userList) {
        if (Boolean.TRUE.equals(user.getIsSuper())) {
            hasSuper = true;
            break;
        }
    }
    return hasSuper;
}
```

精简：

```
public static boolean hasSuper(@NonNull List<UserDO> userList) {
    for (UserDO user : userList) {
        if (Boolean.TRUE.equals(user.getIsSuper())) {
            return true;
        }
    }
    return false;
}
```

5. 利用 static 关键字

利用 static 关键字，可以把字段变成静态字段，也可以把函数变为静态函数，调用时就无需初始化类对象。

普通：

```
public final class GisHelper {
    public double distance(double lng1, double lat1, double lng2, double lat2) {
        // 方法实现代码
    }
}

GisHelper gisHelper = new GisHelper();
double distance = gisHelper.distance(116.178692D, 39.967115D, 116.410778D, 39.899721D);
```

精简：

```
public final class GisHelper {
    public static double distance(double lng1, double lat1, double lng2, double lat2) {
        // 方法实现代码
    }
}

double distance = GisHelper.distance(116.178692D, 39.967115D, 116.410778D, 39.899721D);
```

6. 利用 lambda 表达式

Java 8 发布以后，lambda 表达式大量替代匿名内部类的使用，在简化了代码的同时，更突出了原有匿名内部类中真正有用的那部分代码。

普通：

```
new Thread(new Runnable() {
    public void run() {
        // 线程处理代码
    }
}).start();
```

精简：

```
new Thread(() -> {
    // 线程处理代码
}).start();
```

7. 利用方法引用

方法引用（::），可以简化 lambda 表达式，省略变量声明和函数调用。

普通：

```
Arrays.sort(nameArray, (a, b) -> a.compareToIgnoreCase(b));
List<Long> userIdList = userList.stream()
    .map(user -> user.getId())
    .collect(Collectors.toList());
```

精简：

```
Arrays.sort(nameArray, String::compareToIgnoreCase);
List<Long> userIdList = userList.stream()
    .map(UserDO::getId)
    .collect(Collectors.toList());
```

8. 利用静态导入

静态导入（import static），当程序中大量使用同一静态常量和函数时，可以简化静态常量和函数的引用。

普通：

```
List<Double> areaList = radiusList.stream().map(r -> Math.PI * Math.pow(r,
2)).collect(Collectors.toList());
...
```

精简：

```
import static java.lang.Math.PI;
import static java.lang.Math.pow;
import static java.util.stream.Collectors.toList;

List<Double> areaList = radiusList.stream().map(r -> PI * pow(r, 2)).collect(toList());
...
```

注意：
静态引入容易造成代码阅读困难，所以在实际项目中应该警慎使用。

9. 利用 unchecked 异常

Java 的异常分为两类：Checked 异常和 Unchecked 异常。Unchecked 异常继承了 RuntimeException，特点是代码不需要处理它们也能通过编译，所以它们称作 Unchecked 异常。利用 Unchecked 异常，可以避免不必要的 try-catch 和 throws 异常处理。

普通：

```
@Service
public class UserService {
    public void createUser(UserCreateVO create, OpUserVO user) throws BusinessException {
        checkOperatorUser(user);
        ...
    }
    private void checkOperatorUser(OpUserVO user) throws BusinessException {
        if (!hasPermission(user)) {
            throw new BusinessException("用户无操作权限");
        }
        ...
    }
    ...
}

@RestController
@RequestMapping("/user")
public class UserController {
    @Autowired
    private UserService userService;

    @PostMapping("/createUser")
    public Result<Void> createUser(@RequestBody @Valid UserCreateVO create, OpUserVO user)
throws BusinessException {
        userService.createUser(create, user);
        return Result.success();
    }
    ...
}
```

精简：

```
@Service
public class UserService {
    public void createUser(UserCreateVO create, OpUserVO user) {
        checkOperatorUser(user);
        ...
    }
    private void checkOperatorUser(OpUserVO user) {
        if (!hasPermission(user)) {
            throw new BusinessRuntimeException("用户无操作权限");
        }
        ...
    }
    ...
}

@RestController
@RequestMapping("/user")
public class UserController {
    @Autowired
    private UserService userService;

    @PostMapping("/createUser")
    public Result<Void> createUser(@RequestBody @Valid UserCreateVO create, OpUserVO user) {
        userService.createUser(create, user);
        return Result.success();
    }
    ...
}
```

二、 利用注解

1. 利用 Lombok 注解

Lombok 提供了一组有用的注解，可以用来消除 Java 类中的大量样板代码。

普通：

```
public class UserVO {
    private Long id;
    private String name;
    public Long getId() {
        return this.id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
    ...
}
```

精简：

```
@Getter
@Setter
@ToString
public class UserVO {
    private Long id;
    private String name;
    ...
}
```

2. 利用 Validation 注解

普通：

```
@Getter
@Setter
@ToString
public class UserCreateVO {
    private String name;
    private Long companyId;
}

@Service
public class UserService {
    public Long createUser(UserCreateVO create) {
        // 验证参数
        if (StringUtils.isBlank(create.getName())) {
            throw new IllegalArgumentException("用户名称不能为空");
        }
        if (Objects.isNull(create.getCompanyId())) {
            throw new IllegalArgumentException("公司标识不能为空");
        }

        // TODO: 创建用户
        return null;
    }
}
```

精简：

```
@Getter
@Setter
@ToString
public class UserCreateVO {
    @NotBlank(message = "用户名称不能为空")
    private String name;
    @NotNull(message = "公司标识不能为空")
    private Long companyId;
    ...
}

@Service
@Validated
public class UserService {
    public Long createUser(@Valid UserCreateVO create) {
        // TODO: 创建用户
        return null;
    }
}
```

3. 利用@NonNull 注解

Spring 的@NonNull 注解，用于标注参数或返回值非空，适用于项目内部团队协作。只要实现方和调用方遵循规范，可以避免不必要的空值判断，这充分体现了阿里的“新六脉神剑”提倡的“因为信任，所以简单”。

普通：

```
public List<UserVO> queryCompanyUser(Long companyId) {
    // 检查公司标识
    if (companyId == null) {
        return null;
    }

    // 查询返回用户
    List<UserDO> userList = userDAO.queryByCompanyId(companyId);
    return userList.stream().map(this::transUser).collect(Collectors.toList());
}

Long companyId = 1L;
List<UserVO> userList = queryCompanyUser(companyId);
if (CollectionUtils.isNotEmpty(userList)) {
    for (UserVO user : userList) {
        // TODO: 处理公司用户
    }
}
```

精简：

```
public @NonNull List<UserVO> queryCompanyUser(@NonNull Long companyId) {
    List<UserDO> userList = userDAO.queryByCompanyId(companyId);
    return userList.stream().map(this::transUser).collect(Collectors.toList());
}

Long companyId = 1L;
List<UserVO> userList = queryCompanyUser(companyId);
for (UserVO user : userList) {
    // TODO: 处理公司用户
}
```

4. 利用注解特性

注解有以下特性可用于精简注解声明：

- 当注解属性值跟默认值一致时，可以删除该属性赋值；
- 当注解只有 value 属性时，可以去掉 value 进行简写；
- 当注解属性组合等于另一个特定注解时，直接采用该特定注解。

普通：

```
@Lazy(true);
@Service(value = "userService")
@RequestMapping(path = "/getUser", method = RequestMethod.GET)
```

精简：

```
@Lazy
@Service("userService")
@GetMapping("/getUser")
```

三、 利用泛型

1. 泛型接口

在 Java 没有引入泛型前，都是采用 Object 表示通用对象，最大的问题就是类型无法强校验并且需要强制类型转换。

普通：

```
public interface Comparable {
    public int compareTo(Object other);
}

@Getter
@Setter
@ToString
public class UserVO implements Comparable {
    private Long id;

    @Override
    public int compareTo(Object other) {
        UserVO user = (UserVO)other;
        return Long.compare(this.id, user.id);
    }
}
```

精简：

```
public interface Comparable<T> {
    public int compareTo(T other);
}

@Getter
@Setter
@ToString
public class UserVO implements Comparable<UserVO> {
    private Long id;

    @Override
    public int compareTo(UserVO other) {
        return Long.compare(this.id, other.id);
    }
}
```

2. 泛型类

普通：

```
@Getter
@Setter
@ToString
public class IntPoint {
    private Integer x;
    private Integer y;
}

@Getter
@Setter
@ToString
public class DoublePoint {
    private Double x;
    private Double y;
}
```

精简：

```
@Getter
@Setter
@ToString
public class Point<T extends Number> {
    private T x;
    private T y;
}
```

3. 泛型方法

普通：

```
public static Map<String, Integer> newHashMap(String[] keys, Integer[] values) {
    // 检查参数非空
    if (ArrayUtils.isEmpty(keys) || ArrayUtils.isEmpty(values)) {
        return Collections.emptyMap();
    }

    // 转化哈希映射
    Map<String, Integer> map = new HashMap<>();
    int length = Math.min(keys.length, values.length);
    for (int i = 0; i < length; i++) {
        map.put(keys[i], values[i]);
    }
    return map;
}
...
```

精简：

```
public static <K, V> Map<K, V> newHashMap(K[] keys, V[] values) {
    // 检查参数非空
    if (ArrayUtils.isEmpty(keys) || ArrayUtils.isEmpty(values)) {
        return Collections.emptyMap();
    }

    // 转化哈希映射
    Map<K, V> map = new HashMap<>();
    int length = Math.min(keys.length, values.length);
    for (int i = 0; i < length; i++) {
        map.put(keys[i], values[i]);
    }
    return map;
}
```

四、 利用自身方法

1. 利用构造方法

构造方法，可以简化对象的初始化和设置属性操作。对于属性字段较少的类，可以自定义构造方法。

普通：

```
@Getter
@Setter
@ToString
public class PageDataVO<T> {
    private Long totalCount;
    private List<T> dataList;
}

PageDataVO<UserVO> pageData = new PageDataVO<>();
pageData.setTotalCount(totalCount);
pageData.setDataList(userList);
return pageData;
```

精简：

```
@Getter
@Setter
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class PageDataVO<T> {
    private Long totalCount;
    private List<T> dataList;
}

return new PageDataVO<>(totalCount, userList);
```

注意：

如果属性字段被替换时，存在构造函数初始化赋值问题。比如把属性字段 title 替换为 nickname，由于构造函数的参数个数和类型不变，原有构造函数初始化语句不会报错，导致把原 title 值赋值给 nickname。如果采用 Setter 方法赋值，编译器会提示错误并要求修复。

2. 利用 Set 的 add 方法

利用 Set 的 add 方法的返回值，可以直接知道该值是否已经存在，可以避免调用 contains 方法判断存在。

普通：

以下案例是进行用户去重转化操作，需要先调用 contains 方法判断存在，后调用 add 方法进行添加。

```
Set<Long> userIdSet = new HashSet<>();
List<UserVO> userVOList = new ArrayList<>();
for (UserDO userDO : userDOList) {
    if (!userIdSet.contains(userDO.getId())) {
        userIdSet.add(userDO.getId());
        userVOList.add(transUser(userDO));
    }
}
```

精简：

```
Set<Long> userIdSet = new HashSet<>();
List<UserVO> userVOList = new ArrayList<>();
for (UserDO userDO : userDOList) {
    if (userIdSet.add(userDO.getId())) {
        userVOList.add(transUser(userDO));
    }
}
```

3. 利用 Map 的 computeIfAbsent 方法

利用 Map 的 computeIfAbsent 方法，可以保证获取到的对象非空，从而避免了不必要的空判断和重新设置值。

普通：

```
Map<Long, List<UserDO>> roleUserMap = new HashMap<>();
for (UserDO userDO : userDOList) {
    Long roleId = userDO.getRoleId();
    List<UserDO> userList = roleUserMap.get(roleId);
    if (Objects.isNull(userList)) {
        userList = new ArrayList<>();
        roleUserMap.put(roleId, userList);
    }
    userList.add(userDO);
}
```

精简：

```
Map<Long, List<UserDO>> roleUserMap = new HashMap<>();
for (UserDO userDO : userDOList) {
    roleUserMap.computeIfAbsent(userDO.getRoleId(), key -> new ArrayList<>())
        .add(userDO);
}
```

4. 利用链式编程

链式编程，也叫级联式编程，调用对象的函数时返回一个 this 对象指向对象本身，达到链式效果，可以级联调用。链式编程的优点是：编程性强、可读性强、代码简洁。

普通：

```
StringBuilder builder = new StringBuilder(96);
builder.append("select id, name from ");
builder.append(T_USER);
builder.append(" where id = ");
builder.append(userId);
builder.append(";");
```

精简：

```
StringBuilder builder = new StringBuilder(96);
builder.append("select id, name from ")
    .append(T_USER)
    .append(" where id = ")
    .append(userId)
    .append(";");
```

五、 利用工具方法

1. 避免空值判断

普通：

```
if (userList != null && !userList.isEmpty()) {
    // TODO: 处理代码
}
```

精简：

```
if (CollectionUtils.isNotEmpty(userList)) {
    // TODO: 处理代码
}
```

2. 避免条件判断

普通：

```
double result;
if (value <= MIN_LIMIT) {
    result = MIN_LIMIT;
} else {
    result = value;
}
```

精简：

```
double result = Math.max(MIN_LIMIT, value);
```

3. 简化赋值语句

普通：

```
public static final List<String> ANIMAL_LIST;
static {
    List<String> animalList = new ArrayList<>();
    animalList.add("dog");
    animalList.add("cat");
    animalList.add("tiger");
    ANIMAL_LIST = Collections.unmodifiableList(animalList);
}
```

精简：

```
// JDK流派
public static final List<String> ANIMAL_LIST = Arrays.asList("dog", "cat", "tiger");
// Guava流派
public static final List<String> ANIMAL_LIST = ImmutableList.of("dog", "cat", "tiger");
```

注意：

Arrays.asList 返回的 List 并不是 ArrayList，不支持 add 等变更操作。

4. 简化数据拷贝

普通：

```
UserVO userVO = new UserVO();
userVO.setId(userDO.getId());
userVO.setName(userDO.getName());
...
userVO.setDescription(userDO.getDescription());
userVOList.add(userVO);
```

精简：

```
UserVO userVO = new UserVO();
BeanUtils.copyProperties(userDO, userVO);
userVOList.add(userVO);
```

反例：

```
List<UserVO> userVOList = JSON.parseArray(JSON.toJSONString(userDOList), UserVO.class);
```

精简代码，但不能以过大的性能损失为代价。例子是浅层拷贝，用不着 JSON 这样重量级的武器。

5. 简化异常断言

普通：

```
if (Objects.isNull(userId)) {
  throw new IllegalArgumentException("用户标识不能为空");
}
```

精简：

```
Assert.notNull(userId, "用户标识不能为空");
```

注意：

可能有些插件不认同这种判断，导致使用该对象时会有空指针警告。

6. 简化测试用例

把测试用例数据以 JSON 格式存入文件中，通过 JSON 的 parseObject 和 parseArray 方法解析成对象。虽然执行效率上有所下降，但可以减少大量的赋值语句，从而精简了测试代码。

普通：

```
@Test
public void testCreateUser() {
    UserCreateVO userCreate = new UserCreateVO();
    userCreate.setName("Changyi");
    userCreate.setTitle("Developer");
    userCreate.setCompany("AMAP");
    ...
    Long userId  = userService.createUser(OPERATOR, userCreate);
    Assert.assertNotNull(userId, "创建用户失败");
}
```

精简：

```
@Test
public void testCreateUser() {
    String jsonText = ResourceHelper.getResourceAsString(getClass(), "createUser.json");
    UserCreateVO userCreate = JSON.parseObject(jsonText, UserCreateVO.class);
    Long userId  = userService.createUser(OPERATOR, userCreate);
    Assert.assertNotNull(userId, "创建用户失败");
}
```

建议

JSON 文件名最好以被测试的方法命名，如果有多个版本可以用数字后缀表示。

7. 简化算法实现

一些常规算法，已有现成的工具方法，我们就没有必要自己实现了。

普通：

```
int totalSize = valueList.size();
List<List<Integer>> partitionList = new ArrayList<>();
for (int i = 0; i < totalSize; i += PARTITION_SIZE) {
    partitionList.add(valueList.subList(i, Math.min(i + PARTITION_SIZE, totalSize)));
}
```

精简：

```
List<List<Integer>> partitionList = ListUtils.partition(valueList, PARTITION_SIZE);
```

8. 封装工具方法

一些特殊算法，没有现成的工具方法，我们就只好自己亲自实现了。

普通：

比如，SQL 设置参数值的方法就比较难用，setLong 方法不能设置参数值为 null。

```
/** 设置参数值 */
if (Objects.nonNull(user.getId())) {
  statement.setLong(1, user.getId());
} else {
    statement.setNull(1, Types.BIGINT);
}
...
```

精简：

我们可以封装为一个工具类 SqlHelper，简化设置参数值的代码。

```
/** SQL辅助类 */
public final class SqlHelper {
    /** 设置长整数值 */
    public static void setLong(PreparedStatement statement, int index, Long value) throws
SQLException {
        if (Objects.nonNull(value)) {
            statement.setLong(index, value.longValue());
        } else {
            statement.setNull(index, Types.BIGINT);
        }
    }
    ...
}

/** 设置参数值 */
SqlHelper.setLong(statement, 1, user.getId());
```

六、 利用数据结构

1. 利用数组简化

对于固定上下限范围的 if-else 语句，可以用数组+循环来简化。

普通：

```
public static int getGrade(double score) {
    if (score >= 90.0D) {
        return 1;
    }
    if (score >= 80.0D) {
        return 2;
    }
    if (score >= 60.0D) {
        return 3;
    }
    if (score >= 30.0D) {
        return 4;
    }
    return 5;
}
```

精简：

```
private static final double[] SCORE_RANGES = new double[] {90.0D, 80.0D, 60.0D, 30.0D};
public static int getGrade(double score) {
    for (int i = 0; i < SCORE_RANGES.length; i++) {
        if (score >= SCORE_RANGES[i]) {
            return i + 1;
        }
    }
    return SCORE_RANGES.length + 1;
}
```

思考

上面的案例返回值是递增的，所以用数组简化是没有问题的。但是，如果返回值不是递增的，能否用数组进行简化呢？答案是可以的，请自行思考解决。

2. 利用 Map 简化

对于映射关系的 if-else 语句，可以用 Map 来简化。此外，此规则同样适用于简化映射关系的 switch 语句。

普通：

```
public static String getBiologyClass(String name) {
    switch (name) {
        case "dog" :
            return "animal";
        case "cat" :
            return "animal";
        case "lavender" :
            return "plant";
        ...
        default :
            return null;
    }
}
```

精简：

```
private static final Map<String, String> BIOLOGY_CLASS_MAP
    = ImmutableMap.<String, String>builder()
        .put("dog", "animal")
        .put("cat", "animal")
        .put("lavender", "plant")
        ...
        .build();
public static String getBiologyClass(String name) {
    return BIOLOGY_CLASS_MAP.get(name);
}
```

已经把方法简化为一行代码，其实都没有封装方法的必要了。

3. 利用容器类简化

Java 不像 Python 和 Go，方法不支持返回多个对象。如果需要返回多个对象，就必须自定义类，或者利用容器类。常见的容器类有 Apache 的 Pair 类和 Triple 类，Pair 类支持返回 2 个对象，Triple 类支持返回 3 个对象。

普通：

```
@Setter
@Getter
@ToString
@AllArgsConstructor
public static class PointAndDistance {
    private Point point;
    private Double distance;
}

public static PointAndDistance getNearest(Point point, Point[] points) {
    // 计算最近点和距离
    ...

    // 返回最近点和距离
    return new PointAndDistance(nearestPoint, nearestDistance);
}
```

精简：

```
public static Pair<Point, Double> getNearest(Point point, Point[] points) {
    // 计算最近点和距离
    ...

    // 返回最近点和距离
    return ImmutablePair.of(nearestPoint, nearestDistance);
}
```

4. 利用 ThreadLocal 简化

ThreadLocal 提供了线程专有对象，可以在整个线程生命周期中随时取用，极大地方便了一些逻辑的实现。用 ThreadLocal 保存线程上下文对象，可以避免不必要的参数传递。

普通：

由于 DateFormat 的 format 方法线程非安全（建议使用替代方法），在线程中频繁初始化 DateFormat 性能太低，如果考虑重用只能用参数传入 DateFormat。例子如下：

```
public static String formatDate(Date date, DateFormat format) {
    return format.format(date);
}

public static List<String> getDateList(Date minDate, Date maxDate, DateFormat format) {
    List<String> dateList = new ArrayList<>();
    Calendar calendar = Calendar.getInstance();
    calendar.setTime(minDate);
    String currDate = formatDate(calendar.getTime(), format);
    String maxsDate = formatDate(maxDate, format);
    while (currDate.compareTo(maxsDate) <= 0) {
        dateList.add(currDate);
        calendar.add(Calendar.DATE, 1);
        currDate = formatDate(calendar.getTime(), format);
    }
    return dateList;
}
```

精简：

可能你会觉得以下的代码量反而多了，如果调用工具方法的地方比较多，就可以省下一大堆 DateFormat 初始化和传入参数的代码。

```
private static final ThreadLocal<DateFormat> LOCAL_DATE_FORMAT = new ThreadLocal<DateFormat>
() {
    @Override
    protected DateFormat initialValue() {
        return new SimpleDateFormat("yyyyMMdd");
    }
};

public static String formatDate(Date date) {
    return LOCAL_DATE_FORMAT.get().format(date);
}

public static List<String> getDateList(Date minDate, Date maxDate) {
    List<String> dateList = new ArrayList<>();
    Calendar calendar = Calendar.getInstance();
    calendar.setTime(minDate);
    String currDate = formatDate(calendar.getTime());
    String maxsDate = formatDate(maxDate);
    while (currDate.compareTo(maxsDate) <= 0) {
        dateList.add(currDate);
        calendar.add(Calendar.DATE, 1);
        currDate = formatDate(calendar.getTime());
    }
    return dateList;
}
```

注意：

ThreadLocal 有一定的内存泄露的风险，尽量在业务代码结束前调用 remove 方法进行数据清除。

七、 利用 Optional

在 Java 8 里，引入了一个 Optional 类，该类是一个可以为 null 的容器对象。

1. 保证值存在

普通：

```
Integer thisValue;
if (Objects.nonNull(value)) {
    thisValue = value;
} else {
    thisValue = DEFAULT_VALUE;
}
```

精简：

```
Integer thisValue = Optional.ofNullable(value).orElse(DEFAULT_VALUE);
```

2. 保证值合法

普通：

```
Integer thisValue;
if (Objects.nonNull(value) && value.compareTo(MAX_VALUE) <= 0) {
    thisValue = value;
} else {
    thisValue = MAX_VALUE;
}
```

精简：

```
Integer thisValue = Optional.ofNullable(value)
    .filter(tempValue -> tempValue.compareTo(MAX_VALUE) <= 0).orElse(MAX_VALUE);
```

3. 避免空判断

普通：

```
String zipcode = null;
if (Objects.nonNull(user)) {
    Address address = user.getAddress();
    if (Objects.nonNull(address)) {
        Country country = address.getCountry();
        if (Objects.nonNull(country)) {
            zipcode = country.getZipcode();
        }
    }
}
```

精简：

```
String zipcode = Optional.ofNullable(user).map(User::getAddress)
    .map(Address::getCountry).map(Country::getZipcode).orElse(null);
```

八、 利用 Stream

流（Stream）是 Java 8 的新成员，允许你以声明式处理数据集合，可以看成为一个遍历数据集的高级迭代器。流主要有三部分构成：获取一个数据源→数据转换→执行操作获取想要的结果。每次转换原有 Stream 对象不改变，返回一个新的 Stream 对象，这就允许对其操作可以像链条一样排列，形成了一个管道。流（Stream）提供的功能非常有用，主要包括匹配、过滤、汇总、转化、分组、分组汇总等功能。

1. 匹配集合数据

普通：

```
boolean isFound = false;
for (UserDO user : userList) {
    if (Objects.equals(user.getId(), userId)) {
        isFound = true;
        break;
    }
}
```

精简：

```
boolean isFound = userList.stream()
    .anyMatch(user -> Objects.equals(user.getId(), userId));
```

2. 过滤集合数据

普通：

```
List<UserDO> resultList = new ArrayList<>();
for (UserDO user : userList) {
    if (Boolean.TRUE.equals(user.getIsSuper())) {
        resultList.add(user);
    }
}
```

精简：

```
List<UserDO> resultList = userList.stream()
    .filter(user -> Boolean.TRUE.equals(user.getIsSuper()))
    .collect(Collectors.toList());
```

3. 汇总集合数据

普通：

```
double total = 0.0D;
for (Account account : accountList) {
    total += account.getBalance();
}
```

精简：

```
double total = accountList.stream().mapToDouble(Account::getBalance).sum();
```

4. 转化集合数据

普通：

```
List<UserVO> userVOList = new ArrayList<>();
for (UserDO userDO : userDOList) {
    userVOList.add(transUser(userDO));
}
```

精简：

```
List<UserVO> userVOList = userDOList.stream()
    .map(this::transUser).collect(Collectors.toList());
```

5. 分组集合数据

普通：

```
Map<Long, List<UserDO>> roleUserMap = new HashMap<>();
for (UserDO userDO : userDOList) {
    roleUserMap.computeIfAbsent(userDO.getRoleId(), key -> new ArrayList<>())
        .add(userDO);
}
```

精简：

```
Map<Long, List<UserDO>> roleUserMap = userDOList.stream()
    .collect(Collectors.groupingBy(UserDO::getRoleId));
```

6. 分组汇总集合

普通：

```
Map<Long, Double> roleTotalMap = new HashMap<>();
for (Account account : accountList) {
    Long roleId = account.getRoleId();
    Double total = Optional.ofNullable(roleTotalMap.get(roleId)).orElse(0.0D);
    roleTotalMap.put(roleId, total + account.getBalance());
}
```

精简：

```
roleTotalMap = accountList.stream().collect(Collectors.groupingBy(Account::getRoleId,
Collectors.summingDouble(Account::getBalance)));
```

7. 生成范围集合

Python 的 range 非常方便，Stream 也提供了类似的方法。

普通：

```
int[] array1 = new int[N];
for (int i = 0; i < N; i++) {
    array1[i] = i + 1;
}

int[] array2 = new int[N];
array2[0] = 1;
for (int i = 1; i < N; i++) {
    array2[i] = array2[i - 1] * 2;
}
```

精简：

```
int[] array1 = IntStream.rangeClosed(1, N).toArray();
int[] array2 = IntStream.iterate(1, n -> n * 2).limit(N).toArray();
```

九、 利用程序结构

1. 返回条件表达式

条件表达式判断返回布尔值，条件表达式本身就是结果。

普通：

```
public boolean isSuper(Long userId)
    UserDO user = userDAO.get(userId);
    if (Objects.nonNull(user) && Boolean.TRUE.equals(user.getIsSuper())) {
        return true;
    }
    return false;
}
```

精简：

```
public boolean isSuper(Long userId)
    UserDO user = userDAO.get(userId);
    return Objects.nonNull(user) && Boolean.TRUE.equals(user.getIsSuper());
}
```

2. 最小化条件作用域

最小化条件作用域，尽量提出公共处理代码。

普通：

```
Result result = summaryService.reportWorkDaily(workDaily);
if (result.isSuccess()) {
    String message = "上报工作日报成功";
    dingtalkService.sendMessage(user.getPhone(), message);
} else {
    String message = "上报工作日报失败:" + result.getMessage();
    log.warn(message);
    dingtalkService.sendMessage(user.getPhone(), message);
}
```

精简：

```
String message;
Result result = summaryService.reportWorkDaily(workDaily);
if (result.isSuccess()) {
    message = "上报工作日报成功";
} else {
    message = "上报工作日报失败:" + result.getMessage();
    log.warn(message);
}
dingtalkService.sendMessage(user.getPhone(), message);
```

3. 调整表达式位置

调整表达式位置，在逻辑不变的前提下，让代码变得更简洁。

普通 1：

```
String line = readLine();
while (Objects.nonNull(line)) {
    ... // 处理逻辑代码
    line = readLine();
}
```

普通 2：

```
for (String line = readLine(); Objects.nonNull(line); line = readLine()) {
    ... // 处理逻辑代码
}
```

精简：

```
String line;
while (Objects.nonNull(line = readLine())) {
    ... // 处理逻辑代码
}
```

注意：

有些规范可能不建议这种精简写法。

4. 利用非空对象

在比较对象时，交换对象位置，利用非空对象，可以避免空指针判断。

普通：

```
private static final int MAX_VALUE = 1000;
boolean isMax = (value != null && value.equals(MAX_VALUE));
boolean isTrue = (result != null && result.equals(Boolean.TRUE));
```

精简：

```
private static final Integer MAX_VALUE = 1000;
boolean isMax = MAX_VALUE.equals(value);
boolean isTrue = Boolean.TRUE.equals(result);
```

十、 利用设计模式

1. 模板方法模式

模板方法模式（Template Method Pattern）定义一个固定的算法框架，而将算法的一些步骤放到子类中实现，使得子类可以在不改变算法框架的情况下重定义该算法的某些步骤。

普通

```
@Repository
public class UserValue {
    /** 值操作 */
    @Resource(name = "stringRedisTemplate")
    private ValueOperations<String, String> valueOperations;
    /** 值模式 */
    private static final String KEY_FORMAT = "Value:User:%s";

    /** 设置值 */
    public void set(Long id, UserDO value) {
        String key = String.format(KEY_FORMAT, id);
        valueOperations.set(key, JSON.toJSONString(value));
    }

    /** 获取值 */
    public UserDO get(Long id) {
        String key = String.format(KEY_FORMAT, id);
        String value = valueOperations.get(key);
        return JSON.parseObject(value, UserDO.class);
    }

    ...
}

@Repository
public class RoleValue {
    /** 值操作 */
    @Resource(name = "stringRedisTemplate")
    private ValueOperations<String, String> valueOperations;
    /** 值模式 */
    private static final String KEY_FORMAT = "Value:Role:%s";

    /** 设置值 */
    public void set(Long id, RoleDO value) {
        String key = String.format(KEY_FORMAT, id);
        valueOperations.set(key, JSON.toJSONString(value));
    }

    /** 获取值 */
    public RoleDO get(Long id) {
        String key = String.format(KEY_FORMAT, id);
        String value = valueOperations.get(key);
        return JSON.parseObject(value, RoleDO.class);
    }

    ...
}
```

精简：

```
public abstract class AbstractDynamicValue<I, V> {
    /** 值操作 */
    @Resource(name = "stringRedisTemplate")
    private ValueOperations<String, String> valueOperations;

    /** 设置值 */
    public void set(I id, V value) {
        valueOperations.set(getKey(id), JSON.toJSONString(value));
    }

    /** 获取值 */
    public V get(I id) {
        return                JSON.parseObject(valueOperations.get(getKey(id)),
getValueClass());
    }

    ...

    /** 获取主键 */
    protected abstract String getKey(I id);

    /** 获取值类 */
    protected abstract Class<V> getValueClass();
}

@Repository
public class UserValue extends AbstractValue<Long, UserDO> {
    /** 获取主键 */
    @Override
    protected String getKey(Long id) {
        return String.format("Value:User:%s", id);
    }

    /** 获取值类 */
    @Override
    protected Class<UserDO> getValueClass() {
        return UserDO.class;
    }
}

@Repository
public class RoleValue extends AbstractValue<Long, RoleDO> {
    /** 获取主键 */
    @Override
    protected String getKey(Long id) {
        return String.format("Value:Role:%s", id);
    }

    /** 获取值类 */
    @Override
    protected Class<RoleDO> getValueClass() {
        return RoleDO.class;
    }
}
```

2. 访问者模式

访问者模式（Visitor），封装一些作用于某种数据结构的各元素的操作，它可以在不改变数据结构的前提下定义作用于这些元素的新的操作。

普通：

```
public interface DataHandler<T> {
    /** 解析数据 */
     public T parseData(Record record);

    /** 存储数据 */
     public boolean storeData(List<T> dataList);
}

public <T> long executeFetch(String tableName, int batchSize, DataHandler<T>
dataHandler) throws Exception {
    // 构建下载会话
    DownloadSession session = buildSession(tableName);

    // 获取数据数量
    long recordCount = session.getRecordCount();
    if (recordCount == 0) {
        return 0;
    }

    // 进行数据读取
    long fetchCount = 0L;
    try (RecordReader reader = session.openRecordReader(0L, recordCount,
true)) {
        // 依次读取数据
        Record record;
        List<T> dataList = new ArrayList<>(batchSize);
        while ((record = reader.read()) != null) {
            // 解析添加数据
            T data = dataHandler.parseData(record);
            if (Objects.nonNull(data)) {
                dataList.add(data);
            }

            // 批量存储数据
            if (dataList.size() == batchSize) {
                boolean isContinue = dataHandler.storeData(dataList);
                fetchCount += batchSize;
                dataList.clear();
                if (!isContinue) {
                    break;
                }
            }
        }

        // 存储剩余数据
        if (CollectionUtils.isNotEmpty(dataList)) {
            dataHandler.storeData(dataList);
            fetchCount += dataList.size();
            dataList.clear();
        }
    }

    // 返回获取数量
```

```
    return fetchCount;
}

/** 使用案例 */
long fetchCount = odpsService.executeFetch("user", 5000, new DataHandler() {
    /** 解析数据 */
    @Override
    public T parseData(Record record) {
        UserDO user = new UserDO();
        user.setId(record.getBigint("id"));
        user.setName(record.getString("name"));
        return user;
    }

    /** 存储数据 */
    @Override
    public boolean storeData(List<T> dataList) {
        userDAO.batchInsert(dataList);
        return true;
    }
});
```

精简：

```
public <T> long executeFetch(String tableName, int batchSize, Function<Record,
T> dataParser, Function<List<T>, Boolean> dataStorage) throws Exception {
    // 构建下载会话
    DownloadSession session = buildSession(tableName);

    // 获取数据数量
    long recordCount = session.getRecordCount();
    if (recordCount == 0) {
        return 0;
    }

    // 进行数据读取
    long fetchCount = 0L;
    try (RecordReader reader = session.openRecordReader(0L, recordCount,
true)) {
        // 依次读取数据
        Record record;
        List<T> dataList = new ArrayList<>(batchSize);
        while ((record = reader.read()) != null) {
            // 解析添加数据
            T data = dataParser.apply(record);
            if (Objects.nonNull(data)) {
                dataList.add(data);
            }

            // 批量存储数据
            if (dataList.size() == batchSize) {
                Boolean isContinue = dataStorage.apply(dataList);
                fetchCount += batchSize;
                dataList.clear();
                if (!Boolean.TRUE.equals(isContinue)) {
                    break;
                }
            }
        }

        // 存储剩余数据
        if (CollectionUtils.isNotEmpty(dataList)) {
            dataStorage.apply(dataList);
            fetchCount += dataList.size();
```

```
            dataList.clear();
        }
    }

    // 返回获取数量
    return fetchCount;
}

/** 使用案例 */
long fetchCount = odpsService.executeFetch("user", 5000, record -> {
        UserDO user = new UserDO();
        user.setId(record.getBigint("id"));
        user.setName(record.getString("name"));
        return user;
    }, dataList -> {
        userDAO.batchInsert(dataList);
        return true;
    });
```

普通的访问者模式，实现时需要定义 DataHandler 接口，调用时需要实现 DataHandler 匿名内部类，代码较多较繁琐。而精简后的访问者模式，充分利用了函数式编程，实现时无需定义接口，直接使用 Function 接口；调用时无需实现匿名内部类，直接采用 lambda 表达式，代码较少较简洁。

3. 代理模式

Spring 中最重要的代理模式就是 AOP（Aspect-Oriented Programming，面向切面的编程），是使用 JDK 动态代理和 CGLIB 动态代理技术来实现的。

普通：

```
@Slf4j
@RestController
@RequestMapping("/user")
public class UserController {
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /** 查询用户 */
    @PostMapping("/queryUser")
    public Result<?> queryUser(@RequestBody @Valid UserQueryVO query) {
        try {
            PageDataVO<UserVO> pageData = userService.queryUser(query);
            return Result.success(pageData);
        } catch (Exception e) {
            log.error(e.getMessage(), e);
            return Result.failure(e.getMessage());
        }
    }
    ...
}
```

精简 1：

基于@ControllerAdvice 的异常处理：

```
@RestController
@RequestMapping("/user")
public class UserController {
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /** 查询用户 */
    @PostMapping("/queryUser")
    public Result<PageDataVO<UserVO>> queryUser(@RequestBody @Valid UserQueryVO query) {
        PageDataVO<UserVO> pageData = userService.queryUser(query);
        return Result.success(pageData);
    }
    ...
}

@Slf4j
@ControllerAdvice
public class GlobalControllerAdvice {
    /** 处理异常 */
    @ResponseBody
    @ExceptionHandler(Exception.class)
    public Result<Void> handleException(Exception e) {
        log.error(e.getMessage(), e);
        return Result.failure(e.getMessage());
    }
}
```

精简 2：

基于 AOP 的异常处理：

```
// UserController代码同"精简1"

@Slf4j
@Aspect
public class WebExceptionAspect {
    /** 点切面 */
    @Pointcut("@annotation(org.springframework.web.bind.annotation.RequestMapping)")
    private void webPointcut() {}

    /** 处理异常 */
    @AfterThrowing(pointcut = "webPointcut()", throwing = "e")
    public void handleException(Exception e) {
        Result<Void> result = Result.failure(e.getMessage());
        writeContent(JSON.toJSONString(result));
    }
    ...
}
```

十一、 利用删除代码

“少即是多”，“少”不是空白而是精简，“多”不是拥挤而是完美。删除多余的代码，才能使代码更精简更完美。

1. 删除已废弃的代码

删除项目中的已废弃的包、类、字段、方法、变量、常量、导入、注解、注释、已注释代码、Maven 包导入、MyBatis 的 SQL 语句、属性配置字段等，可以精简项目代码便于维护。

普通：

```
import lombok.extern.slf4j.Slf4j;
@Slf4j
@Service
public class ProductService {
    @Value("discardRate")
    private double discardRate;
    ...
    private ProductVO transProductDO(ProductDO productDO) {
        ProductVO productVO = new ProductVO();
        BeanUtils.copyProperties(productDO, productVO);
        // productVO.setPrice(getDiscardPrice(productDO.getPrice()));
        return productVO;
    }
    private BigDecimal getDiscardPrice(BigDecimal originalPrice) {
        ...
    }
}
```

精简：

```
@Service
public class ProductService {
    ...
    private ProductVO transProductDO(ProductDO productDO) {
        ProductVO productVO = new ProductVO();
        BeanUtils.copyProperties(productDO, productVO);
        return productVO;
    }
}
```

2. 删除接口方法的 public

对于接口（interface），所有的字段和方法都是 public 的，可以不用显式声明为 public。

普通：

```
public interface UserDAO {
    public Long countUser(@Param("query") UserQuery query);
    public List<UserDO> queryUser(@Param("query") UserQuery query);
}
```

精简：

```
public interface UserDAO {
    Long countUser(@Param("query") UserQuery query);
    List<UserDO> queryUser(@Param("query") UserQuery query);
}
```

3. 删除枚举构造方法的 private

对于枚举（menu），构造方法都是 private 的，可以不用显式声明为 private。

普通：

```
public enum UserStatus {
    DISABLED(0, "禁用"),
    ENABLED(1, "启用");
  private final Integer value;
    private final String desc;
    private UserStatus(Integer value, String desc) {
        this.value = value;
        this.desc = desc;
    }
    ...
}
```

精简：

```
public enum UserStatus {
    DISABLED(0, "禁用"),
    ENABLED(1, "启用");
  private final Integer value;
    private final String desc;
    UserStatus(Integer value, String desc) {
        this.value = value;
        this.desc = desc;
    }
    ...
}
```

4. 删除 final 类方法的 final

对于 final 类，不能被子类继承，所以其方法不会被覆盖，没有必要添加 final 修饰。

普通：

```
public final class Rectangle implements Shape {
    ...
    @Override
    public final double getArea() {
        return width * height;
    }
}
```

精简：

```
public final class Rectangle implements Shape {
    ...
    @Override
    public double getArea() {
        return width * height;
    }
}
```

5. 删除基类 implements 的接口

如果基类已 implements 某接口，子类没有必要再 implements 该接口，只需要直接实现接口方法即可。

普通：

```
public interface Shape {
    ...
    double getArea();
}
public abstract class AbstractShape implements Shape {
    ...
}
public final class Rectangle extends AbstractShape implements Shape {
    ...
    @Override
    public double getArea() {
        return width * height;
    }
}
```

精简：

```
...
public final class Rectangle extends AbstractShape {
    ...
    @Override
    public double getArea() {
        return width * height;
    }
}
```

6. 删除不必要的变量

不必要的变量，只会让代码看起来更繁琐。

普通：

```
public Boolean existsUser(Long userId) {
    Boolean exists = userDAO.exists(userId);
    return exists;
}
```

精简：

```
public Boolean existsUser(Long userId) {
    return userDAO.exists(userId);
}
```

后记

古语又云：

> *有道无术，术尚可求也；有术无道，止于术。*

意思是：有“道”而无“术”，“术”还可以逐渐获得；有“术”而无“道”，就可能止步于“术”了。所以，我们不要仅满足于从实践中总结“术”，因为“道”的表现形式是多变的；而应该上升到“道”的高度，因为“术”背后的道理是相通的。当遇到新的事物时，我们可以从理论中找到“道”、从实践中找出“术”，尝试着去认知新的事物。

Java 代码高效之道

导读

世界上只有两种物质：高效率和低效率；世界上只有两种人：高效率的人和低效率的人。

——萧伯纳

同理，世界上只有两种代码：高效代码和低效代码；世界上只有两种编写代码的人：编写高效代码的人和编写低效代码的人。如何编写高效代码，是每个研发团队都面临的一个重大问题。所以，作者根据实际经验，查阅了大量资料，总结了“Java 代码高效之道”，让每一个 Java 程序员都能编写出“高效代码”。

一、 常量&变量

1. 直接赋值常量值，禁止声明新对象

直接赋值常量值，只是创建了一个对象引用，而这个对象引用指向常量值。

反例：

```
Long i = new Long(1L);
String s = new String("abc");
```

正例：

```
Long i = 1L;
String s = "abc";
```

2. 当成员变量值无需改变时，尽量定义为静态常量

在类的每个对象实例中，每个成员变量都有一份副本，而成员静态常量只有一份实例。

反例：

```
public class HttpConnection {
    private final long timeout = 5L;
    ...
}
```

正例：

```
public class HttpConnection {
    private static final long TIMEOUT = 5L;
    ...
}
```

3. 尽量使用基本数据类型，避免自动装箱和拆箱

Java 中的基本数据类型 double、float、long、int、short、char、boolean，分别对应包装类 Double、Float、Long、Integer、Short、Character、Boolean。JVM 支持基本类型与对应包装类的自动转换，被称为自动装箱和拆箱。装箱和拆箱都是需要 CPU 和内存资源的，所以应尽量避免使用自动装箱和拆箱。

反例：

```
Integer sum = 0;
int[] values = ...;
for (int value : values) {
    sum += value; // 相当于result = Integer.valueOf(result.intValue() + value);
}
```

正例：

```
int sum = 0;
int[] values = ...;
for (int value : values) {
    sum += value;
}
```

4. 如果变量的初值会被覆盖，就没有必要给变量赋初值

反例：

```
List<UserDO> userList = new ArrayList<>();
if (isAll) {
    userList = userDAO.queryAll();
} else {
    userList = userDAO.queryActive();
}
```

正例：

```
List<UserDO> userList;
if (isAll) {
    userList = userDAO.queryAll();
} else {
    userList = userDAO.queryActive();
}
```

5. 尽量使用函数内的基本类型临时变量

在函数内，基本类型的参数和临时变量都保存在栈（Stack）中，访问速度较快；对象类型的参数和临时变量的引用都保存在栈（Stack）中，内容都保存在堆（Heap）中，访问速度较慢。在类中，任何类型的成员变量都保存在堆（Heap）中，访问速度较慢。

反例：

```
public final class Accumulator {
    private double result = 0.0D;
    public void addAll(@NonNull double[] values) {
        for(double value : values) {
            result += value;
        }
    }
    ...
}
```

正例：

```
public final class Accumulator {
    private double result = 0.0D;
    public void addAll(@NonNull double[] values) {
        double sum = 0.0D;
        for(double value : values) {
            sum += value;
        }
        result += sum;
    }
    ...
}
```

6. 尽量不要在循环体外定义变量

在老版 JDK 中，建议“尽量不要在循环体内定义变量”，但是在新版的 JDK 中已经做了优化。通过对编译后的字节码分析，变量定义在循环体外和循环体内没有本质的区别，运行效率基本上是一样的。反而，根据“局部变量作用域最小化”原则，变量定义在循环体内更科学更便于维护，避免了延长大对象生命周期导致延缓回收问题。

反例：

```
UserVO userVO;
List<UserDO> userDOList = ...;
List<UserVO> userVOList = new ArrayList<>(userDOList.size());
for (UserDO userDO : userDOList) {
    userVO = new UserVO();
    userVO.setId(userDO.getId());
    ...
    userVOList.add(userVO);
}
```

正例：

```
List<UserDO> userDOList = ...;
List<UserVO> userVOList = new ArrayList<>(userDOList.size());
for (UserDO userDO : userDOList) {
    UserVO userVO = new UserVO();
    userVO.setId(userDO.getId());
    ...
    userVOList.add(userVO);
}
```

7. 不可变的静态常量，尽量使用非线程安全类

不可变的静态常量，虽然需要支持多线程访问，也可以使用非线程安全类。

反例：

```
public static final Map<String, Class> CLASS_MAP;
static {
    Map<String, Class> classMap = new ConcurrentHashMap<>(16);
    classMap.put("VARCHAR", java.lang.String.class);
    ...
    CLASS_MAP = Collections.unmodifiableMap(classMap);
}
```

正例：

```
public static final Map<String, Class> CLASS_MAP;
static {
    Map<String, Class> classMap = new HashMap<>(16);
    classMap.put("VARCHAR", java.lang.String.class);
    ...
    CLASS_MAP = Collections.unmodifiableMap(classMap);
}
```

8. 不可变的成员变量，尽量使用非线程安全类

不可变的成员变量，虽然需要支持多线程访问，也可以使用非线程安全类。

反例：

```
@Service
public class StrategyFactory implements InitializingBean {
    @Autowired
    private List<Strategy> strategyList;
    private Map<String, Strategy> strategyMap;
    @Override
    public void afterPropertiesSet() {
        if (CollectionUtils.isNotEmpty(strategyList)) {
            int size = (int) Math.ceil(strategyList.size() * 4.0 / 3);
            Map<String, Strategy> map = new ConcurrentHashMap<>(size);
            for (Strategy strategy : strategyList) {
                map.put(strategy.getType(), strategy);
            }
            strategyMap = Collections.unmodifiableMap(map);
        }
    }
    ...
}
```

正例：

```
@Service
public class StrategyFactory implements InitializingBean {
    @Autowired
    private List<Strategy> strategyList;
    private Map<String, Strategy> strategyMap;
    @Override
    public void afterPropertiesSet() {
        if (CollectionUtils.isNotEmpty(strategyList)) {
            int size = (int) Math.ceil(strategyList.size() * 4.0 / 3);
            Map<String, Strategy> map = new HashMap<>(size);
            for (Strategy strategy : strategyList) {
                map.put(strategy.getType(), strategy);
            }
            strategyMap = Collections.unmodifiableMap(map);
        }
    }
    ...
}
```

二、 2.对象&类

1. 禁止使用 JSON 转化对象

JSON 提供把对象转化为 JSON 字符串、把 JSON 字符串转为对象的功能，于是被某些人用来转化对象。这种对象转化方式，虽然在功能上没有问题，但是在性能上却存在问题。

反例：

```
List<UserDO> userDOList = ...;
List<UserVO> userVOList = JSON.parseArray(JSON.toJSONString(userDOList), UserVO.class);
```

正例：

```
List<UserDO> userDOList = ...;
List<UserVO> userVOList = new ArrayList<>(userDOList.size());
for (UserDO userDO : userDOList) {
    UserVO userVO = new UserVO();
    userVO.setId(userDO.getId());
    ...
    userVOList.add(userVO);
}
```

2. 尽量不使用反射赋值对象

用反射赋值对象，主要优点是节省了代码量，主要缺点却是性能有所下降。

反例：

```
List<UserDO> userDOList = ...;
List<UserVO> userVOList = new ArrayList<>(userDOList.size());
for (UserDO userDO : userDOList) {
    UserVO userVO = new UserVO();
    BeanUtils.copyProperties(userDO, userVO);
    userVOList.add(userVO);
}
```

正例：

```
List<UserDO> userDOList = ...;
List<UserVO> userVOList = new ArrayList<>(userDOList.size());
for (UserDO userDO : userDOList) {
    UserVO userVO = new UserVO();
    userVO.setId(userDO.getId());
    ...
    userVOList.add(userVO);
}
```

3. 采用 Lambda 表达式替换内部匿名类

对于大多数刚接触 JDK8 的同学来说，都会认为 Lambda 表达式就是匿名内部类的语法糖。实际上，Lambda 表达式在大多数虚拟机中采用 invokeDynamic 指令实现，相对于匿名内部类在效率上会更高一些。

反例：

```
List<User> userList = ...;
Collections.sort(userList, new Comparator<User>() {
    @Override
    public int compare(User user1, User user2) {
        Long userId1 = user1.getId();
        Long userId2 = user2.getId();
        ...
        return userId1.compareTo(userId2);
    }
});
```

正例：

```
List<User> userList = ...;
Collections.sort(userList, (user1, user2) -> {
    Long userId1 = user1.getId();
    Long userId2 = user2.getId();
    ...
    return userId1.compareTo(userId2);
});
```

4. 尽量避免定义不必要的子类

多一个类就需要多一份类加载，所以尽量避免定义不必要的子类。

反例：

```
public static final Map<String, Class> CLASS_MAP =
    Collections.unmodifiableMap(new HashMap<String, Class>(16) {
    private static final long serialVersionUID = 1L;
    {
        put("VARCHAR", java.lang.String.class);
    }
});
```

正例：

```
public static final Map<String, Class> CLASS_MAP;
static {
    Map<String, Class> classMap = new HashMap<>(16);
    classMap.put("VARCHAR", java.lang.String.class);
    ...
    CLASS_MAP = Collections.unmodifiableMap(classMap);
}
```

5. 尽量指定类的 final 修饰符

为类指定 final 修饰符，可以让该类不可以被继承。如果指定了一个类为 final，则该类所有的方法都是 final 的，Java 编译器会寻找机会内联所有的 final 方法。内联对于提升 Java 运行效率作用重大，具体可参见 Java 运行期优化，能够使性能平均提高 50%。

反例：

```
public class DateHelper {
    ...
}
```

正例：

```
public final class DateHelper {
    ...
}
```

注意：

使用 Spring 的 AOP 特性时，需要对 Bean 进行动态代理，如果 Bean 类添加了 final 修饰，会导致异常。

三、 方法

1. 把跟类成员变量无关的方法声明成静态方法

静态方法的好处就是不用生成类的实例就可以直接调用。静态方法不再属于某个对象，而是属于它所在的类。只需要通过其类名就可以访问，不需要再消耗资源去反复创建对象。即便在类内部的私有方法，如果没有使用到类成员变量，也应该声明为静态方法。

反例：

```
public int getMonth(Date date) {
  Calendar calendar = Calendar.getInstance();
  calendar.setTime(date);
  return calendar.get(Calendar.MONTH) + 1;
}
```

正例：

```
public static int getMonth(Date date) {
  Calendar calendar = Calendar.getInstance();
  calendar.setTime(date);
  return calendar.get(Calendar.MONTH) + 1;
}
```

2. 尽量使用基本数据类型作为方法参数类型，避免不必要的装箱、拆箱和空指针判断

反例：

```
public static double sum(Double value1, Double value2) {
    double double1 = Objects.isNull(value1) ? 0.0D : value1;
    double double2 = Objects.isNull(value2) ? 0.0D : value2;
    return double1 + double2;
}
double result = sum(1.0D, 2.0D);
```

正例：

```
public static double sum(double value1, double value2) {
    return value1 + value2;
}
double result = sum(1.0D, 2.0D);
```

3. 尽量使用基本数据类型作为方法返回值类型，避免不必要的装箱、拆箱和空指针判断

在 JDK 类库的方法中，很多方法返回值都采用了基本数据类型，首先是为了避免不必要的装箱和拆箱，其次是为了避免返回值的空指针判断。比如：Collection.isEmpty()和 Map.size()。

反例：

```
public static Boolean isValid(UserDO user) {
    if (Objects.isNull(user)) {
        return false;
    }
  return Boolean.TRUE.equals(user.getIsValid());
}

// 调用代码
UserDO user = ...;
Boolean isValid = isValid(user);
if (Objects.nonNull(isValid) && isValid.booleanValue()) {
    ...
}
```

正例：

```
public static boolean isValid(UserDO user) {
    if (Objects.isNull(user)) {
        return false;
    }
  return Boolean.TRUE.equals(user.getIsValid());
}

// 调用代码
UserDO user = ...;
if (isValid(user)) {
    ...
}
```

4. 协议方法参数值非空，避免不必要的空指针判断

协议编程，可以@NonNull 和@Nullable 标注参数，是否遵循全凭调用者自觉。

反例：

```
public static boolean isValid(UserDO user) {
    if (Objects.isNull(user)) {
        return false;
    }
  return Boolean.TRUE.equals(user.getIsValid());
}
```

正例：

```
public static boolean isValid(@NonNull UserDO user) {
  return Boolean.TRUE.equals(user.getIsValid());
}
```

5. 协议方法返回值非空，避免不必要的空指针判断

协议编程，可以@NonNull 和@Nullable 标注参数，是否遵循全凭实现者自觉。

反例：

```
// 定义接口
public interface OrderService {
    public List<OrderVO> queryUserOrder(Long userId);
}

// 调用代码
List<OrderVO> orderList = orderService.queryUserOrder(userId);
if (CollectionUtils.isNotEmpty(orderList)) {
    for (OrderVO order : orderList) {
        ...
    }
}
```

正例：

```
// 定义接口
public interface OrderService {
    @NonNull
    public List<OrderVO> queryUserOrder(Long userId);
}

// 调用代码
List<OrderVO> orderList = orderService.queryUserOrder(userId);
for (OrderVO order : orderList) {
    ...
}
```

6. 被调用方法已支持判空处理，调用方法无需再进行判空处理

反例：

```
UserDO user = null;
if (StringUtils.isNotBlank(value)) {
    user = JSON.parseObject(value, UserDO.class);
}
```

正例：

```
UserDO user = JSON.parseObject(value, UserDO.class);
```

7. 尽量避免不必要的函数封装

方法调用会引起入栈和出栈，导致消耗更多的 CPU 和内存，应当尽量避免不必要的函数封装。当然，为了使代码更简洁、更清晰、更易维护，增加一定的方法调用所带来的性能损耗是值得的。

反例：

```
// 函数封装
public static boolean isVip(Boolean isVip) {
    return Boolean.TRUE.equals(isVip);
}

// 使用代码
boolean isVip = isVip(user.getVip());
```

正例：

```
boolean isVip = Boolean.TRUE.equals(user.getVip());
```

8. 尽量指定方法的 final 修饰符

方法指定 final 修饰符，可以让方法不可以被重写，Java 编译器会寻找机会内联所有的 final 方法。内联对于提升 Java 运行效率作用重大，具体可参见 Java 运行期优化，能够使性能平均提高 50%。

注意：
所有的 private 方法会隐式地被指定 final 修饰符，所以无须再为其指定 final 修饰符。

反例：

```
public class Rectangle {
    ...
    public double area() {
        ...
    }
}
```

正例：

```
public class Rectangle {
    ...
    public final double area() {
        ...
    }
}
```

注意：

使用 Spring 的 AOP 特性时，需要对 Bean 进行动态代理，如果方法添加了 final 修饰，将不会被代理。

9. 尽量使用方法传递代替值传递

尽量使用方法传递代替值传递，可以避免不必要的方法计算。比如 Optional 的 orElse(T value)方法和 orElseGet(Supplier supplier)方法：其中 orElse(T value)方法无论前面 Optional 容器值是 null 还是 nonNull，都会提前执行 orElse 里的方法；然而 orElseGet（Supplier supplier）方法并不会，只会在 Optional 容器值为 null 时才调用 orElseGet 里的方法。

反例：

```
public static String getNewValue() {
    log.info("调用getNewValue");
    ... // 耗时代码
    return "newValue";
}

// 会调用getNewValue
String oldValue = "oldValue";
String value = Optional.ofNullable(oldValue).orElse(getNewValue());
```

正例：

```
// 同上getNewValue

// 不会调用getNewValue
String oldValue = "oldValue";
String value = Optional.ofNullable(oldValue).orElseGet(() -> getNewValue());
```

四、 表达式

1. 尽量减少方法的重复调用

反例：

```
List<UserDO> userList = ...;
for (int i = 0; i < userList.size(); i++) {
    ...
}
```

正例：

```
List<UserDO> userList = ...;
int userLength = userList.size();
for (int i = 0; i < userLength; i++) {
    ...
}
```

2. 尽量避免不必要的方法调用

反例：

```
List<UserDO> userList = userDAO.queryActive();
if (isAll) {
    userList = userDAO.queryAll();
}
```

正例：

```
List<UserDO> userList;
if (isAll) {
    userList = userDAO.queryAll();
} else {
    userList = userDAO.queryActive();
}
```

3. 尽量使用移位来代替正整数乘除

用移位操作可以极大地提高性能。对于乘除 2^n（n 为正整数）的正整数计算，可以用移位操作来代替。

反例：

```
int num1 = a * 4;
int num2 = a / 4;
```

正例：

```
int num1 = a << 2;
int num2 = a >> 2;
```

4. 提取公共表达式，避免重复计算

提取公共表达式，只计算一次值，然后重复利用值。

反例：

```
double distance = Math.sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
```

正例：

```
double dx = x2 - x1;
double dy = y2 - y1;
double distance = Math.sqrt(dx * dx + dy * dy);
或
double distance = Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
```

5. 尽量不在条件表达式中用!取反

使用!取反会多一次计算，如果没有必要则优化掉。

反例：

```
if (!(a >= 10)) {
    ... // 条件处理1
} else {
    ... // 条件处理2
}
```

正例：

```
if (a < 10) {
    ... // 条件处理1
} else {
    ... // 条件处理2
}
```

6. 对于多常量选择分支，尽量使用 switch 语句而不是 if-else 语句

if-else 语句，每个 if 条件语句都要加装计算，直到 if 条件语句为 true 为止。switch 语句进行了跳转优化，Java 中采用 tableswitch 或 lookupswitch 指令实现，对于多常量选择分支处理效率更高。经过试验证明：在每个分支出现概率相同的情况下，低于 5 个分支时 if-else 语句效率更高，高于 5 个分支时 switch 语句效率更高。

反例：

```
if (i == 1) {
    ...; // 分支1
} else if (i == 2) {
    ...; // 分支2
} else if (i == ...) {
    ...; // 分支n
} else {
    ...; // 分支n+1
}
```

正例：

```
switch (i) {
    case 1 :
        ... // 分支1
        break;
    case 2 :
        ... // 分支2
        break;
    case ... :
        ... // 分支n
        break;
    default :
        ... // 分支n+1
        break;
}
```

备注

如果业务复杂，可以采用 Map 实现策略模式。

五、 字符串

1. 尽量不要使用正则表达式匹配

正则表达式匹配效率较低，尽量使用字符串匹配操作。

反例：

```
String source = "a::1,b::2,c::3,d::4";
String target = source.replaceAll("::", "=");
Stringp[] targets = source.spit("::");
```

正例：

```
String source = "a::1,b::2,c::3,d::4";
String target = source.replace("::", "=");
Stringp[] targets = StringUtils.split(source, "::");
```

注意：

StringUtils.split 函数不保留空字符串。

2. 尽量使用字符替换字符串

字符串的长度不确定，而字符的长度固定为 1，查找和匹配的效率自然提高了。

反例：

```
String source = "a:1,b:2,c:3,d:4";
int index = source.indexOf(":");
String target = source.replace(":", "=");
```

正例：

```
String source = "a:1,b:2,c:3,d:4";
int index = source.indexOf(':');
String target = source.replace(':', '=');
```

3. 尽量使用 StringBuilder 进行字符串拼接

String 是 final 类，内容不可修改，所以每次字符串拼接都会生成一个新对象。StringBuilder在初始化时申请了一块内存,以后的字符串拼接都在这块内存中执行,不会申请新内存和生成新对象。

反例：

```
String s = "";
for (int i = 0; i < 10; i++) {
    if (i != 0) {
        s += ',';
    }
    s += i;
}
```

正例：

```
StringBuilder sb = new StringBuilder(128);
for (int i = 0; i < 10; i++) {
    if (i != 0) {
        sb.append(',');
    }
    sb.append(i);
}
```

4. 不要使用""+转化字符串

使用""+进行字符串转化，使用方便但是效率低，建议使用 String.valueOf.

反例：

```
int i = 12345;
String s = "" + i;
```

正例：

```
int i = 12345;
String s = String.valueOf(i);
```

5. 尽量预编译正则表达式

众所周知，Pattern.compile 方法的性能开销很大。但很不幸的是，这个方法可能隐藏在一些我们认为很方便的方法里。比如：String.matches、String.replaceAll、String.split 等函数。例如：

```
public String replaceAll(String regex, String replacement) {
    return Pattern.compile(regex).matcher(this).replaceAll(replacement);
}
```

对于多次调用这些方法，可以考虑预编译正则表达式以提高执行效率，并参考原有实现代码编写优化后的代码。

反例：

```
// 以下代码将调用N次
String source = "a:1|b:2|c:3|d:4";
boolean isMatched = source.matches("^(\\w+:\\d+\\|)*(\\w+:\\d+)$"); // true
String target = source.replaceAll("(\\w+):(\\d+)", "$1=$2"); // a=1|b=2|c=3|d=4
String[] targets = source.split("\\|"); // ["a:1","b:2","c:3","d:4"]
```

正例：

```
// 预编译正则表达式
final Pattern PATTERN1 = Pattern.compile("^(\\w+:\\d+\\|)*(\\w+:\\d+)$");
final Pattern PATTERN2 = Pattern.compile("(\\w+):(\\d+)");
final Pattern PATTERN3 = Pattern.compile("\\|");

// 以下代码将调用N次
String source = "a:1|b:2|c:3|d:4";
boolean isMatched = PATTERN1.matcher(source).matches();
String target = PATTERN2.matcher(source).replaceAll("$1=$2");
String[] targets = PATTERN3.split(source);
```

六、 数组

1. 不要使用循环拷贝数组，尽量使用 System.arraycopy 拷贝数组

推荐使用 System.arraycopy 拷贝数组，也可以使用 Arrays.copyOf 拷贝数组。

反例：

```
int[] sources = new int[] {1, 2, 3, 4, 5};
int[] targets = new int[sources.length];
for (int i = 0; i < targets.length; i++) {
    targets[i] = sources[i];
}
```

正例：

```
int[] sources = new int[] {1, 2, 3, 4, 5};
int[] targets = new int[sources.length];
System.arraycopy(sources, 0, targets, 0, targets.length);
```

2. 集合转化为类型 T 数组时，尽量传入空数组 T[0]

将集合转换为数组有 2 种形式：toArray（new T[n]）和 toArray（new T[0]）。在旧的 Java 版本中，建议使用 toArray（new T[n]），因为创建数组时所需的反射调用非常慢。在 OpenJDK6 后，反射调用是内在的，使得性能得以提高，toArray（new T[0]）比 toArray（new T[n]）效率更高。此外，toArray（new T[n]）比 toArray（new T[0]）多获取一次列表大小，如果计算列表大小耗时过长，也会导致 toArray（new T[n]）效率降低。

反例：

```
List<Integer> integerList = Arrays.asList(1, 2, 3, 4, 5, ...);
Integer[] integers = integerList.toArray(new Integer[integerList.size()]);
```

正例：

```
List<Integer> integerList = Arrays.asList(1, 2, 3, 4, 5, ...);
Integer[] integers = integerList.toArray(new Integer[0]); // 勿用new Integer[]{}
```

建议

集合应该提供一个 toArray(Class<T> clazz)方法，避免无用的空数组初始化（new T[0]）。

3. 集合转化为 Object 数组时，尽量使用 toArray()方法

转化 Object 数组时，没有必要使用 toArray[new Object[0]]，可以直接使用 toArray()。避免了类型的判断，也避免了空数组的申请，所以效率会更高。

反例：

```
List<Object> objectList = Arrays.asList(1, "2", 3, "4", 5, ...);
Object[] objects = objectList.toArray(new Object[0]);
```

正例：

```
List<Object> objectList = Arrays.asList(1, "2", 3, "4", 5, ...);
Object[] objects = objectList.toArray();
```

七、 集合

1. 初始化集合时，尽量指定集合大小

Java 集合初始化时都会指定一个默认大小，当默认大小不再满足数据需求时就会扩容，每次扩容的时间复杂度有可能是 O（n）。所以，尽量指定预知的集合大小，就能避免或减少集合的扩容次数。

反例：

```
List<UserDO> userDOList = ...;
Set<Long> userSet = new HashSet<>();
Map<Long, UserDO> userMap = new HashMap<>();
List<UserVO> userList = new ArrayList<>();
for (UserDO userDO : userDOList) {
    userSet.add(userDO.getId());
    userMap.put(userDO.getId(), userDO);
    userList.add(transUser(userDO));
}
```

正例：

```
List<UserDO> userDOList = ...;
int userSize = userDOList.size();
Set<Long> userSet = new HashSet<>(userSize);
Map<Long, UserDO> userMap = new HashMap<>((int) Math.ceil(userSize * 4.0 / 3));
List<UserVO> userList = new ArrayList<>(userSize);
for (UserDO userDO : userDOList) {
    userSet.add(userDO.getId());
    userMap.put(userDO.getId(), userDO);
    userList.add(transUser(userDO));
}
```

2. 不要使用循环拷贝集合，尽量使用 JDK 提供的方法拷贝集合

JDK 提供的方法可以一步指定集合的容量，避免多次扩容浪费时间和空间。同时，这些方法的底层也是调用 System.arraycopy 方法实现，进行数据的批量拷贝效率更高。

反例：

```
List<UserDO> user1List = ...;
List<UserDO> user2List = ...;
List<UserDO> userList = new ArrayList<>(user1List.size() + user2List.size());
for (UserDO user1 : user1List) {
    userList.add(user1);
}
for (UserDO user2 : user2List) {
    userList.add(user2);
}
```

正例：

```
List<UserDO> user1List = ...;
List<UserDO> user2List = ...;
List<UserDO> userList = new ArrayList<>(user1List.size() + user2List.size());
userList.addAll(user1List);
userList.addAll(user2List);
```

3. 尽量使用 Arrays.asList 转化数组为列表

原理与“不要使用循环拷贝集合，尽量使用 JDK 提供的方法拷贝集合”类似。

反例：

```
List<String> typeList = new ArrayList<>(8);
typeList.add("Short");
typeList.add("Integer");
typeList.add("Long");

String[] names = ...;
List<String> nameList = ...;
for (String name : names) {
    nameList.add(name);
}
```

正例：

```
List<String> typeList = Arrays.asList("Short", "Integer", "Long");

String[] names = ...;
List<String> nameList = ...;
nameList.addAll(Arrays.asList(names));
```

4. 直接迭代需要使用的集合

直接迭代需要使用的集合，无需通过其它操作获取数据。

反例：

```
Map<Long, UserDO> userMap = ...;
for (Long userId : userMap.keySet()) {
    UserDO user = userMap.get(userId);
    ...
}
```

正例：

```
Map<Long, UserDO> userMap = ...;
for (Map.Entry<Long, UserDO> userEntry : userMap.entrySet()) {
    Long userId = userEntry.getKey();
    UserDO user = userEntry.getValue();
    ...
}
```

5. 不要使用 size 方法检测空，必须使用 isEmpty 方法检测空

使用 size 方法来检测空逻辑上没有问题，但使用 isEmpty 方法使得代码更易读，并且可以获得更好的性能。任何 isEmpty 方法实现的时间复杂度都是 O（1），但是某些 size 方法实现的时间复杂度有可能是 O（n）。

反例：

```
List<UserDO> userList = ...;
if (userList.size() == 0) {
    ...
}
Map<Long, UserDO> userMap = ...;
if (userMap.size() == 0) {
    ...
}
```

正例：

```
List<UserDO> userList = ...;
if (userList.isEmpty()) {
    ...
}
Map<Long, UserDO> userMap = ...;
if (userMap.isEmpty()) {
    ...
}
```

6. 非随机访问的 List，尽量使用迭代代替随机访问

对于列表，可分为随机访问和非随机访问两类，可以用是否实现 RandomAccess 接口判断。随机访问列表，直接通过 get 获取数据不影响效率。而非随机访问列表，通过 get 获取数据效率极低。

反例：

```
LinkedList<UserDO> userDOList = ...;
int size = userDOList.size();
for (int i = 0; i < size; i++) {
    UserDO userDO = userDOList.get(i);
    ...
}
```

正例：

```
LinkedList<UserDO> userDOList = ...;
for (UserDO userDO : userDOList) {
    ...
}
```

其实，不管列表支不支持随机访问，都应该使用迭代进行遍历。

7. 尽量使用 HashSet 判断值存在

在 Java 集合类库中，List 的 contains 方法普遍时间复杂度是 O（n），而 HashSet 的时间复杂度为 O（1）。如果需要频繁调用 contains 方法查找数据，可以先将 List 转换成 HashSet。

反例：

```
List<Long> adminIdList = ...;
List<UserDO> userDOList = ...;
List<UserVO> userVOList = new ArrayList<>(userDOList.size());
for (UserDO userDO : userDOList) {
    if (adminIdList.contains(userDO.getId())) {
        userVOList.add(transUser(userDO));
    }
}
```

正例：

```
Set<Long> adminIdSet = ...;
List<UserDO> userDOList = ...;
List<UserVO> userVOList = new ArrayList<>(userDOList.size());
for (UserDO userDO : userDOList) {
    if (adminIdSet.contains(userDO.getId())) {
        userVOList.add(transUser(userDO));
    }
}
```

8. 避免先判断存在再进行获取

如果需要先判断存在再进行获取，可以直接获取并判断空，从而避免了二次查找操作。

反例：

```
public static UserVO transUser(UserDO user, Map<Long, RoleDO> roleMap) {
    UserVO userVO = new UserVO();
    userVO.setId(user.getId());
    ...
    if (roleMap.contains(user.getRoleId())) {
        RoleDO role = roleMap.get(user.getRoleId());
        userVO.setRole(transRole(role));
    }
}
```

正例：

```
public static UserVO transUser(UserDO user, Map<Long, RoleDO> roleMap) {
    UserVO userVO = new UserVO();
    userVO.setId(user.getId());
    ...
    RoleDO role = roleMap.get(user.getRoleId());
    if (Objects.nonNull(role)) {
        userVO.setRole(transRole(role));
    }
}
```

八、 异常

1. 直接捕获对应的异常

直接捕获对应的异常，避免用 instanceof 判断，效率更高代码更简洁。

反例：

```
try {
    saveData();
} catch (Exception e) {
    if (e instanceof IOException) {
        log.error("保存数据IO异常", e);
    } else {
        log.error("保存数据其它异常", e);
    }
}
```

正例：

```
try {
    saveData();
} catch (IOException e) {
    log.error("保存数据IO异常", e);
} catch (Exception e) {
    log.error("保存数据其它异常", e);
}
```

2. 尽量避免在循环中捕获异常

当循环体抛出异常后，无需循环继续执行时，没有必要在循环体中捕获异常。因为，过多的捕获异常会降低程序执行效率。

反例：

```
public Double sum(List<String> valueList) {
    double sum = 0.0D;
    for (String value : valueList) {
        try {
            sum += Double.parseDouble(value);
        } catch (NumberFormatException e) {
            return null;
        }
    }
    return sum;
}
```

正例：

```
public Double sum(List<String> valueList) {
    double sum = 0.0D;
    try {
        for (String value : valueList) {
            sum += Double.parseDouble(value);
        }
    } catch (NumberFormatException e) {
        return null;
    }
    return sum;
}
```

3. 禁止使用异常控制业务流程

相对于条件表达式，异常的处理效率更低。

反例：

```
public static boolean isValid(UserDO user) {
    try {
        return Boolean.TRUE.equals(user.getIsValid());
    } catch(NullPointerException e) {
        return false;
    }
}
```

正例：

```
public static boolean isValid(UserDO user) {
    if (Objects.isNull(user)) {
        return false;
    }
  return Boolean.TRUE.equals(user.getIsValid());
}
```

九、 缓冲区

1. 初始化时尽量指定缓冲区大小

初始化时，指定缓冲区的预期容量大小，避免多次扩容浪费时间和空间。

反例：

```
StringBuffer buffer = new StringBuffer();
StringBuilder builder = new StringBuilder();
```

正例：

```
StringBuffer buffer = new StringBuffer(1024);
StringBuilder builder = new StringBuilder(1024);
```

2. 尽量重复使用同一缓冲区

针对缓冲区，Java 虚拟机需要花时间生成对象，还要花时间进行垃圾回收处理。所以，尽量重复利用缓冲区。

反例：

```
StringBuilder builder1 = new StringBuilder(128);
builder1.append("update t_user set name = '").append(userName).append("' where id =
").append(userId);
statement.executeUpdate(builder1.toString());
StringBuilder builder2 = new StringBuilder(128);
builder2.append("select id, name from t_user where id = ").append(userId);
ResultSet resultSet = statement.executeQuery(builder2.toString());
...
```

正例：

```
StringBuilder builder = new StringBuilder(128);
builder.append("update t_user set name = '").append(userName).append("' where id =
").append(userId);
statement.executeUpdate(builder.toString());
builder.setLength(0);
builder.append("select id, name from t_user where id = ").append(userId);
ResultSet resultSet = statement.executeQuery(builder.toString());
...
```

其中，使用 setLength 方法让缓冲区重新从 0 开始。

3. 尽量设计使用同一缓冲区

为了提高程序运行效率，在设计上尽量使用同一缓冲区。

反例：

```
// 转化XML(UserDO)
public static String toXml(UserDO user) {
    StringBuilder builder = new StringBuilder(128);
    builder.append("<UserDO>");
    builder.append(toXml(user.getId()));
    builder.append(toXml(user.getName()));
    builder.append(toXml(user.getRole()));
    builder.append("</UserDO>");
    return builder.toString();
}
// 转化XML(Long)
public static String toXml(Long value) {
    StringBuilder builder = new StringBuilder(128);
    builder.append("<Long>");
    builder.append(value);
    builder.append("</Long>");
    return builder.toString();
}
...

// 使用代码
UserDO user = ...;
String xml = toXml(user);
```

正例：

```
// 转化XML(UserDO)
public static void toXml(StringBuilder builder, UserDO user) {
    builder.append("<UserDO>");
    toXml(builder, user.getId());
    toXml(builder, user.getName());
    toXml(builder, user.getRole());
    builder.append("</UserDO>");
}
// 转化XML(Long)
public static void toXml(StringBuilder builder, Long value) {
    builder.append("<Long>");
    builder.append(value);
    builder.append("</Long>");
}
...

// 使用代码
UserDO user = ...;
StringBuilder builder = new StringBuilder(1024);
toXml(builder, user);
String xml = builder.toString();
```

去掉每个转化方法中的缓冲区申请，申请一个缓冲区给每个转化方法使用。从时间上来说，节约了大量缓冲区的申请释放时间；从空间上来说，节约了大量缓冲区的临时存储空间。

4. 尽量使用缓冲流减少 IO 操作

使用缓冲流 BufferedReader、BufferedWriter、BufferedInputStream、BufferedOutputStream 等，可以大幅减少 IO 次数并提升 IO 速度。

反例：

```
try (FileInputStream input = new FileInputStream("a");
    FileOutputStream output = new FileOutputStream("b")) {
    int size = 0;
    byte[] temp = new byte[1024];
    while ((size = input.read(temp)) != -1) {
        output.write(temp, 0, size);
    }
} catch (IOException e) {
    log.error("复制文件异常", e);
}
```

正例：

```
try (BufferedInputStream input = new BufferedInputStream(new FileInputStream("a"));
    BufferedOutputStream output = new BufferedOutputStream(new FileOutputStream("b"))) {
    int size = 0;
    byte[] temp = new byte[1024];
    while ((size = input.read(temp)) != -1) {
        output.write(temp, 0, size);
    }
} catch (IOException e) {
    log.error("复制文件异常", e);
}
```

其中，可以根据实际情况手动指定缓冲流的大小，把缓冲流的缓冲作用发挥到最大。

十、 线程

1. 在单线程中，尽量使用非线程安全类

使用非线程安全类，避免了不必要的同步开销。

反例：

```
StringBuffer buffer = new StringBuffer(128);
buffer.append("select * from ").append(T_USER).append(" where id = ?");
```

正例：

```
StringBuilder buffer = new StringBuilder(128);
buffer.append("select * from ").append(T_USER).append(" where id = ?");
```

2. 在多线程中，尽量使用线程安全类

使用线程安全类，比自己实现的同步代码更简洁更高效。

反例：

```
private volatile int counter = 0;
public void access(Long userId) {
    synchronized (this) {
        counter++;
    }
    ...
}
```

正例：

```
private final AtomicInteger counter = new AtomicInteger(0);
public void access(Long userId) {
    counter.incrementAndGet();
    ...
}
```

3. 尽量减少同步代码块范围

在一个方法中，可能只有一小部分的逻辑是需要同步控制的，如果同步控制了整个方法会影响执行效率。所以，尽量减少同步代码块的范围，只对需要进行同步的代码进行同步。

反例：

```
private volatile int counter = 0;
public synchronized void access(Long userId) {
  counter++;
    ... // 非同步操作
}
```

正例：

```
private volatile int counter = 0;
public void access(Long userId) {
    synchronized (this) {
        counter++;
    }
    ... // 非同步操作
}
```

4. 尽量合并为同一同步代码块

同步代码块是有性能开销的，如果确定可以合并为同一同步代码块，就应该尽量合并为同一同步代码块。

反例：

```
// 处理单一订单
public synchronized handleOrder(OrderDO order) {
    ...
}

// 处理所有订单
public void handleOrder(List<OrderDO> orderList) {
    for (OrderDO order : orderList) {
        handleOrder(order);
    }
}
```

正例：

```
// 处理单一订单
public handleOrder(OrderDO order) {
    ...
}

// 处理所有订单
public synchronized void handleOrder(List<OrderDO> orderList) {
    for (OrderDO order : orderList) {
        handleOrder(order);
    }
}
```

5. 尽量使用线程池减少线程开销

多线程中两个必要的开销：线程的创建和上下文切换。采用线程池，可以尽量地避免这些开销。

反例：

```
public void executeTask(Runnable runnable) {
    new Thread(runnable).start();
}
```

正例：

```
private static final ExecutorService EXECUTOR_SERVICE = Executors.newFixedThreadPool(10);
public void executeTask(Runnable runnable) {
    executorService.execute(runnable);
}
```

后记

作为一名长期奋战在业务一线的“IT 民工”，没有机会去研究什么高深莫测的“理论”，只能专注于眼前看得见摸得着的“技术”，致力于做到“干一行、爱一行、专一行、精一行”。

看到重庆一农民工携子举包扛货照片，仿佛看到了自己，不禁填词一首：

《如梦令·携子举包扛货》

携子举包扛货，皆是生活所迫。
虽苦也觉甜，惟愿小儿安妥。
沉默，沉默，长大可别学我。

Java 函数优雅之道

导语

随着软件项目代码的日积月累，系统维护成本变得越来越高，是所有软件团队面临的共同问题。持续地优化代码，提高代码的质量，是提升系统生命力的有效手段之一。软件系统思维有句话“Less coding，more thinking（少编码、多思考）”，也有这么一句俚语“Think more，code less（思考越多，编码越少）”。所以，我们在编码中多思考多总结，努力提升自己的编码水平，才能编写出更优雅、更高质、更高效的代码。

本文总结了一套与 Java 函数相关的编码规则，旨在给广大 Java 程序员一些编码建议，有助于大家编写出更优雅、更高质、更高效的代码。这套编码规则，通过在高德采集部门的实践，已经取得了不错的成效。

一、 使用通用工具函数

案例 1：字符串比较

现象描述：

不完善的写法

```
thisName != null && thisName.equals(name);
```

更完善的写法

```
(thisName == name) || (thisName != null && thisName.equals(name));
```

建议方案：

```
Objects.equals(name, thisName);
```

案例 2：判断列表为空

现象描述：

```
!(list == null || list.isEmpty());
```

建议方案：

```
import org.apache.commons.collections4.CollectionUtils;
CollectionUtils.isNotEmpty(list);
```

主要收益：

- 函数式编程，业务代码减少，逻辑一目了然；
- 通用工具函数，逻辑考虑周全，出问题概率低。

二、 拆分超大函数

当一个函数超过 80 行后，就属于超大函数，需要进行拆分。

案例 1：每一个代码块都可以封装为一个函数

每一个代码块必然有一个注释，用于解释这个代码块的功能。如果代码块前方有一行注释，就是在提醒你——可以将这段代码替换成一个函数，而且可以在注释的基础上给这个函数命名。如果函数有一个描述恰当的名字，就不需要去看内部代码究竟是如何实现的。

现象描述：

```
// 每日生活函数
public void liveDaily() {
    // 吃饭
    // 吃饭相关代码几十行

    // 编码
    // 编码相关代码几十行

    // 睡觉
    // 睡觉相关代码几十行
}
```

建议方案：

```
// 每日生活函数
public void liveDaily() {
    // 吃饭
    eat();

    // 编码
    code();

    // 睡觉
    sleep();
}

// 吃饭函数
private void eat() {
    // 吃饭相关代码
}

// 编码函数
private void code() {
    // 编码相关代码
}

// 睡觉函数
private void sleep() {
    // 睡觉相关代码
}
```

案例 2：每一个循环体都可以封装为一个函数

现象描述：

```
// 生活函数
public void live() {
    while (isAlive) {
        // 吃饭
        eat();

        // 编码
        code();

        // 睡觉
        sleep();
    }
}
```

建议方案：

```
// 生活函数
public void live() {
    while (isAlive) {
        // 每日生活
        liveDaily();
    }
}

// 每日生活函数
private void liveDaily() {
    // 吃饭
    eat();

    // 编码
    code();

    // 睡觉
    sleep();
}
```

案例 3：每一个条件体都可以封装为一个函数

现象描述：

```
// 外出函数
public void goOut() {
    // 判断是否周末
    // 判断是否周末：是周末则游玩
    if (isWeekday()) {
        // 游玩代码几十行
    }
    // 判断是否周末：非周末则工作
    else {
        // 工作代码几十行
    }
}
```

建议方案：

```
// 外出函数
public void goOut() {
    // 判断是否周末
    // 判断是否周末：是周末则游玩
    if (isWeekday()) {
        play();
    }
    // 判断是否周末：非周末则工作
    else {
        work();
    }
}

// 游玩函数
private void play() {
    // 游玩代码几十行
}

// 工作函数
private void work() {
    // 工作代码几十行
}
```

主要收益：

- 函数越短小精悍，功能就越单一，往往生命周期较长；
- 一个函数越长，就越不容易理解和维护，维护人员不敢轻易修改；
- 在过长函数中，往往含有难以发现的重复代码。

三、 同一函数内代码块级别尽量一致

案例 1：代码块级别尽量保持一致

现象描述：

```
// 每日生活函数
public void liveDaily() {
    // 吃饭
    eat();

    // 编码
    code();

    // 睡觉
    // 睡觉相关代码几十行
}
```

很明显，睡觉这块代码块，跟 eat（吃饭）和 code（编码）不在同一级别上，显得比较突兀。如果把写代码比作写文章，eat（吃饭）和 code（编码）是段落大意，而睡觉这块代码块属于一个详细段落。而在 liveDaily（每日生活）这个函数上，只需要写出主要流程（段落大意）即可。

建议方案：

```
public void liveDaily() {
    // 吃饭
    eat();

    // 编码
    code();

    // 睡觉
    sleep();
}

// 睡觉
private void sleep() {
    // 睡觉相关代码
}
```

主要收益：

- 函数调用表明用途，函数实现表达逻辑，层次分明便于理解；
- 不用层次的代码块放在一个函数中，容易让人觉得代码头重脚轻。

四、 封装相同功能代码为函数

案例 1：封装相同代码为函数

现象描述：

```
// 禁用用户函数
public void disableUser() {
    // 禁用黑名单用户
    List<Long> userIdList = queryBlackUser();
    for (Long userId : userIdList) {
        User userUpdate = new User();
        userUpdate.setId(userId);
        userUpdate.setEnable(Boolean.FALSE);
        userDAO.update(userUpdate);
    }

    // 禁用过期用户
    userIdList = queryExpiredUser();
    for (Long userId : userIdList) {
        User userUpdate = new User();
        userUpdate.setId(userId);
        userUpdate.setEnable(Boolean.FALSE);
        userDAO.update(userUpdate);
    }
}
```

建议方案：

```
// 禁用用户函数
public void disableUser() {
    // 禁用黑名单用户
    List<Long> userIdList = queryBlackUser();
    for (Long userId : userIdList) {
        disableUser(userId);
    }

    // 禁用过期用户
    userIdList = queryExpiredUser();
    for (Long userId : userIdList) {
        disableUser(userId);
    }
}

// 禁用用户函数
private void disableUser(Long userId) {
    User userUpdate = new User();
    userUpdate.setId(userId);
    userUpdate.setEnable(Boolean.FALSE);
    userDAO.update(userUpdate);
}
```

案例 2：封装相似代码为函数

封装相似代码为函数，差异性通过函数参数控制。

现象描述：

```
// 通过工单函数
public void adoptOrder(Long orderId) {
    Order orderUpdate = new Order();
    orderUpdate.setId(orderId);
    orderUpdate.setStatus(OrderStatus.ADOPTED);
    orderUpdate.setAuditTime(new Date());
    orderDAO.update(orderUpdate);
}

// 驳回工单函数
public void rejectOrder(Long orderId) {
    Order orderUpdate = new Order();
    orderUpdate.setId(orderId);
    orderUpdate.setStatus(OrderStatus.REJECTED);
    orderUpdate.setAuditTime(new Date());
    orderDAO.update(orderUpdate);
}
```

建议方案：

```
// 通过工单函数
public void adoptOrder(Long orderId) {
    auditOrder(orderId, OrderStatus.ADOPTED);
}

// 驳回工单函数
public void rejectOrder(Long orderId) {
    auditOrder(orderId, OrderStatus.REJECTED);
}

// 审核工单函数
private void auditOrder(Long orderId, OrderStatus orderStatus) {
    Order orderUpdate = new Order();
    orderUpdate.setId(orderId);
    orderUpdate.setStatus(orderStatus);
    orderUpdate.setAuditTime(new Date());
    orderDAO.update(orderUpdate);
}
```

主要收益：

- 封装公共函数，减少代码行数，提高代码质量；
- 封装公共函数，使业务代码更精炼，可读性可维护性更强。

五、 封装获取参数值函数

案例 1：把获取参数值封装为函数

现象描述：

```
// 是否通过函数
public boolean isPassed(Long userId) {
    // 获取通过阈值
    double thisPassThreshold = PASS_THRESHOLD;
    if (Objects.nonNull(passThreshold)) {
        thisPassThreshold = passThreshold;
    }

    // 获取通过率
    double passRate = getPassRate(userId);

    // 判读是否通过
    return passRate >= thisPassThreshold;
}
```

建议方案：

```
// 是否通过函数
public boolean isPassed(Long userId) {
    // 获取通过阈值
    double thisPassThreshold = getPassThreshold();

    // 获取通过率
    double passRate = getPassRate(userId);

    // 判读是否通过
    return passRate >= thisPassThreshold;
}

// 获取通过阈值函数
private double getPassThreshold() {
    if (Objects.nonNull(passThreshold)) {
        return passThreshold;
    }
    return PASS_THRESHOLD;
}
```

主要收益：

- 把获取参数值从业务函数中独立，使业务逻辑更清晰；
- 封装的获取参数值为独立函数，可以在代码中重复使用。

六、 通过接口参数化封装相同逻辑

案例 1：通过接口参数化封装相同逻辑

现象描述：

```
// 发送审核员结算数据函数
public void sendAuditorSettleData() {
    List<WorkerSettleData> settleDataList = auditTaskDAO.statAuditorSettleData();
    for (WorkerSettleData settleData : settleDataList) {
        WorkerPushData pushData = new WorkerPushData();
        pushData.setId(settleData.getWorkerId());
        pushData.setType(WorkerPushDataType.AUDITOR);
        pushData.setData(settleData);
        pushService.push(pushData);
    }
}

// 发送验收员结算数据函数
public void sendCheckerSettleData() {
    List<WorkerSettleData> settleDataList = auditTaskDAO.statCheckerSettleData();
    for (WorkerSettleData settleData : settleDataList) {
        WorkerPushData pushData = new WorkerPushData();
        pushData.setId(settleData.getWorkerId());
        pushData.setType(WorkerPushDataType.CHECKER);
        pushData.setData(settleData);
        pushService.push(pushData);
    }
}
```

建议方案：

```
// 发送审核员结算数据函数
public void sendAuditorSettleData() {
    sendWorkerSettleData(WorkerPushDataType.AUDITOR, () ->
auditTaskDAO.statAuditorSettleData());
}

// 发送验收员结算数据函数
public void sendCheckerSettleData() {
    sendWorkerSettleData(WorkerPushDataType.CHECKER, () ->
auditTaskDAO.statCheckerSettleData());
}

// 发送作业员结算数据函数
public void sendWorkerSettleData(WorkerPushDataType dataType, WorkerSettleDataProvider
dataProvider) {
    List<WorkerSettleData> settleDataList = dataProvider.statWorkerSettleData();
    for (WorkerSettleData settleData : settleDataList) {
        WorkerPushData pushData = new WorkerPushData();
        pushData.setId(settleData.getWorkerId());
        pushData.setType(dataType);
        pushData.setData(settleData);
        pushService.push(pushData);
    }
}

// 作业员结算数据提供者接口
private interface WorkerSettleDataProvider {
    // 统计作业员结算数据
    public List<WorkerSettleData> statWorkerSettleData();
}
```

主要收益：

- 把核心逻辑从各个业务函数中抽析，使业务代码更清晰更易维护；
- 避免重复性代码多次编写，精简重复函数越多收益越大。

七、 减少函数代码层级

如果要使函数优美，建议函数代码层级在 1-4 之间，过多的缩进会让函数难以阅读。

案例 1：利用 return 提前返回函数

现象描述：

```
// 获取用户余额函数
public Double getUserBalance(Long userId) {
    User user = getUser(userId);
    if (Objects.nonNull(user)) {
        UserAccount account = user.getAccount();
        if (Objects.nonNull(account)) {
            return account.getBalance();
        }
    }
    return null;
}
```

建议方案：

```
// 获取用户余额函数
public Double getUserBalance(Long userId) {
    // 获取用户信息
    User user = getUser(userId);
    if (Objects.isNull(user)) {
        return null;
    }

    // 获取用户账户
    UserAccount account = user.getAccount();
    if (Objects.isNull(account)) {
        return null;
    }

    // 返回账户余额
    return account.getBalance();
}
```

案例 2：利用 continue 提前结束循环

现象描述：

```
// 获取合计余额函数
public double getTotalBalance(List<User> userList) {
    // 初始合计余额
    double totalBalance = 0.0D;

    // 依次累加余额
    for (User user : userList) {
        // 获取用户账户
        UserAccount account = user.getAccount();
        if (Objects.nonNull(account)) {
            // 累加用户余额
            Double balance = account.getBalance();
            if (Objects.nonNull(balance)) {
                totalBalance += balance;
            }
        }
    }

    // 返回合计余额
    return totalBalance;
}
```

建议方案：

```
// 获取合计余额函数
public double getTotalBalance(List<User> userList) {
    // 初始合计余额
    double totalBalance = 0.0D;

    // 依次累加余额
    for (User user : userList) {
        // 获取用户账户
        UserAccount account = user.getAccount();
        if (Objects.isNull(account)) {
            continue;
        }

        // 累加用户余额
        Double balance = account.getBalance();
        if (Objects.nonNull(balance)) {
            totalBalance += balance;
        }
    }

    // 返回合计余额
    return totalBalance;
}
```

特殊说明：

其它方式：在循环体中，先调用案例 1 的函数 getUserBalance（获取用户余额），再进行对余额进行累加。

在循环体中，建议最多使用一次 continue。如果需要有使用多次 continue 的需求，建议把循环体封装为一个函数。

案例 3：利用条件表达式函数减少层级

请参考下一章的“案例 2：把复杂条件表达式封装为函数”

主要收益：

- 代码层级减少，代码缩进减少；
- 模块划分清晰，方便阅读维护。

八、 封装条件表达式函数

案例 1：把简单条件表达式封装为函数

现象描述：

```
// 获取门票价格函数
public double getTicketPrice(Date currDate) {
    if (Objects.nonNull(currDate) && currDate.after(DISCOUNT_BEGIN_DATE)
        && currDate.before(DISCOUNT_END_DATE)) {
        return TICKET_PRICE * DISCOUNT_RATE;
    }
    return TICKET_PRICE;
}
```

建议方案：

```
// 获取门票价格函数
public double getTicketPrice(Date currDate) {
    if (isDiscountDate(currDate)) {
        return TICKET_PRICE * DISCOUNT_RATE;
    }
    return TICKET_PRICE;
}

// 是否折扣日期函数
private static boolean isDiscountDate(Date currDate) {
    return Objects.nonNull(currDate) &&
currDate.after(DISCOUNT_BEGIN_DATE)
        && currDate.before(DISCOUNT_END_DATE);
}
```

案例 2：把复杂条件表达式封装为函数

现象描述：

```
// 获取土豪用户列表
public List<User> getRichUserList(List<User> userList) {
    // 初始土豪用户列表
    List<User> richUserList = new ArrayList<>();

    // 依次查找土豪用户
    for (User user : userList) {
        // 获取用户账户
        UserAccount account = user.getAccount();
        if (Objects.nonNull(account)) {
            // 判断用户余额
            Double balance = account.getBalance();
            if (Objects.nonNull(balance) && balance.compareTo(RICH_THRESHOLD) >= 0) {
                // 添加土豪用户
                richUserList.add(user);
            }
        }
    }

    // 返回土豪用户列表
    return richUserList;
}
```

建议方案：

```
// 获取土豪用户列表
public List<User> getRichUserList(List<User> userList) {
    // 初始土豪用户列表
    List<User> richUserList = new ArrayList<>();

    // 依次查找土豪用户
    for (User user : userList) {
        // 判断土豪用户
        if (isRichUser(user)) {
            // 添加土豪用户
            richUserList.add(user);
        }
    }

    // 返回土豪用户列表
    return richUserList;
}

// 是否土豪用户
private boolean isRichUser(User user) {
    // 获取用户账户
    UserAccount account = user.getAccount();
    if (Objects.isNull(account)) {
        return false;
    }

    // 获取用户余额
    Double balance = account.getBalance();
    if (Objects.isNull(balance)) {
        return false;
    }

    // 比较用户余额
    return balance.compareTo(RICH_THRESHOLD) >= 0;
}
```

以上代码也可以用采用流式（Stream）编程的过滤来实现。

主要收益：

- 把条件表达式从业务函数中独立，使业务逻辑更清晰；
- 封装的条件表达式为独立函数，可以在代码中重复使用。

九、 尽量避免不必要的空指针判断

本章只适用于项目内部代码，并且是自己了解的代码，才能够尽量避免不必要的空指针判断。对于第三方中间件和系统接口，必须做好空指针判断，以保证代码的健壮性。

案例 1：调用函数保证参数不为空，被调用函数尽量避免不必要的空指针判断

现象描述：

```
// 创建用户信息
User user = new User();
... // 赋值用户相关信息
createUser(user);

// 创建用户函数
private void createUser(User user){
    // 判断用户为空
    if(Objects.isNull(user)) {
        return;
    }

    // 创建用户信息
    userDAO.insert(user);
    userRedis.save(user);
}
```

建议方案：

```
// 创建用户信息
User user = new User();
... // 赋值用户相关信息
createUser(user);

// 创建用户函数
private void createUser(User user){
    // 创建用户信息
    userDAO.insert(user);
    userRedis.save(user);
}
```

案例 2：被调用函数保证返回不为空，调用函数尽量避免不必要的空指针判断

现象描述：

```
// 保存用户函数
public void saveUser(Long id, String name) {
    // 构建用户信息
    User user = buildUser(id, name);
    if (Objects.isNull(user)) {
        throw new BizRuntimeException("构建用户信息为空");
    }

    // 保存用户信息
    userDAO.insert(user);
    userRedis.save(user);
}

// 构建用户函数
private User buildUser(Long id, String name) {
    User user = new User();
    user.setId(id);
    user.setName(name);
    return user;
}
```

建议方案：

```
// 保存用户函数
public void saveUser(Long id, String name) {
    // 构建用户信息
    User user = buildUser(id, name);

    // 保存用户信息
    userDAO.insert(user);
    userRedis.save(user);
}

// 构建用户函数
private User buildUser(Long id, String name) {
    User user = new User();
    user.setId(id);
    user.setName(name);
    return user;
}
```

案例 3：赋值逻辑保证列表数据项不为空，处理逻辑尽量避免不必要的空指针判断

现象描述：

```
// 查询用户列表
List<UserDO> userList = userDAO.queryAll();
if (CollectionUtils.isEmpty(userList)) {
    return;
}

// 转化用户列表
List<UserVO> userVoList = new ArrayList<>(userList.size());
for (UserDO user : userList) {
    UserVO userVo = new UserVO();
    userVo.setId(user.getId());
    userVo.setName(user.getName());
    userVoList.add(userVo);
}

// 依次处理用户
for (UserVO userVo : userVoList) {
    // 判断用户为空
    if (Objects.isNull(userVo)) {
        continue;
    }

    // 处理相关逻辑
    ...
}
```

建议方案：

```
// 查询用户列表
List<UserDO> userList = userDAO.queryAll();
if (CollectionUtils.isEmpty(userList)) {
    return;
}

// 转化用户列表
List<UserVO> userVoList = new ArrayList<>(userList.size());
for (UserDO user : userList) {
    UserVO userVo = new UserVO();
    userVo.setId(user.getId());
    userVo.setName(user.getName());
    userVoList.add(userVo);
}

// 依次处理用户
for (UserVO userVo : userVoList) {
    // 处理相关逻辑
    ...
}
```

案例 4：MyBatis 查询函数返回列表和数据项不为空，可以不用空指针判断

MyBatis 是一款优秀的持久层框架，是在项目中使用的最广泛的数据库中间件之一。通过对 MyBatis 源码进行分析，查询函数返回的列表和数据项都不为空，在代码中可以不用进行空指针判断。

现象描述：

这种写法没有问题，只是过于保守了。

```
// 查询用户函数
public List<UserVO> queryUser(Long id, String name) {
    // 查询用户列表
    List<UserDO> userList = userDAO.query(id, name);
    if (Objects.isNull(userList)) {
        return Collections.emptyList();
    }

    // 转化用户列表
    List<UserVO> voList = new ArrayList<>(userList.size());
    for (UserDO user : userList) {
        // 判断对象为空
        if (Objects.isNull(user)) {
            continue;
        }

        // 添加用户信息
        UserVO vo = new UserVO();
        BeanUtils.copyProperties(user, vo);
        voList.add(vo);
    }

    // 返回用户列表
    return voList;
}
```

建议方案：

```
// 查询用户函数
public List<UserVO> queryUser(Long id, String name) {
    // 查询用户列表
    List<UserDO> userList = userDAO.query(id, name);

    // 转化用户列表
    List<UserVO> voList = new ArrayList<>(userList.size());
    for (UserDO user : userList) {
        UserVO vo = new UserVO();
        BeanUtils.copyProperties(user, vo);
        voList.add(vo);
    }

    // 返回用户列表
    return voList;
}
```

主要收益：

- 避免不必要的空指针判断，精简业务代码处理逻辑，提高业务代码运行效率；
- 这些不必要的空指针判断，基本属于永远不执行的 Death 代码，删除有助于代码维护。

十、 内部函数尽量使用基础类型

案例 1：内部函数参数尽量使用基础类型

现象描述：

```
// 调用代码
double price = 5.1D;
int number = 9;
double total = calculate(price, number);

// 计算金额函数
private double calculate(Double price, Integer number) {
    return price * number;
}
```

建议方案：

```
// 调用代码
double price = 5.1D;
int number = 9;
double total = calculate(price, number);

// 计算金额函数
private double calculate(double price, int number) {
    return price * number;
}
```

案例 2：内部函数返回值尽量使用基础类型

现象描述：

```
// 获取订单总额函数
public double getOrderAmount(List<Product> productList) {
    double amount = 0.0D;
    for (Product product : productList) {
        if (Objects.isNull(product) || Objects.isNull(product.getPrice())
            || Objects.isNull(product.getNumber())) {
            continue;
        }
        amount += calculate(product.getPrice(), product.getNumber());
    }
    return amount;
}

// 计算金额函数
private Double calculate(double price, double number) {
    return price * number;
}
```

建议方案：

```
// 获取订单总额函数
public double getOrderAmount(List<Product> productList) {
    double amount = 0.0D;
    for (Product product : productList) {
        if (Objects.isNull(product) || Objects.isNull(product.getPrice())
            || Objects.isNull(product.getNumber())) {
            continue;
        }
        amount += calculate(product.getPrice(), product.getNumber());
    }
    return amount;
}

// 计算金额函数
private double calculate(double price, double number) {
    return price * number;
}
```

主要收益：

- 内部函数尽量使用基础类型，避免了隐式封装类型的打包和拆包；
- 内部函数参数使用基础类型，用语法上避免了内部函数的参数空指针判断；
- 内部函数返回值使用基础类型，用语法上避免了调用函数的返回值空指针判断。

十一、 尽量避免返回的数组和列表为 null

案例 1：尽量避免返回的数组为 null，引起不必要的空指针判断

现象描述：

```
// 调用代码
UserVO[] users = queryUser();
if (Objects.nonNull(users)) {
    for (UserVO user : users) {
        // 处理用户信息
    }
}

// 查询用户函数
private UserVO[] queryUser() {
    // 查询用户列表
    List<UserDO> userList = userDAO.queryAll();
    if (CollectionUtils.isEmpty(userList)) {
        return null;
    }

    // 转化用户数组
    UserVO[] users = new UserVO[userList.size()];
    for (int i = 0; i < userList.size(); i++) {
        UserDO user = userList.get(i);
        users[i] = new UserVO();
        users[i].setId(user.getId());
        users[i].setName(user.getName());
    }

    // 返回用户数组
    return users;
}
```

建议方案：

```
// 调用代码
UserVO[] users = queryUser();
for (UserVO user : users) {
    // 处理用户信息
}

// 查询用户函数
private UserVO[] queryUser() {
    // 查询用户列表
    List<UserDO> userList = userDAO.queryAll();
    if (CollectionUtils.isEmpty(userList)) {
        return new UserVO[0];
    }

    // 转化用户数组
    UserVO[] users = new UserVO[userList.size()];
    for (int i = 0; i < userList.size(); i++) {
        UserDO user = userList.get(i);
        users[i] = new UserVO();
        users[i].setId(user.getId());
        users[i].setName(user.getName());
    }

    // 返回用户数组
    return users;
}
```

案例 2：尽量避免返回的列表为 null，引起不必要的空指针判断

现象描述：

```
// 调用代码
List<UserVO> userList = queryUser();
if (Objects.nonNull(userList)) {
    for (UserVO user : userList) {
        // 处理用户信息
    }
}

// 查询用户函数
private List<UserVO> queryUser(){
    // 查询用户列表
    List<UserDO> userList = userDAO.queryAll();
    if(CollectionUtils.isEmpty(userList)) {
        return null;
    }

    // 转化用户列表
    List<UserVO> userVoList = new ArrayList<>(userList.size());
    for(UserDO user : userList) {
        UserVO userVo = new UserVO();
        userVo.setId(user.getId());
        userVo.setName(user.getName());
        userVoList.add(userVo);
    }

    // 返回用户列表
    return userVoList;
}
```

建议方案：

```
// 调用代码
List<UserVO> userList = queryUser();
for (UserVO user : userList) {
   // 处理用户信息
 }

// 查询用户函数
private List<UserVO> queryUser(){
    // 查询用户列表
    List<UserDO> userList = userDAO.queryAll();
    if(CollectionUtils.isEmpty(userList)) {
        return Collections.emptyList();
    }

    // 转化用户列表
    List<UserVO> userVoList = new ArrayList<>(userList.size());
    for(UserDO user : userList) {
        UserVO userVo = new UserVO();
        userVo.setId(user.getId());
        userVo.setName(user.getName());
        userVoList.add(userVo);
    }

    // 返回用户列表
    return userVoList;
}
```

主要收益：

- 保证返回的数组和列表不为 null，避免调用函数的空指针判断。

十二、 封装函数传入参数

案例 1：当传入参数过多时，应封装为参数类

Java 规范不允许函数参数太多，不便于维护也不便于扩展。

现象描述：

```
// 修改用户函数
public void modifyUser(Long id, String name, String phone, Integer age,
    Integer sex, String address, String description) {
    // 具体实现逻辑
}
```

建议方案：

```
// 修改用户函数
public void modifyUser(User user) {
    // 具体实现内容
}

// 用户类
@Getter
@Setter
@ToString
private class User{
    private Long id;
    private String name;
    private String phone;
    private Integer age;
    private Integer sex;
    private String address;
    private String description;
}
```

案例 2：当传入成组参数时，应封装为参数类

既然参数成组出现，就需要封装一个类去描述这种现象。

现象描述：

```
// 获取距离函数
public double getDistance(double x1, double y1, double x2, double y2) {
    // 具体实现逻辑
}
```

建议方案：

```
// 获取距离函数
public double getDistance(Point point1, Point point2) {
    // 具体实现逻辑
}

// 点类
@Getter
@Setter
@ToString
private class Point{
    private double x;
    private double y;
}
```

主要收益：

- 封装过多函数参数为类，使函数更便于扩展和维护；
- 封装成组函数参数为类，使业务概念更明确更清晰。

十三、 尽量用函数替换匿名内部类的实现

Java 匿名内部类的优缺点：

- 在匿名内部类（包括 Lambda 表达式）中可以直接访问外部类的成员，包括类的成员变量、函数的内部变量。正因为可以随意访问外部变量，所以会导致代码边界不清晰。
- 首先推荐用 Lambda 表达式简化匿名内部类，其次推荐用函数替换复杂的 Lambda 表达式的实现。

案例 1：尽量用函数替换匿名内部类（包括 Lambda 表达式）的实现

现象描述：

```
// 发送结算数据
sendWorkerSettleData(WorkerPushDataType.CHECKER, () -> {
  Date beginDate = DateUtils.addDays(currDate, -aheadDays);
  Date endDate = DateUtils.addDays(currDate, 1);
  return auditTaskDAO.statCheckerSettleData(beginDate, endDate);
});
```

建议方案：

```
// 发送结算数据
sendWorkerSettleData(WorkerPushDataType.CHECKER, () -> statCheckerSettleData(currDate,
aheadDays));

// 统计验收员结算数据函数
private List<WorkerSettleData> statCheckerSettleData(Date currDate, int aheadDays) {
    Date beginDate = DateUtils.addDays(currDate, -aheadDays);
    Date endDate = DateUtils.addDays(currDate, 1);
    return auditTaskDAO.statCheckerSettleData(beginDate, endDate);
}
```

其实，还有一个更简单的办法。在调用函数 sendWorkerSettleData（发送作业员结算数据）之前计算开始日期、结束日期，就直接可以用函数 auditTaskDAO.statCheckerSettleData（beginDate，endDate）代替匿名内部类实现。

案例 2：拆分复杂匿名内部类实现接口为多个函数式接口

如果一个匿名内部类实现的接口几个函数间关联性不大，可以把这个接口拆分为几个函数式接口，便于使用 Lambda 表达式。

现象描述：

```
// 清除过期数据
cleanExpiredData("用户日志表", new CleanExpiredDataOperator() {
    @Override
    public List<Date> queryExpiredDate(Integer remainDays) {
        return userDAO.queryExpiredDate(remainDays);
    }
    @Override
    public void cleanExpiredData(Date expiredDate) {
        userDAO.cleanExpiredData(expiredDate);
    }
});

// 清除过期数据函数
private void cleanExpiredData(String tableName, CleanExpiredDataOperator ,
cleanExpiredDataOperator) {
    // 功能实现代码
}

// 清除过期操作接口
interface CleanExpiredDataOperator {
    // 查询过期日期
    public List<Date> queryExpiredDate(Integer remainDays);
    // 清除过期数据
    public void cleanExpiredData(Date expiredDate);
}
```

建议方案：

```
// 清除过期数据
cleanExpiredData("用户日志表", userDAO::queryExpiredDate,userDAO::cleanExpiredData);

// 清除过期数据函数
private void cleanExpiredData(String tableName, QueryExpiredDateOperator
queryExpiredDateOperator, CleanExpiredDataOperator cleanExpiredDataOperator) {
    // 功能实现代码
}

// 查询过期日期接口
interface QueryExpiredDateOperator {
    // 查询过期日期
    public List<Date> queryExpiredDate(Integer remainDays);
}

// 清除过期操作接口
interface CleanExpiredDataOperator {
    // 清除过期数据
    public void cleanExpiredData(Date expiredDate);
}
```

主要收益：

- 定义函数并指定参数，明确规定了匿名内部类的代码边界；
- 利用 Lambda 表达式简化匿名内部类实现，使代码更简洁。

十四、 利用 return 精简不必要的代码

案例 1：删除不必要的 if

现象描述：

```
// 是否通过函数
public boolean isPassed(Double passRate) {
    if (Objects.nonNull(passRate) && passRate.compareTo(PASS_THRESHOLD) >= 0) {
        return true;
    }
    return false;
}
```

建议方案：

```
// 是否通过函数
public boolean isPassed(Double passRate) {
    return Objects.nonNull(passRate) && passRate.compareTo(PASS_THRESHOLD) >= 0;
}
```

案例 2：删除不必要的 else

现象描述：

```
// 结算工资函数
public double settleSalary(Long workId, int workDays) {
    // 根据是否合格处理
    if (isQualified(workId)) {
        return settleQualifiedSalary(workDays);
    } else {
        return settleUnqualifiedSalary(workDays);
    }
}
```

建议方案：

```
// 结算工资函数
public double settleSalary(Long workId, int workDays) {
    // 根据是否合格处理
    if (isQualified(workId)) {
        return settleQualifiedSalary(workDays);
    }
    return settleUnqualifiedSalary(workDays);
}
```

案例 3：删除不必要的变量

现象描述：

```
// 查询用户函数
public List<UserDO> queryUser(Long id, String name) {
    UserQuery userQuery = new UserQuery();
    userQuery.setId(id);
    userQuery.setName(name);
    List<UserDO> userList = userDAO.query(userQuery);
    return userList;
}
```

建议方案：

```
// 查询用户函数
public List<UserDO> queryUser(Long id, String name) {
    UserQuery userQuery = new UserQuery();
    userQuery.setId(id);
    userQuery.setName(name);
    return userDAO.query(userQuery);
}
```

主要收益：

- 精简不必要的代码，让代码看起来更清爽。

十五、 利用临时变量优化代码

在一些代码中，经常会看到 a.getB().getC()...getN()的写法，姑且叫做“函数的级联调用”，代码健壮性和可读性太差。

建议：杜绝函数的级联调用，利用临时变量进行拆分，并做好对象空指针检查。

案例 1：利用临时变量厘清逻辑

现象描述：

```
// 是否土豪用户函数
private boolean isRichUser(User user) {
    return Objects.nonNull(user.getAccount())
        && Objects.nonNull(user.getAccount().getBalance())
        && user.getAccount().getBalance().compareTo(RICH_THRESHOLD) >= 0;
}
```

这是精简代码控的最爱，但是可读性实在太差。

建议方案：

```
// 是否土豪用户函数
private boolean isRichUser(User user) {
    // 获取用户账户
    UserAccount account = user.getAccount();
    if (Objects.isNull(account)) {
        return false;
    }

    // 获取用户余额
    Double balance = account.getBalance();
    if (Objects.isNull(balance)) {
        return false;
    }

    // 比较用户余额
    return balance.compareTo(RICH_THRESHOLD) >= 0;
}
```

这个方案，增加了代码行数，但是逻辑更清晰。有时候，当代码的精简性和可读性发生冲突时，个人更偏向于保留代码的可读性。

案例 2：利用临时变量精简代码

现象描述：

```
// 构建用户函数
public UserVO buildUser(UserDO user) {
    UserVO vo = new UserVO();
    vo.setId(user.getId());
    vo.setName(user.getName());
    if (Objects.nonNull(user.getAccount())) {
        vo.setBalance(user.getAccount().getBalance());
        vo.setDebt(user.getAccount().getDebt());
    }
    return vo;
}
```

这么写，大概是为了节约一个临时变量吧。

建议方案：

```
// 构建用户函数
public UserVO buildUser(UserDO user) {
    UserVO vo = new UserVO();
    vo.setId(user.getId());
    vo.setName(user.getName());
    UserAccount account = user.getAccount();
    if (Objects.nonNull(account)) {
        vo.setBalance(account.getBalance());
        vo.setDebt(account.getDebt());
    }
    return vo;
}
```

主要收益：

- 利用临时变量厘清逻辑，显得业务逻辑更清晰；
- 利用临时变量精简代码，看变量名称即知其义，减少了大量无用代码；
- 如果获取函数比较复杂耗时，利用临时变量可以提高运行效率；
- 利用临时变量避免函数的级联调用，可有效预防空指针异常。

十六、 仅保留函数需要的参数

案例 1：删除多余的参数

现象描述：

```
// 修改用户状态函数
private void modifyUserStatus(Long userId, Integer status, String unused) {
    userCache.modifyStatus(userId, status);
    userDAO.modifyStatus(userId, status);
}
```

其中，unused 参数是无用参数。

建议方案：

```
// 修改用户状态函数
private void modifyUserStatus(Long userId, Integer status) {
    userCache.modifyStatus(userId, status);
    userDAO.modifyStatus(userId, status);
}
```

案例 2：用属性取代对象

现象描述：

```
// 删除用户函数
private void deleteUser(User user) {
    userCache.delete(user.getId());
    userDAO.delete(user.getId());
}
```

建议方案：

```
// 删除用户函数
private void deleteUser(Long userId) {
    userCache.delete(userId);
    userDAO.delete(userId);
}
```

调用函数时，参数对象不需要专门构建，而函数使用其属性超过 3 个，可以不必使用该规则。

主要收益：

- 仅保留函数需要的参数，明确了调用时需要赋值的参数，避免了调用时还要去构造些无用参数。

十七、 仅在需要时才定义变量

案例 1：仅在需要时才定义变量

一些具有 VB、C++、PLSQL 编程经验的人，习惯于把函数所需的变量定义在函数体最前面。

现象描述：

```
// 冒泡排序函数
public void bubbleSort(int[] array) {
    // 定义相关变量
    boolean flag;
    int i, j, length, temp;

    // 检查不用排序
    if (Objects.isNull(array) || array.length <= 1) {
        return;
    }

    // 进行冒泡排序
    length = array.length;
    for (i = 0; i < length; ++i) {
        flag = false;
        for (j = 0; j < length - i - 1; ++j) {
            if (array[j] > array[j + 1]) {
                temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
                flag = true;
            }
        }
        if (!flag) {
            break;
        }
    }
}
```

建议方案：

```
// 冒泡排序函数
public void bubbleSort(int[] array) {
    // 检查不用排序
    if (Objects.isNull(array) || array.length <= 1) {
        return;
    }

    // 进行冒泡排序
    int length = array.length;
    for (int i = 0; i < length; ++i) {
        boolean flag = false;
        for (int j = 0; j < length - i - 1; ++j) {
            if (array[j] > array[j + 1]) {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
                flag = true;
            }
        }
        if (!flag) {
            break;
        }
    }
}
```

主要收益：

- 变量在使用时定义，更容易让人理解变量的含义；
- 如果函数存在提前返回语句，避免了一些不必要的变量定义和初始化。

后记

“众人拾柴火焰高”。如果有更多更好的观点，亦或有更好的代码案例，欢迎大家进行补充说明。

笔者希望以此文抛砖引玉，如果最终形成一套完善的 Java 编码规范，善莫大焉。

最完美的 Lambda 表达式只有一行

前言

民国佛学大师李叔同在《晚晴集》中有言：

> *世界是个回音谷，念念不忘必有回响。你大声喊唱，山谷雷鸣，音传千里，一叠一叠，一浪一浪，彼岸世界都收到了。*

是的，一个好的想法，只要你念念不忘，定然必有回响。阿里巴巴的前辈孤尽、夕华，因为对 Java 的挚爱，有了自己对 Java 的见解，于是便有了《阿里巴巴 Java 规范》。

这里，对于 Java 8 推出的 Lambda 表达式，我很赞同“最完美的 Lambda 表达式只有一行”的观点。于是，便有了这一篇文章，希望这个观点得到更多的人赞同和推广。

一、 Lambda 表达式简介

Lambda 表达式，也被称为闭包，是 Java8 最重要的新特性之一。Lambda 这一单词，来源于学术界出来的一套用来描述计算的 λ 演算法。Lambda 表达式例子如下：

```
(int a, int b) -> {
    int sum = a + b;
    return sum;
}
```

Lambda 表达式本质上是一个匿名方法，实现了由函数式接口定义的唯一抽象方法。Lambda 表达式具有匿名、函数、传递、简洁这 4 大特性：

- **匿名**：Lambda 表达式不需要指定名称；
- **函数**：Lambda 表达式具有参数列表、函数主体和返回类型，还可能有可抛出的异常集合；
- **传递**：Lambda 表达式可以作为对象，作为参数对象传递给方法或存储于变量中；
- **简洁**：Lambda 表达式无需向匿名类那样写很多的样板代码。

总之，使用 Lambda 表达式，可以写出更简洁、更灵活、更紧凑的代码，从而使的 Java 语言的表达能力得到提升。

二、 Lambda 表达式语法

最完整的 Lambda 表达式语法如下：

- **表达式语法**

```
(Type1 parameter1, Type2 parameter2, ..., TypeN parameterN) -> {
    statement1;
    statement2;
    ...
    statementN;
}
```

普通表达式语法主要包括以下内容：

a) 参数列表：用小括号（）包括起来的参数列表，跟函数式接口的方法参数保持一致；
b) 分隔符：用箭头符号->分隔参数列表和方法体；
c) 方法体：用花括符{}包括起来的代码块，代码块中包括多条语句，最后可能存在一条 return 语句。

- **表达式案例**

```
(int index, int total, UserDO user) -> {
    log.info("{}/{}:开始处理用户({})...", index, total, user.getId());
    ... // 处理用户代码
    log.info("{}/{}:完成处理用户({}).", index, total, user.getId());
    return true;
}
```

普通 Lambda 表达式语法还具有以下特征：

a) **可选的类型声明**：不需要声明参数类型，编译器可以自动识别参数类型；
b) **可选的参数圆括号**：只有一个参数无需定义圆括号，但多个参数需要定义圆括号；
c) **可选的主体大括号**：如果主体只有一条语句，就不需要定义主体的大括号；

d) **可选的返回关键字**：如果主体只有一条返回语句，就不需要定义返回关键字。

根据 Lambda 表达式以上特征，可以得到以下几种变种表达式语法。

1. 无类型表达式语法

根据普通 Lambda 表达式语法的特征——“可选的类型声明”，可以去掉参数类型的声明，编译器可以自动识别参数类型。

- **表达式语法**

```
(parameter1, parameter2, ..., parameterN) -> {
    statement1;
    statement2;
    ...
    statementN;
}
```

- **表达式案例**

```
(index, total, user) -> {
    log.info("{}/{}:开始处理用户({})...", index, total, user.getId());
    ... // 处理用户代码
    log.info("{}/{}:完成处理用户({}).", index, total, user.getId());
    return true;
}
```

2. 单参数表达式语法

根据普通 Lambda 表达式语法的特征——“可选的参数圆括号”，可以去掉单参数类型的圆括号。

- **表达式语法**

```
parameter -> {
    statement1;
    statement2;
    ...
    statementN;
}
```

- **表达式案例**

```
user -> {
    log.info("开始处理用户({})...", user.getId());
    ... // 处理用户代码
    log.info("完成处理用户({}).", user.getId());
    return true;
}
```

3. 单语句表达式语法

根据普通 Lambda 表达式语法的特征——“可选的主体大括号”和“可选的返回关键字”，可以形成单语句 Lambda 表达式。

- **表达式语法**

```
(parameter1, parameter2, ..., parameterN) -> statement
```

- **表达式案例**

```
user -> handleUser(user)
() -> System.out.println("Lambda表达式")
(index, total, user) -> handleUser(index, total, user)
```

三、 Lambda 表达式方法引用

方法引用是一种“Lambda 表达式用来直接访问类或实例的已存在的普通方法或构造方法”的方法，提供了一种“只引用而不执行方法”的方式。

Java8 对方法引用的支持，只是 Java 编译器层面的支持，Java 虚拟机仍然执行的是函数接口的实现。当 Java 编译器遇到方法引用时，Java 编译器会自动推断出并将方法引用还原成函数接口实现。可以这样认为，方法引用只是 Java8 提供的语法糖。

方法引用仍然是一个 Lambda 表达式，其中方法引用的操作符是双冒号“::”。主要有以下几种方法引用语法：

1. 构造方法引用语法

当 Lambda 表达式参数与调用的构造方法参数一致时，可以采用构造方法引用语法。

- **表达式语法**

```
ClassName::new
```

- **原始代码**

```
Map<String, String> roleMap = Optional.ofNullable(roleService.getRoleMap())
    .orElseGet(()->new HashMap<>());
```

- **优化代码**

```
Map<String, String> roleMap = Optional.ofNullable(roleService.getRoleMap())
    .orElseGet(HashMap::new);
```

2. 静态方法引用语法

当 Lambda 表达式参数与调用的静态方法参数一致时，可以采用静态方法引用语法。

- **表达式语法**

```
ClassName::staticMethodName
```

- **原始代码**

```
valueList.forEach(value -> System.out.println(value));
Collections.sort(valueList, (value1, value2) -> String.compare(value1, value2));
```

- **优化代码**

```
valueList.forEach(System.out::println);
Collections.sort(valueList, String::compare);
```

3. 实例方法引用语法

当 Lambda 表达式参数与调用的对象实例方法参数一致时，可以采用实例方法引用语法。

- **表达式语法**

```
objectName::instanceMethodName
```

- **原始代码**

```
int loginCount = Optional.ofNullable(user.getDeviceId())
    .map(deviceId -> deviceService.getLoginCount(deviceId))
    .orElse(0);
```

- **优化代码**

```
int loginCount = Optional.ofNullable(user.getDeviceId())
    .map(deviceService::getLoginCount)
    .orElse(0);
```

注意：
如果 getLoginCount 在本身所在的类中，可以把 deviceService 修改为 this。

4. 参数类方法引用语法

当 Lambda 表达式只有一个参数且调用该参数的无参类方法时，可以采用参数类方法引用语法。

- **表达式语法**

```
ClassName::instanceMethodName
```

- **原始代码**

```
String zipcode = Optional.ofNullable(user)
    .map(user -> user.getAddress())
    .map(address -> address.getCountry())
    .map(country -> country.getZipcode())
    .orElse(null);
```

- **优化代码**

```
String zipcode = Optional.ofNullable(user)
    .map(User::getAddress)
    .map(Address::getCountry)
    .map(Country::getZipcode)
    .orElse(null);
```

四、 Lambda 表达式知识点

此外，Lambda 表达式还涉及到函数式接口、类型推断、this 指向对象、变量作用域等知识点。

1. 函数式接口

函数式接口是只有一个抽象方法的接口，用作表示 Lambda 表达式的类型。比如 Java 标准库中的 java.lang.Runnable 和 java.util.Comparator 都是典型的函数式接口。

Java8 提供@FunctionalInterface 作为注解，这个注解是非必须的，只要接口符合函数式接口的标准（即只包含一个方法的接口），虚拟机会自动判断。但是，最好在接口上使用注解@FunctionalInterface 进行声明，以避免团队的其他人员错误地往该接口中添加新的方法。举例如下：

```
@FunctionalInterface
public interface MessageHandler
    boolean handleMessage(Message message);
}
```

2. 类型推断

1) 类型推断

Java 编译器会从上下文中推断出用什么函数式接口来配合 Lambda 表达式。

Java 编译器类型推断步骤如下：

a) 首先，根据 Lambda 表达式对应的方法、参数和返回值，确定使用了哪个函数式接口；
b) 然后，Java 编译器根据这个函数式接口，获取到唯一抽象方法的函数描述符（参数和返回值类型）；
c) 最后，Java 编译器通过函数描述符推断出 Lambda 表达式的参数类型。

2) 类型检查

利用 Java 编译器推断出来的函数描述符（参数和返回值类型），验证 Lambda 表达式参数是否合法。

比如，我们要从一堆苹果中筛选出直径大于等于 80 毫米的合格苹果：

```
List<Apple> qualifiedAppleList = appleList.stream()
    .filter(apple -> apple.getDiameter() >= 80)
    .collect(Collectors.toList());
```

通过简单推断，该 Lambda 表达式命中了 Stream 的方法：

```
Stream<T> filter(Predicate<? super T> predicate);
```

其中，命中的函数式接口 Predicate 定义为：

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

那么，我们可以推断出 Lambda 表达式的输入参数类型为 Apple，返回值类型为 boolean。

3. this 指向对象

总所周知，Lambda 表达式可以用来取代唯一抽象方法的内部匿名类的。但是，对于 this 指针指向对象，却是完全不一样的：

- 对于 Java 中的匿名内部类，编译器会自动生成它的类名（外部类类名$数字）。而匿名内部类中的 this，将指向的是这个内部类对象本身。
- 对于 Java 中的 Lambda 表达式中的 this，指向的是 Lambda 表达式所在类的对象。也就是说，Lambda 表达式中的 this，与普通表达式中的 this，没有任何区别。

执行下面方法：

```
public void testThis() {
    System.out.printf("普通类this:%s%n", this);
    new Thread(new Runnable() {
        @Override
        public void run() {
            System.out.printf("匿名内部类this:%s%n", this);
        }
    }).start();
    new Thread(() -> System.out.printf("Lambda表达式this:%s%n", this)).start();
}
```

得到以下结果：

```
普通类this:com.amap.aos.order.LambdaThisTest@23223dd8
匿名内部类this:com.amap.aos.order.LambdaThisTest$1@10a16904
Lambda表达式this:com.amap.aos.order.LambdaThisTest@23223dd8
```

可以看出，Lambda 表达式 this 指向对象与普通类 this 指向对象一致。如果把其中的“this”替换为“this.getClass().getName()”，还能打印出 this 指向对象类的名称。

4. 变量作用域

Java 局部类和匿名类都存在变量捕获（Captured Variable）和变量隐藏（Shadow Variable），但 Lambda 表达式却有些许不同——只存在变量捕获，不存在变量隐藏。

也就是说，Lambda 表达式的作用域：

- Lambda 表达式不会从超类继承或引入新级别的作用域；
- Lambda 表达式中的声明变量和普通封闭程序块中的一样。

```
String string = "Hello world";
new Thread(() -> System.out.println(string)).start();
```

Lambda 表达式可以无限制地捕获变量或常量，但是局部变量必须定义为 final 或准 final 型（不允许修改）。这是因为，Lambda 表达式只通过 this 指针捕获一次局部变量值，后续局部变量发生更改将无法得知。所以，干脆禁止这些局部变量的更改，期望这些局部变量被定义为 final 或准 final 型，否则会出现编译错误。

```
String string = "Hello world"; // 定义为准final型
new Thread(() -> {
    System.out.println(string);
    string = "inner"; // 报错,不允许修改
}).start();
string = "outer"; // 报错,不允许修改
```

五、 丑陋的 Lambda 表达式

由于 Lambda 表达式比较简洁和自由，所以写出来的代码天马星空，很容易造成代码可阅读性、可维护性差的问题。下面，作者总结了一些丑陋的 Lambda 表达式，并给出一些整改解决方案。

1. 无法理解的 Lambda 表达式

请看下面的 Lambda 表达式，不仔细阅读 Lambda 表达式的代码，并不知道这段 Lambda 表达式是干啥的。

```
int[] primes = IntStream.rangeClosed(1, 100).filter(n -> {
    if (n <= 3) {
        return n > 1;
    }
    int mid = n / 2;
    for (int i = 2; i < mid; i++) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}).toArray();
```

如果抽象出一个私有静态“isPrime”（是否质数）方法，这段代码将会变得更容易理解。

```
int[] primes = IntStream.rangeClosed(1, 100).filter(TestClass::isPrime).toArray();
```

2. 无法复用的 Lambda 表达式

假设，在代码中存在大量的重复 Lambda 表达式，将是代码变得更加臃肿和庞大。

```
int[] primes = IntStream.rangeClosed(1, 100).filter(n -> {
    ...
    return true;
}).toArray();
int[] otherPrimes = IntStream.rangeClosed(1, 100).filter(n -> {
    ...
    return true;
}).toArray();
```

如果抽象出一个公共“PrimeHelper.isPrime”（ 是否质数）方法，这两段代码可以复用一个方法并变得更简洁。

```
int[] primes = IntStream.rangeClosed(1, 100).filter(PrimeHelper::isPrime).toArray();
int[] otherPrimes = IntStream.rangeClosed(1, 100).filter(PrimeHelper::isPrime).toArray();
```

3. 无法单测的 Lambda 表达式

如果 Lambda 表达式的执行对象被 Mock，就会导致单元测试中的 Lambda 表达式无法被执行。因此，该 Lambda 表达式将无法被单元测试所覆盖。

```
List<EmployeeDO> employeeList = odpsService.executeQuery(sql, record -> {
    EmployeeDO employee = new EmployeeDO();
    employee.setId(OdpsHelper.getLong(record, "id"));
    ...
    return employee;
});
```

如果抽象出一个私有“parseEmployee”（解析雇员）方法，可以利用下列方法进行单元测试：

```
@Test
public void testParseEmployee() throws Exception {
    Record record = Mockito.mock(Record.class);
    ... // 模拟record属性值
    EmployeeDO employee = Whitebox.invokeMethod(employeeService, "parseEmployee", record);
    ... // 验证employee属性值
}
```

注意：

如果抽象出私有静态方法，只需要把“Whitebox.invokeMethod”方法第一个参数改为“EmployeeService.class”即可。

4. 无法确定入参的 Lambda 表达式

Lambda 表达式跟匿名内部类一样，可以无限制地获取方法中的 final 或准 final 局部变量。这就导致，如果不完全阅读 Lambda 表达式的代码，就无法知道——Lambda 表达式除了声明的输入参数外，还把多少方法输入参数或局部变量作为输入参数。

```
public List<EmployeeVO> queryEmployee(Long companyId) {
    Map<String, String> positionMap = positionService.getPositionMap(companyId);
    List<EmployeeDO> employeeList = employeeDAO.queryByCompanyId(companyId);
    return employeeList.stream()
        .map(employeeDO -> {
            EmployeeVO employeeVO = new EmployeeVO();
            employeeVO.setId(employeeDO.getId());
            employeeVO.setCompanyId(companyId);
            employeeVO.setPositionName(positionMap.get(employeeDO.getPositionId()));
            ...
            return employeeVO;
        })
        .collect(Collectors.toList());
}
```

如果抽象出一个“buildEmployee”（构建雇员）方法，将会明确这个 Lambda 表达式的输入参数。

```
public List<EmployeeVO> queryEmployee(Long companyId) {
    Map<String, String> positionMap = positionService.getPositionMap(companyId);
    List<EmployeeDO> employeeList = employeeDAO.queryByCompanyId(companyId);
    return employeeList.stream()
        .map(employee -> buildEmployee(companyId, employee, positionMap))
        .collect(Collectors.toList());
}
private static EmployeeVO buildEmployee(Long companyId, EmployeeDO employeeDO, Map<String,
String> positionMap) {
    EmployeeVO employeeVO = new EmployeeVO();
    employeeVO.setId(employeeDO.getId());
    employeeVO.setCompanyId(companyId);
    employeeVO.setPositionName(positionMap.get(employeeDO.getPositionId()));
    ...
    return employeeVO;
}
```

5. 代码量大的 Lambda 表达式

在阿里巴巴的《Java 开发手册》中有这样的规定：

【推荐】单个方法的总行数不超过 80 行。

如果一个 Lambda 表达式代码量过于庞大，不光会影响代码的阅读行，也将会影响本身方法的代码总行数。

```
public List<EmployeeVO> queryEmployee(Long companyId) {
    List<EmployeeDO> employeeList = employeeDAO.queryByCompanyId(companyId);
    return employeeList.stream()
        .map(employeeDO -> {
            EmployeeVO employeeVO = new EmployeeVO();
            employeeVO.setId(employeeDO.getId());
            ... // 几十上百行
            return employeeVO;
        })
        .collect(Collectors.toList());
}
```

如果抽象出一个“buildEmployee”（构建雇员）方法，将会大大降低本身方法的代码总行数。

```
public List<EmployeeVO> queryEmployee(Long companyId) {
    List<EmployeeDO> employeeList = employeeDAO.queryByCompanyId(companyId);
    return employeeList.stream()
        .map(EmployeeService::buildEmployee)
        .collect(Collectors.toList());
}
private static EmployeeVO buildEmployee(EmployeeDO employeeDO) {
    EmployeeVO employeeVO = new EmployeeVO();
    employeeVO.setId(employeeDO.getId());
    ... // 几十上百行
    return employeeVO;
}
```

6. 套娃一样的 Lambda 表达式

Lambda 表达式套 Lambda 表达式，一层一层地嵌套下去，只会让代码变得难以阅读和维护。

```
boolean hasRight = userList.stream().filter(user -> {
   List<Role> roleList = user.getRoleList();
   return roleList.stream().filter(rold -> {
       List<Integer> rightIdList = role.getRightIdList();
       return rightIdList.stream().filter(RIGHT_ID_SET::contains);
   }).findAny().isPresent();
}).findAny().isPresent();
```

如果每层 Lambda 表达式都抽象出一个方法，将会使代码变得更便于阅读和维护。

```
boolean hasRight = userList.stream().filter(UserService::hasRight).findAny().isPresent();
private static boolean hasRight(User user) {
    List<Role> roleList = user.getRoleList();
    return roleList.stream().filter(UserService::hasRight).findAny().isPresent();
}
private static boolean hasRight(Role role) {
    List<Integer> rightIdList = role.getRightIdList();
    return rightIdList.stream().filter(RIGHT_ID_SET::contains);
}
```

六、 完美的 Lambda 表达式

简短的 Lambda 表达式是提高代码可读性，这是函数式编程的重要好处之一。但是，复杂的 Lambda 表达式具有相反的效果，会让代码变得更加杂乱且难以阅读。如果将复杂的 Lambda 表达式变成一行，这样就很容易避免这些问题。这里，作者提倡一个理念——“最完美的 Lambda 表达式只有一行”。

如何把 Lambda 表达式简化为一行，作者总结了以下方法供大家参考。

1. 利用省略语法来简化 Lambda 表达式

利用 Lambda 省略语法，尽量把 Lambda 表达式精简为单语句的表达式，使其成为只有一行的最完美的 Lambda 表达式。常见的精简 Lambda 表达式的省略语法有：

- **去掉参数类型**：Lambda 表达式不需要声明参数类型，编译器可以自动识别参数类型；
- **去掉参数圆括号**：Lambda 表达式只有一个参数无需定义圆括号，但多个参数需要定义圆括号；
- **去掉主体大括号**：如果 Lambda 表达式主体只有一条语句，就不需要定义主体的大括号；
- **去掉返回关键字**：如果 Lambda 表达式主体只有一条返回语句，就不需要定义返回关键字。

下面，将以“求 1 到 100 的偶数和”为例说明如何“精简为单语句表达式”。

- **原始代码**

```
int sum = IntStream.rangeClosed(1, 100)
    .filter((int e) -> {
        return e % 2 == 0;
    })
    .sum();
```

- **优化代码**

```
int sum = IntStream.rangeClosed(1, 100)
    .filter(e -> e % 2 == 0)
    .sum();
```

2. 利用方法引用来简化 Lambda 表达式

通过上一章方法优化的单语句 Lambda 表达式，还有可能存在继续优化的空间。比如，可以替换为方法引用 Lambda 表达式（包括构造方法引用、静态方法引用、实例方法引用和参数类方法引用），会让 Lambda 表达式显得更精简。

- **原始代码**

```
Set<Long> studentIdSet = studentList.stream()
    .filter(student -> Objects.nonNull(student))
    .map(student -> student.getId())
    .collect(Collectors.toCollection(() -> new TreeSet<>()));
```

- **优化代码**

```
Set<Long> studentIdSet = studentList.stream()
    .filter(Objects::nonNull)
    .map(Student::getId)
    .collect(Collectors.toCollection(TreeSet::new));
```

在上面优化代码中，同时采用了三种方法引用语法：

- Objects::nonNull 采用了静态方法引用语法；
- Student::getId 采用了参数类方法引用语法；
- TreeSet::new 采用了构造方法引用语法。

3. 利用提取方法来简化 Lambda 表达式

对于无法简化为单语句或方法引用的多语句 Lambda 表达式，可以先提取 Lambda 表达式为一个方法，然后采用静态方法引用语法或实例方法引语法用来简化 Lambda 表达式。

- **原始代码**

```
public List<EmployeeVO> queryEmployee(Long companyId) {
    List<EmployeeDO> employeeList = employeeDAO.queryByCompanyId(companyId);
    return employeeList.stream()
        .map(employeeDO -> {
            EmployeeVO employeeVO = new EmployeeVO();
            employeeVO.setId(employeeDO.getId());
            ...
            return employeeVO;
        })
        .collect(Collectors.toList());
}
```

- **优化代码**

```
public List<EmployeeVO> queryEmployee(Long companyId) {
    List<EmployeeDO> employeeList = employeeDAO.queryByCompanyId(companyId);
    return employeeList.stream()
        .map(EmployeeService::buildEmployee)
        .collect(Collectors.toList());
}
private static EmployeeVO buildEmployee(EmployeeDO employeeDO) {
    EmployeeVO employeeVO = new EmployeeVO();
    employeeVO.setId(employeeDO.getId());
    ...
    return employeeVO;
}
```

在上面例子中，提取 Lambda 表达式为一个私有静态方法，然后采用静态方法引用简化 Lambda 表达式。为什么不提取 Lambda 表达式为一个私有普通方法呢？因为提取的 buildEmployee 方法非常独立，不依赖与 EmployeeService 类中的任何属性字段，一般建议定义为私有静态方法。

七、 Lambda 表达式实践

上面介绍了这么多知识，这章将详细介绍 Lambda 表达式的实践。这里，将以 ODPS 查询封装为例子，详细地介绍如何使用和优化 Lambda 表达式。

1. 查询 ODPS 数据代码

这里，有一段查询公司雇员的方法：

```
public List<EmployeeDO> queryEmployee(Long companyId) {
    try {
        // 组装SQL语句
        String sql = ...;

        // 执行ODPS查询
        Odps odps = getOdps();
        Instance instance = SQLTask.run(odps, DEFAULT_TASK_NAME);
        instance.waitForSuccess();

        // 获取查询结果
        List<Record> recordList = SQLTask.getResult(instance);
        if (CollectionUtils.isEmpty(recordList)) {
            return Collections.emptyList();
        }

        // 依次读取数据
        List<EmployeeDO> employeeList = new ArrayList<>();
        for (Record record : recordList) {
            EmployeeDO employee = new EmployeeDO();
            employee.setId(OdpsHelper.getLong(record, "id"));
            ... // 获取雇员属性
            employeeList.add(employee);
        }

        // 返回查询结果
        return employeeList;
    } catch (OdpsException e) {
        log.error("查询公司雇员异常:companyId={}", companyId, e);
        throw new BizRuntimeException("查询公司雇员异常", e);
    }
}
```

类似地，还有几段从 ODPS 查询数据的代码。

2. 利用内部类进行优化

因为上面代码的重复性，我们可以采用设计模式——模板方法模式（Template Method Pattern），把相同的部分固化下来，并把不同的部分抽象出接口。那么，我们就抽象出一个接口 DataParser（数据解析器），并把 ODPS 数据查询固化到 OdpsService 的 executeQuery 方法中。

```
public interface DataParser<T>
    T parse(Record record);
}

public class OdpsService {
    public <T> List<T> executeQuery(String sql, DataParser<T> dataParser) {
      try {
            // 执行ODPS查询
            Odps odps = getOdps();
            Instance instance = SQLTask.run(odps, sql);
            instance.waitForSuccess();

            // 获取查询结果
            List<Record> recordList = SQLTask.getResult(instance);
            if (CollectionUtils.isEmpty(recordList)) {
                return Collections.emptyList();
            }

            // 依次读取数据
            List<T> dataList = new ArrayList<>();
            for (Record record : recordList) {
                T data = dataParser.parse(record);
                if (Objects.nonNull(data)) {
                    dataList.add(data);
                }
            }

            // 返回查询结果
            return dataList;
        } catch (OdpsException e) {
            log.warn("执行ODPS数据查询异常:sql={}", sql, e);
            throw new BizRuntimeException("执行ODPS数据查询异常", e);
        }
  }
}
```

那么，采用匿名内部类的编写方法，可以把 queryEmployee 改为：

```
public List<EmployeeDO> queryEmployee(Long companyId) {
    // 组装SQL语句
    String sql = ...;

    // 查询雇员列表
    return odpsService.executeQuery(sql, new DataParser<EmployeeDO>() {
        @Override
      public EmployeeDO parse(Record record) {
            EmployeeDO employee = new EmployeeDO();
            employee.setId(OdpsHelper.getLong(record, "id"));
            ... // 获取雇员属性
            return employee;
        }
    });
}
```

3. 利用 Lambda 表达式优化

从上面例子可以看出，利用内部类实现 queryEmployee 方法，由于模式代码比较多，显得代码比较啰嗦。这里，利用 Lambda 表达式进行优化。

```
public List<EmployeeDO> queryEmployee(Long companyId) {
    // 组装SQL语句
    String sql = ...;

    // 查询雇员列表
    return odpsService.executeQuery(sql, record -> {
        EmployeeDO employee = new EmployeeDO();
        employee.setId(OdpsHelper.getLong(record, "id"));
        ... // 获取雇员属性
        return employee;
    });
}
```

可以看出，代码量大大地减少了。

4. 去掉自定义的函数式接口

为了减少代码中自定义函数式接口，可以利用 Java 提供的 4 个基本函数式接口。这里，我们采用函数接口“Function<Record，T>”替换掉“DataParser<T>”。

```
public class OdpsService {
    public <T> List<T> executeQuery(String sql, Function<Record, T> dataParser) {
        ... // 与代码一致，只是把"dataParser.parse"替换为"dataParser.apply"
    }
}
```

5. 利用提取方法优化 Lambda 表达式

上面的例子中，最大的问题就是——Lambda 表达式理解困难，没有一个名称或注释，需要看完 Lambda 表达式后才能理解。通过第 6 章的方法，可以把 Lambda 表达式抽象成一个方法，从而达到优化 Lambda 表达式的目的。

```
public List<EmployeeDO> queryEmployee(Long companyId) {
    // 组装SQL语句
    String sql = ...;

    // 查询雇员列表
    return odpsService.executeQuery(sql, record -> parseEmployee(record));
}
private static EmployeeDO parseEmployee(Record record) {
    EmployeeDO employee = new EmployeeDO();
    employee.setId(OdpsHelper.getLong(record, "id"));
    ... // 获取雇员属性
    return employee;
}
```

6. 利用方法引用优化 Lambda 表达式

我们使用 Lambda 表达式，就是希望代码更精简。上面例子中，Lambda 表达式还可以采用方法引用的方式进行精简，从而最终达到“完美的 Lambda 表达式只有一行”的目标。

```
public List<EmployeeDO> queryEmployee(Long companyId) {
    // 组装SQL语句
    String sql = ...;

    // 查询雇员列表
    return odpsService.executeQuery(sql, EmployeeService::parseEmployee);
}
```

7. 变化 1：需要传递参数或变量

如果需要把方法参数 companyId 传给每一个 EmployeeDO 对象，最终的代码如下：

```
public List<EmployeeDO> queryEmployee(Long companyId) {
    // 组装SQL语句
    String sql = ...;

    // 查询雇员列表
    return odpsService.executeQuery(sql, record -> parseEmployee(companyId, record));
}
private static EmployeeDO parseEmployee(Long companyId, Record record) {
    EmployeeDO employee = new EmployeeDO();
    employee.setId(OdpsHelper.getLong(record, "id"));
    employee.setCompanyId(companyId);
    ... // 获取雇员属性
    return employee;
}
```

8. 变化 2：需要抽象出多个方法

如果需要抽象出多个方法，还想使用函数式编程，就需要声明多个函数式接口参数。比如：分批查询并保存数据。

```
// 定义代码示例
public class OdpsService {
    public <T> void executeQuery(String sql, int batchSize, Function<Record, T> dataParser,
Predicate<List<T>> dataStorage) {
        ... // 具体实现代码省略
    }
}

// 调用代码示例
odpsService.executeQuery(sql, 100, EmployeeService::parseEmployee, this::storageEmployee);
```

注意：

“parseEmployee” 被定义为 static 方法，方法引用为“EmployeeService::parseEmployee”；而“storageEmployee”被定义为 private 方法，方法引用为“this::storageEmployee”。具体原因，请参考第 3 章。

9. 总结：由繁至简，由简至明

通过这个过程，可以总结出一个使用 Lambda 表达式的历程——“由繁至简，由简至明”：

1) 由繁至简

通过 Lambda 表达式，消除了很多模式代码，把复杂的代码进行简化；但是，也正由于代码过于简略，失去了代码可读性和可维护行。

2) 由简至明

为了提高代码的可读性和可维护行，又不得不损失一些简化，利用提取方法等机制，让 Lambda 表达式变得更明确。

八、 Lambda 表达式使用经验

下面，将介绍函数式编程和 Lambda 表达式的一些使用经验。

1. 规避传入可变参数

由于 Lambda 表达式只接受 final 不可变参数，或没有 final 修饰的无修改参数。如果希望传入的参数可修改，可以利用对象（容器对象不可变，但容器内容可变）解决。

```
int total = userList.size();
AtomicInteger index = new AtomicInteger(0);
userList.forEach(user -> {
    log.info("{}/{}:开始处理用户({})...", index.incrementAndGet(), total, user.getName());
    ...
});
```

如上所示，利用 AtomicInteger 解决了序号递增的问题。

2. 避免自定义函数式接口

为了减少代码中自定义函数式接口，可以利用 Java 提供的 4 个基本函数式接口：

- Supplier<T>（提供者）：没有输入参数，返回一个 T 类型的值；
- Consumer<T>（消费者）：接收一个 T 类型的参数，并且没有返回值；
- Function<T，R>（函数）：接收一个 T 类型的参数，返回一个 R 类型的值；
- Predicate<T>（断言）：接收一个 T 类型的参数，返回一个 boolean 类型的值。

在包“java.util.function”中，Java 提供了大量的函数式接口，大家可以实际需求选择使用。

- **原始代码**

```
@FunctionalInterface
public interface DataParser<T>
    T parse(Record record);
}

public <T> List<T> executeQuery(String sql, DataParser<T> dataParser) {
    ... // 代码逻辑
}
```

- **优化代码**

```
public <T> List<T> executeQuery(String sql, Function<Record, T> dataParser) {
    ... // 代码逻辑
}
```

3. 解决延迟调用问题

尽量使用方法引用传递代替值传递，可以让方法在真正使用时才调用，避免一些不必要的方法计算，我们称之为“延迟调用”。

在下面代码中，无论“localValue”是否为空，都将执行“getRemoteValue”方法获取值。

```
Double localValue = ...;
Double value = Optional.ofNullable(localValue).orElse(getRemoteValue());
```

那么，我们可以利用“Optional”的“orElseGet”方法实现延迟调用：

```
Double value = Optional.ofNullable(localValue).orElseGet(() -> getRemoteValue());
```

4. 解决初始化计算问题

根据模板工厂设计模式，需要抽象出一个或多个方法，并实现一个固定的算法流程。如果在算法流程中，循环地调用某一个方法，并且在这个方法中都要计算同一套参数，就存在一个初始化计算问题——应该提前初始化这些参数。

如果这些参数固定（每个实现类都用这套参数），我们可以抽象出一个初始化参数的虚方法，利用这个方法计算得出参数值，然后再把这些参数传入对应的方法。但是，如果这些参数不固定（根据实现类不同而不同），就不能采用这个方法了。

这里，作者给出一个基于 Lambda 表达式（函数式编程）的解决方案。

```
public abstract class AbstractUserFilter {
    public List<User> filterUser(FilterContext context, List<User> userList) {
        Predicate<User> checker = getChecker(context);
        return userList.stream().filter(checker).collect(Collectors.toList());
    }
    public abstract Predicate<User> getChecker(FilterContext context);
}

public class SimpleUserFilter extends AbstractUserFilter {
    @Override
    public Predicate<User> getChecker(FilterContext context) {
        int k1 = getK1(context);
        int k2 = getK2(context);
        return user -> isValid(k1, k2, user);
    }
    private int getK1(FilterContext context) {
        ... // 根据上下文计算参数k1
        return 0;
    }
    private int getK2(FilterContext context) {
        ... // 根据上下文计算参数k2
        return 0;
    }
    private boolean isValid(int k1, int k2, User user) {
        ... // 根据k1,k2判断user是否有效
        return true;
    }
}
```

如上所示：首先计算出参数 k1、k2 值，再返回一个基于这些参数实现的方法引用。主流程使用这个方法引用判断用户是否有效，从而避免了每次调用对参数 k1、k2 值的计算。

5. 解决类型推断冲突问题

有时候，重载的两个方法（方法名完全一致），为了区分是否有返回值，需要定义两个不同的参数类型。如果，这两个参数类型是函数式接口，且这两个函数式接口定义完全一致，在使用 Lambda 表达式时，就会出现类型推断冲突的问题。

```
public interface JedisExecutor {
    void execute(JedisExecutorWithoutResult executor);
    <T> List<T> execute(JedisExecutorWithResult executor);
}

@FunctionalInterface
public interface JedisExecutorWithoutResult {
    void execute(Pipeline pipeline);
}

@FunctionalInterface
public interface JedisExecutorWithResult {
    void execute(Pipeline pipeline);
}
```

通常，我们会如下编写 Lambda 表达式：

```
jedisExecutor.execute(pipeline -> {
  ...
});
```

但是，这个时候编辑器就会报出以下错误：

```
The method execute(JedisExecutorWithoutResult) is ambiguous for the type JedisExecutor
```

那么，我们需要加上类型强制转换：

```
jedisExecutor.execute((JedisExecutorWithoutResult)pipeline -> {
  ...
});
```

但是，SonarLint 还会出现以下警告：

```
Remove this unnecessary cast to "JedisExecutorWithoutResult".
```

最后，给出最完美的解决方案：

```
JedisExecutorWithoutResult executor = pipeline -> {
  ...
};
jedisExecutor.execute(executor);
```

6. 简化常用的设计模式

在软件开发过程中，软件设计模式无处不在。通过函数式编程，减少了不必要接口声明及类实现，能够让设计模式的运用更灵活。

比如下面的代码中，明显有很多的通用代码。如果按照模板方法模式（Template Method Pattern）进行封装实现，将会编写大量的模式代码。

```
@Service
public class OrderMockPriceService {
    @Resource
    private MockConfig mockConfig;

    public Integer getMainMockPrice(String userId, Integer orderPrice) {
        Set<String> whiteSet = mockConfig.getWhiteSet();
        if (Objects.nonNull(whiteSet) && whiteSet.contains(userId)) {
            return orderPrice;
        }
        return Optional.ofNullable(mockConfig.getMainPrice()).orElse(orderPrice);
    }
    public Integer getMiniMockPrice(String userId, Integer orderPrice) {
        Set<String> whiteSet = mockConfig.getWhiteSet();
        if (Objects.nonNull(whiteSet) && whiteSet.contains(userId)) {
            return orderPrice;
        }
        return Optional.ofNullable(mockConfig.getMiniPrice()).orElse(orderPrice);
    }
    // 其它类似方法...
}
```

如果通过函数式编程和 Lambda 表达式实现，代码量将会降到最低。

```
@Service
public class OrderMockPriceService {
    @Resource
    private MockConfig mockConfig;

    public Integer getMainMockPrice(String userId, Integer orderPrice) {
        return getMockPrice(userId, orderPrice, mockConfig::getMainPrice);
    }
    public Integer getMiniMockPrice(String userId, Integer orderPrice) {
        return getMockPrice(userId, orderPrice, mockConfig::getMiniPrice);
    }
    // 其它类似方法...

    private Integer getMockPrice(String userId, Integer orderPrice, Supplier<Integer>
mockPriceSupplier) {
        Set<String> whiteSet = mockConfig.getWhiteSet();
        if (Objects.nonNull(whiteSet) && whiteSet.contains(userId)) {
            return orderPrice;
        }
        return Optional.ofNullable(mockPriceSupplier.get()).orElse(orderPrice);
    }
}
```

7. 优化多表达式条件语句

有时候，我们会判断很多条件，需求用&&（或||）连接多个条件表达式。

```
private static AuditResult getAuditResult(AuditDataVO data) {
    boolean isPassed = isPassed(data.getAuditItem1());
    isPassed = isPassed && isPassed(data.getAuditItem2());
    ...
    isPassed = isPassed && isPassed(data.getAuditItem11());
    if (isPassed) {
        return AuditResult.PASSED;
    }
    return AuditResult.REJECTED;
}
```

通过 SonarLint 插件扫描，会存在 2 个复杂度问题：

- Expressions should not be too complex（表达式不能太复杂）；
- Methods should not be too complex（方法不能太复杂）。

这里，可以采用 Lambda 表达式进行优化，降低代码复杂度并避免上述问题。

```
private static final List<Predicate<AuditDataVO>> AUDIT_RESULT_PREDICATE_LIST =
    Collections.unmodifiableList(Arrays.asList(
        data -> isPassed(data.getAuditItem1()),
        data -> isPassed(data.getAuditItem2()),
        ...
        data -> isPassed(data.getAuditItem11())));

private static AuditResult getAuditResult(AuditDataVO data) {
    for (Predicate<AuditDataVO> predicate : AUDIT_RESULT_PREDICATE_LIST) {
        if (!predicate.test(data)) {
            return AuditResult.REJECTED;
        }
    }
    return AuditResult.PASSED;
}
```

后记

《如梦令·迎雪夜归》

冬日寒风砭骨，皓月冷光铺路。
一骑夜归迟，迎雪下车徒步。
何苦？何苦？即兴作诗吟赋！

卷尾语

古语又云：

> *有道无术，术尚可求也；有术无道，止于术。*

意思是：有“道”而无“术”，“术”还可以逐渐获得；有“术”而无“道”，就可能止步于“术”了。

所以，我们不要仅满足于从实践中总结“术”，因为“道”的表现形式是多变的；而应该上升到“道”的高度，因为“术”背后的道理是相通的。当遇到新的事物时，我们可以**从理论中找到“道”、从实践中找到“术”**，尝试着利用已知事物去认知新的事物。

寻求同道之人

《鹧鸪天·归乐至居》

老朽归来乐至居，
闲得无事钓虾鱼。
小桥流水成仙境，
佛塔香阁隐太虚。
食夏果，品秋菊，
挎篮背篓采桑榆。
人生雅趣当如是，
妻子相随共晚愉。